



Department of Computer Science

Data Quality and Reasoning in Graph Databases

Human-in-the-Loop, Machine Learning, and Causal Approaches

Supervisors:

Prof. Angela Bonifati

Prof. Andrea Mauri

Defended By:

Amedeo Pachera

Reviewers:

Prof. Gianmaria Silvello

Prof. Evaggelia Pitoura

Examiners:

Prof. Hamida Seba

Prof. Oscar Romero

Abstract

Property graphs have become the predominant model for interconnected data, yet the systems built around them remain confined to associational reasoning over data whose quality is largely taken for granted. This thesis studies data quality and reasoning over property graphs under the assumption that this knowledge is supplied by users, and is organized in three parts. Part I addresses the repair of property graphs under graph denial constraints. We formalize an interactive repair process in which violations are detected by compiling each constraint into a query and their interactions are materialized in a dependency graph, the Graph Repair Dependency Graph, which allows independent violations to be assigned to concurrent users. We define a local safety condition on accepted repairs and prove that it is necessary and sufficient for convergence, with a bound on the number of iterations that is independent of user competence. We then relax the assumptions that users are interchangeable and that their effort is free. A heterogeneous graph attention network estimates the difficulty of each violation and repairs the easiest ones automatically, users are modelled by skill, cognitive capacity and cost, and the resulting allocation problem is shown to be NP-hard by reduction from Multiple Knapsack and solved by Lagrangian relaxation with a closed-form dual certificate. On four datasets, two of them real, the first framework improves average repair quality over automated and interactive baselines, and the budgeted method matches or exceeds that quality while involving fewer users. Part II addresses reasoning over hypothetical modifications of a graph. We define the what-if query over the predictions of a trained graph neural network under a disposable update, and propose two exact maintenance procedures: a rank-one correction of the cached forward state for attribute updates, and a locality-bounded recomputation for structural updates. Both leave the base graph unmodified and compose across an exploration session, so that a sequence of queries pays only for the increment between retained states. We then extend the property graph model with causal structure, representing causal variables as hypervertices over the observational data together with the meta-properties that carry conditional and interventional information, and identify the query-language constructs required for extraction, intervention and counterfactual reasoning. Part III reports an empirical study of how people formulate queries over property graphs. We derive a taxonomy of query formulation errors organized around Cypher constructs, analyse them quantitatively and qualitatively, and examine the role of prior experience with other query languages. We conclude with an analysis of language models and agents as interfaces to graph data, and of the primitives required for such interfaces to operate over a constrained graph.

Resumé

Les graphes de propriétés sont devenus le modèle prédominant pour représenter les données interconnectées. Pourtant, les systèmes qui les exploitent restent largement limités à un raisonnement associatif sur des données dont la qualité est généralement considérée comme acquise. Cette thèse étudie la qualité des données et le raisonnement sur les graphes de propriétés en partant de l’hypothèse que les connaissances nécessaires à ces tâches sont fournies par les utilisateurs. Elle s’articule en trois parties.

La première partie porte sur la réparation des graphes de propriétés soumis à des contraintes de déni sur les graphes (graph denial constraints). Nous formalisons un processus interactif de réparation dans lequel les violations sont détectées en compilant chaque contrainte sous la forme d’une requête, tandis que leurs interactions sont matérialisées dans un graphe de dépendances, le Graph Repair Dependency Graph (graphe de dépendances pour la réparation des graphes), qui permet d’attribuer les violations indépendantes à des utilisateurs travaillant en parallèle. Nous définissons une condition locale de sûreté sur les réparations acceptées et démontrons qu’elle est nécessaire et suffisante pour garantir la convergence, avec une borne sur le nombre d’itérations qui est indépendante du niveau de compétence des utilisateurs. Nous levons ensuite les hypothèses selon lesquelles les utilisateurs sont interchangeable et leur effort gratuit. Un réseau neuronal hétérogène à attention sur les graphes (heterogeneous graph attention network) estime la difficulté de chaque violation et répare automatiquement les violations les plus faciles. Les utilisateurs sont modélisés en fonction de leurs compétences, de leur capacité cognitive et de leur coût, et le problème d’allocation qui en résulte est démontré NP-difficile par une réduction à partir du problème du sac à dos multiple (Multiple Knapsack). Il est résolu au moyen d’une relaxation lagrangienne accompagnée d’un certificat dual en forme fermée. Sur quatre jeux de données, dont deux réels, le premier cadre améliore la qualité moyenne des réparations par rapport aux approches automatisées et interactives de référence. La méthode prenant en compte le budget atteint ou dépasse cette qualité tout en mobilisant moins d’utilisateurs.

La deuxième partie porte sur le raisonnement concernant des modifications hypothétiques d’un graphe. Nous définissons une requête de type what-if portant sur les prédictions d’un réseau neuronal de graphes (Graph Neural Network, GNN) entraîné, dans le contexte d’une mise à jour temporaire. Nous proposons deux procédures exactes de maintenance : une correction de rang un de l’état de propagation (forward state) mis en cache pour les mises à jour d’attributs, et un recalcul local borné par la localité pour les mises à jour structurelles. Ces deux procédures laissent le graphe de base inchangé et peuvent être composées au cours d’une session d’exploration, de sorte qu’une séquence de requêtes ne paie que le coût correspondant à l’incrément entre les états conservés. Nous étendons ensuite le modèle des graphes de propriétés en lui ajoutant une structure

causale, en représentant les variables causales sous forme d’hyper-sommets (hypervertices) au-dessus des données observationnelles, ainsi que les méta-propriétés qui portent les informations conditionnelles et interventionnelles. Nous identifions les constructions du langage de requêtes nécessaires à l’extraction, à l’intervention et au raisonnement contrefactuel.

La troisième partie présente une étude empirique de la manière dont les utilisateurs formulent des requêtes sur les graphes de propriétés. Nous élaborons une taxonomie des erreurs de formulation des requêtes, organisée autour des constructions de Cypher, que nous analysons à la fois quantitativement et qualitativement. Nous examinons également le rôle de l’expérience préalable avec d’autres langages de requêtes. Nous concluons par une analyse des modèles de langage et des agents en tant qu’interfaces avec les données en graphe, ainsi que des primitives nécessaires à ces interfaces pour fonctionner sur un graphe soumis à des contraintes.

Contents

Abstract	i
Resumé	iii
1 Introduction	1
1.1 Motivation: Data Quality and Reasoning in Graph Databases	1
1.2 A Motivating Scenario	2
1.3 Outline and Contributions	4
1.4 Publications	6
2 Background and Preliminaries	9
2.1 The Property Graph Model	9
2.1.1 Querying a Property Graph	10
2.1.2 Graph Query Languages	12
2.1.3 PGM Extension	13
2.2 Constraints for Graphs	14
2.2.1 PG-Keys	17
2.2.2 Graph Functional Dependencies and Graph Entity Dependencies	19
2.2.3 Denial Constraints	20
2.3 Graph Neural Networks	21
2.3.1 Graph Convolution and R-GCN	22
2.3.2 Graph Attention and hGAT	24
2.4 Causality	25
2.4.1 From Correlation to Causation	25
2.4.2 The Ladder of Causation	26
2.4.3 Structural Causal Models	27
2.4.4 Graphical Structure and d-Separation	28
2.4.5 Interventions and the do-Operator	30
2.4.6 Counterfactuals	33
2.5 Human-in-the-Loop Data Management	35
2.5.1 A Taxonomy of Human Involvement	35
2.5.2 Human Effort, Cost, and Cognitive Limitations	37
2.5.3 Crowdsourcing and the Economics of Human Effort	38
2.5.4 Human Involvement Across Data Management Tasks	38

I	Cleaning Property Graphs	41
3	User-Centric Interactive Repair	43
3.1	Context	43
3.2	Related Work	46
3.3	Constraints and Violations	47
3.4	The graph repair problem	48
3.5	User-centric property graph repairs	49
3.5.1	Graph Repair Dependency Graph	51
3.5.2	Properties of the Graph Repairs	52
3.5.3	Property Graph Repair Process	55
3.5.4	Complexity	59
3.6	Experimental Evaluation	60
3.6.1	Settings	61
3.6.2	Quality Assessment	63
3.6.3	Scalability	68
3.7	Grafixer: A Tool for User-Centric Repair	69
3.8	Conclusion	72
4	Difficulty-Aware, Budgeted Repair with GNNs	73
4.1	Context	73
4.2	Related Work	75
4.3	GNN Difficulty Annotation	76
4.4	Graph Repair Framework	79
4.4.1	User Model	79
4.4.2	Problem Formulation	80
4.4.3	Automated Repairs with GNN	83
4.4.4	Lagrangian-Relaxation Assignment	84
4.5	Experimental Evaluation	88
4.5.1	Settings	88
4.5.2	Validating the difficulty estimation	89
4.5.3	Balancing human involvement	91
4.5.4	Comparison with baselines	92
4.5.5	LLMs for Data Repairs	94
4.5.6	Testing Different Constraint Types	95
4.5.7	Illustrative Examples	96
4.6	Conclusion	97
5	Conclusion of Part I	99
5.1	Summary of Part I	99
5.2	Lessons Learned	100
5.3	Limitations	102
5.4	Future Work	103

5.5	From Cleaning to Reasoning	104
II	Reasoning and Causal Analysis over Graphs	105
6	What-If Queries over GNN Predictions	107
6.1	Context	107
6.2	Related Work	110
6.3	What-if Queries	111
6.3.1	Interventions	112
6.3.2	What-if Query	113
6.3.3	The Incremental Evaluation Problem	114
6.4	Answering What-if Queries Incrementally	115
6.4.1	Attribute Update: Low-Rank Maintenance	116
6.4.2	Structure Update: Locality Maintenance	118
6.4.3	Composability of the Maintenance	120
6.5	Experimental Evaluation	121
6.5.1	Setup	121
6.5.2	Incremental vs. From-Scratch Evaluation	123
6.5.3	Comparison with other baselines	125
6.5.4	Use Case: An Interactive What-If Session	127
6.6	Conclusion	129
7	Causal Analysis with Graph Databases	131
7.1	Context	131
7.2	Graph Model	133
7.3	Causal Property Graph Queries	135
7.3.1	Causal Path Queries	136
7.3.2	Interventional Query	137
7.3.3	Counterfactual Query	137
7.3.4	Causal DAG Maintenance	138
7.4	Related Approaches	138
7.5	Prototype and Experimental study	140
7.6	Extension: Towards a Causal World Ecosystem	141
7.6.1	Architecture.	141
7.6.2	Using the system	143
7.7	Conclusion	144
8	Conclusion of Part II	145
8.1	Summary of Part II	145
8.2	Lessons Learned	146
8.3	Limitations	147
8.4	Future Work	148
8.5	Towards the Human Dimension	149

III	The Human Dimension	151
9	Understanding How Users Query Graphs	153
9.1	Introduction	153
9.2	Related Work	156
9.3	Property Graphs and the Cypher Graph Query Language	157
9.4	Methods	158
9.4.1	Dataset and Questions	159
9.4.2	Research Procedure	160
9.4.3	Participants	161
9.4.4	Error Taxonomy and Annotation of the Responses	162
9.5	Quantitative Analysis (RQ1)	163
9.5.1	Schema Discovery Queries	163
9.5.2	Analytical Queries	165
9.5.3	Aggregate Queries	167
9.6	Qualitative analysis (RQ1)	170
9.6.1	The use of <code>COLLECT</code>	170
9.6.2	The use of <code>WITH</code>	170
9.6.3	Missing and Wrong Conditions	171
9.6.4	Pattern Definition	172
9.6.5	Schema Misunderstanding	173
9.6.6	Syntax Errors	173
9.7	Investigation on the cause of the errors (RQ2)	174
9.7.1	Correlation Analysis	175
9.7.2	Understanding the overuse of <code>WITH</code>	176
9.7.3	Understanding the overuse of <code>COLLECT</code>	176
9.7.4	Understanding the comprehension of the use of the schema	177
9.7.5	Understanding the misconceptions when working with complex patterns	178
9.7.6	Discussion (RQ2)	178
9.7.7	Best Practices for Teaching (Future) Graph Query Languages	179
9.8	Limitations	180
9.9	Conclusion	181
10	Conclusion and Future Perspectives	183
10.1	Language Models and Agents as Interfaces to Graph Data	184
10.2	Directions for Future Research	186
A	Appendix for Ch. 3	189
A.1	Survey for the user study	189
A.2	Table shapes	191
A.3	Evaluation of different assignment policies with Unsafe Repairs	191
A.4	Number of Waitings with Unsafe repairs	192

B	Appendix for Ch. 4	193
B.1	Time Complexity of LARA	193
B.2	Constraints	194
B.2.1	LDBC Finbench	194
B.2.2	LDBC SNB	195
B.2.3	FAERS (Real)	195
B.2.4	ICIJ (Real)	196
B.3	Different Type of Constraints	196
B.3.1	ICIJ	196
B.3.2	FAERS (Real)	197
C	Appendix for Ch. 6	199
C.1	Additional Experimental Results	199
C.1.1	Depth Scaling at All Intervention Sizes	199
C.1.2	Absolute Latency at All Intervention Sizes	199
C.1.3	Exactness Across All GNN Tasks	200
C.2	Workload Specification	201
C.3	Full Store-Backed Comparison	202
D	Appendix for Ch. 9	207
D.1	Informed Consent	207
D.2	Demographic Information and Knowledge Self-Assessment	208
D.3	Queries Ground Truth	208
D.3.1	Schema Discovery Queries	209
D.3.2	Analytical Queries	209
D.3.3	Aggregation Queries	210
D.4	Semi-Structured Interview	211
D.5	Table extended versions	213
	Bibliography	B1

Chapter 1

Introduction

Property graphs are used to represent interconnected data in domains such as healthcare, finance, cybersecurity and social networks, where entities of different kinds and the relationships between them are stored in a single artefact. Two requirements recur wherever they are deployed: the stored data has to satisfy the integrity constraints that define its intended meaning, and the analyses computed over it have to support predictions and decision making. Neither requirement can be met from the content of the database alone. This chapter states the problem that the two requirements raise, develops a single scenario in which both appear together, and presents the organisation of the thesis, its contributions and the publications on which it is based.

1.1 Motivation: Data Quality and Reasoning in Graph Databases

The property graph model represents data as a directed multigraph in which nodes and edges carry labels and key-value properties. It is supported by the major graph database systems and has been standardised, with GQL as an ISO query language and PG-Schema as a companion schema formalism [1, 2]. It is also equipped with a constraint theory: graph keys, graph functional dependencies, graph entity dependencies and graph denial constraints define what a valid instance is, and several of these families subsume their relational counterparts for common data quality tasks [3, 4].

Two problems recur wherever property graphs are used in practice. The first is data quality. Graph data accumulates incorrect property values, mislabelled nodes and edges, and violations of the constraints above, and practitioners of graph processing systems report data cleaning as one of the most time-consuming activities in graph applications [5, 6]. Detecting a violation is a matter of pattern matching, since a constraint compiles into a query whose answers are the offending subgraphs. Repairing it is not: a single violation admits several repairs that all restore consistency while corresponding to different readings of the data, and the information needed to choose among them is frequently absent from the graph. This is the reason automated repair methods plateau well below the quality their benchmarks define [7], and the reason the choice is usually delegated to an expert user.

The second problem is that the analyses computed over graph data have moved beyond retrieval. Graph neural networks predict properties of nodes, edges and subgraphs from the combination of topology and attributes [8, 9], and their predictions are increasingly consumed by

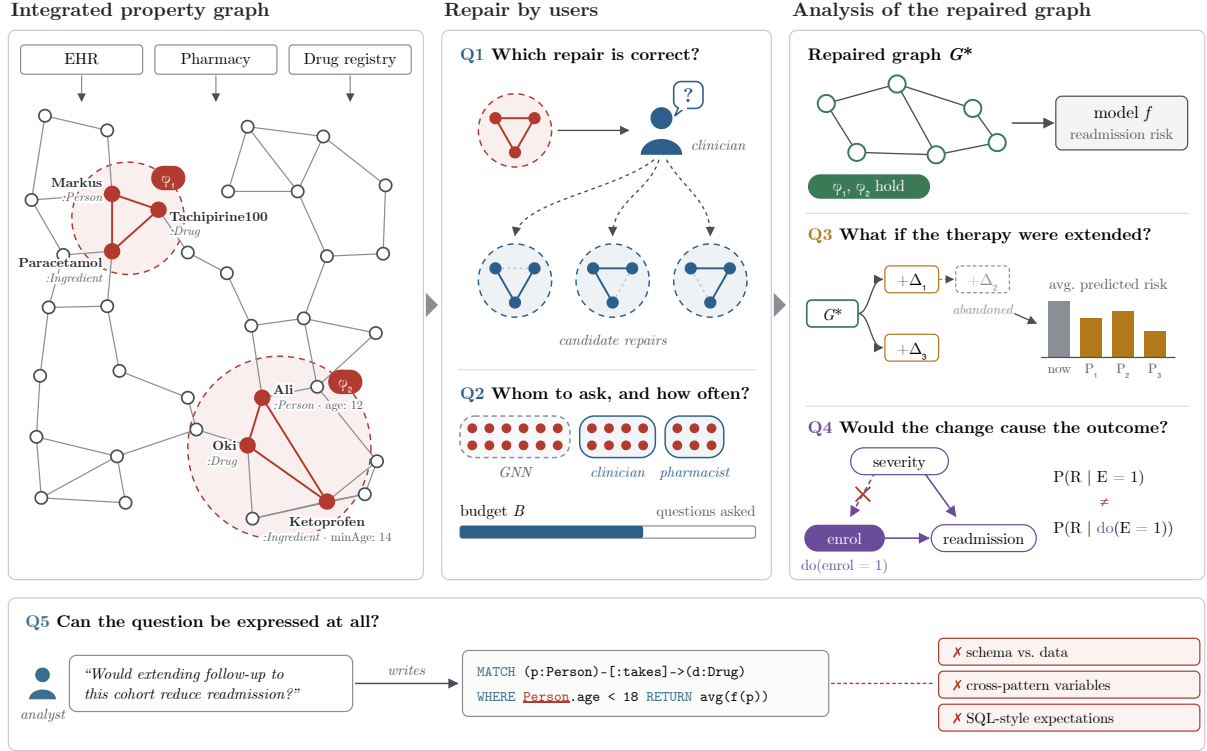


Figure 1.1: Motivating Scenario

users who have to act on them. Acting requires more than a prediction: it requires knowing what the analysis would return if the data were different, and, further, whether a change to the data would cause the outcome to change. These are the interventional and counterfactual levels of the ladder of causation [10, 11], and neither is answered by a model trained on observational data. They also share a structural requirement with repair: the alternative worlds under consideration are modifications of the graph that are never committed to it.

Both problems place a user inside the system. In the first, the user supplies the knowledge that decides between admissible repairs; in the second, the user poses the hypothesis and interprets its effect. The literature treats this user as an oracle with unlimited availability. However, two observations contradict this assumption: (i) expertise, attention and time are finite and unequally distributed, and (ii) the formalism through which questions are expressed (e.g., a graph query language) is itself a source of error. We therefore treat what users do when they query and repair property graphs, and where they fail, as an input to the formulation of both problems.

1.2 A Motivating Scenario

We consider the scenario illustrated in Figure 1.1 a hospital that maintains a property graph about patients and their treatments, assembled from an electronic health record, a pharmacy system and a national drug registry. Nodes carry labels such as *:Person*, *:Drug* and *:Ingredient* together with properties in the form of key-value pairs, and edges carry labels such as *:takes*, *:hasIngredient* and *:hasAllergyTo*. Two graph denial constraints are declared over it: ϕ_1 states that a patient cannot be treated with a drug containing an ingredient they are allergic to, and

ϕ_2 states that a patient cannot be treated with a drug whose ingredient specifies a minimum age above the age of the patient. Each source was consistent in isolation. Once they are merged, the graph is not: *Markus* is recorded as allergic to *Paracetamol* and as taking *Tachipirine100*, which contains it, and *Ali* is recorded as taking *Oki*, whose ingredient carries a minimum age above his own. The same graph is also the input of a model that predicts readmission risk, and the basis on which the hospital decides which patients to enroll in a follow-up therapy. Five questions arise over this single artefact, and they are the questions this thesis addresses.

Which repair is the correct one. The violation involving *Markus* can be removed in at least four ways: (a) updating the drug node to a different drug, (b) deleting the *:takes* edge, (c) deleting the *:hasIngredient* edge, or (d) deleting the *:hasAllergyTo* edge. All four restore consistency and each of them asserts something different about what happened, so the graph does not determine the choice. A clinician who has seen the patient record does, and a pharmacist knows whether *Oki* contains *Paracetamol* at all. Repair therefore requires allocating a decision to the user who can make it. Moreover, repairs interact: fixing one violation can resolve or introduce others, so two users editing overlapping parts of the graph can invalidate each other's work, and the allocation has to account for this. This is the subject of Chapter 3.

Whom to ask, and how often. The hospital graph does not contain two violations but several thousand, while the clinicians who can resolve them are few, unequally competent across the constraints involved, able to answer a bounded number of questions, and paid for the time they spend. Some violations do not need them: a country code that contradicts the rest of a well-populated neighbourhood is recoverable from the data, whereas an allergy that contradicts a prescription is not. Deciding which violations are answerable automatically, and which of the remaining ones is put to which user within a fixed budget, is the subject of Chapter 4.

What the analysis would return under a change that has not been made. Once the graph is consistent, the hospital does not want a prediction but the effect of a decision: how the predicted readmission risk over a cohort would change if a follow-up therapy were extended to it, and how that compares with three variants of the same therapy. The candidate changes are hypothetical and must not be written to the graph, the analyst proposes them in sequence, abandons some and returns to earlier ones, and the number shown has to be the number the model would produce and not an approximation of it. Evaluating queries over the predictions of a trained model under such updates, exactly and without recomputing what the unmodified graph already determines, is the subject of Chapter 6.

Whether the change would cause the outcome. A model trained on the hospital data reports that patients enrolled in the therapy are readmitted less often. This is an associational statement, and acting on it requires an interventional one: whether enrolling a patient causes the reduction, or whether both are explained by the severity of the initial diagnosis. Answering it requires the causal structure relating treatment, severity and outcome to be represented alongside the observational data, and the query language to provide the constructs for extraction, intervention and counterfactual reasoning. This is the subject of Chapter 7.

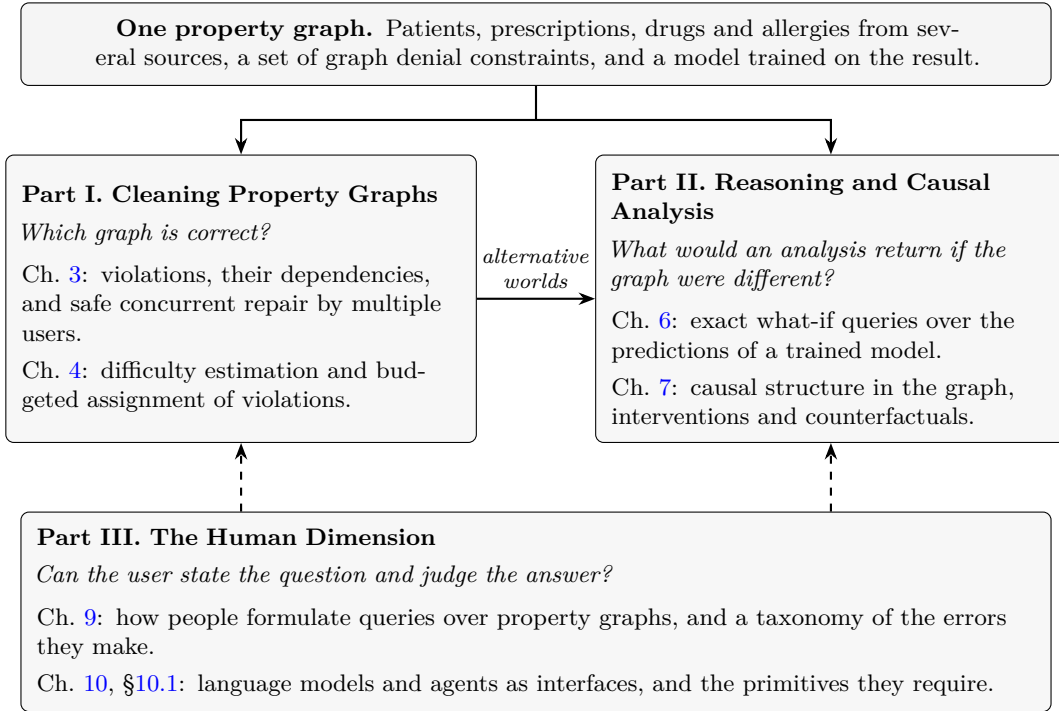


Figure 1.2: Organisation of the thesis over the scenario of Section 1.2. Part I decides which graph the data denotes, Part II computes over graphs that are never committed, and Part III studies the user on whom both depend.

Whether the question can be expressed at all. Each of the four questions above assumes a user able to state it. The clinician who accepts a repair reads a violation expressed as a subgraph, and the analyst who poses a hypothesis writes it as a query. Both steps fail in practice: users confuse the schema with the data, misuse the constructs that bind variables across patterns, and transfer expectations from other query languages that do not hold for graphs. What these errors are, and what follows for the interfaces through which the previous four questions are posed, is the subject of Chapter 9; what changes when the query is produced by a language model or an agent is discussed in Section 10.1.

The five questions concern one graph and one set of decisions taken over it. Figure 1.2 summarises which of them each part of the thesis addresses, and on what the answer of each part depends.

1.3 Outline and Contributions

The remainder of the thesis is organised in three parts. Each part opens with a short statement of the question it addresses; Parts I and II are closed by a chapter that consolidates what their chapters jointly establish, while Part III consists of a single chapter whose conclusion plays the same role. Chapter 2 introduces the notions used throughout: the property graph model and its query languages, the constraint families defined over it and the graph denial constraints adopted in the thesis, graph neural networks and the two architectures used in the following chapters, structural causal models together with the do-operator, and the literature on human-in-the-loop data management.

Part I - Cleaning Property Graphs. Part I concerns the state of the data, and treats the user as the one who decides what the data should say. A violation of a constraint usually admits several repairs, all of which restore consistency and only one of which corresponds to the ground truth, so the choice between them requires knowledge that the graph does not contain. The two chapters of this part ask for that knowledge: the first establishes that multiple, fallible users can be coordinated over an inconsistent property graph with formal guarantees on what the process will do, while the second establishes that this can be done under the conditions where users are unequal, their attention is finite and their time is a cost.

Chapter 3 formalises the interactive repair of property graphs under graph denial constraints. Violations are detected by compiling each constraint into a query and are organised in the Graph Repair Dependency Graph, which records where repairs interfere and allows independent violations to be assigned to concurrent users. We define a local safety condition on accepted repairs and prove that it is necessary and sufficient for convergence, with a bound on the number of iterations that does not depend on user competence [12, 13]. We then implement the framework as an interactive tool, comprising an administrator panel for configuring a repair session and a repair panel through which users resolve the violations assigned to them [14] (Section 3.7).

Chapter 4 removes the two assumptions that the previous design allows itself, namely that users are interchangeable and that their effort is free. We propose a difficulty-aware and budgeted repair method: a heterogeneous graph attention network scores how hard each violation is to repair and resolves the easiest ones itself, users are modelled by skill, cognitive capacity and cost, and the resulting assignment problem is shown to be NP-hard and solved by Lagrangian relaxation with a closed-form dual certificate [15].

Chapter 5 consolidates what the two chapters jointly establish, states what they leave unsettled, and gathers the directions they open.

Part II - Reasoning and Causal Analysis over Graphs. Part II concerns what is computed over the data, and the user moves from deciding to planning and analysing. Its object is the set of graphs that a deliberate change to the current one would produce. An analyst explores this set interactively: they propose a change, read its effect, propose a variant and return to an earlier state. The two chapters address the two questions such an exploration raises: (i) what a trained model would predict under a hypothetical update, and (ii) what would happen under an intervention. The first question is answered with the model held fixed and belongs to the associational level of the ladder of causation; the second requires the causal structure to be represented alongside the data.

Chapter 6 defines the what-if query over the predictions of a trained graph neural network under a hypothetical update, and gives two exact procedures for answering it incrementally: (i) a low-rank correction for attribute updates and (ii) a locality-bounded recomputation for structural ones. Both are applied as an overlay that leaves the base graph untouched, and we show that they compose across an exploration session, so that a branching, backtracking sequence of hypotheticals pays only for the increment between candidates.

Chapter 7 moves from what a model would predict to what would happen. We introduce a causal property graph model, in which causal variables are hypervertices over the observational data and conditional, interventional and structural information is carried as meta-properties, we

identify the constructs that current query languages lack for extraction and for do-calculus, and we validate the model with a proof-of-concept implementation [16]. We then extend the same principle to a multi-source causal ecosystem, in which one explicit and inspectable causal layer serves analysts, learning systems and agents at once [17] (Section 7.6).

Chapter 8 consolidates the two chapters, states the lessons that generalise beyond the specific systems, and sets out what the part leaves open.

Part III - The Human Dimension. The first two parts assume a user who is knowledgeable about the domain and fallible about the formalism. This part studies the second half of that assumption.

Chapter 9 reports an empirical study of how people who have been taught a graph query language formulate queries over property graphs and where they fail. We contribute a taxonomy of graph query errors organised around Cypher constructs, quantitative and qualitative analyses of those errors, interviews on their causes, and the implications for teaching [18]; the conclusion of the chapter relates the error families to the interfaces of Parts I and II.

Finally, Chapter 10 concludes the thesis. It summarises its contributions, discusses what language models and agents as interfaces to graph data change for the results of the thesis and what they do not (Section 10.1), drawing on a workshop on human-model collaboration for data processing that we co-organised [19], and states directions for future research.

1.4 Publications

This thesis is based on the following publications.

- Amedeo Pachera, Angela Bonifati, Andrea Mauri. Towards User-Centric Graph Repairs. *IEEE International Conference on Data Engineering Workshops (ICDEW)*, 2024 [12]. (Chapter 3.)
- Amedeo Pachera, Angela Bonifati, Andrea Mauri. User-Centric Property Graph Repairs. *Proceedings of the ACM on Management of Data (SIGMOD)*, 3(1), 2025 [13]. (Chapter 3.)
- Amedeo Pachera, Angela Bonifati, Andrea Mauri. Grafixer: Enabling User-Centric Repairs for Property Graphs. *Companion of the International Conference on Management of Data (SIGMOD/PODS)*, 2025 [14]. (Section 3.7.)
- Amedeo Pachera, Angela Bonifati, Laks V. S. Lakshmanan, Andrea Mauri. Budgeted Interactive Property Graph Repair with Graph Neural Networks. *ACM SIGMOD International Conference on Management of Data*, 2027, to appear [15]. (Chapter 4.)
- Amedeo Pachera, Laks V. S. Lakshmanan, Angela Bonifati, Andrea Mauri. Incremental Evaluation of What-If Queries over Graph Neural Networks. Under submission. (Chapter 6.)
- Amedeo Pachera, Mattia Palmiotto, Angela Bonifati, Andrea Mauri. WhatIf: Causal Analysis with Graph Databases. *Proceedings of the VLDB Endowment*, 18(11), 2025 [16]. (Chapter 7.)

- Dazhuo Qiu, Yingli Zhou, Amedeo Pachera, Angela Bonifati, Andrea Mauri. Toward a Causal Data Management Ecosystem for Decision Making and Agentic AI. 2026 [17]. (Section 7.6.)
- Amedeo Pachera, Stefania Dumbrava, Angela Bonifati, Andrea Mauri. Understanding Student Errors in Graph Query Formulation. *ACM Transactions on Computing Education*, 25(3), 2025 [18]. (Chapter 9.)
- Amedeo Pachera, Andrea Mauri, Kashif Imteyaz, Jie Yang, Eric Umuhoza, Angela Bonifati, Michal Lahav, Nitesh Goyal. Co-Data: Cultivating Effective Human-LLM Collaboration for Collaborative Data Processing. *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 2026 [19]. (Section 10.1.)

Chapter 2

Background and Preliminaries

This chapter consolidates the definitions and background that the remainder of the thesis relies on. Section 2.1 describes the property graph data model, graph patterns, the semantics of query evaluation and the landscape of graph query languages; it is used throughout. Section 2.2 surveys the constraint families defined over property graphs and motivates the choice of graph denial constraints adopted in Part I. Section 2.3 introduces graph neural networks and the two architectures used in Chapters 4 and 6, namely R-GCN and hGAT. Section 2.4 presents structural causal models, the ladder of causation and the do-operator, which frame Part II and are used directly in Chapter 7. Finally, Section 2.5 reviews human-in-the-loop data management, the literature in which the user model of Part I and the study of Part III are positioned.

2.1 The Property Graph Model

The **property graph model (PGM)** represents data as a directed, labelled, attributed multi-graph. Both vertices and edges are treated as first-class objects that can carry (i) a set of *labels*, which describe what kind of real-world entity the object stands for, and (ii) a set of *properties* expressed as key-value pairs, which hold the actual data [20]. Two features distinguish the PGM from simpler graph models. First, it is a multi-graph: any number of edges may connect the same ordered pair of vertices. Second, it is multi-labelled and key-value attributed, so an object may carry zero, one, or many labels, and may instantiate an arbitrary set of properties independently of its labels. Labels are purely descriptive and imply no schema over properties [20]. This combination of expressiveness and simplicity is what has made the PGM the predominant model in modern graph database systems [21, 22].

Definition 2.1 (Property Graph Model [20]). *Let O be a set of objects, L a finite set of labels, K a set of property keys, and N a set of values, all pairwise disjoint. A **property graph** is a structure $G = (V, E, \eta, \lambda, \nu)$, where*

- $V \subseteq O$ is a finite set of objects called **vertices**;
- $E \subseteq O$ is a finite set of objects called **edges**;
- $\eta : E \rightarrow V \times V$ assigns to each edge an ordered pair of vertices (its source and target);
- $\lambda : V \cup E \rightarrow \mathcal{P}(L)$ assigns to each object a finite set of labels, where $\mathcal{P}(S)$ denotes the finite powerset of S ; and

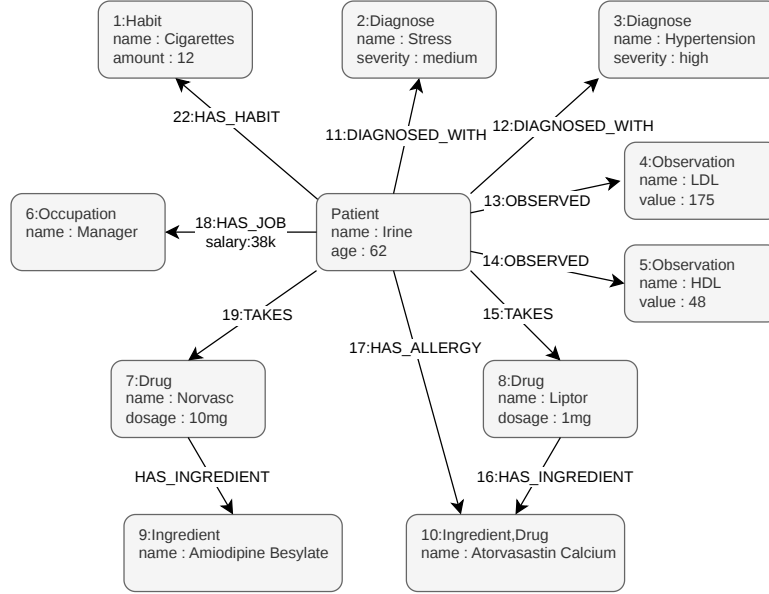


Figure 2.1: Example of a Property Graph for an healthcare scenario.

- $\nu : (V \cup E) \times K \rightarrow N$ is a partial function assigning property values to objects,

subject to the constraints that the object sets are disjoint ($V \cap E = \emptyset$) and that ν is defined on only finitely many argument pairs.

Figure 2.1 shows in details the PG of the running example. It models an healthcare scenario. Every node and edge has an id and a label (with the notation $ID : \langle LABEL \rangle$) and a list of key-value properties. For instance, node 21 represents a 62 years old patient named *Irine*. The edge 11 links it to the node 2, modeling a diagnose of *Stress* with medium severity. The edge 18 holds a property, indicating the salary of the patient for their occupation. Finally, node 10 has two labels. The *Atorvasastin Calcium* indeed is both an ingredient and a generic drug.

2.1.1 Querying a Property Graph

The key components of property graph queries are graph patterns. A graph pattern refers to a visual or structural template used to query the PG.

Definition 2.2 (Graph pattern [20]). Let $\mathcal{N}$ and $\mathcal{E}$ be disjoint sets of vertex variables and edge variables. A graph pattern is a directed graph $Q[\bar{x}] = (N_Q, E_Q, \lambda, \mu)$ where N_Q is a finite set of graph pattern vertices, E_Q is a finite set of graph pattern edges, $\lambda : N_Q \cup E_Q \rightarrow \mathcal{L}$ is a partial function assigning a label to each vertex and edge, $\bar{x}$ is a list of variables such that one of the following holds:

- (i) $\bar{x} \subset \mathcal{N}$ with $|\bar{x}| = |N_Q|$ and $\mu : \bar{x} \rightarrow N_Q$ is a bijective function assigning distinct variables in $\bar{x}$ to vertices in N_Q ; or
- (ii) $\bar{x} \subset \mathcal{E}$ $|\bar{x}| = |E_Q|$ and $\mu : \bar{x} \rightarrow E_Q$ is a bijective function assigning distinct variables in $\bar{x}$ to edges in E_Q .

Graph patterns can be represented in three (equivalent) forms.

- (i) Graphical form: 
- (ii) Logic form: $Q := Patient(x) \wedge Observation(y) \wedge Diagnosis(z) \wedge OBSERVED(x, y) \wedge DIAGNOSED_WITH(x, z)$
- (iii) Cypher-like form¹:

$Q := (Observation) <- [OBSERVED] - (Patient) - [DIAGNOSED_WITH] -> (Diagnosis)$

The pattern Q in this example looks for all the patient with an observation and a diagnosis.

A pattern by itself only describes a shape; querying is the act of *matching* it against the data. Graph patterns are matched against a property graph to identify all subgraphs that satisfy their structural and labeling conditions [20, 3]. Matching is typically defined through graph homomorphisms [3], although subgraph isomorphism is also considered in the literature.

Definition 2.3 (Graph Pattern Matching). *Given a property graph G and a graph pattern $Q[\bar{x}]$, a match of Q in G is a homomorphism $h : Q \rightarrow G$. That is, h maps every vertex (resp. edge) of Q to a vertex (resp. edge) of G such that:*

- (i) *labels are preserved: for every $z \in N_Q \cup E_Q$ on which λ is defined, $\lambda(z)$ is among the labels assigned to $h(z)$ in G ; and*
- (ii) *incidence is preserved: whenever Q contains an edge $x - e \rightarrow y$, then $h(e)$ is an edge of G from $h(x)$ to $h(y)$.*

Each match binds the output variables $\bar{x}$ of the pattern to concrete objects of G : the object bound to a variable x is $h(\mu(x))$. The **answer** (or result) of evaluating $Q[\bar{x}]$ over G collects these bindings across all matches,

$$\{Q[\bar{x}]\}_G = \{ (h(\mu(x)))_{x \in \bar{x}} \mid h \text{ is a match of } Q \text{ in } G \}, \quad (2.1)$$

so a query returns a set of tuples, one component per output variable. Because matching is homomorphism-based, the map h need not be injective: distinct pattern variables may be sent to the same object, and a single data vertex can therefore play several roles in one match.

For instance, evaluating Q against the property graph of Figure 2.1 yields four matches. Each binds the patient variable to node 21 and the observation and diagnosis variables to the pairs (2, 4), (2, 5), (3, 4) and (3, 5); seen as subgraphs, these are $(2) \leftarrow (21) \rightarrow (4)$, $(2) \leftarrow (21) \rightarrow (5)$, $(3) \leftarrow (21) \rightarrow (4)$ and $(3) \leftarrow (21) \rightarrow (5)$.

Patterns of this kind, whose edges each carry a single label, correspond to **conjunctive queries (CQ)**: they test for the presence of a fixed structural fragment. Their reach is limited to a fixed topology, so they cannot follow paths of a priori unbounded length. *Navigational* queries lift this restriction.

The most basic navigational primitive is the **regular path query (RPQ)**, which asks for all pairs of vertices connected by at least one path whose sequence of edge labels spells a word in the language of a given regular expression over the label alphabet L [20]. RPQs are built from single labels using inverse, concatenation ($/$), union ($+$), and transitive closure ($^+$). Evaluating an RPQ g over a graph G yields a binary relation $\{g\}_G \subseteq V \times V$ of connected vertex pairs, defined

¹Cypher [23] is one of the most common graph query languages.

recursively over the operators [20]. RPQs capture reachability, the characteristic strength of graph querying over relational querying.

Combining pattern matching with navigation gives the **conjunctive regular path query (CRPQ)**: a graph pattern in which each edge is labelled by a regular path expression, so that one pattern edge is satisfied by an entire path rather than a single edge. Formally, a match h must now satisfy, for every path predicate $\alpha(x, y)$ of the pattern, $(h(x), h(y)) \in \{\alpha\}_G$, while the answer is still given by Equation (2.1) over the output variables $\bar{x}$. Because RPQs and CQs are both special cases of CRPQs, the CRPQ machinery uniformly captures both path querying and subgraph pattern matching [20]. Adding disjunction across an arity-consistent set of CRPQs yields **unions of conjunctive regular path queries (UCRPQ)**, the fragment that most closely matches the expressive power of today’s practical languages [20].

2.1.2 Graph Query Languages

The formal fragments above underpin the query languages used in practice. A common yardstick is how close a language comes to a closed and composable core over property graphs, that is, one that supports the full navigational power of regular path queries together with graph construction, so that the output of a query is itself a graph that can feed another query [20].

Cypher is the most widely known property graph language, originating with Neo4j. In the version described by Bonifati et al. (Cypher 9), it is broadly comparable to UCRPQ but not identical: it uses edge-isomorphism matching, and its path semantics enumerate all paths under bag semantics rather than returning reachable vertex pairs under set semantics. Cypher 9 does not support the full power of RPQs, lacks graph construction, and is not closed and composable over graphs, though it is composable over tables and offers relational-style selection, projection, aggregation, and ordering [20, 22]. Cypher has continued to evolve through the openCypher process, and many of these gaps have been addressed in later versions.

GQL is the outcome of the standardisation effort that Bonifati et al. anticipated [20]. Building on the convergence of Cypher, Oracle’s PGQL, and G-CORE, the ISO/IEC committee responsible for SQL (JTC 1/SC 32/WG 3) published GQL as ISO/IEC 39075:2024 in April 2024, the first entirely new ISO database language standard since SQL in 1987 [24]. GQL is a declarative database language for property graphs: it standardises visual ASCII-art graph patterns and defines facilities for creating, querying, and maintaining property graphs, with the explicit goals of skill portability and reduced vendor lock-in [24]. It is designed as a sibling to SQL rather than a replacement.

G-CORE is a research and consortium language designed by an industry-academia group (LDBC) to capture, in a natural syntax, the graph-querying features that matter most in practice [21]. Unlike Cypher 9, it is closed and composable over graphs, since its **CONSTRUCT** capability lets queries build new graphs, and it foresees interoperability with tabular (SQL) data. It served as one of the principal inputs to the GQL standard [20, 21].

Other relevant languages include **PGQL** (Oracle), which likewise targets property graphs and fed into the standardisation process [22]; **Gremlin** (Apache TinkerPop), a more imperative, traversal-oriented language that can also express UCRPQ but with a programming-interface feel and bag semantics for paths [20]; and **SPARQL**, the W3C-standardised language for RDF

graphs, which is comparable to UCRPQ and offers graph construction (via **CONSTRUCT**) but is not composable in the sense above [20, 25].

2.1.3 PGM Extension

The plain property graph of Section 2.1 keeps vertices as atomic, first-order objects, while edges are second-order objects that make statements about ordered pairs of vertices. Many application domains, however, need to reason about a whole group of vertices and edges as a single unit: a team within a social network, an interview linking several people, or a community inside a larger graph. Just as importantly, one often wants to relate such a group to other groups, or to the very members it contains. The base PGM cannot express this directly, because there is no object that stands for a subgraph.

The **hypervertex property graph model (HVPGM)** closes this gap by letting a (hyper)vertex denote a, possibly empty, subgraph [20]. A hypervertex that denotes the empty subgraph behaves exactly like a vertex in the traditional sense, whereas a hypervertex that denotes a non-empty subgraph is a genuine higher-order object. Because hypervertices remain vertices, they can be connected by ordinary edges, so the model can link one subgraph to another subgraph, or link a subgraph to a member nested inside it. For instance, a hypervertex labelled **:Novice** might denote an empty subgraph and thus act as a plain vertex, while a hypervertex labelled **:Interview** could denote a subgraph made of two hypervertices and their connecting edges, and at the same time carry outgoing edges that point to another subgraph and to one of its own contained members [20]. Since a hypervertex need not have any adjacent edges, the HVPGM strictly generalises the objectified-subgraph model (and hence the base PGM) [20].

Definition 2.4 (hypervertex property graph model (HVPGM) [20]). *Formally, an HVPGM graph extends the property graph of to a structure $G = (V, E, \eta, \gamma, \lambda, \nu)$, where*

- $V \subseteq O$ is a finite set of objects, now called **hypervertices**;
- $E \subseteq O$ is a finite set of edges;
- $\gamma : V \rightarrow \mathcal{P}(V) \times \mathcal{P}(E)$ is a total function assigning to each hypervertex the subgraph it denotes, given as a pair of a hypervertex set and an edge set; and
- η , λ , and ν are as in Definition 2.1, with $\eta : E \rightarrow V \times V$, $\lambda : V \cup E \rightarrow \mathcal{P}(L)$, and $\nu : (V \cup E) \times K \rightarrow N$ a partial function,

subject to the following constraints:

- the object sets are disjoint, $V \cap E = \emptyset$;
- every hypervertex denotes a well-formed graph, meaning that each edge in the denoted subgraph is adjacent only to hypervertices of that same subgraph, that is, for every $e \in \pi_2(\gamma(v))$ it holds that $\eta(e) \in \pi_1(\gamma(v)) \times \pi_1(\gamma(v))$; and
- ν is defined on only finitely many argument pairs,

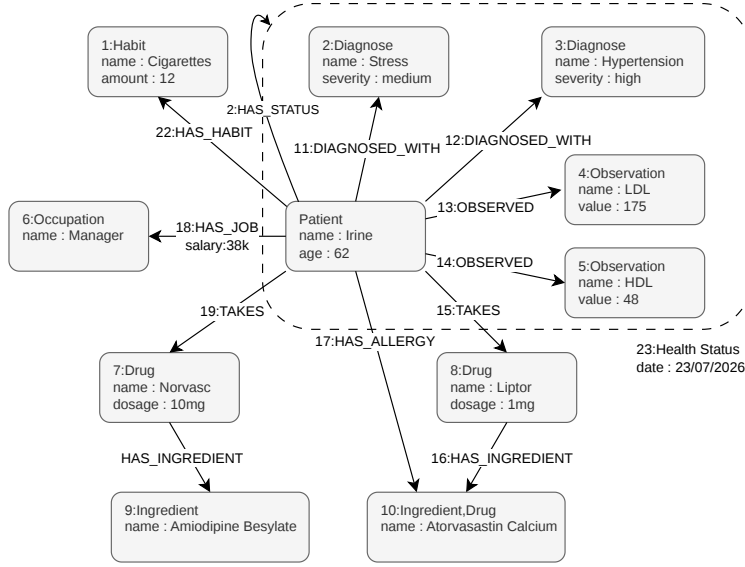


Figure 2.2: Example of a Hypervertex Property Graph for an healthcare scenario.

where π_1 and π_2 denote the first and second component of a pair. A hypervertex v with $\gamma(v) = (\emptyset, \emptyset)$ is a vertex in the traditional sense, whereas any v with a non-empty denotation is a higher-order object.

Figure 2.2 shows an example of a Hypervertex Property Graph, where the subgraph 23 models the health status of *Irine* and is linked to the patient node through the edge 24.

2.2 Constraints for Graphs

Integrity constraints are the classical mechanism by which a data model expresses what its instances are allowed to look like. In the relational world this vocabulary is settled: keys, foreign keys, functional dependencies (FDs), conditional functional dependencies (CFDs) [26], matching dependencies [27] and denial constraints (DCs) [28, 29] form a well-understood hierarchy with known reasoning costs. Graphs resist this transfer for two reasons. First, real-life graphs are typically schemaless: entities of the same conceptual type need not carry the same attributes, and the same property may be reached from two entities along different paths, so there is no fixed set X of attributes on which two objects can be compared [30]. Second, in a graph the relevant unit of constraint is rarely a single object: what identifies or invalidates a node is often its neighbourhood, i.e. a topological pattern around it.

A graph constraint language therefore has to say two things at once: *where* in the graph the constraint applies, and *what* must then hold. The design space is essentially the product of these two choices.

Concretely, the formalisms surveyed below differ along four axes:

1. **Scope language.** How the constrained objects are selected: by a single label, by a graph pattern matched via homomorphism or subgraph isomorphism, by a restricted pattern shape (star, tree, dual star), or by a full query including regular path expressions.

2. **Dependency language.** What predicates may be combined: equality of attributes and constants only, identity (`id`) literals, built-in comparisons $\{\neq, <, \leq, >, \geq\}$, edge literals, temporal orders, or even machine-learning classifiers used as Boolean predicates.
3. **Targets.** Whether nodes, edges, property values, or the topology itself can be constrained and, symmetrically, what a repair is allowed to change.
4. **Validation cost.** The complexity of deciding whether a given graph satisfies a set of constraints, which drives error detection at scale.

Expressiveness and cost trade off sharply here. Because subgraph isomorphism and graph homomorphism are both NP-complete [31], any formalism whose scope is an unrestricted graph pattern inherits intractable validation, and the restrictions that buy tractability: star-shaped patterns, at most one non-constant predicate per leaf are exactly the ones that limit what can be said [30].

A second, orthogonal distinction cuts across the whole area and matters for how each formalism is used in practice: whether the constraints are declared or mined.

Declared (prescriptive) constraints are authored by a designer or an expert (i.e. the database administrator translates the stakeholders or domain-specific requirements), derived from a conceptual model, or carried over from a relational source during migration, and are then enforced by the system, ideally at write time. PG-Keys [32], the uniqueness/existence/key constraints shipped by Neo4j, TigerGraph and others, and shape languages such as SHACL [33] and ShEx [34] sit here. Their design goals are usability, declarativeness, and cheap (ideally incremental) validation; the number of such constraints is small, they are assumed correct, and a violation is by definition a data error.

Discovered (descriptive) constraints are mined from an instance and are statements about how the data in fact behave. This is the dominant mode for Graph Functional Dependencies (GFDs), Graph Entity Dependencies (GEDs), Graph Association Rules (GARs), Graph Cleaning Rules (GCRs), and for Denial Constraints (DCs), for the simple reason that nobody can enumerate by hand the thousands of heterogeneous topological patterns that a schemaless graph exhibits. Discovery introduces its own machinery about quality measures [35, 36], redundancy [35, 30] and cost [37, 30] which are out of scope for this thesis.

The two modes are complementary: declared keys give the identification backbone (and are what a standard can realistically mandate), while mined dependencies supply the long tail of instance-specific regularities used for error detection, entity resolution and imputation. Table 2.1 summarises the formalisms discussed below along the axes above.

Definition 2.2 returns, through Equation (2.1), a binding for every variable of the pattern, and those bindings are always objects of O . Constraint languages need two further capabilities: they must refer to property values (elements of N , obtained through ν), and they must project onto a chosen sublist of outputs. We therefore fix the following vocabulary, used uniformly in the remainder of the chapter.

Definition 2.5 (Terms). *Let $Q[\bar{x}]$ be a graph pattern. A **term** over $Q[\bar{x}]$ is either a variable $x \in \bar{x}$ or a **property term** $x.k$ with $x \in \bar{x}$ and $k \in K$. Given a match h of Q in G , terms are*

Table 2.1: Constraint formalisms for graphs; PTIME entries denote tractable validation.

Formalism	Scope	Predicates	Targets	Validation
PG-Keys [32]	query (parameter)	scope/descriptor equality	nodes, edges, properties	query evaluation
Keys for graphs [38]	recursive pattern	id equality	nodes	intractable
GFDs [3]	generic pattern	$x.A=y.B, x.A=c$	attribute values	coNP-c
GEDs [39]	generic pattern	+ id literals	values, entities	coNP-c
NGFDs [40]	generic pattern	+ linear arithmetic	numeric values	coNP-c
GARs [41]	generic pattern	+ ML, edge literals	values, entities, links	coNP-c
GCRs [42]	dual star	equality, ML	ER, CR	PTIME
GCR ⁺ _s [30]	dual star	+ $\oplus, \preceq / \prec$, edge	ER, CR, TD, MI	PTIME
GDC [43]	query + recursion	comparisons, negation	topology (deletions)	query evaluation
Neighborhood [44]	adjacent pair	allowed label pairs	labels, edges	linear
StarFDs [45]	star + CRPQ	value dependency	attribute values	PTIME

interpreted by

$$\{x\}_h = h(\mu(x)), \quad \{x.k\}_h = \nu(h(\mu(x)), k),$$

the latter being defined only when ν is defined on the pair $(h(\mu(x)), k)$. Object terms denote elements of O ; property terms denote elements of N .

Definition 2.6 (Projected query). Let $\bar{t} = (t_1, \dots, t_n)$ be a list of terms over $Q[\bar{x}]$ and let θ be a (possibly empty) Boolean combination of comparison atoms $t \oplus t'$ and $t \oplus c$, where t, t' are terms, $c \in N$, and $\oplus \in \{=, \neq, <, \leq, >, \geq\}$. A **projected query** is written

$$q = \bar{t} \text{ WITHIN } Q[\bar{x}] \text{ [WHERE } \theta],$$

and its answer over G is

$$\{q\}_G = \left\{ (\{t_1\}_h, \dots, \{t_n\}_h) \mid h \text{ a match of } Q \text{ in } G, \{t_i\}_h \text{ defined for all } i, h \models \theta \right\}.$$

We write $q(\bar{t})$ to expose the output terms and say that $q(\bar{r})$ holds when $\bar{r} \in \{q\}_G$.

Equation (2.1) is the special case $\bar{t} = \bar{x}$ with θ empty. Two consequences of Definition 2.5 recur below. Since ν is partial, a match that leaves some output term undefined simply does not contribute to $\{q\}_G$; this is the semantics of a missing property, and it is what makes the MANDATORY mode of §2.2.1 non-vacuous. And since λ maps objects to *sets* of labels while a pattern assigns at most one label per pattern element, a scope such as $(x:\text{Ingredient})$ selects every object whose label set *contains* **Ingredient**, including node 10 of Figure 2.1, which is simultaneously an **Ingredient** and a **GenericDrug**.

Scope of this thesis. Of the formalisms above we concentrate on graph denial constraints (GDCs), referring to PG-Keys, GFDs and GEDs only where a comparison demands it. Two reasons motivate the choice. First, the pair is jointly complete with respect to the rest: denial constraints subsume the forbidding fragment implication dependencies such as GFDs and CFDs are the subclass whose body is $X \wedge \neg Y$, and GDCs additionally admit comparison operators and recursion over paths, while PG-Keys subsume the identifying fragment, i.e. the id literals of GEDs and the recursive keys of [38], which generate equalities rather than forbid configurations.

Second, both are data-quality constraints in the strict sense: they partition instances into clean and dirty, every violation is by definition an error, and satisfying a constraint asserts no

new fact about the world. This sets them apart from GARs and from the ML-augmented GCR⁺s, whose edge literals in the consequence and embedded classifiers make them knowledge-generating, they are ranked by support and confidence, they predict links and impute values, and their output enriches the graph rather than certifying it. That difference in purpose changes what one wants from the formalism: for generation, coverage and predictive accuracy; for quality, declarativeness, cheap validation, and a well-behaved notion of repair.

2.2.1 PG-Keys

PG-Keys [32] is the key-constraint proposal of the LDBC Property Graph Schema Working Group, developed with an eye on the ISO GQL standardisation effort. It is the clearest representative of the *declared* end of the spectrum. A property graph is taken as a tuple $G = (N, E, \rho, \lambda, \pi)$ with nodes N , edges E , an incidence function ρ , a labelling λ into sets of labels, and a partial function π mapping (element, property name) pairs to values.

Every key has a *scope*, e.g. the set of objects it applies to, and a *descriptor*, which says how to obtain the key value of an object in scope. Both are expressed as queries, and the query language is a parameter of the formalism: any language mapping graphs to tables of bindings over nodes, edges and property values can be plugged in, so different vendors can conform while retaining their own syntax. We instantiate that parameter with the projected queries of Definition 2.6.

Definition 2.7 (PG-Key). *A **PG-Key** is an expression*

$$\kappa = \text{FOR } p(t) \text{ } M \text{ } q(t, \bar{u}),$$

where p is a projected query with a single output term t , called the **scope**; q is a projected query with output list $(t, \bar{u})$ for some $\bar{u} = (u_1, \dots, u_n)$, $n \geq 1$, called the **descriptor**; and the **mode** M is a non-empty subset of $\{\text{EXCLUSIVE}, \text{MANDATORY}, \text{SINGLETON}\}$ containing EXCLUSIVE.

Before stating when a graph satisfies a PG-Key we fix the two sets the key talks about, and the relation between them that the modes constrain.

Definition 2.8 (Targets, key values, key-value relation). *Let $\kappa = \text{FOR } p(t) \text{ } M \text{ } q(t, \bar{u})$ be a PG-Key with $|\bar{u}| = n$, and let G be a property graph.*

- The **target set** of κ in G is the set of objects or values returned by the scope,

$$T_\kappa(G) = \{o \mid (o) \in \{p\}_G\} \subseteq O \cup N.$$

Its elements o are the **targets**: the things the key is supposed to identify. Since by Definition 2.5 the output term t of the scope may be an object term or a property term, a target is a vertex, an edge, or a property value.

- The **key-value space** is $\mathcal{V}_\kappa = (O \cup N)^n$. Its elements $\bar{r} = (r_1, \dots, r_n)$ are the **key values**: the tuples the descriptor may return for a target. Each component r_i is again an object or a value, according to the sort of the term u_i .

Table 2.2: The four PG-Key modes as properties of the key-value relation $\text{kv}_\kappa(G) \subseteq T_\kappa(G) \times \mathcal{V}_\kappa$.

Mode	Conditions	$\text{kv}_\kappa(G)$ is ...
EXCLUSIVE	(2.2)	injective relation
EXCLUSIVE MANDATORY	(2.2), (2.3)	injective, total relation
EXCLUSIVE SINGLETON	(2.2), (2.4)	injective partial function
IDENTIFIER	(2.2), (2.3), (2.4)	injective total function

- The **key-value relation** of κ in G is

$$\text{kv}_\kappa(G) = \{ (o, \bar{r}) \in T_\kappa(G) \times \mathcal{V}_\kappa \mid (o, \bar{r}) \in \{q\}_G \},$$

and we write $\text{kv}_\kappa(G)[o] = \{\bar{r} \mid (o, \bar{r}) \in \text{kv}_\kappa(G)\}$ for the set of key values of a target o .

A PG-Key is thus a requirement on the shape of the binary relation $\text{kv}_\kappa(G)$, and the three keywords of Definition 2.7 name three standard properties a binary relation may have. Dropping the subscripts where κ and G are clear:

Definition 2.9 (Satisfaction of a PG-Key). *Let κ and G be as in Definition 2.8. Consider the following three conditions on $\text{kv} = \text{kv}_\kappa(G)$:*

$$\begin{aligned} (\text{EXC}) \quad & \forall o_1, o_2 \in T_\kappa(G), \quad \forall \bar{r} \in \mathcal{V}_\kappa : \\ & (o_1, \bar{r}) \in \text{kv} \wedge (o_2, \bar{r}) \in \text{kv} \implies o_1 = o_2, \end{aligned} \tag{2.2}$$

$$(\text{MAN}) \quad \forall o \in T_\kappa(G), \quad \exists \bar{r} \in \mathcal{V}_\kappa : (o, \bar{r}) \in \text{kv}, \tag{2.3}$$

$$\begin{aligned} (\text{SGL}) \quad & \forall o \in T_\kappa(G), \quad \forall \bar{r}_1, \bar{r}_2 \in \mathcal{V}_\kappa : \\ & (o, \bar{r}_1) \in \text{kv} \wedge (o, \bar{r}_2) \in \text{kv} \implies \bar{r}_1 = \bar{r}_2. \end{aligned} \tag{2.4}$$

Then $G \models \kappa$ iff every condition named in the mode M holds, i.e. iff (2.2) holds, and (2.3) holds whenever MANDATORY $\in M$, and (2.4) holds whenever SINGLETON $\in M$. The mode $M = \{\text{EXCLUSIVE}, \text{MANDATORY}, \text{SINGLETON}\}$ is abbreviated IDENTIFIER.

In words: (2.2) says that no key value is shared by two distinct targets, so a key value *refers* to at most one target; (2.3) says that every target has at least one key value, so every target *can* be referred to; and (2.4) says that no target has two key values, so the key value of a target is *canonical*. In relational terms, $\text{kv}_\kappa(G)$ is required to be respectively left-unique (injective), left-total, and right-unique (single-valued); the four admissible modes are therefore exactly the four combinations in Table 2.2.

Two remarks follow from this reading. First, an IDENTIFIER key makes $\text{kv}_\kappa(G)$ a bijection between $T_\kappa(G)$ and its image, so the key value of a target determines the target and vice versa : the key value is the identity of the object, which is the property required of a reference scheme. Second, this explains why EXCLUSIVE is mandatory in the mode (a constraint we imposed in Definition 2.7 without justification): (2.3) and (2.4) alone restrict how many key values a target has, which is a participation or cardinality constraint on the descriptor, not a statement about identification. Such constraints, for instance “every post belongs to exactly one forum”, are deliberately outside the scope of PG-Keys [32].

Example 2.1. Over the graph of Figure 2.1, the key

FOR (x:Diagnosis) EXCLUSIVE MANDATORY x.name

has scope $p(x) = x \text{ WITHIN } Q_1[x]$ with Q_1 the single-vertex pattern labelled *Diagnosis*, and descriptor $q(x, u) = (x, x.\text{name}) \text{ WITHIN } Q_1[x]$. It requires every diagnosis vertex to carry a name and no two to share one, so node 2 (*Stress*) may not be duplicated. Combining topology with property values, a key on edges states that a patient is diagnosed with a given condition at most once per date:

FOR y WITHIN (:Patient)-[y:DIAGNOSED_WITH]->(:Diagnosis)

EXCLUSIVE x, y.date, z WITHIN (x:Patient)-[y]->(z:Diagnosis)

Here the scope output is an edge variable, so $\text{tgt}_p(G) \subseteq E$, and the descriptor mixes two object terms with one property term. Edge 11 of Figure 2.1 is a target; a second *DIAGNOSED_WITH* edge from node 21 to node 2 with the same date would violate (2.2).

Validation. Each of (2.2)-(2.4) is equivalent to the emptiness of a query built from p and q , so validation reduces to query evaluation and reuses the engine's existing machinery. For (2.2) one takes two copies of the scope with distinct target variables and two copies of the descriptor sharing the key value, and asks for pairs with $t_1 \neq t_2$; for (2.3) one asks for targets in $\{p\}_G$ admitting no descriptor binding; for (2.4) one takes one copy of the scope and two copies of the descriptor with distinct key-value variables. Note that, by Definition 2.5, matching a descriptor already entails that every property term it mentions is defined, so no separate IS NOT NULL test is required. Incremental validation is far less settled: it is manageable for the near-relational fragments, but once reachability or regular path expressions appear it requires auxiliary structures and datalog-maintenance techniques [46].

2.2.2 Graph Functional Dependencies and Graph Entity Dependencies

The GFD/GED line takes the opposite starting point: a constraint is a graph pattern plus an attribute dependency, and the intended use is error detection, entity resolution and repair over dirty graphs.

GFDs. A graph functional dependency [3] is $Q[\bar{x}](X \rightarrow Y)$, where Q is a graph pattern with variables $\bar{x}$ and X, Y are conjunctions of literals of the form $x.A = y.B$ (variable literals) and $x.A = c$ (constant literals). The semantics of GFD is defined over all matches (homomorphisms) h of Q in G : for every match, if $h \models X$ then $h \models Y$. GFDs thus combine a topological condition with a value dependency and subsume CFDs as the special case where Q degenerates to a single node.

GEDs. Graph entity dependencies [39] extend GFDs with id literals $x.id = y.id$, which assert that two matched vertices denote the same entity. This single addition unifies two previously separate classes: keys for graphs [38], which identify entities by recursive patterns, and GFDs, which catch inconsistencies. GEDs come with a revised chase that is shown to be Church-Rosser, giving a deterministic repair semantics: all chase sequences converge to the same result irrespective of the order in which rules are applied.

Numeric and ML extensions. NGFDs [40] add linear arithmetic expressions to catch numeric inconsistencies, at the cost of pushing satisfiability and implication to Σ_2^P -complete and Π_2^P -complete. GARs [41] extend GEDs with edge literals and with *ML predicates* (i.e. pre-trained classifiers returning a Boolean, e.g. BERT-based similarity or a neural entity-resolution model) thereby unifying logical deduction with learned prediction in a single rule format. GARs support association and link prediction as well as cleaning, but error detection with GARs takes exponential time in the worst case [41, 30]. In a parallel line, graph generating dependencies [47] adapt tuple- and equality-generating dependencies to property graphs, combining a source and a target graph pattern with attribute constraints.

Discovery. This family is discovery-driven throughout. GFD discovery [35] introduces reduced GFDs and topological support, and gives parallel algorithms for mining and for computing covers. GAR mining [36] uses sampling and application-driven predicate selection. GCR^+ discovery [30] departs from the usual interleaved “vertical/horizontal expansion” by generating patterns and predicates jointly from rule units, uses an LSTM language model over label sequences to select candidate paths, and a GNN with order embeddings to pre-estimate support and confidence with a probabilistic accuracy guarantee before exact verification.

2.2.3 Denial Constraints

Denial constraints are the most expressive of the classical integrity-constraint families. Relationally, a DC is a universally quantified first-order sentence

$$\forall t_1, \dots, t_n \neg(P_1 \wedge \dots \wedge P_m),$$

where each predicate P_i compares two cells or a cell and a constant using a built-in operator from $\{=, \neq, <, \leq, >, \geq\}$ [28]. A DC simply *forbids* a configuration; it does not say what should hold instead. This gives DCs the ability to subsume FDs, CFDs, order and metric dependencies in one syntax, and makes them the target of a substantial discovery literature [29, 48] as well as the basis for repair and consistent query answering.

Transposing DCs to graphs replaces the tuple variables and the implicit join by a graph pattern (or, in the standard-oriented proposals, by a graph query), while keeping the conjunction of comparison predicates that must not all hold simultaneously. Two observations follow.

First, several of the formalisms of §2.2.2 already contain DCs as a fragment. Once comparison operators are admitted, GCRs (and hence GCR^+ s) support the primitives of the relational cleaning rules CFDs, DCs and matching dependencies [42, 30]; an implication-form rule $Q[\bar{x}](X \rightarrow p_0)$ becomes a denial constraint when p_0 is taken to be unsatisfiable. This is also where the complexity boundary lies: it is exactly the addition of built-in comparisons that turns satisfiability NP-hard and implication coNP-hard even for star patterns [30].

Second, a standard-facing line has emerged around *PG-Constraints*, the constraint language accompanying GQL and SQL/PGQ. A significant fragment of PG-Constraints encodes denial constraints and, notably, admits recursion (constraints over paths of unbounded length such as “no cyclic references” or “every document has a higher access level than any document it references, directly or indirectly”) while still permitting automata-based structural analysis of

errors [43]. Because a DC only forbids, the repair space is large and under-determined: repairs are formulated as node, edge, or label deletions and computed by integer linear programming or LP-guided greedy strategies, with label deletion turning out to be markedly less destructive than node/edge deletion. This highlights the feature that distinguishes graph DCs from the FD-style formalisms: repairs may legitimately change the topology and not just attribute values.

Neighborhood Constraints

Neighborhood constraints are the minimal, purely structural instance of the DC idea, and predate the property-graph formulations. Song et al. [49, 44] consider graphs in which data values appear as vertex labels (i.e. similarity networks, workflow and event graphs, protein networks) where both the labels and the adjacency structure may be dirty. A neighborhood constraint is itself a graph: a constraint graph $\mathcal{S}(L, N)$ over the label alphabet L , whose edges N enumerate the label pairs permitted on adjacent vertices. An instance graph G with labelling λ satisfies $\mathcal{S}$ iff for every edge $(u, v) \in E$ the pair $(\lambda(u), \lambda(v))$ either consists of the same label or is an edge of $\mathcal{S}$; an edge whose endpoint labels are not permitted is a violation. Equivalently, a neighborhood constraint is a family of binary denial constraints $\neg \exists u, v (edge(u, v) \wedge \lambda(u) = l_1 \wedge \lambda(v) = l_2)$ for every disallowed pair (l_1, l_2) .

The interest lies in the repair problem rather than in detection, which is a linear scan of the edges. Repairs may modify vertex labels, or modify the neighborhood itself by inserting and deleting edges, and the objective is to reach a satisfying graph. The problem is NP-hard in general, which motivates approximation: greedy heuristics, a contraction-based method, and a hybrid of the two, together with the identification of tractable special cases (notably *star* and *clique* constraint graphs, which admit efficient exact solutions [44]).

Neighborhood constraints are deliberately less expressive, and two works extend them towards the expressiveness of the formalisms above. StarFDs [45] condition value dependencies on an entity together with its relevant neighbours, where the neighbourhood is delimited by a star pattern built from conjunctive regular path queries. The recursive fragment of PG-Constraints [43] can be read as the other generalisation: neighbourhoods of unbounded radius specified by regular paths, with attributes and comparisons restored.

2.3 Graph Neural Networks

The constraint formalisms of §2.2 are deductive: a GFD, a key or a denial constraint states what must hold, and validation is a matching problem with a definite answer. A large part of what one wants to do with a graph, however, is not deducible from the data, e.g. whether two vertices denote the same entity, whether an edge is missing, whether a patient is at risk of readmission. These are inductive questions, answered by generalising from observed regularities, and the dominant machinery for answering them over graph-structured data is the graph neural network (GNN).

The common ground is representation learning: we learn an encoder mapping each vertex $v \in V$ to a vector $z_v \in \mathbb{R}^d$ such that vertices playing similar structural and attribute-level roles receive similar embeddings, and then we solve the downstream task with a classifier on top of z_v .

Early encoders include DeepWalk [50] and node2vec [51], that sample random walks and fit a skip-gram objective, so that the encoder is a lookup table with one free vector per vertex. Such encoders are transductive: a vertex absent at training time has no embedding, so the model must be refitted whenever the graph G changes, and they see only the edge set E , ignoring the nodes' properties. Both are disqualifying in an application setting, where the graph is repeatedly modified and where the attribute values are precisely what the task is about.

Graph neural networks replace the lookup table with a parameterised function of a vertex's neighbourhood, evaluated layer by layer. Almost every architecture in current use is an instance of the message-passing scheme [52]: writing $h_v^{(0)} = x_v$ for the input features of v and $\mathcal{N}(v)$ for its neighbourhood, a message passing layer that computes a new node representation $h_v^{(l+1)}$ at layer $l + 1$ is

$$h_v^{(l+1)} = \phi^{(l)} \left(h_v^{(l)}, \bigoplus_{u \in \mathcal{N}(v)} \psi^{(l)}(h_v^{(l)}, h_u^{(l)}, h_{e_{uv}}^{(l)}) \right), \quad (2.5)$$

where $\psi^{(l)}$ is a learnable message function, $\bigoplus$ a permutation-invariant aggregator (sum, mean, max), and $\phi^{(l)}$ a learnable update. After L layers, $z_v = h_v^{(L)}$ summarises the L -hop neighbourhood of v . Since the parameters are shared across all vertices, the encoder is inductive: the same architecture applies to vertices and to graphs never seen during training [53].

The standard applications of GNN include three tasks: *vertex-level* prediction (classification or regression on z_v), *edge-level* prediction (a score $f(z_u, z_v)$ for a candidate edge, used for link prediction), and *graph-level* prediction (a readout or classification over a whole graph or subgraph).

We now describe the two architectures used in this thesis, which are the most common employed models in standard applications [54], namely Graph Convolutional Networks (GCNs) and Graph Attention Networks (GATs). Many variations of both have since been proposed, but almost all of them retain one of the two aggregation principles these models introduced, a neighbour weighting fixed by the topology or one learned from the endpoint features. We therefore present each in its original homogeneous formulation before developing the heterogeneous extension we actually employ, R-GCN in §2.3.1 and hGAT in §2.3.2.

2.3.1 Graph Convolution and R-GCN

The graph convolutional network (GCN) [8] is the simplest useful instantiation of (2.5). A vertex updates its state by averaging the states of its neighbours together with its own, and the average is passed through a single linear map shared by the whole layer. The name is borrowed from image convolution: a convolutional filter replaces a pixel by a weighted combination of the pixels around it, and a GCN layer replaces a vertex state by a weighted combination of the states around it in G . The difference is that a pixel has a fixed number of neighbours in a fixed spatial arrangement, whereas a graph neighbourhood has neither a fixed size nor an ordering, so the combination must be a permutation-invariant average.

Let $A \in \{0, 1\}^{|V| \times |V|}$ be the adjacency matrix of a graph G , $\tilde{A} = A + I$ the adjacency with self-loops added (the node considers its own state), and $\tilde{D}$ the corresponding degree matrix. The vertices' features at each layer l are store in a feature matrix $H^{(l)} \in \mathbb{R}^{|V| \times d_l}$, one row per vertex, where d_l is the output dimension of the layer l . Each layer has a its own parameters, or weights,

stores in the weight matrix $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$. A GCN layer is

$$H^{(l+1)} = \sigma\left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(l)} W^{(l)}\right), \quad (2.6)$$

or, per vertex,

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_u \tilde{d}_v}} W^{(l)} h_u^{(l)}\right), \quad (2.7)$$

with $\tilde{d}_v$ the degree of v in $\tilde{A}$ and σ an elementwise nonlinearity (i.e. RELU or LeakyRELU).

Read against (2.5), the message function is a linear map shared by all neighbours, the aggregator is a sum, and the update is the identity, with the self term folded into the sum by the added self-loop. The symmetric normalisation $(\tilde{d}_u \tilde{d}_v)^{-1/2}$ keeps the spectral radius bounded and prevents high-degree vertices from dominating.

Each layer therefore performs one round of localised averaging over the one-hop neighbourhood, and L stacked layers give every vertex a receptive field of radius L . The whole layer is described by a single matrix $W^{(l)}$, so the parameter count is independent of $|V|$ and $|E|$.

The first layer is the input layer ($l = 0$), and it corresponds to the feature matrix $X \in \mathbb{R}^{|V| \times d_0}$, where d_0 is the dimension of the embedding space for vertices:

$$H^{(0)} = X, \quad h_v^{(0)} = x_v, \quad (2.8)$$

Three remarks on (2.8) carry through the rest of the section. First, X presupposes that every vertex is described in one common space $\mathbb{R}^{d_0}$. Second, the weight attached to a neighbour is fixed by the topology alone, so the model cannot learn that one neighbour matters more than another. Third, $\tilde{A}$ records only whether an edge exists, collapsing the edge set E to a single relation, so that in Example 2.1 a PRESCRIBED neighbour and a DIAGNOSED_WITH neighbour are aggregated through the same $W^{(l)}$ and differ only in their degrees.

The heterogeneous extension: RGCN. RGCN [55] addresses the third problem by splitting the neighbourhood by edge type and giving each relation its own weight matrix. Writing $\mathcal{N}_r(v) = \{u \in V : \exists e \in E, \eta(e) = (u, v), \rho(e) = r\}$ for the neighbours of v connected with an edge of type r , the RGCN layer is

$$h_v^{(l+1)} = \sigma\left(W_0^{(l)} h_v^{(l)} + \sum_{r \in \mathcal{R}} \sum_{u \in \mathcal{N}_r(v)} \frac{1}{c_{v,r}} W_r^{(l)} h_u^{(l)}\right), \quad (2.9)$$

where $W_0^{(l)}$ is a self-loop matrix retaining the vertex's own state and $c_{v,r}$ is a normalisation constant, taken to be $|\mathcal{N}_r(v)|$.

R-GCN inherits the first problem of (2.7). The weight $1/c_{v,r}$ depends only on degree, so within a given relation every neighbour contributes equally. An RGCN can learn that PRESCRIBED matters more than CONTAINS, but not that one particular prescription matters more than another.

2.3.2 Graph Attention and hGAT

The graph attention network (GAT) [9] makes the aggregation weights learnable, replacing the fixed $(\tilde{d}_u \tilde{d}_v)^{-1/2}$ of (2.7) by a coefficient computed from the vertices' features themselves.

A GAT layer is defined per vertex, since the coefficient attached to an edge now depends on the two vertices it connects. Let $h_v^{(l)} \in \mathbb{R}^{d_l}$ be the features vector of vertex v at layer l , that is, the row of $H^{(l)}$ indexed by v . As in a GCN, each layer has a weight matrix $W^{(l)} \in \mathbb{R}^{d_l \times d_{l+1}}$ shared by every vertex, and in addition a learnable attention vector $a^{(l)} \in \mathbb{R}^{2d_{l+1}}$, whose length is twice the output dimension because it is applied to a pair of projected nodes. Let $\parallel$ denote concatenation, so that $[W^{(l)}h_v^{(l)} \parallel W^{(l)}h_u^{(l)}] \in \mathbb{R}^{2d_{l+1}}$, and let $\mathcal{N}(v)$ be the neighbourhood of v with v itself included, playing the role that the self-loops of $\tilde{A}$ play in (2.6). Each neighbour is first assigned a scalar score

$$e_{vu}^{(l)} = \text{LeakyReLU}(a^{(l)\top} [W^{(l)}h_v^{(l)} \parallel W^{(l)}h_u^{(l)}]) \in \mathbb{R}, \quad (2.10)$$

where LeakyReLU is the leaky rectifier $x \mapsto \max(x, \gamma x)$ with a small fixed slope γ , which keeps the score sensitive to pairs that would otherwise be clipped to zero. The score $e_{vu}^{(l)}$ measures how relevant u is to v and is not symmetric. Scores are then normalised over the neighbourhood and used as aggregation weights,

$$\alpha_{vu}^{(l)} = \frac{\exp(e_{vu}^{(l)})}{\sum_{w \in \mathcal{N}(v)} \exp(e_{vw}^{(l)})}, \quad h_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(l)} W^{(l)}h_u^{(l)}\right), \quad (2.11)$$

with σ an elementwise nonlinearity. The softmax in (2.11) is taken over $\mathcal{N}(v)$ alone, so the coefficients satisfy $\alpha_{vu}^{(l)} \in (0, 1)$ and $\sum_{u \in \mathcal{N}(v)} \alpha_{vu}^{(l)} = 1$ for every v , and are therefore comparable across neighbourhoods of different sizes. The attention is typically computed by M independent heads whose outputs are concatenated, and averaged, which stabilises training and lets different heads specialise on different aspects of the neighbourhood. Setting α_{vu} to a constant recovers a mean-aggregating convolution, so GAT strictly generalises the fixed-weight scheme.

GATs carry the same gaps as GCNs: Wh_u doesn't consider node types. Moreover a single softmax over $\mathcal{N}(v)$ pools neighbours of every type into one normalisation, so the relative importance of a kind of evidence cannot be separated from the importance of an individual neighbour.

The heterogeneous extension: hGAT. Heterogeneous attention networks [56, 57] resolve both points with two attention mechanisms stacked on a type-specific projection. First, each vertex is mapped into a common space by a projection indexed by its type,

$$h'_v = W_{\tau(v)} x_v \in \mathbb{R}^d, \quad (2.12)$$

which is the minimal fix for the typing problem and is what all heterogeneous models share. *Node-level attention* then aggregates, separately for each type $t \in \mathcal{T}$, the type- t neighbours of v ,

with coefficients computed by a type-specific attention vector μ_t :

$$\alpha_{vu}^t = \text{softmax}_{u \in \mathcal{N}_t(v)} \left(\sigma(\mu_t^\top [h'_v \parallel h'_u]) \right), \quad z_v^t = \sum_{u \in \mathcal{N}_t(v)} \alpha_{vu}^t h'_u, \quad (2.13)$$

which is exactly (2.11) restricted to one type at a time, and yields one representation z_v^t per neighbour type. Type-level attention then weighs these against each other. With q a learnable attention vector and (W, b) a shared MLP,

$$w_t = \frac{1}{|V|} \sum_{v \in V} q^\top \tanh(W z_v^t + b), \quad \beta_t = \frac{\exp(w_t)}{\sum_{t' \in \mathcal{T}} \exp(w_{t'})}, \quad h_v = \sum_{t \in \mathcal{T}} \beta_t z_v^t. \quad (2.14)$$

The two levels answer complementary questions. The coefficient α_{vu}^t asks which neighbour of a given type is informative for v , and β_t asks which type of evidence is informative for the task. Note that α is global, i.e., one value per type averaged over V . It can therefore be read as a summary of the model's inductive bias.

2.4 Causality

The preceding sections describe how graph data is modelled, queried, and matched. All of those operations remain associational: they report what is present in the data. Chapters 6 and 7 instead ask what would happen under a change, which is a question of a different kind and requires a different formalism. This section introduces that formalism, namely the structural theory of causation developed by Pearl [58], and fixes the notation used throughout Part II.

2.4.1 From Correlation to Causation

Statistical learning operates on a joint distribution $P(\mathbf{V})$ over a set of variables $\mathbf{V} = \{V_1, \dots, V_n\}$, and return quantities such as $P(y \mid x)$ or $\mathbb{E}[Y \mid X = x]$, that is, summaries of how Y is distributed within the subpopulation in which $X = x$ is observed. These describe how beliefs about Y change upon observing X , but they say nothing about how Y would change were one to act on X . The distinction is not only philosophical: an associational model cannot separate $X \rightarrow Y$ from $Y \rightarrow X$ or from $X \leftarrow Z \rightarrow Y$, since all three mechanisms can induce the same statistical dependence.

If X is not statistically independent from Y ($X \not\perp Y$), then either X causes Y , or Y causes X , or there is a common cause Z that renders the two variables independent upon conditioning (Reichenbach's common cause principle [59]). The practical consequences are familiar. For instance, under confounding a shared cause makes two variables move together although neither influences the other. Under selection bias, the association is an artefact of how the data came to be recorded rather than of the domain itself. And in Simpson's paradox, an association that holds within every stratum of a population changes once the strata are pooled, so that the aggregate and the parts support opposite conclusions. In a data management setting this is more than a modelling inconvenience. An aggregation over the wrong grouping attribute, or over a graph pattern that silently conditions on a collider (Section 2.4.4), yields a query answer that is

Layer	Typical query	Question answered
1. Association	$P(y \mid x)$	What does observing $X = x$ tell me about Y ?
2. Intervention	$P(y \mid do(x))$	What happens to Y if I set X to x ?
3. Counterfactual	$P(y_x \mid x', y')$	Given that $X = x'$ and $Y = y'$ occurred, what would Y have been had X been x ?

Table 2.3: The three layers of the causal hierarchy.

statistically correct and causally misleading, with nothing in the query semantics to signal the problem [60].

Answering questions of the form “what if” therefore requires a language in which causal assumptions can be stated explicitly, held apart from statistical assumptions, and combined with data.

2.4.2 The Ladder of Causation

Pearl organises causal questions into a three level hierarchy, the **ladder of causation**, in which each step requires strictly more information than the one below it [10]. The three layers are summarised in Table 2.3, where lower case letters denote values taken by the corresponding upper case variables, so that $P(y \mid x)$ abbreviates $P(Y = y \mid X = x)$.

Layer 1, association. The quantity $P(y \mid x)$ is the ordinary conditional distribution, obtained by restricting attention to the subpopulation in which $X = x$ was observed and reading off the distribution of Y there. This layer is determined entirely by the observational distribution, and it is the layer at which conventional querying and supervised learning operate. A question such as “among the patients recorded as having received treatment x , what proportion recovered?” is an association question, and it is answered by a pattern match followed by an aggregation.

Layer 2, intervention. The expression $P(y \mid do(x))$ is not a conditional distribution, and the operator $do(\cdot)$, defined formally in Section 2.4.5, is what marks the difference. Instead of selecting the subpopulation in which $X = x$ happened to hold, it describes the distribution of Y in a modified world in which X has been forced to the value x for everyone, leaving the rest of the system as it was. The corresponding question is “if treatment x were administered to a patient, what would the probability of recovery be?”, which is not the same question as before: if clinicians prescribe x preferentially to the more severely ill, then the observed recovery rate among treated patients reflects their severity as much as the treatment. Layer 2 is the target of randomised experiments and of policy evaluation.

Layer 3, counterfactuals. The subscript in y_x denotes the value that Y would take under the intervention $do(x)$, so that $P(y_x \mid x', y')$ conditions on facts that actually obtained, namely $X = x'$ and $Y = y'$, while asking about an antecedent that contradicts them. The question is “this patient received treatment x' and did not recover; would they have recovered under treatment x ?”, and answering it requires holding fixed everything specific to that individual while varying only the treatment. Because such questions are about a single unit and not a population, Layer 3 is what underlies attribution, responsibility, and individual level explanation, including the question of which particular edit was responsible for a given outcome.

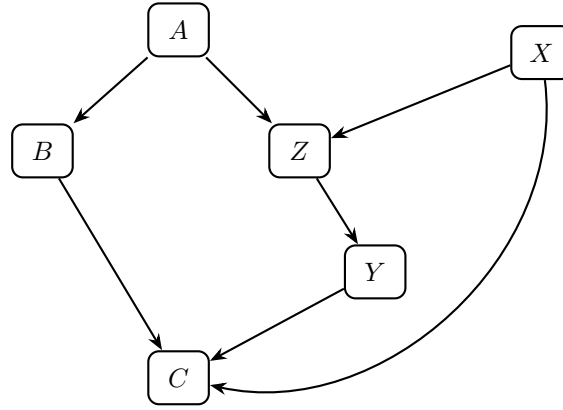


Figure 2.3: A causal directed acyclic graph over six variables.

The Causal Hierarchy Theorem [61] establishes that questions at a higher layer cannot be answered from lower layer information alone: no volume of observational data determines an interventional distribution, and no volume of experimental data determines a counterfactual quantity, in the absence of further structural assumptions. This is the formal reason why a causal system requires a model of the process that generated the data and not a model of the data alone. A property graph on its own is insufficient and must be augmented with causal structure (Chapter 7).

2.4.3 Structural Causal Models

One of the most established theories of causality is represented by *structural causal models* (SCMs) [58]. An SCM has two components: a causal graph, which records the qualitative assumption of which variables act on which, and a set of structural equations, which specify how they act. Causal graphs are a special class of Bayesian networks in which edges carry causal meaning, so they inherit the well defined conditional independence criteria of that family [62].

Formally, a causal graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is a directed graph where $\mathcal{V}$ is the node set and $\mathcal{E}$ is the edge set. Each node in $\mathcal{V}$ represents a random variable, while an edge $x \rightarrow y$ represents a causal effect between two variables $x, y \in \mathcal{V}$. Given an edge $x \rightarrow y$ we call x the *exposure* and y the *outcome*. The node set $\mathcal{V}$ contains both the observed and the unobserved variables.

A *directed path* is a sequence of directed edges that point in the same direction. We consider causal directed acyclic graphs (cDAGs), that is, causal graphs that contain no cycles [63]. Acyclicity is what guarantees that the structural equations below admit a unique solution, and it is assumed throughout this thesis. Figure 2.3 shows an example of a cDAG with six variables. The path $A \rightarrow B \rightarrow C$ is a *causal path*, representing a causal relation between the variables A and C through B , which is called a *mediator*. The variables A and X are *confounders*, that is, each of them is the cause of two variables simultaneously. Finally, the variables Z and C are called *colliders*, because they have more than one incoming edge. Note that Z is a collider and a mediator at the same time, on two different paths, so these roles are properties of a node relative to a path and not of the node alone.

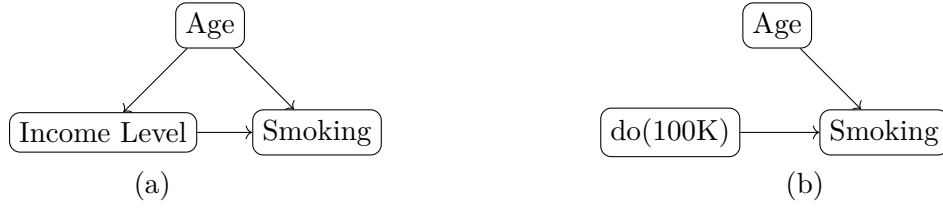


Figure 2.4: SCM without (a) and with (b) intervention.

The second component of a structural causal model consists of the *structural equations*. These are non-parametric equations that specify, for each variable, the mechanism by which its direct causes and an unobserved disturbance determine its value.

For instance, the structural equations for the SCM in Figure 2.4(a) are:

$$Age = f_{Age}(\epsilon_{Age})$$

$$Income\ Level = f_{Income\ Level}(Age, \epsilon_{Income\ Level})$$

$$Smoking = f_{Smoking}(Age, Income\ Level, \epsilon_{Smoking}),$$

where ϵ_x denotes the noise of the observed variable x . Each equation takes as arguments exactly the parents of its left hand side variable in $\mathcal{G}$, so the graph and the equations carry the same qualitative information, the equations adding the mechanism itself. Note that the functions $f_x()$ are not invertible: they are to be read as assignments and not as algebraic identities, since the mechanism runs from the right hand side to the left hand side and not in the reverse direction. This asymmetry, which is absent both from regression equations and from database views, is what carries the causal content of the model. When the noise terms ϵ_x are mutually independent the model is called *Markovian*; a dependence between two noise terms represents an unobserved common cause, is drawn as a bidirected edge, and yields a *semi-Markovian* model. Almost every model built over observational data is semi-Markovian. Hence, identification (Section 2.4.5) is a substantive problem.

2.4.4 Graphical Structure and d-Separation

Every path decomposes into the three shapes shown in Figure 2.5, and it is worth examining each in turn, because the third behaves in the opposite way to the first two.

In the chain of Figure 2.5(a), conditioning on the intermediate variable *Income Level* blocks the path and so d-separates *Age* and *Smoking*. Intuitively, once the income level is known, age carries no further information about the smoking habit, since whatever influence age exerted was exerted through income. The same applies to the confounding path of Figure 2.5(b): conditioning on *Age* blocks the path and allows the causal relationship between the income level and the smoking habit to be studied free of the confounding bias that age would otherwise introduce.

The collider path of Figure 2.5(c) behaves in the opposite way. Here *Age* and *Income Level* are already d-separated, and therefore statistically independent, without conditioning on anything. Conditioning on the collider *Smoking* opens the path and induces a spurious association between them, even though neither is a cause of the other. This is the mechanism behind selection bias:

restricting attention to smokers creates a dependence between age and income level that does not exist in the population at large. The same happens when conditioning on any descendant of a collider, which makes this the case most easily overlooked in practice, since it means that adjusting for additional variables can create bias rather than remove it.

Conditional independence is guaranteed in causal graphs through *dependency-separation* (d-separation) [58], a purely graphical criterion from the structure of $\mathcal{G}$ alone, which independences must hold in any distribution the graph describes.

Definition 2.10 (d-separation). *Let $X, Y \in \mathcal{V}$ and let $\mathcal{Z} \subseteq \mathcal{V} \setminus \{X, Y\}$. A path p (directed or undirected) between X and Y is blocked by $\mathcal{Z}$ if either*

1. *p contains a chain $i \rightarrow m \rightarrow j$ or a fork $i \leftarrow m \rightarrow j$ such that $m \in \mathcal{Z}$, or*
2. *p contains a collider $i \rightarrow m \leftarrow j$ such that $m \notin \mathcal{Z}$ and no descendant of m belongs to $\mathcal{Z}$.*

X and Y are d-separated by $\mathcal{Z}$, written $(X \perp\!\!\!\perp Y \mid \mathcal{Z})_{\mathcal{G}}$, if $\mathcal{Z}$ blocks every path between them.

The definition treats the three shapes of Figure 2.5 asymmetrically, and it is worth taking them in turn, because the third behaves in the opposite way to the first two. In the chain of Figure 2.5(a), conditioning on the intermediate variable *Income Level* blocks the path and so d-separates *Age* and *Smoking*. Intuitively, age acts on the smoking habit only by way of the income it produces, so once the income level is known, age carries no further information about smoking. Note that the causal influence of age has not disappeared: what disappears is the informational dependence, which is the only thing the data can see.

The fork of Figure 2.5(b) behaves the same way for a different reason. Here *Income Level* and *Smoking* are dependent although neither causes the other, because both are driven by *Age*. Conditioning on the common cause blocks the path and removes the confounding bias. Thus, comparisons drawn within age groups are informative, while comparisons drawn over the whole population are not.

The collider of Figure 2.5(c) reverses the pattern. The path is blocked by default, so *Age* and *Income Level* are d-separated by the empty set and are therefore independent without any conditioning at all. Conditioning on the collider *Smoking* opens the path and induces a dependence between two variables neither of which causes the other. The mechanism is one of explaining away: among smokers, learning that an individual is young makes a high income a less plausible account of why they smoke, so the two variables become informative about each other within that subpopulation even though they were unrelated outside it. This is the shape behind selection bias, and clause 2 of Definition 2.10 extends it to any *descendant* of the collider, since conditioning on a proxy for a common effect leaks the same information. The practical consequence deserves emphasis: conditioning on additional variables can create bias rather than remove it, so adjustment sets must be chosen from the structure of $\mathcal{G}$ and not by including whatever covariates happen to be available.

Two assumptions connect this graphical criterion to the joint distribution $P(\mathcal{V})$ induced by the structural equations. The *Markov condition* states that every variable is independent of its non-descendants given its parents, which is equivalent to requiring that d-separation in $\mathcal{G}$ imply conditional independence in $P(\mathcal{V})$. It holds by construction whenever the noise terms

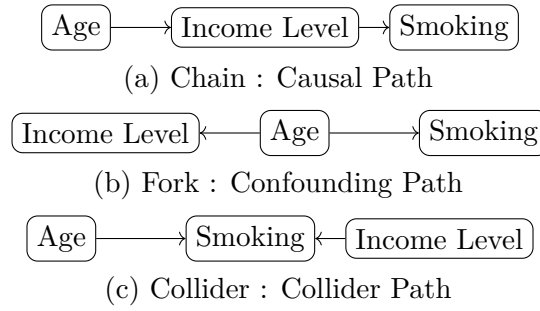


Figure 2.5: DAGs for conditional independence.

are mutually independent, and it licenses the direction of reasoning that matters in practice: from an assumed graph to independences that the data must exhibit. The converse direction requires *faithfulness* [64], which states that $P(\mathcal{V})$ exhibits no independences beyond those that d-separation implies. Faithfulness can fail, but only through exact numerical coincidence, as when the influences carried by two paths cancel precisely and leave no net dependence; such models occupy a set of measure zero in the space of models compatible with $\mathcal{G}$, so the assumption is standard, although it is an assumption.

Under both assumptions the graph and the independence structure of the distribution are two views of the same object, and d-separation is moreover *complete*: whenever X and Y are not d-separated by $\mathcal{Z}$, there is some distribution compatible with $\mathcal{G}$ in which they are dependent given $\mathcal{Z}$. The criterion therefore extracts exactly the independences that follow from the structure.

The whole framework depends on this correspondence, for two reasons. First, it makes causal assumptions testable. A graph asserts a specific list of conditional independences, each of which can be checked against the data, and an independence that the graph predicts but the data contradicts refutes the graph. This is the basis of constraint-based structure learning [64]. Second, it reduces the question of which variables must be conditioned on to a question about which paths exist and which of them are blocked, that is, to a reachability problem over a graph. The reduction is more than a change of vocabulary: although the number of paths between two nodes may be exponential in $|\mathcal{V}|$, d-separation can be decided in time linear in the size of $\mathcal{G}$ by propagating reachability from the queried nodes.

2.4.5 Interventions and the do-Operator

An *intervention* is an external act that forces a variable to take a value, overriding whatever mechanism ordinarily determines it. The distinction between observing a value and imposing it is the whole content of the difference between the first and the second layer of the hierarchy, and it is easiest to see on an example.

Consider the model of Figure 2.4(a), in which *Age* influences both *Income Level* and *Smoking*, and *Income Level* in turn influences *Smoking*. Two questions can be asked about income and smoking. The first is: among the individuals who earn 100K, what fraction smoke? This question selects a subpopulation, namely those individuals whose income mechanism happened to output 100K, and reports what is observed inside it. The second is: if every individual were given an income of 100K, what fraction would smoke? This question is not about any subpopulation of the recorded data, because no such intervention was ever performed. It asks about a different

system, one in which the mechanism that ordinarily sets the income level has been replaced by an external decision. The two questions have different answers, and only the second one is what a policy maker considering an income subsidy would want to know.

Structural equations provide a quantitative way to represent an intervention on a variable of a cDAG. The *do-calculus* [58] introduced the $do(x')$ operator, which denotes the intervention of setting the value of a variable x to x' . The operator is realised in the model by replacing the structural equation of x with the constant assignment $x := x'$ and leaving every other equation intact. Figure 2.4(b) shows an example, where $do(100K)$ sets the value of the variable *Income Level* to 100K. The system of equations of Section 2.4.3 becomes

$$Age = f_{Age}(\epsilon_{Age})$$

$$Income\ Level = 100K$$

$$Smoking = f_{Smoking}(Age, Income\ Level, \epsilon_{Smoking}).$$

The mechanism producing income has been removed, since income is now decided externally, whereas the mechanism producing the smoking habit is untouched: individuals still respond to their income in the same way, they merely have a different income to respond to.

The graphical counterpart of the same operation is the deletion of all edges directed into the intervened variable, an operation known as *graph surgery*. We write $\mathcal{G}_{\overline{X}}$ for the resulting mutilated graph. In the example, the edge $Age \rightarrow Income\ Level$ disappears, since age no longer has any bearing on an income that is assigned by decree. The model obtained in this way is called a *submodel* and denoted $\mathcal{M}_{x'}$, and the distribution it induces is the *interventional distribution*, written $P(y \mid do(x'))$. For the example,

$$P(Smoking \mid do(100K)) = \sum_{age} P(Smoking \mid 100K, age) P(age),$$

which is the distribution of *Smoking* generated by the equations above once the noise terms have been marginalised out.

It is important to note that $P(y \mid do(x))$ and $P(y \mid x)$ are not the same quantity. Their difference is the *confounding bias*, and in the example it can be exhibited exactly. Decomposing both quantities over the confounder gives

$$P(y \mid x) = \sum_z P(y \mid x, z) P(z \mid x), \quad P(y \mid do(x)) = \sum_z P(y \mid x, z) P(z). \quad (2.15)$$

The two expressions share their summands and differ only in the weights. The observational quantity weights each stratum of *Age* by how prevalent that stratum is *among individuals of the given income*, so an income of 100K that is more common at one age drags the answer toward the smoking behaviour typical of that age. The interventional quantity weights each stratum by how prevalent it is in the population, because an externally assigned income carries no information about age. In Figure 2.4(a), therefore, $P(Smoking \mid Income\ Level)$ results from combining the causal effect $P(Smoking \mid do(Income\ Level))$ with the statistical association carried by the confounding path $Income\ Level \leftarrow Age \rightarrow Smoking$. The two coincide when X

has no incoming edges in $\mathcal{G}$, since then surgery removes nothing and there is no backdoor path to carry association. This is what randomisation achieves physically, and in practice verifying the absence of confounding bias amounts to verifying that $P(y \mid do(x)) = P(y \mid x)$.

Obtaining an unbiased estimate therefore requires removing the confounding bias, an operation known as *causal identification*. A causal quantity is *identifiable* from a graph $\mathcal{G}$ together with an observational distribution $P(\mathcal{V})$ if it can be expressed uniquely in terms of $P(\mathcal{V})$, so that it can be computed from data that no experiment produced. The best known sufficient condition is the *backdoor criterion*, and it is stated directly in the vocabulary of Section 2.4.4.

Definition 2.11 (Backdoor criterion). *A set $\mathcal{Z} \subseteq \mathcal{V}$ satisfies the backdoor criterion relative to an ordered pair (X, Y) if no node in $\mathcal{Z}$ is a descendant of X , and $\mathcal{Z}$ blocks every path between X and Y that contains an edge directed into X .*

Paths of the second kind are called *backdoor paths*, since they leave X through the back door, against the direction of causation, and they are precisely the routes by which association flows without causation. Blocking all of them while leaving the directed paths intact isolates the causal effect, and yields the *adjustment formula*

$$P(y \mid do(x)) = \sum_{\mathbf{z}} P(y \mid x, \mathbf{z}) P(\mathbf{z}). \quad (2.16)$$

The exclusion of descendants of X matters: conditioning on a variable that lies on a directed path from X to Y would block part of the effect being measured, and conditioning on a descendant of a collider would open a path rather than close one. In the running example, $\mathcal{Z} = \{Age\}$ satisfies the criterion, since *Age* is not a descendant of *Income Level* and blocks the only backdoor path, and Equation 2.16 then recovers the interventional distribution computed above. The intuition behind the formula is that the causal effect is estimated within strata that are homogeneous with respect to the confounder, where income no longer varies on account of age, and the stratum specific estimates are then averaged over the age distribution of the population rather than that of the treated group.

Adjustment is sufficient but not necessary. When the confounder is unobserved there may be no admissible set, and yet the effect may still be identifiable by other means: the *front-door criterion* exploits a variable that fully mediates the effect of X on Y and is itself unconfounded with Y , identifying the effect through the mediator in two stages. More generally, *do-calculus* [65] provides three rules that govern the insertion and deletion of observations, the exchange of an action for an observation, and the insertion and deletion of actions, in expressions that mix $do(\cdot)$ with ordinary conditioning. Each rule is licensed by a d-separation condition checked in a suitably mutilated graph, so the entire apparatus of Section 2.4.4 is reused here. Repeated application rewrites an expression containing $do(\cdot)$ into one that does not, at which point the quantity has been reduced to observational terms and is identified.

Do-calculus is sound, and it is also complete [66, 67]: any effect identifiable from $\langle \mathcal{G}, P(\mathcal{V}) \rangle$ can be derived by finitely many applications of its rules, and any effect that cannot be so derived is not identifiable from that pair, so that failure to find a derivation is itself an answer rather than an inconclusive result. Completeness is what licenses treating identification as a mechanisable

procedure, and therefore as something that a query engine can be asked to carry out on the user's behalf and not something the user must derive by hand.

2.4.6 Counterfactuals

An intervention asks what would happen to a population if a variable were forced to a value. A *counterfactual* asks a narrower and more demanding question: what would have happened to *this particular unit* if the variable had been forced to a different value from the one it actually took. The question is retrospective, and its antecedent contradicts the record.

Return to the model of Figure 2.4(a) and consider a single individual, observed to be 45 years old, to earn 30K, and to smoke. Three questions can now be distinguished, one per layer of the hierarchy. At Layer 1 one asks what fraction of individuals earning 30K smoke. At Layer 2 one asks what fraction of the population would smoke if everyone were given an income of 100K. At Layer 3 one asks whether *this* individual, who earns 30K and smokes, would still have smoked had their income been 100K. The third question is not answered by the second, because the population average speaks about nobody in particular, whereas the counterfactual is about a person with a known age and a known disposition to smoke, both of which must be held fixed while the income alone is varied.

What makes this well posed is that a structural causal model is deterministic once the noise is fixed. All the idiosyncrasy of a unit, everything that distinguishes one individual from another with the same observed parents, resides in the realisation of the noise terms. We write ϵ for the vector of all noise terms of the model, and note that a realisation ϵ determines the value of every variable in $\mathcal{V}$ through the structural equations. A unit, in this framework, just is a value of ϵ .

Definition 2.12 (Potential outcome). *Let $\mathcal{M}$ be an SCM and let ϵ be a realisation of its noise terms. For $X, Y \in \mathcal{V}$, the potential outcome $Y_x(\epsilon)$ is the value taken by Y in the submodel $\mathcal{M}_x$ when the noise is fixed to ϵ .*

Because ϵ determines every variable, and because the submodel $\mathcal{M}_x$ is defined regardless of what value X takes in $\mathcal{M}$ itself, the quantity $Y_x(\epsilon)$ is well defined even when the antecedent $X = x$ contradicts the observed record.

The individual noise realisation is of course not observed. What is observed is evidence e , a set of assignments to some of the variables in $\mathcal{V}$, and the evidence constrains which noise realisations are compatible with the unit at hand. This yields the three step procedure for evaluating a counterfactual.

Definition 2.13 (Counterfactual evaluation). *Given an SCM $\mathcal{M}$, evidence e , and an antecedent $X = x$, the counterfactual quantity $P(y_x | e)$ is obtained as follows:*

1. **Abduction:** *update the distribution of the noise terms to $P(\epsilon | e)$, retaining only the realisations that could have produced the evidence;*
2. **Action:** *form the submodel $\mathcal{M}_x$ by replacing the structural equation of X with the assignment $X := x$;*
3. **Prediction:** *compute the distribution of Y in $\mathcal{M}_x$ under $P(\epsilon | e)$.*

Applied to the individual above, with $e = \{Age = 45, Income\ Level = 30K, Smoking = yes\}$ and the antecedent $Income\ Level = 100K$, the three steps read as follows. Abduction pins the noise of Age and of $Income\ Level$ to the values that reproduce 45 and 30K, and restricts $\epsilon_{Smoking}$ to those realisations satisfying $f_{Smoking}(45, 30K, \epsilon_{Smoking}) = yes$, which is the formal counterpart of the phrase “a person disposed to smoke at that age and that income”. Action replaces the income equation by $Income\ Level := 100K$, which by graph surgery also removes the edge $Age \rightarrow Income\ Level$. Prediction then evaluates

$$Smoking_{100K} = f_{Smoking}(45, 100K, \epsilon_{Smoking}), \quad \epsilon_{Smoking} \sim P(\epsilon_{Smoking} | e).$$

If the surviving set of noise realisations yields the same outcome throughout, the counterfactual has a determinate answer; otherwise it has a probability, and the residual uncertainty reflects what the evidence failed to pin down about the unit.

Two features of this computation deserve emphasis, because together they delineate Layer 3 from Layer 2. First, surgery and abduction pull in opposite directions and both are needed: surgery discards the mechanism that generated the income, while abduction retains the background conditions of the individual, including the age that the discarded mechanism had used. Averaging over the population instead of abducting would answer the Layer 2 question and lose the individual. Second, the factual and the counterfactual value of $Smoking$ are evaluated in two different submodels, $\mathcal{M}$ and $\mathcal{M}_{100K}$, under one and the same realisation ϵ . It is this shared noise that makes the two worlds concern the same person, and a quantity of this kind is accordingly called *cross-world*.

Cross-world dependence is also the source of the central difficulty. A counterfactual quantity may constrain the joint distribution of Y_x and $Y_{x'}$ for $x \neq x'$, and no experiment can ever observe that joint distribution, since a unit can be assigned only one of the two treatments. Two models may therefore agree on every interventional distribution, and hence be indistinguishable by any randomised experiment, while disagreeing on counterfactuals: agreeing on the marginals of Y_x and $Y_{x'}$ leaves their coupling free. Consequently the quantities used for attribution are in general not identifiable even from experimental data. The *effect of treatment on the treated*,

$$ETT = P(y_x | x', y'),$$

which asks what would have become of the units that actually received x' had they received x instead, and the *probability of necessity*,

$$PN = P(Y_{x'} = 0 | X = x, Y = 1),$$

which asks whether the observed cause was necessary for the observed effect and is therefore the natural formal counterpart of attributing responsibility, can typically only be bounded from data [68, 69]. Point identification requires additional assumptions, such as monotonicity of the response, which excludes units that react to treatment in the opposite direction to the rest, or knowledge of the functional form of the equations $f_x()$, which determines the coupling outright. This is the sense in which the third layer of the hierarchy of Section 2.4.2 is strictly stronger than the second, and the gap recurs throughout Part II.

Finally, the construction above subsumes the potential outcome notation of the Neyman and Rubin framework [70, 71], which takes the symbols Y_x as primitives and reasons about them under assumptions of ignorability. In the structural account they are instead derived objects, obtained from the equations by Definition 2.12. The two frameworks are logically equivalent in what they assume, and differ in what they take as given: one posits the counterfactuals and constrains them, the other posits the mechanisms and computes them.

2.5 Human-in-the-Loop Data Management

A recurring assumption in the work surveyed so far is that a data management task can be fully specified before it is executed. For instance, a query has a fixed semantics, a constraint set has a fixed satisfaction condition, and an algorithm optimises a fixed objective. For a large class of tasks this assumption does not hold, and it fails in a characteristic way: the specification is underdetermined. Many mappings reconcile the same pair of schemas, many partitions are consistent with the same similarity scores, many repairs satisfy the same constraints, and many queries fit the same informal intent. Resolving the residual ambiguity requires information that is not present in the data and not expressible in the formalism, because it resides with the people who produced the data or who will use it.

Human-in-the-loop data management refers to systems in which human judgements, actions, or feedback are incorporated into an otherwise computational data management process. It covers a crowd worker comparing two records, a domain scientist writing a heuristic that labels thousands of records, an analyst rejecting a proposed schema mapping, and a data steward deciding that a detected anomaly is in fact correct. What unites these cases is a shift in the object of design: the system is no longer only an algorithm that computes an answer, but a protocol that decides what to ask, whom to ask, when to ask, and what to do with the reply.

The motivation for this shift is usually argued from one of two directions. The first is empirical, namely that unaided automation is insufficient for the task at hand. The second is conceptual, namely that automation and human agency are complementary design resources and that the interesting systems combine them rather than maximising one at the expense of the other [72, 73]. The second argument is the stronger one, because it does not depend on the accuracy of current methods: even a perfect predictor cannot supply a preference that was never recorded. Surveys of the resulting design space exist both in the machine learning literature [74] and, under the heading of human computation, in the human-computer interaction literature [75].

2.5.1 A Taxonomy of Human Involvement

The literature is fragmented across communities that use different vocabularies for similar arrangements. Three orthogonal questions are enough to organise most of it: why the human is consulted, at what point in the process, and in what capacity.

By purpose. The reason for the interaction determines what can be asked and what the answer is worth.

- *Providing labels or annotations.* The human supplies ground truth that the system lacks, as in supervised learning and in annotation pipelines [76].
- *Correcting system outputs.* The system produces a candidate and the human amends it, which presumes the output is close enough to be worth editing and not discarding [77].
- *Resolving ambiguity.* Several outputs are equally admissible under the formalism and the human selects among them. Here the human is not correcting an error but supplying a preference that the specification never contained.
- *Validating results.* The human confirms or rejects an output without proposing an alternative, which is the cheapest form of interaction and the one most amenable to statistical treatment [78].
- *Providing domain knowledge.* The human contributes reusable knowledge: a constraint, a rule, or a labelling heuristic that the system then applies at scale [79].
- *Configuring system behaviour.* The human sets thresholds, weights, or objectives, thereby steering the automated component without touching individual decisions.

The distinction between the first two purposes and the fifth is consequential. Labels and corrections are consumed once and scale linearly with the size of the problem; knowledge is consumed repeatedly and therefore amortises. Systems that leverage knowledge obtain a fundamentally better return on the same human effort, which is the central design argument of weak supervision [79].

By timing. When the human enters determines how much of the computation can be conditioned on the answer.

- **Human in the loop.** The human is consulted during processing and the computation cannot proceed without the reply, so latency of the reply is latency of the system [80].
- **Human on the loop.** The system runs autonomously while a human supervises, intervening by exception. The control is retained without the human being on the critical path.
- **Human after the loop.** The system completes its work and the human validates the result post hoc, which makes the human a quality gate and means errors are detected only after they have propagated.
- **Interactive and iterative workflows.** The interaction is a sequence of rounds in which each answer reshapes the next question, so neither the questions nor their number are known in advance [81, 82, 83].

By system role. The same person can occupy structurally different positions with respect to the system, and each position implies a different treatment of their answers.

- **Human as oracle.** The human answers a well-formed question correctly by assumption, which is the model used in active learning and in the theoretical analysis of query complexity [84].
- **Human as validator.** The human accepts or rejects proposals and is assumed reliable but fallible, so answers are aggregated and not trusted individually [85].
- **Human as decision maker.** Authority rests with the human and the system is advisory, which relocates the design problem to how evidence is presented [86].
- **Human as domain expert.** The human contributes with semantics that is unavailable to the system, it is scarce and costly, and cannot be replaced by redundancy [87].
- **Human as data curator.** The human holds continuing responsibility for a data asset, so the interaction is longitudinal and the system must accumulate and reuse what has been established [88].

These roles are frequently conflated in the literature, and the conflation matters, because an argument valid for one role is often invalid for another. A result about the number of oracle queries needed to identify a target does not transfer to a setting in which the human is a fallible validator, and a redundancy-based quality mechanism is meaningless when the human is the sole authority on the answer.

2.5.2 Human Effort, Cost, and Cognitive Limitations

Much early work treats the human as an unlimited and reliable source of ground truth. This idealisation is convenient for analysis. However, it is misleading in practice, and the discrepancy between the two is what the literature reviewed below addresses.

Human input is costly along several distinct axes, and they do not reduce to one another. There is direct cost, whether monetary as in paid crowd work or opportunity cost as in expert time. There is latency, since a reply arrives orders of magnitude later than a computation. There is an expertise requirement: many data management questions cannot be answered without knowledge of how the data was produced, which restricts the population of admissible respondents, sometimes to a single person. And expert availability is not merely limited but unpredictable, so a system cannot assume that the question it wants to ask will be answered at the moment it wants to ask it.

Beyond cost, human answers have properties that a formal model of an oracle does not. Cognitive load imposes a bound on how much context can be considered in a single judgement, and exceeding it degrades accuracy rather than merely slowing it down [89]. Sustained annotation produces fatigue effects, and measurements in crowd settings show accuracy that varies with worker, task position, and task design [90]. Judgements are inconsistent, both between annotators and within a single annotator over time. Hence, a statistical treatment of annotator disagreement is a prerequisite for using annotation at scale [91]. Some questions are irreducibly subjective, so disagreement reflects a genuine plurality of correct answers, and averaging them destroys information. Moreover, humans simply make mistakes, at rates that are frequently comparable to the automated component they are supposed to correct [86].

Taken together these observations support a conclusion that reframes the design problem. The objective of a human-in-the-loop system is not to involve humans, but to involve them efficiently and meaningfully: to ask few questions, to ask the ones whose answers change the outcome, to ask them of people positioned to answer, and to treat the answers as evidence rather than as truth.

2.5.3 Crowdsourcing and the Economics of Human Effort

Crowdsourcing is the branch of the field in which the cost of human effort is handled most rigorously, for the straightforward reason that it is priced. Its characteristic move is declarative: human computation is exposed as an operator that a system can plan around. CrowdDB extends SQL so that missing values and even missing tuples can be obtained from workers during query execution [80], and comparable systems develop the operator algebra with different vocabularies [92, 93]. One consequence is that the closed-world assumption no longer holds: a crowd-populated relation has no predetermined cardinality, so enumerating or counting its members becomes a problem of statistical estimation [94]. The area is surveyed by Li et al. [95] and, from a practice-oriented perspective, by Marcus and Parameswaran [96]; Doan et al. give the earlier and broader account [97].

Because cost, latency, and quality are all measurable here, operator design becomes an explicit optimisation problem, and the resulting algorithmic literature is the most developed treatment of the dimensions listed above. Batching reduces the number of paid tasks needed to cover a candidate set [98]. Filtering admits algorithms that bound the number of questions subject to a target error rate [78], and the same has been done for sorting and joining [99]. Where the task has exploitable semantics one answer constrains others, so question selection becomes the problem of choosing the questions whose answers imply the most about the rest [100, 101]. Applied to data quality, the crowd has served as the adjudicator of last resort for cases an automated source cannot settle [102], as the disambiguation signal in probabilistic entity linking [103], and, more ambitiously, as a source of knowledge absent from the database altogether, where the queries concern the workers' own practices [104].

Quality is obtained through redundancy combined with inference: a task is issued to several workers and the latent truth, together with per-worker reliability, is estimated from the disagreement pattern [85, 105]. This mechanism is what distinguishes crowd settings from expert settings, and it rests on an assumption that is easy to overlook: that workers are noisy but interchangeable. When the human in the loop is a domain expert or the owner of the data, that assumption fails on three counts. The task may not be answerable without context the worker does not have; the data may not be shareable with anonymous workers; and the respondent may not be replicable, so an answer must be treated as authoritative rather than as one vote among many.

2.5.4 Human Involvement Across Data Management Tasks

Table 2.4 summarises how the human involvement recurs across the data management pipeline: an automated component produces candidates that are individually plausible and collectively

Table 2.4: Human involvement across data management tasks. The automated component and the human role differ, but the structure of the interaction and the nature of the difficulty recur.

Task	Automated component	Human role	Characteristic challenge
Data exploration	Suggest queries and views	Steer the session	Intent is revealed only through interaction
Query formulation	Synthesise a query from examples	Provide and label examples	Users recognise answers more easily than they write queries
Schema matching	Propose correspondences	Validate mappings	Correctness depends on domain knowledge
Data integration	Identify overlapping sources	Resolve conflicts	Heterogeneity and disagreement between sources
Entity resolution	Generate candidate matches	Confirm ambiguous pairs	Quadratic candidate space under a fixed budget
Data annotation	Predict labels	Correct labels and supply heuristics	Label noise and annotator disagreement
Data cleaning	Detect violations	Validate or choose among repairs	Many repairs satisfy the same constraints
Data quality	Discover rules and issues	Prioritise what to fix	More candidates than can be inspected

too numerous or too ambiguous to accept unexamined, and a human is enlisted in a specific and limited capacity.

In annotation the human is an oracle or a validator, answers are cheap and redundant, and the dominant difficulty is noise. In schema matching and mapping specification the human is a domain expert, answers are expensive and authoritative, and the difficulty is that a validated mapping must generalise from the handful of examples the expert was willing to inspect [106, 87, 107]. Entity resolution sits between the two. Its questions are simple enough for non-experts, and it became the canonical crowd application [98, 100, 108], ; however, its candidate space is quadratic, so selective querying is indispensable. Data exploration and query formulation invert the direction of the loop, since the human supplies the goal and the system supplies the candidates [83, 109, 110], an arrangement motivated by the observation that formulating a correct query is itself a documented source of user error [18]. Data cleaning and repair combine the hardest features of the others: violations are detected automatically, while their resolution is both ambiguous and knowledge-dependent. Interactive designs therefore appeared early in this area and persist [81, 77, 111, 102].

Two observations follow. First, human-in-the-loop denotes a class of system architectures: results obtained for one task transfer to another only insofar as the roles, costs and propagation mechanisms match. Second, the tasks in which interaction is least dispensable are those in which the ambiguity is semantic and not statistical, since more data or a better model does not reduce it. That observation motivates the setting studied in the remainder of this thesis, where the

candidate space is large, the ambiguity is semantic, and the human whose judgement resolves it is a scarce expert.

Part I

Cleaning Property Graphs

Chapter 3

User-Centric Interactive Repair

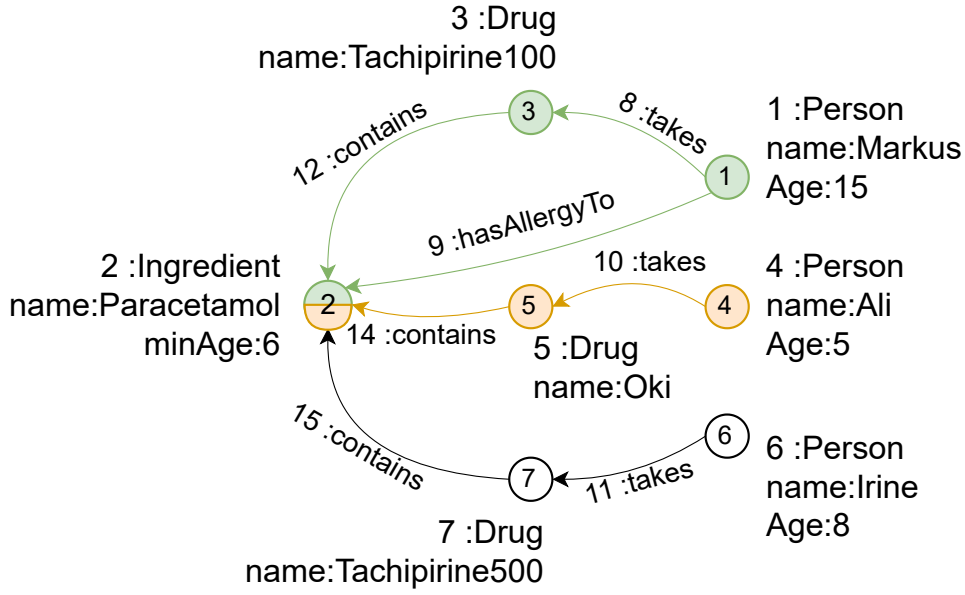
This chapter presents the user-centric interactive repair of property graphs that forms the foundation of Part I. It introduces the model, inconsistency detection by query, the graph repair dependency graph, and multi-user assignment via independent sets, together with its safety and convergence guarantees and a full evaluation that includes a study with real, non-oracle users. This Chapter is the result of two publications: A full research paper titled *User-Centric Property Graph Repairs* [13] and a demonstration titled *Grafixer: Enabling User-Centric Repairs for Property Graphs* [14] both presented at SIGMOD 2025.

3.1 Context

The quality and accuracy of graph-structured data are often compromised by errors and inconsistencies, which can appear under the form of incorrect property values and erroneous node/edge labels. The prevalence of errors in graph data is well-documented [6, 112, 113]. In a recent survey, users and practitioners of graph processing systems confirmed that data cleaning is one of the most time-consuming activity when working with graph applications [5].

Several existing methods to detect glitches and errors in graphs and to find the corresponding repairs adopt *constraints* from relational databases [114, 115] and redefine them for graphs (e.g., formally defining Graph Functional Dependencies [3], Graph Entity Dependencies [4], and Graph Denial Constraints [3, 44, 45, 4]). However, these works make the assumption that the knowledge necessary to fix the errors is readily available (e.g., under the form of graph repair rules) and typically focus on the efficiency of the repair algorithm [116]. None of these works have considered the problem of interactive repairs on property graphs where the knowledge to repair graph data lies with the users.

The challenge of repairing a property graph is further exacerbated by the inherent complexity of the data model, raising the following challenges: (i) an error on property graph data when treated with a repair might trigger new errors (e.g., in the presence of multiple labels); (ii) the same errors can be fixed in several different ways, considering that a property graph includes labels and properties on both nodes and edges (e.g., a repair on an edge property X is different from a repair on a node property with the same name). To address the above challenge, in this chapter we study a novel interactive approach to property graph repair, where repairs are verified by multiple users and lead to fix the different denial constraint violations.



$$Q_1(x_1, y_1, z_1, z_2) \leftarrow :hasAllergyTo(x_1, z_2) \wedge :takes(x_1, y_1) \\ \wedge :hasIngredients(y_1, z_1)$$

$$\phi_1 = (Q_1(x_1, y_1, z_1, z_2), \\ id(z_1) = id(z_2) \rightarrow \perp)$$

$$Q_2(x_1, y_1, z_1) \leftarrow :takes(x_1, y_1) \\ \wedge :contains(y_1, z_1)$$

$$\phi_2 = (Q_2(x_1, y_1, z_1), \\ x_1.age < z_1.minAge \rightarrow \perp)$$

Figure 3.1: A property graph from a healthcare scenario. In green and yellow are the two violations.

Example. The property graph depicted in Figure 3.1 exemplifies the problem of repairing a property graph about patients and their treatments, represented with property and edge labels (such as *:Person* and *:hasAllergyTo*) and properties as key-value pairs (*name=Aspirin*).

The property graph reflects a possible real-world state of a database containing patients, *Markus*, who is allergic to *Paracetamol*, and *Irine* who takes *Tachipirine500*. Plausible property graph denial constraints ϕ_1 and ϕ_2 in this example state that 1) patients cannot be treated with a medicine they are allergic to, and 2) a patient cannot be treated with a medicine if they are younger than the minimum age specified in the ingredient of the drug.

The subgraphs highlighted in green and orange showcase respectively two violations of the above constraints, namely (1) *Markus* is taking *Tachipirine100* that contains *Paracetamol* while

being allergic to it, and (2) *Ali* taking *Oki* that contains *Paracetamol* while being too young. These two violations can be fixed applying different graph transformations: (a) one can modify the node with name *Tachipirine100* with the one of another *Drug* (and consequentially change the ingredient) - corresponding to a property update - (b) the edge between patient and medication can be deleted, (c) the edge between the drug and the ingredient can be removed (for instance, *Oki* does not contain *Paracetamol* but contains *Ibuprofene*), (d) or the edge *hasAllergyTo* can be suppressed because the patient was not allergic to *Paracetamol*.

These repairs imply different degrees and levels of domain knowledge. Repairs (a) and (b) imply to have domain knowledge of what happened with the patients, while (c) is a factual information (i.e., the key ingredient of a drug), and (d) requires either knowing the patient healthcare record or to perform further tests (e.g., an allergy test). Approaches based on fixed graph repair rules as input [116] would require to know each repair in advance, while this is not feasible in practice when the repairs depend on the context (on the particular patient, medication, allergy as data values of the properties).

Preferred repairs approaches [117] can be tuned to prioritize a repair over another. However, this preference model would be applied to every violation globally for the entire graph and not on a case-by-case basis. For instance, applying a repair policy that prefers deleting the edges *takes* to the example in Figure 3.1 would solve both the green and the orange violations. However, the orange violation should be repaired by deleting the edge *contains* to reflect the fact that *Oki* does not contain *Paracetamol*. A schema-level approach could help in limiting the possible repair choices (e.g., it could exclude the deletion of the *:hasIngredient* edge, because it would enforce that a *Drug* needs an *Ingredient*). However, many real-world property graphs do not possess a schema [5] and standardization around graph schemas is still in progress [2].

On the other hand, users would be able to evaluate each case independently because they are either domain experts (e.g., they know Alice was given a drug containing ibuprofen), possess previous knowledge (e.g., they know *Oki* does not contain paracetamol) or reason on the data to provide insight for the repair.

Previous works have already advocated the need for humans to verify repairs [118], and studied how to involve users in the repair of knowledge bases [119, 120] and graphs [121], they only partially addressed the challenges related to repair property graphs mentioned before. They oversimplify the characteristics of the users (e.g., considering them only as oracle), and they utilize a very simple graph model, constraints and user interaction [121] that are not suitable for the property graph case.

Thus, our first research question is to understand how users with domain knowledge perform on repairing expressive property graphs in the presence of complex denial constraints with respect to automated approaches. Second, we investigate how is it possible to involve multiple users in the repair process, which raises the further challenge of managing conflicts due to simultaneous repairs on interlinked graph data. A repair indeed may solve multiple violations at once or introduce several new ones. Therefore, the challenge lies in assigning users to violations in such a way that we can apply repairs in bulk without introducing new conflicts and avoiding to re-traverse the entire graph for newly introduced violations.

In summary, our main contributions are the following.

- We propose a novel interactive repairing approach for property graphs, in the presence of **multiple** users possessing diverse knowledge (Section 3.4).
- We develop a theoretical framework (Section 3.5) including a query-based graph inconsistency detection mechanism, a dependency graph and an assignment algorithm to allow multiple users to repair the graph by leveraging independent sets of neighboring nodes, along with a set of algorithms to efficiently manage repair propagation in the graph.
- We study the complexity of the above algorithms and derive two properties of our framework, namely safety and convergence along with their relationships.
- We evaluate our approach on both accuracy and efficiency with several experiments on real-world datasets with both injected and real-world inconsistencies (Section 3.6). We consider different user expertise (e.g, from oracles to random users and their variants thereof) against both interactive and non interactive baselines. We also performed a user study to assess how real users perform graph repairs. The results show that our approach outperforms the baselines for different sets of constraints by 30% on average repair quality even without oracle users. Moreover, our algorithms gracefully scale with respect to dataset size and number of users.

3.2 Related Work

Graph repairs. Graph repairs include various methods for cleaning data that violate integrity constraints. Multiple constraints have been proposed with different expressivity, including Graph Functional Dependencies [3], Graph Entity Dependencies and Graph Denial Constraints [4]. For what concerns the repairs, the deletion model [122] has been proposed to remove erroneous data, while the update model [123] applies transformations to rectify data and ensure compliance with constraints. In this work, we employ a hybrid approach, combining both deletion and update models. Graph database repair spans across different dependency classes [45, 20, 4]. In a prior work [44], repairs involve both deletion and update operations for neighborhood constraints, addressing conflicts through relabeling (update) and edge deletion. However, this work does not study user-centric repairing techniques and is limited to a special class of constraints. Other works used rules [116] to repair graphs, but they assume that the rules are known in advance, or are derived from a graph that is already error-free.

Collaborative Data Management. The collaborative aspects of data management have earned considerable attention in the literature, particularly in the context of databases and collaborative editing. In the database scenario, collaboration is addressed by DBMSs exploiting concurrency control protocols [124] and transaction scheduling. Approaches like the Graph-Based protocol [125] allow a DBMS to order the incoming operations using directed dependency graphs, encoding the dependencies between physical resources requested by the transactions. The dependency graph we use in our approach is necessarily undirected since it has to tackle asynchronous concurrent repairing operations from multiple users.

Conflict-Free Replicated Data Types (CRDTs) [126] are noteworthy to mention for easing collaborative work. These data structures enable concurrent updates across multiple replicas

without requiring synchronization, ensuring eventual convergence to a consistent state. CRDTs could be Operation-based (data replicas share only the operation between each other) or State-based (the entire state of the data is shared). For both, only commutative operations are allowed, which makes them unsuitable for collaborative graph repairs. Finally, human-in-the-loop approaches have been applied to data cleaning [111, 77, 127] to ensure better quality fixes with respect to automated repairs, or to discover constraints from dirty datasets [128, 129]. Interactive graph repairing has also found application in prior work [121], particularly in the context of repairing labeled graphs under neighborhood constraints. In this chapter, our focus is on more expressive graph data models and constraints (i.e. property graphs and denial constraints). Moreover, for the first time, we are studying a user interaction model targeting multiple users.

3.3 Constraints and Violations

In this Section, we introduce the preliminary notions needed as basic ingredients in our framework. For this Section, we adopt the property graph model defined in Section 2.1. Figure 3.1 shows an example of a property graph. For instance, the node with identifier 0 has the label *:Person* and the value *Alice* for the property *name*.

In this Section, we focus on Graph Denial Constraints (GDCs) defined in Section 2.2.3, which allow the definition of the highest expressive dependencies and constraints on property graphs. For example, given the graph pattern Q_1 over the property graph in Figure 3.1, the following GDC ϕ_1 states that a person cannot take a drug with an ingredient to which they are allergic.

$$\begin{aligned} Q_1(x_1, y_1, z_1, z_2) &\leftarrow :hasAllergyTo(x_1, z_2) \wedge :takes(x_1, y_1) \\ &\quad \wedge :hasIngredients(y_1, z_1) \\ \phi_1 &= (Q_1(x_1, y_1, z_1, z_2), id(z_1) = id(z_2) \rightarrow \perp) \end{aligned} \tag{3.1}$$

Given the definition of *matching* as in [3], that is to find all subgraphs $h(Q)$ of a graph G that are homomorphic to the graph pattern Q , we define a constraint violation as follow:

Definition 3.1 (Violation). *For a graph G and a denial constraint ϕ , a subgraph I satisfies ϕ iff there does not exist a match $h(I)$ of Q in G . Otherwise, I is a violation of the constraint ϕ .*

In our approach, we need to detect all the patterns that cause violations. To reach this goal, we need to map the constraints into queries in order to retrieve all the violations as subgraphs. Conjunctive Graph Queries (CGQs) allow to identify and query substructures in a graph (see Section 2.1). Informally, CGQs select all the subgraphs that match a given graph pattern. We introduce Property Graph Logic (PGLog) rules, a simplification of RPGLog rules introduced in [20] that correspond to unions of conjunctive queries on a property graph, augmented with edge variables. PGLog rules are introduced in order to allow the specification of simple graph patterns (see Section 3.6) without recursion and nested subqueries.

Definition 3.2 (PGLog rule). *Given a set of vertex variables $\mathcal{V}$, a set of edge variables $\mathcal{E}$, and $\mathcal{K}$ be the set of property keys. A PGLog rule has the form*

$$\text{head} \leftarrow \text{body}_1, \dots, \text{body}_n, \text{constraint}_1, \dots, \text{constraint}_m$$

for some $n > 0$ and $m \geq 0$ where body_i is a graph pattern of the form $p(x, y)$ as e , with $e \in \mathcal{E}$ being an edge variable, $p \in \mathcal{L}$ is an edge label and $x, y \in \mathcal{V}$ are vertex variables. Alternatively, body_i is a graph pattern of the form $p(x)$, where $p \in \mathcal{L}$ is a node label and $x \in \mathcal{V}$ is a vertex variable. Moreover, constraint_i is a θ -literal over the variables of body_i and head is of the form $p(x, y)$ or $\text{results}(x_1, \dots, x_n)$.

A PGLog query is a finite non-empty non-recursive set of rules such that at least one rule has a head predicate result and all result predicates have the same arity.

A GDC mapped into a PGLog query is called a *Constraint query*, defined as follows:

Definition 3.3 (Constraint query). *Given a GDC $\phi = (Q[\bar{x}], X \rightarrow Y)$, the constraint query CQ_ϕ is a PGLog query whose body predicates match the graph pattern $Q[\bar{x}]$ of ϕ and its constraints are the literals X, Y of ϕ .*

The Graph Denial Constraint 3.1 will be mapped in the following PGLog query :

$$\begin{aligned} \text{result}(x_1, x_2, x_3) \leftarrow & \text{hasAllergyTo}(x_1, x_2), : \text{takes}(x_1, x_3), \\ & : \text{hasIngredients}(x_3, x_2), \text{id}(x_2) = \text{id}(x_3) \quad (3.2) \end{aligned}$$

If we apply the above query to the instance graph in Figure 3.1, it leads to the violation highlighted in green.

3.4 The graph repair problem

Given a property graph G that violates a set of graph denial constraints, as surfaced by the constraint queries of 3.2, the repair problem is :

Definition 3.4 (Graph Repair Problem). *Given a property graph G , a list of denial constraints Σ producing a set of violations I , the graph repair problem consists of returning a graph G^* with no violations to Σ .*

Two facts make the problem hard. First, the number of repairs for a violation is not bounded a priori and may be infinite. For instance, a constraint requiring $x.A = y.B$ is satisfied by any value pair that makes them equal. Second, a repair that removes one violation may *introduce* others, so repairs interact.

Moreover, the right choice among repairs often depends on knowledge absent from the data. This issue makes automatic repair unreliable on violations whose resolution depends on such knowledge [121]. *Ambiguity* is a second challenge: a single violation typically admits many repairs, and choosing among them requires the judgement of a domain expert. Rule-based repair assumes the fixes are known in advance as rules [130], which is infeasible when the correct fix depends on the specific patient, drug, and allergy. Preference-based repair applies one global

policy rather than deciding case by case. Schema-level restrictions could prune repairs, but many property graphs have no schema and graph-schema standardisation is only now maturing (2.1).

Humans, by contrast, can evaluate each case on its merits, as domain experts, from prior knowledge, or by reasoning over the data, a need that is long recognised in data cleaning [77, 118].

For this reason, we reframe the problem in an interactive fashion:

Definition 3.5 (Interactive Graph Repair Problem). *Given a property graph G , a list of denial constraints Σ producing a set of violations I and a set of users U , the interactive graph repair problem consists of asking the users for a set of repairs R that return a graph G^* with no violations to Σ .*

3.5 User-centric property graph repairs

In this section, we present our User-Centric graph repair approach that allows concurrent interactive repairs on property graphs to solve the problem of Definition 3.5. Figure 3.2 shows a high-level view of our pipeline. It consists of four phases: Inconsistency Detection, Dependency Analysis, User Assignment, and User-Centric Question-Answer-Repair.

In the Inconsistency Detection phase, we use concepts described in Section 3.3 to detect inconsistencies (Phase 1). Given a set of constraints, we create the corresponding constraint queries and use them to retrieve the violations in the graph.

In the second phase, we build the *Graph Repair Dependency graph* (GRDG) to analyze the dependencies between violations.

In the third phase, we assign the violations to users while the last phase consists of the uQAR (User-Centric Question-Answer-Repair) framework, an interactive mechanism allowing multiple users to repair the inconsistencies collaboratively.

The repair process iterates over the last three phases until the graph is repaired. This cycle is necessary because we may not be able to repair all the graph in one iteration (i.e., we cannot repair some violations because they might conflict with other violations).

Before introducing the concepts of graph repair formally, we establish the graph transformations allowed in our approach.

Definition 3.6 (Graph transformations). *Let $G = (V, E, \eta, \lambda, \nu)$ be a property graph. A Graph Transformation is a function $\mathcal{T} : G \rightarrow G'$ where $G' = (V, E', \eta', \lambda', \nu')$ is an LPG with $E' \subseteq E$ and $\eta' : E' \rightarrow V \times V$ is a function assigning an ordered pair of vertices to each edge in E' , $\lambda' : V \cup E' \rightarrow \mathcal{P}(\mathcal{L}')$ be a labeling function assigning to a vertex in V or an edge in E' a finite set of labels and $\nu' : (V \cup E') \times K \rightarrow N$ is a partial function assigning values for properties to vertices and edges such that $V \cap E' = \emptyset$ and the set of domain values where ν is define is finite.*

According to the definition above, there are three types of transformation: (i) edge-deletion iff the edge set $E' \subset E$, (ii) relabeling iff $\lambda \neq \lambda'$ and (iii) property-update iff $\nu \neq \nu'$.

We can now define a repair as follows:

Definition 3.7 (Repair). *Given a graph instance G and a violation I , a repair r is a sequence of transformations $(\mathcal{T}_0, \dots, \mathcal{T}_m)$ with $m \geq 0$ returning a graph $G' = (V, E', \eta', \lambda', \nu')$ with no match $h(I)$ of I in G' .*

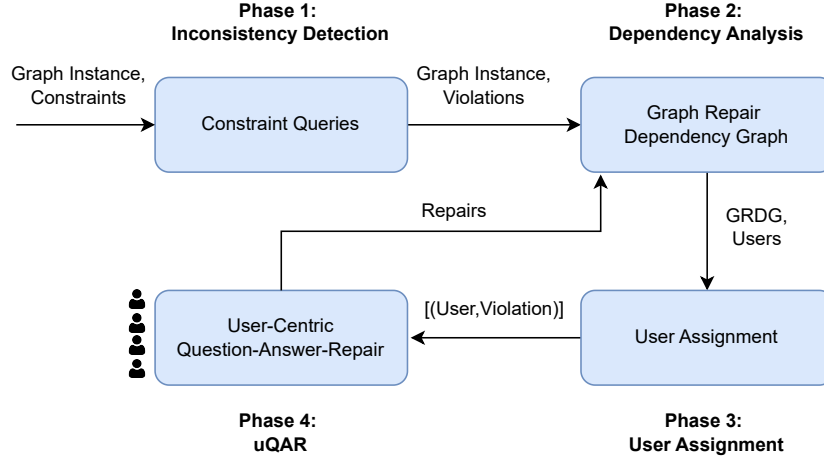


Figure 3.2: High level view of our pipeline, composed by the inconsistency detection phase (1), the dependencies analysis (2), the assignment of the violations to the users (3), and the collaborative repair phase (4).

Repairs based on edge deletions may cause a loss of information because they might affect node reachability in the graph. However, differently from tabular data cleaning approaches [131, 132], some inconsistencies in the graphs can only be solved by edge deletions. For instance, the green violation in Figure 3.1 violates a constraint that forbids its own graph pattern. In this case, deleting one of the edges, e.g. edges 8, 9 or 12, are the only possible repairs for the inconsistency. Hence, updating any of the property values would not have repaired the violation, e.g. changing the name of node 3 representing the Drug.

Notice that a graph repair might solve a violation while introducing new ones. Moreover, it is not possible to determine a priori the number of possible repairs for a violation V . For example, if a violation consists of two properties that need to be equal but are not (e.g., the constraint state that $x.A == y.B$, but $x.A > y.B$), there exists an infinite number of repairs that solve the inconsistency (e.g., any possible pair of properties A and B such that their values are equals).

Since the graph repairs are provided by humans, we need to define questions and answers.

Definition 3.8 (Question, Answer). *Given a violation I and a user u , a question q is a tuple $q = (I, u)$ assigning the violation I to user u and an answer a is a tuple $a = (q, r)$ where r is a repair for the violation I as defined in Definition 3.7*

For instance, given the example illustrated in Figure 3.3, the question given to user u_1 who is repairing the violation highlighted in green (h_{17}) and its answer that deletes edge 6 are: $q_1 = (h_{17}, u_1)$, $a_1 = (q_1, [\mathcal{T}_0])$ where $\mathcal{T}_0$ is an edge deletion on the edge 6 :*hasAllergyTo*, i.e. a graph transformation returning a graph G' such that the edge set $E' = E \setminus \{e_6\}$ where E is the edge set of the input graph G .

The approach is agnostic on how the users are asked to provide the graph repairs. They can either select graph transformations from a given list, they can be asked if a specific sequence of transformations is a repair, or they can manually provide each transformation.

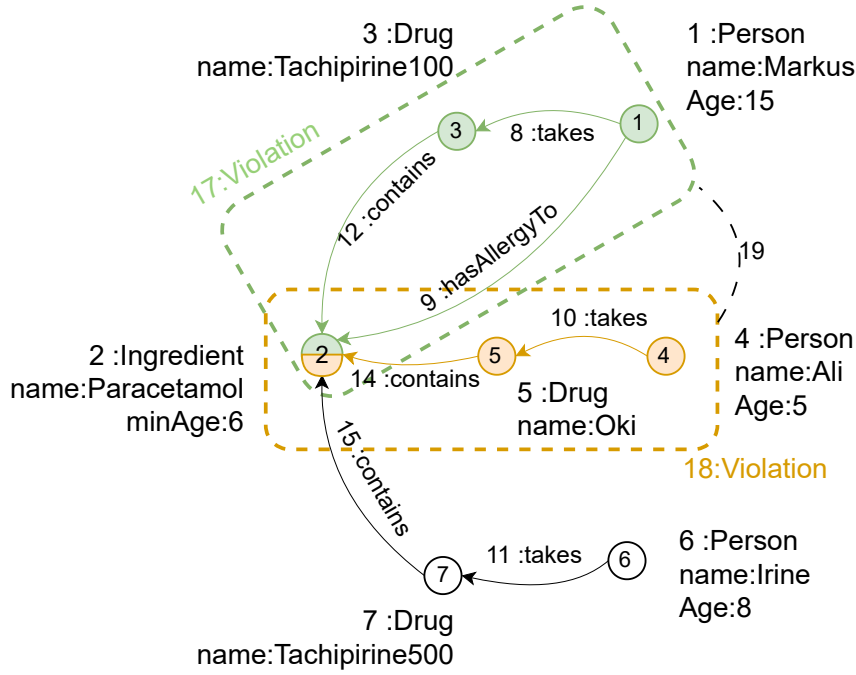


Figure 3.3: Example of two violations detected in the graph instance with the related Graph Repair Dependency Graph (in dashed lines) overlaid on the Instance Graph.

3.5.1 Graph Repair Dependency Graph

In the process of repairing inconsistencies in a graph, it is possible for a single repair to have effects on multiple violations. A transformation that solves a violation indeed may introduce new inconsistencies or, conversely, resolve multiple ones simultaneously. To manage this complexity throughout the entire repair process, we leverage the concept of a dependency graph.

After having detected the inconsistencies by mapping the constraints into queries as specified in Definition 3.3 and retrieving the violations as a list of subgraphs, we build a dependency graph to assign the violations to the user for repairs, while at the same time avoiding conflicts (i.e., repairs on adjacent parts of the graph). Since the detected violations may correspond to overlapping subgraphs, the dependency graph is modeled as a hypergraph having as nodes the violations, and edges representing the intersection between subgraphs in the graph instance. Given two nodes representing violations I_i and I_j , an edge between them means that a repair on I_i has also effect on I_j . For instance, the green and orange violations in Figure 3.3 overlap on node 2. Hence, any updates on that node may affect both violations at the same time.

To define the dependency graph, we propose a new graph model by extending LPG with hypervertices and undirected edges.

Definition 3.9 (Hypervertex Labelled Property Graph). *A Hypervertex Labelled Property Graph extends a PGM graph as defined in Definition 2.1 to a structure $(V, H, E, F, \eta, \delta, \gamma, \lambda, \nu)$, where H is a finite set of hypervertices, F is a finite set of edges with $F \cap E = \emptyset$, $\gamma : V \rightarrow V \times E$ is a total function assigning every hypervertex a pair of vertex set and edge set such that the object*

sets V and E are disjoint and $\delta : F \rightarrow H \times H$ is a total function assigning every edge in F a pair of vertices in H .

Following the definition, we build the *Graph Repair Dependency Graph* (in short GRDG), a Hypervertex Labelled Property Graph that maps the results of the pattern matching performed in the inconsistency detection phase to a dependency graph with violations as hypervertices. The sets of the hypervertices H is the union of the results of the constraints queries applied to the instance graph, formally $H = \{results_{CQ_i} \mid CQ_i \in C\}$. The set F of undirected edges contains an edge for each overlap between the vertex sets of hypervertices pair, formally $(h_i, h_j) \in F$ iff $V_{h_i} \cap V_{h_j} \neq \emptyset$.

Due to the use of hypervertices, the GRDG is built as an overlay on top of the actual graph instance and each hypervertex contains the subgraph corresponding to its violation. Figure 3.3 shows an example of a graph repair dependency graph. Hypervertices 17 and 18 overlay the two subgraphs related to two different inconsistencies that share node 2, therefore the undirected edge 19 links the two hypervertices in the GRDG and represents the dependency between the two violations. Finally, we call *independent* violations, two violations that do not overlap (i.e. they do not share any node).

To build the GRDG we designed Algorithm 1. It takes as input the result of the Constraint Queries from the inconsistency detection phase. In lines 1-3, it adds a hypervertex for each detected violation. To compute the overlaps between the violations, instead of looping twice over the list of hypervertices, we leverage the graph query in line 4 that returns the nodes belonging to pair of violations by scanning the nodes of the instance graph and checking if there are more than one outgoing edge. If two violations have indeed overlapping nodes, the algorithm adds an edge between the two corresponding hypervertices.

Algorithm 1 Build GRDG

Require: Graph G , list of Violations I .

```

1: for  $i$  in  $I$  do
2:    $G.addHyperVertex(i)$ 
3: end for
4:  $overlaps \leftarrow G.query(: Violation(i_1), : Violation(i_2),$ 
    $(x, i_1), (x, i_2),$ 
    $id(i_1) \neq id(i_2) );$ 
5: for  $overlap$  in  $overlaps$  do
6:    $G.addEdgeIfNotExists(overlap.i_1, overlap.i_2)$ 
7: end for
```

3.5.2 Properties of the Graph Repairs

We now define properties that characterize the repairs and the overall framework. For what concerns the repairs, we first define the *safety* property, which determines whether a repair introduces new violations in the graph.

Definition 3.10 (Safety). *Given an instance graph G and a repair r , r is safe if its graph transformations do not introduce new violations in G , otherwise it is unsafe. A repair framework has the safety property if it allows only safe repairs.*

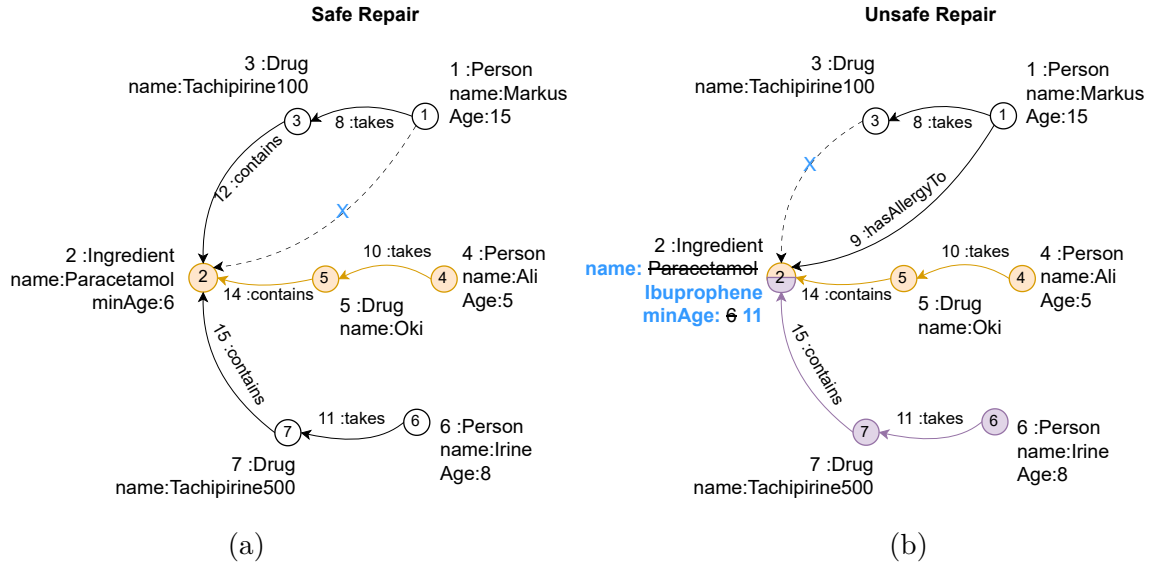


Figure 3.4: Example of a safe (a) and unsafe repair (b). The colors indicate the presence of a violation, and in blues are the edits on the graph.

Let's consider the scenario in Figure 3.4, showing both safe and unsafe repair for our original running example. An example of *safe* repairs consists in removing the edge `:hasAllergyTo`, as shown in Figure 3.4(a). This is a *safe* repair because it does not introduce **new** violations. However, another possible repair - shown in Figure 3.4(b) - is to modify the `:Ingredient` name to `Ibuprophene`, which also has a higher minimum age requirement (`minAge:11`). This repair is *unsafe* because it fixes the first violation while introducing a new one, since *Irine* is too young to take *Ibuprophene*.

Note that Definition 3.10 concerns the repairs and not the single graph transformations. A repair could still be safe even if one of its internal transformations creates violations.

Given the Definition 3.10 of *safety*, and the Definition 3.7 of repair, we can prove the following:

Theorem 3.1. *An edge deletion is always a safe repair.*

Proof. A violation consists of a subgraph that matches a PGLog query. The body predicates of a query express a pattern as a string P of vertex and edge variables (see Definition 3.2). Removing an edge produces a subgraph that matches a new pattern. This corresponds to removing an edge variable E from P (a character of the string). The resulting pattern is expressed with a string P'_1 such that $P = E + P'_1$, or $P = P'_1 + E$, or with two strings P'_1, P'_2 such that $P = P'_1 + E + P'_2$. Both strings P'_1 and P'_2 express patterns that differ from the pattern selected by the PGLog query for the detected violation. Moreover, if they had violated other constraints, they would already have been detected as violations. Thus, no violations are introduced by removing the edge. $\square$

When applying safe repairs to *independent* violations in a graph, the order in which the repairs are applied does not affect the final outcome. Since each repair resolves a specific violation without introducing new ones, the result remains the same regardless of the sequence of repairs. We now present the properties of the framework. We denote as $G_R = \{G'_1, G'_2, \dots, G'_n\}$ the set of graphs that do not contain violations and that are the result of a sequence of repairs.

Definition 3.11 (Convergence). *A user-centric Graph Repair framework has the convergence property iff exists a list of repairs $R = \{r_1, \dots, r_n\}$ applied on the graph G that results in a graph in G_R .*

Ideally, the framework should possess the convergence property. In the remainder, we prove that the safety property guarantees the convergence of the framework.

Theorem 3.2. *The safety property is a necessary and sufficient condition for the convergence property, that is $\text{Safety} \leftrightarrow \text{Convergence}$.*

Proof. Given a graph G with n violations and any list of *safe* repairs R , we know that by definition, the application of *safe* repairs guarantees that the number of violations is strictly monotonic decreasing among the iterations because they solve violations without introducing new ones. Iterating the over violations and progressively applying repairs in R we reduce the violations eventually reaching a graph in G_R . Thus, if a framework has the safety property, it converges. On the contrary, if a framework does not terminate, it surely does not have the safety property because by iterating over the violations and progressively applying repairs in R we reduce the violations eventually reaching a graph in G_R . $\square$

Notice that the convergence property differs from the termination property of the chase algorithm [4]. The chase algorithm can terminate with an unsolved violation due to equality-generating dependencies, equating two different constants, thus in an invalid state, whereas our framework converges only when it obtains a graph without violations. Also, contrary to previous work on human-in-the-loop data cleaning [127], where the cleaning algorithm is forced to terminate with a halt instruction - defined by some empirically defined variables [128] - our framework terminates *by design* - i.e., its convergence is guaranteed by its properties. As a consequence, neither the *convergence* property nor its relation to *safety* depends on the knowledge of the users involved in the repair of the graph.

Lemma 3.3. *For a user-centric Graph Repair framework with the safety property, the number of iterations for repairing the graph is bounded by the number of violations $|I|$.*

The proof of Lemma 3.3 is straightforward and comes directly from the definition of safe repair. Each repair solves a violation, if it is safe it does not introduce new ones. With only one user solving one inconsistency at each iteration, it will take at most $|I|$ iterations to repair the entire graph.

Although ensuring convergence, the safety property restricts the list of potential repairs. For instance, only the repair in Figure 3.4(a) is allowed for convergence. This has an impact on the outcome of the repairs: a knowledgeable user for example, being a domain expert, might want to update node 2 - as in Figure 3.4(b) - knowing it would introduce new violations but with the intention to repair them afterward. However, the latter repair is not allowed with the *safety* property.

This highlights a trade-off between the convergence and the quality of the repaired graph. In Section 3.6, we investigate how different levels of domain knowledge impact the quality of the repairs both in the safe and unsafe case.

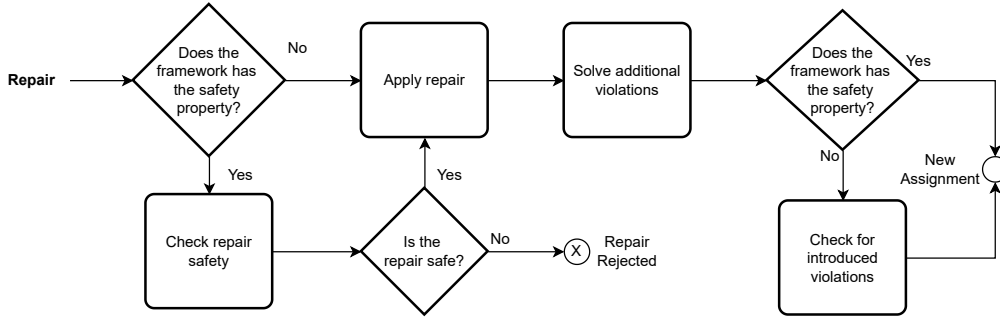


Figure 3.5: Maintenance workflow of the GRDG and propagation of effects of repairs.

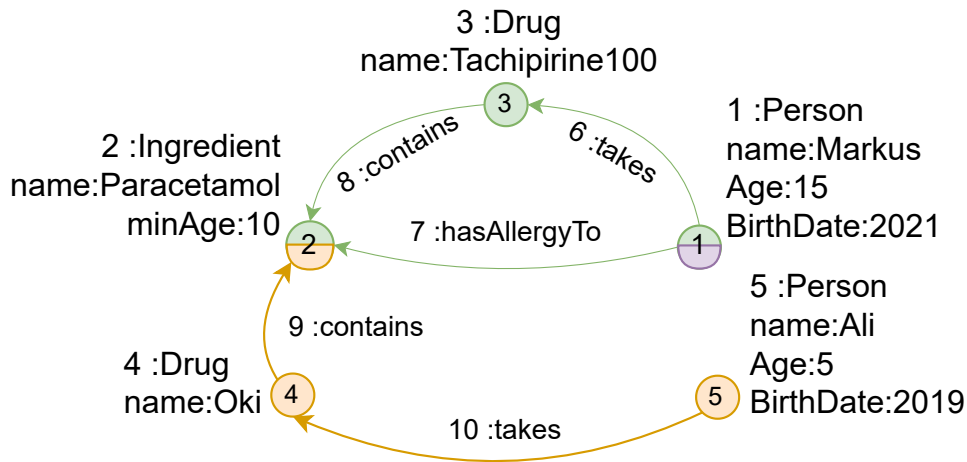


Figure 3.6: Example of a safe repairs conflict.

3.5.3 Property Graph Repair Process

In this section, we describe in detail the repair process, from the assignment of the violations to the users, to the application of the repairs on the graph and the propagation of its effects. The process starts with the users being assigned to independent violations and continues after they choose their repair. Figure 3.5 exemplifies the steps that follow. If the framework has the *safety* property the process checks if the repair chosen by the user introduces a new violation. If yes, the repair is rejected and the user needs to choose another one. If not, or if the framework doesn't have the safety property, the repair is applied. Then, before proceeding to a new repair iteration, the effect of the repair is propagated in the GDRG. First, it checks if additional violations are solved or, on the contrary - if the framework allows unsafe repair - if new violations are introduced. In the following, we elaborate on each step.

User Assignment To avoid conflicts due to the propagation of repairs, we need to assign independent hypervertices to different users. To do so, we assign violations to users, only if the sets of neighbors of the respective hypernodes on the GRDG are disjoint, otherwise the repair of one could conflict with the other when the framework has the safety property. For example, the property graph in Figure 3.6 contains three violations. We assign the orange violation (i.e. the

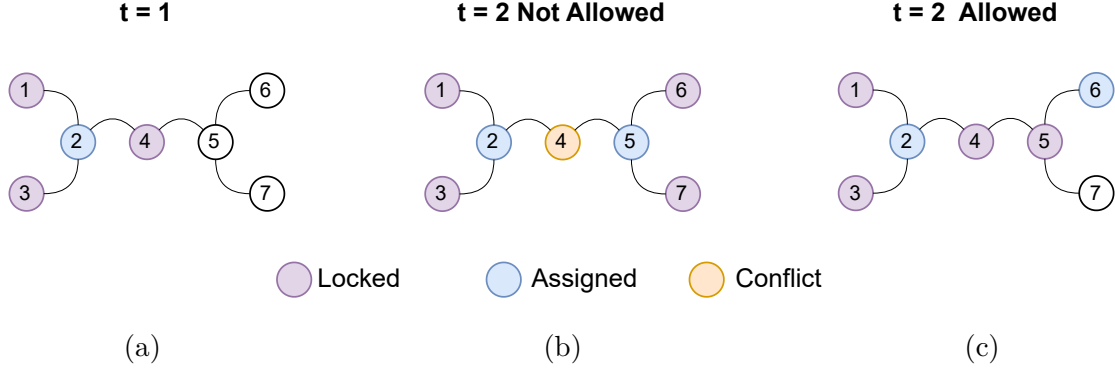


Figure 3.7: Example of two assignment iterations on the GRDG with seven violations.

minimum age for paracetamol is greater than Ali's age) and the purple violation (i.e. Markus cannot be 15 years old if born in 2021) to two different users. If a user chooses to repair the orange violation by decreasing the minAge to 5 (safe repair) while the other repairs the purple violation by decreasing the age of Markus to 4 (safe repair), one of the two repairs is no longer safe when the other is applied, introducing a new violation consisting of Markus taking paracetamol being too young. To assign several users at once we must select violations that can be repaired concurrently without interfering, and independence of the selected hypervertices is not sufficient for this. As Figure 3.6 shows, two *safe* repairs applied to non-adjacent violations that share a neighbour can each be safe in isolation and jointly introduce a new violation. What is required is that the selected hypervertices have pairwise disjoint closed neighbourhoods: writing $N(h)$ for the neighbours of h in the GRDG and $N[h] = \{h\} \cup N(h)$ for its closed neighbourhood, a set $S \subseteq H$ is *neighbour-independent* if $N[h_i] \cap N[h_j] = \emptyset$ for all distinct $h_i, h_j \in S$. This is exactly the condition Algorithm 2 enforces: selecting h locks h together with $N(h)$, and a later candidate h' is assigned only if neither h' nor any of its neighbours is locked, which is $N[h'] \cap N[h] = \emptyset$ for every already-selected h .

Figure 3.7 shows an example of user assignment for a GRDG with 7 violations. In the first iteration (Figure 3.7(a)), violation 2 (in light blue) is assigned and its neighbors (in purple) are locked. In the second iteration, the assignment on Figure 3.7(b) is not allowed because it would produce a conflict on node 4. Instead, Figure 3.7(c) shows a possible allowed assignment (node 6).

Equivalently, S is neighbour-independent if and only if $\text{dist}_D(h_i, h_j) \geq 3$ for all distinct $h_i, h_j \in S$. Indeed, if the distance is 1 then each vertex lies in the other's closed neighbourhood, and if it is 2 then the two share a neighbour; conversely, a vertex $w \in N[h_i] \cap N[h_j]$ satisfies $\text{dist}_D(h_i, w) \leq 1$ and $\text{dist}_D(w, h_j) \leq 1$, placing h_i and h_j at distance at most 2. Sets of this kind are known as 2-packings, or distance-3 independent sets [133], and are precisely the independent sets of the square D^2 of the dependency graph. The assignment step asks for a largest such set; its decision version, which we call NEIGHBOUR-INDEPENDENT SET (NIS), asks whether a GRDG D admits a neighbour-independent set of size at least k , that is, whether a single assignment iteration can serve k users at once.

Theorem 3.4. NEIGHBOUR-INDEPENDENT SET is NP-complete.

Proof. Membership. A set S is a certificate: checking $|S| \geq k$ and, for each of the $\binom{|S|}{2}$ pairs, that $N[h_i] \cap N[h_j] = \emptyset$, takes time polynomial in $|H| + |F|$.

Hardness. We reduce from INDEPENDENT SET, which is NP-complete [134, 135]. Let (G, k) be an instance with $G = (V, E)$ and $k \geq 1$. We may assume $E \neq \emptyset$ and that G has no isolated vertex: an isolated vertex belongs to some maximum independent set, so removing all of them and decreasing k accordingly is a polynomial-time equivalence-preserving preprocessing step, and an instance with $E = \emptyset$ is decided immediately. Build the graph $D = (H, F)$ with

$$H = \{c_v : v \in V\} \cup \{m_e : e \in E\}, \quad F = \{\{c_v, m_e\} : v \in e\} \cup \{\{m_e, m_f\} : e \neq f\},$$

that is, the incidence graph of G with the edge side turned into a clique. It has $|V| + |E|$ vertices and $2|E| + \binom{|E|}{2}$ edges and is built in polynomial time. Three distance properties follow directly from the construction. (a) $\text{dist}_D(m_e, m_f) = 1$ for all $e \neq f$, since the edge vertices form a clique. (b) $\text{dist}_D(c_v, m_e) \leq 2$ for every $v \in V$ and $e \in E$: if $v \in e$ they are adjacent; otherwise v is not isolated, so some $f \in E$ contains v with $f \neq e$, and $c_v m_f m_e$ is a path of length two. (c) For distinct $u, v \in V$, $\text{dist}_D(c_u, c_v) = 2$ if $\{u, v\} \in E$ and $\text{dist}_D(c_u, c_v) = 3$ otherwise. Centre vertices are pairwise non-adjacent, so the distance is at least two; if $e = \{u, v\} \in E$ then m_e is a common neighbour and it is exactly two. If $\{u, v\} \notin E$, a common neighbour would be some m_e with $u, v \in e$, i.e. $e = \{u, v\} \in E$, which is excluded, so the distance is at least three; it is exactly three because choosing $e \ni u$ and $f \ni v$ gives $e \neq f$ (otherwise $\{u, v\} \in E$) and the path $c_u m_e m_f c_v$. We claim D has a neighbour-independent set of size at least k if and only if G has an independent set of size at least k .

($\Leftarrow$) Let I be independent in G with $|I| \geq k$ and put $S = \{c_v : v \in I\}$. Distinct $u, v \in I$ are non-adjacent, so $\text{dist}_D(c_u, c_v) = 3$ by (c), and S is neighbour-independent by the characterisation above.

($\Rightarrow$) Let S be neighbour-independent with $|S| \geq k \geq 1$; its elements are pairwise at distance at least three. By (a), S contains at most one edge vertex, and by (b) an edge vertex is at distance at most two from *every* centre vertex; hence if S contains an edge vertex it contains nothing else and $|S| = 1$, in which case $k = 1$ and any single vertex of G is an independent set of that size. Otherwise $S \subseteq \{c_v : v \in V\}$, and $I = \{v : c_v \in S\}$ is independent in G by (c), with $|I| = |S| \geq k$. The reduction runs in polynomial time, which establishes NP-hardness. $\square$

Note that the hardness is not a consequence of the dense dependency graph used in the proof: under the characterisation above the problem is distance-3 independent set, which remains NP-complete on planar bipartite graphs of maximum degree three [133], that is, on exactly the sparse regime in which the GRDGs produced by our framework sit.

An exact solver is therefore out of reach at interactive latencies. Exact algorithms for maximum independent set with polynomial space exist [136, 137], but they target low-degree graphs, and the relevant degree here is that of D^2 rather than D , which squares the degree bound. Since the assignment step sits inside the interaction loop (e.g. a user waits for it before every repair) a fast approximate answer is worth more than an optimal one, and a greedy strategy [138] additionally lets us stop as soon as every available user has been assigned a violation instead of computing a maximal set.

Algorithm 2 implements the static-order variant of this strategy: violations are sorted once by their degree in D (line 1), then it assigns the first node to the user and locks the chosen node together with its neighbours (lines 10–12). Locking nodes instead of removing them from the graph allows us to perform fewer write operations on the graph. If further users are available, the algorithm looks for the next free (not-locked) node, and assigns it only if its neighbours are not locked either, which guarantees that the assigned set is neighbour-independent. If no node is available, the user waits for the next iteration of the framework, after the current repairs are completed.

Algorithm 2 GRDG User Assignment Algorithm

Require: Graph repair dependency graph D , users list U

```

1:  $W \leftarrow \text{sort\_by\_degree}(D.\text{vertices})$ 
2: while True do
3:   if  $W.\text{isEmpty}() \vee U.\text{isEmpty}()$  then break
4:   end if
5:    $\text{chosen} \leftarrow W.\text{pop}(0)$ 
6:   if  $\text{chosen.locked} = \text{True}$  then
7:     continue
8:   else
9:      $\text{neighbors\_locked} \leftarrow \text{False}$ 
10:    for  $nh$  in  $D.\text{neighbors}(\text{chosen})$  do
11:       $\text{neighbors\_locked} \leftarrow \text{neighbors\_locked} \vee nh.\text{locked}$ 
12:    end for
13:    if  $\text{neighbors\_locked}$  then
14:      continue
15:    else
16:       $\text{chosen.locked} \leftarrow \text{True}$ 
17:      for  $nh$  in  $D.\text{neighbors}(\text{chosen})$  do
18:         $nh.\text{locked} \leftarrow \text{True}$ 
19:       $\text{user} \leftarrow U.\text{pop}(0)$ 
20:       $\text{assignToUser}(\text{chosen}, \text{user})$ 
21:    end for
22:  end if
23: end if
24: end while

```

After assigning violations to users and collecting the answers, we apply the repairs to the graph. The order in which we perform the updates is not important since the assigned violations and their neighbors are independent.

Check Safety of a Repair If the framework has the *safety* property, we need to check if the repairs do not introduce new violations. To do so, we apply the constraint query on the pattern that contains the nodes updated by the repair. Figure 3.8(a) shows an example of a repair where node 2 is updated and violation 18 is solved. To check if the repair is safe, we look for the graph patterns contained in the constraints Q_1 and Q_2 because node 2 is relevant for both, resulting in the nodes marked in red. If the path already constitutes a violation, as in the case of Violation 17, no further actions are taken since new violations cannot be introduced, while we need to check the path containing nodes 7 and 6. If new violations are detected as a result of the repair, the transformation is reverted and the user is asked for a different repair. Finally, notice there is no need to verify edge-deletion repairs because they are always safe (see Theorem 3.1).

Check for Additional Solved Violations Figure 3.8(b) shows an example of the process of checking if additional violations are solved. After the Violation 18 is solved with the repair to node 2, the algorithm visits the neighbor violations of the solved inconsistency - in this case

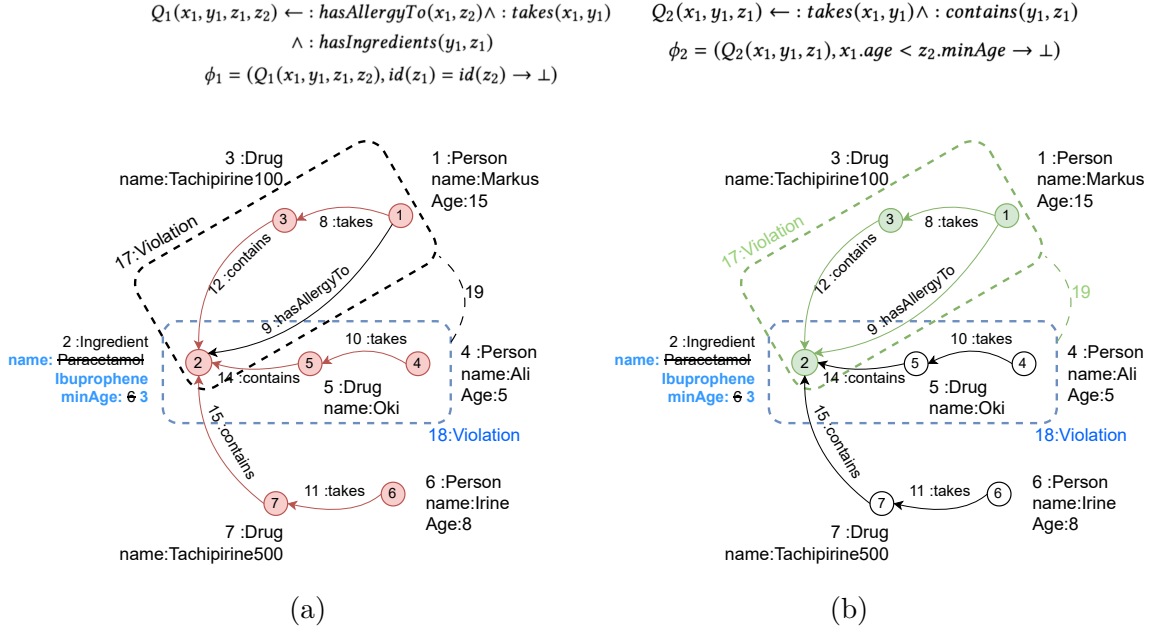


Figure 3.8: Example of (a) checking if additional violation are solved and (b) checking if new violation are introduced

- the hypernode 17. Then, it applies the constraint query corresponding to the violation (Q_1) to the nodes and edges included in the hypernode. If the query returns an empty result, the violation has been solved. In this way, thanks to the GRDG, we avoid unnecessary exploration of unrelated parts of the graph.

Check for Introduced Violations To check if new violations are introduced, we adopt the same approach to check if a repair is safe. The two operations are indeed mutually exclusive: if the framework has the safety property we check if a repair does not introduce new violations, otherwise we look for the introduced ones.

3.5.4 Complexity

In this section, we discuss the computational complexity of our approach concerning the size of the graph $G(V, E)$, and the number of constraints C .

Detecting Violations First, we consider the complexity of detecting the violations in a graph G . The complexity of a graph query evaluated using the product-graph approach based on finite state automaton [139], is $O(|V||\mathcal{D}||q|)$ where $\mathcal{D} = \max(|V|, |E|)$ and $|q|$ is the size of a query q written as a string of vertex or edge variables. Since $|q|$ is constant, the upper bound for the complexity of applying $|C|$ constraint queries is $O(|C||V||\mathcal{D}|)$.

Building the GRDG The complexity of Algorithm 1 lies on the query that checks if two violations overlap (line 4). As for the constraints queries, its complexity is $O(|V||\mathcal{D}||q|)$. Since $|q|$ is constant, the upper bound for the complexity of building the GRDG is $O(|C||V||\mathcal{D}|)$.

Assignment Algorithm Second, we consider the complexity of assigning the violations to the users. Given a GDRG $D(H, F)$, the complexity of the assignment algorithm depends on 1) the node sorting and on 2) looking for the nodes that are free to be assigned. Computing the degree for each node costs $O(|H| + |F|)$, in the worst case where the graph is fully connected $O(|H|^2)$, while ordering the list has complexity $O(|H| \log |H|)$, resulting in $O(|H|^2)$.

Then, checking if the neighbors of a node are locked takes in the worst case $O(|H|)$, because the graphs could have the shape of a star with the chosen node as the center, for a total of $O(|H|^2)$. If we consider the worst-case scenario, where each vertex V is a violation, this becomes $O(|V|^2)$. At most, there are $|V|$ iterations, in the case where only one user repairs every single node using safe repairs, resulting in a complexity of $O(|V|^3)$.

Applying the repairs and maintaining the GDRG Finally, we discuss the complexity of applying the repair and updating the GRDG. This includes checking if neighbors' violations are solved and checking for introduced violations. The neighbors of a hypervertex are at most $|H|$ (if the GDRG is fully connected), hence $O(|V|)$ in the worst case. Applying the constraint queries to the neighbors has a complexity of $O(1)$ since the patterns have fixed length and there is no need to scan the entire graph. Checking for introduced violations instead involves applying the $|C|$ constraint queries over a portion of the graph and thus the complexity is the same as Algorithm 1, that is $O(|C||V||\mathcal{D}|)$. Therefore the complexity of applying the repair is $O(|V|) + O(|C||V||\mathcal{D}|) = O(|C||V||\mathcal{D}|)$.

Overall Complexity Summarizing, the complexity of our approach is related to the complexity of building and maintaining the Graph Repair Dependency Graph ($O(2|C||V||\mathcal{D}|) = O(|C||V||\mathcal{D}|)$), and assigning the users ($O(|V|^3)$). The first one dominates over the latter, resulting in a complexity of $O(|C||V||\mathcal{D}|)$.

Note that the complexity remains polynomial and is cubic in the worst case. However, this complexity refers to the worst case and depends on the following factors: the number of violations in the graph, that are assigned in the worst case to a single user, giving thus $O(|V|)$ iterations (Section 3.5.4) and the computation of the degrees of the nodes (line 1 in Alg. 2), which gives $O(|V|^2)$. In practice, the complexity is bearable thanks to the fact that there are multiple users repairing several violations and that the latter decrease at each iteration. Our empirical evaluation confirms the practicality of our approach in terms of performance and runtimes.

3.6 Experimental Evaluation

In this section, we present a comprehensive experimental evaluation to assess the effectiveness of our approach. We tested our framework on different datasets and settings to assess both the quality and the efficiency of our method. We also conducted a user study with real users having different expertise on a real-world use case involving collaborative repairs. We used Neo4j [140] (Community Edition version 5.12) for the simulated experiments and AuraDB [141] for the user study. All the material necessary to reproduce the results together with the hardware specifications are available on our online repository [142].

3.6.1 Settings

Datasets

Dataset	Nodes	Edges	Node Labels	Edge Labels
Star Wars	268	638	6	5
StackOverflow	1395	3821	5	6
wwc_2019	2400	16596	5	9
Fincen	7061	117452	3	6
ICIJ	1908466	3193390	5	14
Synthea	5104	20242	13	15
Synthea50	11844	48033	13	15
Synthea75	19025	66679	13	15
Synthea100	23770	100320	13	15
Synthea200	37473	165235	13	15

Table 3.1: Size of the datasets in term of number of nodes, edges, node and edge labels

The datasets used in our experiments are illustrated in Table 3.1 and include **StarWars** [143], a small graph modeling characters, starships, and planets from the Star Wars universe, **StackOverflow** [144], which represents questions, answers, and comments from the StackOverflow platform, and **wwc2019** [145], focused on data from the Women’s FIFA World Cup 2019. Additionally, we used **Fincen** [146], representing 18,153 suspicious financial transactions, and **Synthea** [147], a synthetic patient simulator from which we generated health records for various patient populations. We generated data for 15 patients, 50, 75, 100 and 200 patients. Finally, we also used a real-world dataset, the Offshore Leaks Database and guide from the International Consortium of Investigative Journalists *ICIJ* [148].

In our experiments, we considered the pattern shapes listed in [149] that reflect the shapes found in a large corpus of query logs with different lineage and sizes, from DBPedia to BioPortal and Wikidata. As shown in Figure 3.9, we used chain queries of length 2 and 3, that are the most common lengths found in real-world graph query logs; two types of stars, with two and three external nodes; flowers, and cycles. The statistics about the patterns’ shape of our datasets are reported in the Appendix A.2.

To conduct the simulated experiments systematically (i.e., to bound the number of possible repairs and to be able to control the error injection), we added a synthetic property to each node. We leveraged this property to generate inconsistencies according to probability distributions.

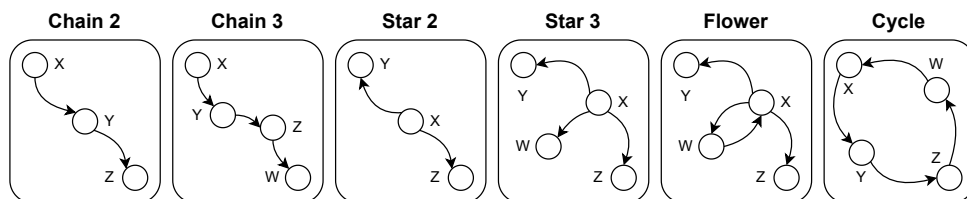


Figure 3.9: Different shapes of graph patterns.

Constraints and repairs

First, we crafted the constraints for each dataset as defined in Section 2.2.3 by using a pattern for each shape of Figure 3.9 and all the logical operators for the literals. Then, we mapped the constraints into queries as defined in Definition 3.3. The choice for a pattern in a constraint query determines the number of subgraphs in the graph instance that could potentially become violations (after the inconsistencies injection). To control the following error generation phase, we chose the patterns for each dataset by avoiding overlaps between labels. Finally, we listed all the possible repairs for each constraint, including the best repair (i.e., the ground truth). To bind the number of repairs per violation, we set the constraints exclusively on the added synthetic properties. Therefore, the repairs concern updates on the synthetic properties and deletions of the edges. For ease of implementation, we implemented node relabeling as property updates, adding the property *label* to each node. Finally, the list of possible repairs for the users contains the atomic repairs (single updates on the properties or the single edge deletions) and the bulk repairs made by all the possible combinations of atomic transformations.

Inconsistencies injection

Following the “silver standard” [150] assumption, we considered our datasets as the ground truth and synthetically injected the inconsistencies. As previously done in the literature [116, 45, 151] we introduced $20\%|V|$ violations in each datasets, where $|V|$ is the number of nodes in the instance graph. To maintain the shape distribution over the inconsistencies, we injected a proportional number of violations for each type of constraint. For example, if a cycle is present 10 times in the instance graph and the total number of violations to be injected is 50, we introduce an inconsistency on 5 subgraphs having the cycle pattern. Note that introducing an inconsistency could generate additional violations in the graph since a node could belong to multiple and different patterns. To control the error generation process and ensure fixed error distributions over the different datasets, we injected the inconsistencies on the synthetic properties and edges.

Metrics

To evaluate the quality we used the **F1-score** considering as True Positives the injected errors correctly modified, False Positives, are the properties (or edge) modified (or deleted) in a way different from the ground truth, and, finally False Negatives are injected errors that weren’t fixed. To evaluate the efficiency, we counted the number of iterations of the user assignment **#iteration** and an iteration skip due to the locks on the violation (**#wait**) in our framework. These metrics represent how costly it is to involve the users, either in repairing the graph or by keeping them on hold because all the violations are being repaired by others.

Simulated users

We model user behavior as follows. Each user has a probability p to choose the repair corresponding to the ground truth. For instance, an oracle will have $p = 1$, a malicious user will have $p = 0$, and a knowledgeable user will have $0.5 \leq p \leq 0.99$. The objective is to evaluate how our approach performs in the best and worst cases, and with knowledgeable users in the

middle. At each interaction, users can choose the repair corresponding to the ground truth according to the probability p . If they don't, they will pick one of the other repairs at random.

3.6.2 Quality Assessment

Baselines

For our experiments we consider three groups of baselines. (1) First, we adapted the preferred repair framework [117] to the property graph case, producing four variants: *update preferred* (U-pref) which ranks the repair by ascending number of property updates (edge deletion only repairs are last in the ranking), *delete preferred* (D-pref) which ranks the repair by ascending number of edge deletions (update only repairs are last in the ranking), *number of operation preferred* (N-pref) which ranks the repair by ascending number of operations (edge deletion or property update) and *label aware* (L-pref) which orders the repairs considering which the node and edge labels are going to modify (e.g., modifying the *:hasIngredient* label in our running example has more impact than modifying a *:takes*, because we can imagine the information about a drug to be more reliable). For the latter, we manually ranked the possible repairs based on the constraints and the cause of the inconsistencies. For instance, if a violation is due to the presence of a specific edge, the L-pref approach will choose to delete that edge. The preferred repair approach iteratively retrieves a violation from the instance graph, chooses a repair with the highest rank, and applies the transformation until the graph is repaired. (2) Second, we compared our approach with Graph Repairing Rule (GRR-B1), mining the logical rules using AMIE [152] as it was done in [116] and translating them into graph queries. It is important to notice that logical rules do not consider edge deletions, thus we improved the baseline by manually adding rules for edge deletions (GRR-B2). On the contrary, GRRs include edge insertions, so we labeled them as false negatives. Furthermore, in [116] GRRs are applied for one iteration, we instead run the set of repairing rules until the graphs are repaired. (3) Third, we considered the case of neighborhood constraints in labeled graphs, by evaluating our approach with neighborhood constraints on property graphs, comparing the results with both non-interactive and interactive approaches. In particular, we used the automated algorithms *AlterGC* and *Grepair* from [44] and the interactive approach *IGR* from [121] as baselines. To generate the violations we relabeled the nodes and used the original graph schema of our datasets as neighborhood constraint graphs.

Comparison with Preferred Repairs in Isolation

To investigate the effectiveness of user repairs with respect to automated ones, we evaluated them in isolation (i.e. repairing single violations without diramations), comparing the choices of repairs made by our approach against those of the baselines. Table 3.2 summarizes the F1 scores of our approach across different datasets, comparing it with four baseline methods: D-Pref, U-Pref, N-Pref, and L-Pref. We didn't compare with the GRR baselines because they cannot be applied on a single violation. We tested our approach under five different configurations of user expertise, denoted as *uQAR-x*, where x ranges from 0 (non-expert users) to 1 (oracle users). The results indicate that our approach consistently outperforms the baselines across all datasets, especially as the user expertise increases. Even with less knowledgeable users (e.g., *uQAR-0.25*),

	Star Wars	Stackoverflow	wwc_2019	Synthea	Fincen
D-Pref	0.431	0.293	0.3	0.321	0.703
U-Pref	0.434	0.574	0.311	0.359	0.544
N-Pref	0.204	0.264	0.098	0.233	0.450
L-Pref	0.434	0.573	0.31	0.359	0.580
uQAR-0	0.319	0.386	<u>0.396</u>	0.314	0.485
uQAR-0.25	<u>0.454</u>	0.486	<u>0.486</u>	<u>0.455</u>	0.624
uQAR-0.5	<u>0.520</u>	<u>0.584</u>	<u>0.555</u>	<u>0.472</u>	<u>0.707</u>
uQAR-0.75	<u>0.565</u>	<u>0.638</u>	<u>0.635</u>	<u>0.553</u>	<u>0.743</u>
uQAR-1	0.633	0.672	0.67	0.608	0.776

Table 3.2: F1 score of our approach vs baselines tested in isolation. Bold represents the best, while underlined values beat the baselines.

our approach surpasses the baselines in most cases. For instance, with uQAR-0.25, the F1 scores on *Star Wars*, *wwc_2019*, and *Synthea* datasets are all higher than those of the baselines. The only baseline that comes close to our method is *L-Pref*, especially on the *Stackoverflow* dataset, where it achieves 0.573 compared to 0.584 for uQAR-0.5.

Comparison with Preferred Repairs and Graph Repairing Rules

Then, we evaluated the quality of the repairs obtained with our approach considering the whole process against six baselines (preferred repairs and graph repair rules). In particular,

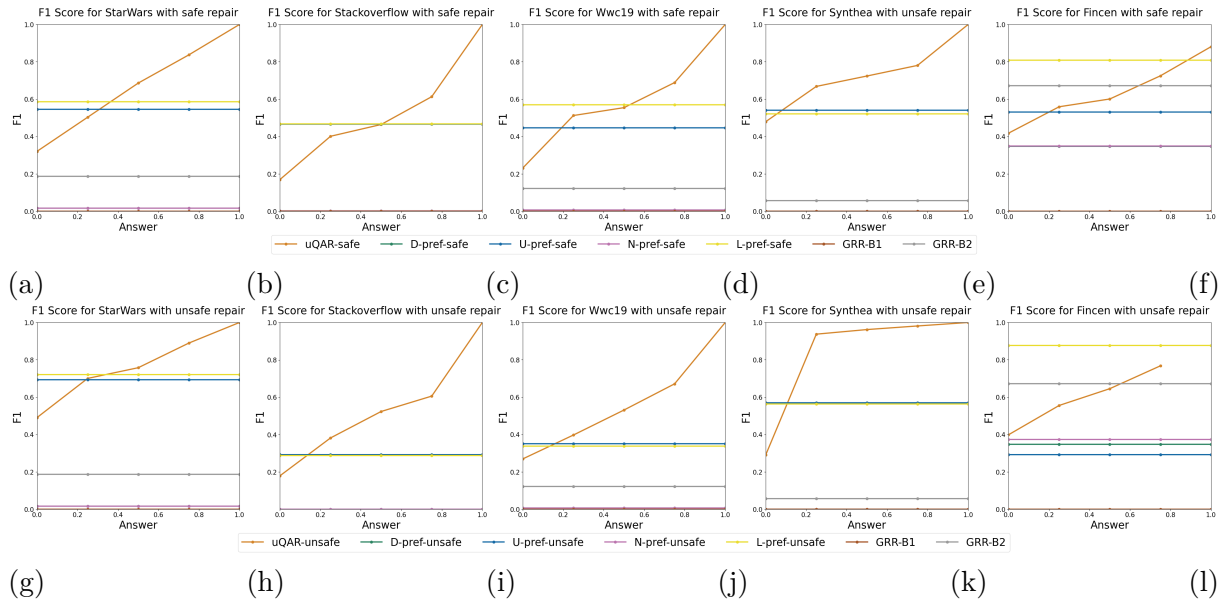


Figure 3.10: Average F1 score of our approach vs the baselines in relation with the probability of the users to pick the correct repair (lack of a data point indicates a non-converging experiment).

we compare our approach in the safe and unsafe configurations (uQAR-safe and uQAR-unsafe) against the safe and unsafe version of the baselines. Figure 3.10 summarizes the results. First, our approach outperforms the baselines even with users who do not possess oracle knowledge. The only exception is the *Fincen* dataset, where *L-pref* beats the non-oracle users. This is because the *Fincen* dataset contains only three labels and the majority of the patterns cover only two of them, thus a preferred repair approach based on the labels has good results in terms of the

F1-score. Similar comments can be made for **GRR-B2**. Our approach outperforms it even with non-oracle users with the exception of the **Fincen** dataset where our approach performs better only with more expert users. We hypothesize it's because **Fincen** has the lowest number of labels and thus the smaller set of rules. Moreover, **GRR-B1** never converges, since it does not repair violations involving edge deletions, while **GRR-B2** does not terminate with the **Stackoverflow** dataset. This result shows that a **GRR** approach, while it is appropriate to improve the overall quality of a graph, cannot address denial constraints. In general, we can see that the *safe* and *unsafe* variant of our approach have similar performance, with the *safe* variance performing better with less knowledgeable users in **StackOverflow** and **Synthea** datasets. Moreover, the **Fincen** dataset is the only one in which the ground truth is not reached, showing that the safety property does not guarantee to reach the ground truth and confirming our theoretical results. However, without the safety property, the framework does not always converge.

We also report the runtime of the approach, that is the time needed for the algorithms to create the GDRG, assign a user, apply the repair, and update the GDRG (without considering the time of a simulated user to provide the repair). In Figure 3.11, we can see that the task that takes most of the time is the creation of the GDRG (hence the peak at iteration 0). This doesn't happen for the **Star Wars** dataset because it's very small. For the others, especially for the bigger ones (e.g., **Synthea** and **Fincen**) it is evident how the maintenance cost in terms of time drops to just some seconds. As described in Section 3.5.4, despite this theoretical complexity, the behavior in practice shows that the overhead of the initial iterations decreases with the following iterations, as multiple users repair multiple violations. Moreover, our implementation leverages off-the-shelf indexing mechanisms in the underlying graph database to improve performances.

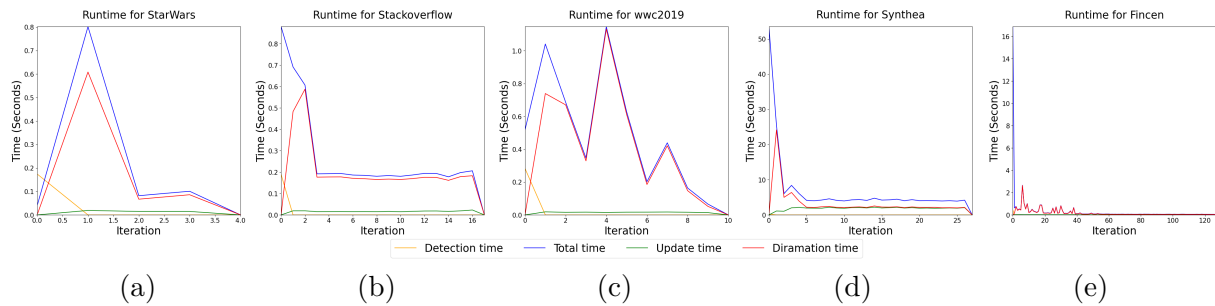


Figure 3.11: Runtime in seconds for each iteration. Notice that the y axes have different scales.

Evaluation of different assignment policies

To evaluate the effectiveness of the assignment algorithm, we compare it with two alternative policies: *Density-Based Assignment* (DBA) and *Relevance-Based Assignment* (RBA). The DBA policy prioritizes violations that are more interconnected in the graph, assigning those with the highest degree first, ensuring that violations affecting many nodes are addressed early. The RBA policy assigns violations based on their importance within the GRDG, with those having the highest PageRank score prioritized. The PageRank reflects the relevance of a node, so addressing more influential violations first may increase the propagation of the repair early on. As shown in Figure 3.12 in our experiment with the safe configuration we didn't find any significant difference

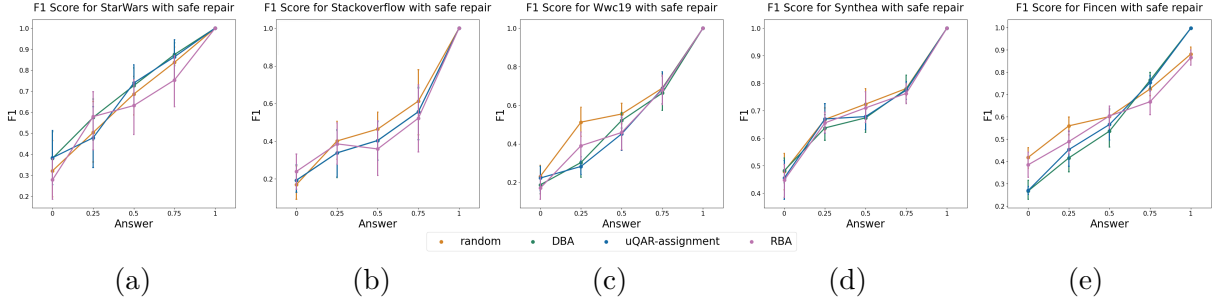


Figure 3.12: Average F1 score of our approach using different assignment policies. The error bars represent the standard deviations.

in terms of quality of the results. However, in Section 3.6.3 we discuss their impact on efficiency. We found the same result with the unsafe configuration, reported in the Appendix A.3.

Comparison with Neighborhood Constraints

	Star Wars	Stackoverflow	wwc_2019	Synthea	Fincen
AlterGC	0.4	0.217	0.600	0.286	0.308
Grepair	0.6	0.463	0.533	0.046	0.338
IGR-Random	0.333	0.217	0.599	0.105	0.308
uQAR-0	0.085	0.219	0.044	0.074	0.245
uQAR-0.25	0.475	0.453	0.537	0.236	<u>0.605</u>
uQAR-0.5	<u>0.735</u>	<u>0.748</u>	<u>0.789</u>	<u>0.429</u>	<u>0.752</u>
uQAR-0.75	0.868	0.892	0.898	0.825	0.912
IGR-Oracle	0.8	0.889	1	0.952	0.359
uQAR-1	1	1	1	1	1

Table 3.3: F1 measure of our approach vs baselines for the neighborhood constraints case. Bold represents the overall best, while in underline are the values that beat the baselines.

	Star Wars	Stackoverflow	wwc_2019	Synthea	Fincen
AlterGC	9	12	26	21	15
Grepair	21	23	32	25.8	14
IGR-Random	30	12	48	23	21
uQAR-0	746	24	1229.67	350	<u>5.3</u>
uQAR-0.25	21.9	<u>10.6</u>	26.4	<u>14.3</u>	<u>4.4</u>
uQAR-0.5	<u>8.4</u>	<u>5.2</u>	<u>7.4</u>	<u>7.2</u>	<u>2.6</u>
uQAR-0.75	6.7	3.2	4	2.9	2.4
IGR-Oracle	9	13	26	21	15
uQAR-1	5	2	2	1	1

Table 3.4: Average number of iterations of our approach vs baselines for the neighborhood constraints case. Bold represents the overall best, while in underline are the values that beat the baselines.

In this case, we evaluated our approach in its unsafe version since that was the nature of the baselines, considering as constraints the schema of our datasets. As before, we considered different users' expertise. Table 3.3 summarizes the results. Our approach indicated as uQAR-x, where x refers to the type of users simulated, beats all the baselines, both interactive and not. Notice also that we beat the baselines even without oracle users: uQAR-0.75 and uQAR-0.5

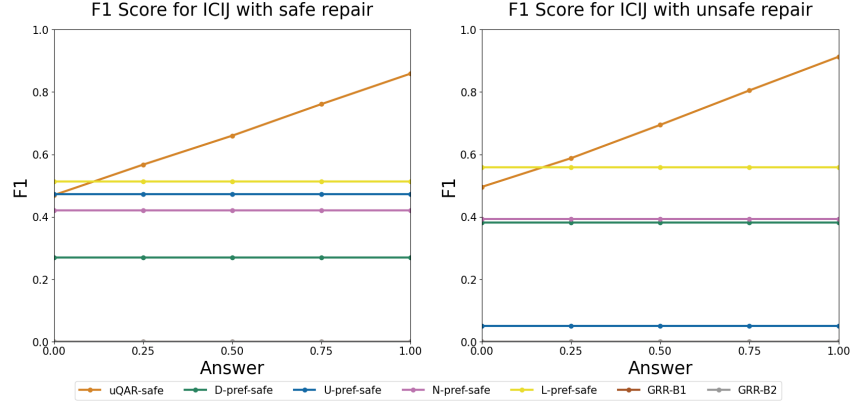


Figure 3.13: Average F1 score of our approach vs the baselines in relation with the probability of the users to pick the correct repair with the ICIJ dataset.

achieve the best performances on each dataset, while on **Fincen** uQAR-0.25 is enough. Similar insights are obtained for the efficiency of the approach. Table 3.4 shows the average number of iterations needed to repair the graph. As before, our approach outperforms all the baselines, even with non-oracle users. Notably, on **Fincen**, uQAR-0.25 is both faster and has a higher F1 score than the baselines while this happens with uQAR-0.5 on all datasets.

Experiments with real world inconsistencies

We evaluate our approach also in case of real world (i.e., not injected) inconsistencies. We use the ICIJ graph dataset containing information regarding the offshore leaks. We found the inconsistencies by manually examining the data, and in particular they were due to mismatches between country codes, overlapping dates and entity status discrepancies. We selected in total a sample of 2000 equally distributed violations. As shown in Figure 3.13, our approach outperforms all the baselines even with non-oracle users with a margin greater than the experiment with the injected inconsistencies. Notice also that the baselines GRR-B1 and GRR-B2 do not converge.

User Study

Exp.	Users	F1-score	#int	#wait	Avg. Repair Time (s)	Duration (s)
1	5	0.589	21	17	28.78	636
2	5	0.631	17	12	49.93	867
3	10	0.643	18	34	42.27	797
L-pref	-	0.234	21.2	0	-	3.21
D-pref	-	0.096	26.9	0	-	4.16
U-pref	-	0.103	17	0	-	2.59
N-pref	-	0.145	22.6	0	-	3.63

Table 3.5: Results of the user study vs the baselines

Finally, we also experimented with real users, to investigate how the findings from the synthetic experiments translate to a real use case. For this qualitative experiment, we focus on the **Star Wars** dataset, which is sufficiently small and understandable by real users. We created a small web application using Flask [153] that shows the users the violations and asks them to choose

an appropriate repair. We recruited 20 users involving people from our laboratory (including Master, PhD students, and postdocs) and from our connections, aiming for a mix of people with and without a database background. Before the experiment, they were briefly instructed on what their task was. We collected their expertise on databases, graphs, and Star Wars using a 10-level Likert scale. The average knowledge about databases is 5.65, on constraints is 4.35, on graphs is 4.65, and on Star Wars it is 5. In Appendix A.1 we report both the questions and a summary of the answers collected with the survey. The participants were split into two groups of 5 and one group of 10 (to evaluate the approach with different numbers of users) ensuring the same distribution of the knowledge about the Star Wars universe. To perform the experiments, we injected the noise into the dataset by adding edges and modifying existing properties. Then, we defined the following constraints resulting in 43 violations: i) A *Character* that pilots a *Starship* that appeared in a film should have appeared in that film, ii) A *Character* that pilots a *Vehicle* that appeared in a film should have appeared in that film, iii) A *Character* that pilots a *Starship* should be of a species with an average height smaller than the *Starship*'s height.

During the experiments, we measured the total duration of the process, the average time per repair, the number of interactions and waitings, and finally the F1 score. We also tested the preferred repair baselines on the dataset. Table 3.5 shows the results we collected. As for the simulated experiments, increasing the number of users does not impact the number of interactions but increases the number of times that the users wait for a violation to be repaired. Despite users reporting an average knowledge of the Star Wars universe of 5 (in the 1-10 range), the average F1-score outperforms the score obtained by the baselines. Moreover, the time that users took to repair the graph is close to the total duration time of the experiments, proving that the final bottleneck of the interactive framework is the human and the time for the algorithms to update the GRDG and assign the violations is significantly less than the human interactions.

3.6.3 Scalability

To assess the scalability of our approach, we tested our repair framework in its safe version (because we wanted to ensure the termination property) on the Synthea dataset generated with an increasing number of patients (15, 50, 75, 100, and 200). Additionally, we considered different numbers of (oracle) users, from 5 to 100. Figure 3.14 shows the number of iterations for each dataset. We noticed that the number of iterations remains almost stable, within a range of 10-30, across the different datasets. Notice from Figure 3.14(a) that the iterations for the smallest

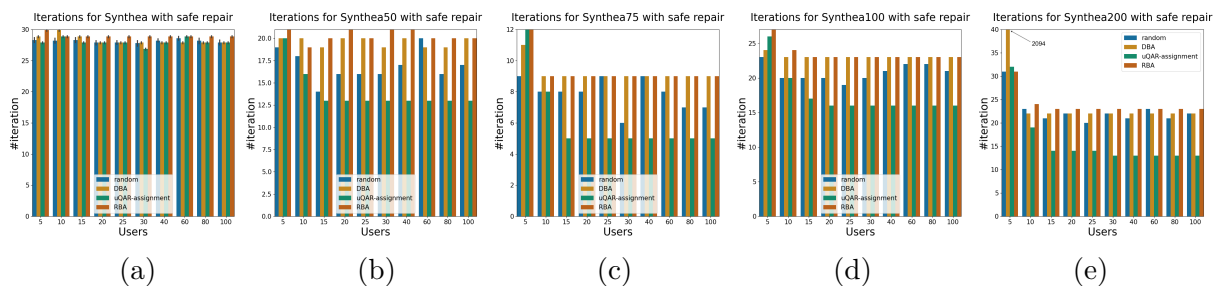


Figure 3.14: Number of interactions for each dataset vs users number and the function used to rank the nodes in the GDRG. The y axes have different scales.

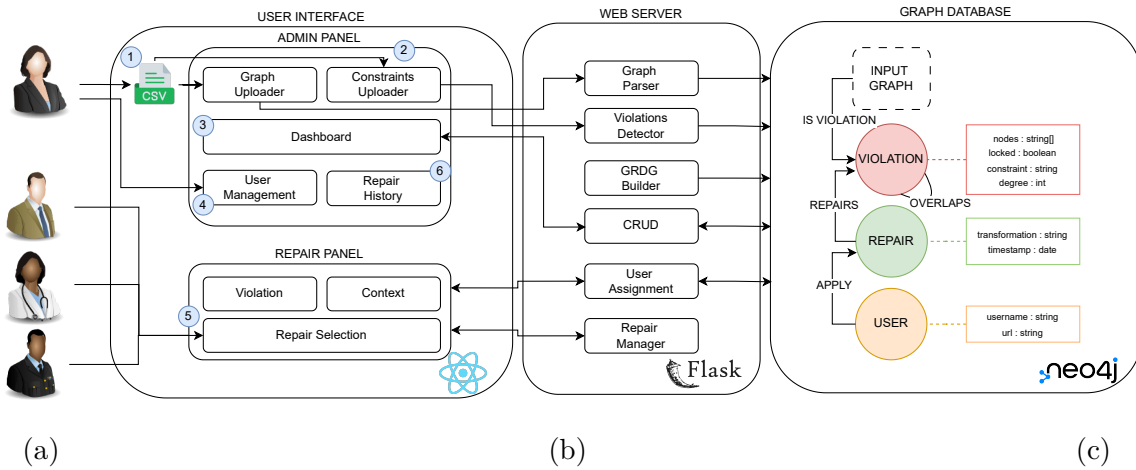


Figure 3.15: Overview of the **Grafixer** architecture: the user interface (a), the web server managing the GRDG (b), and the graph data model (c).

dataset are higher (~ 30) than for the larger ones. This happens because, given the graph's structure, with just five users, we assign all the free violations to be repaired. We also observed that increasing the number of users decreases the number of iterations until a plateau is reached. This plateau depends on the size and structure of the graph. As the size of the dataset increases, more users can be involved before reaching the plateau. We can see that for **Synthea50** and **Synthea75**, it is reached with 15 users, for **Synthea100** with 20 users, and for **Synthea200**, it needs 30 users. Once a plateau is reached, adding more users does not reduce the iterations required, but it increases the number of times the users need to wait. In Appendix A.4 we report the chart showing the behavior of the waitings w.r.t the size of the dataset. Moreover, we compared the different assignment policies listed in Section 3.6.2. Results show that the **uQAR-assignment** results in fewer repair iterations when increasing the number of users. Notice that the last plot has an outlier for the DBA assignment with 5 users.

3.7 Grafixer: A Tool for User-Centric Repair

The framework described so far has been implemented in **Grafixer**, a web application that lets several users repair a property graph collaboratively and that was demonstrated at SIGMOD 2025 [14]. The tool follows the model-view-controller paradigm: it is implemented in Python with Flask as web server, the interface is built with ReactJS, and all data is stored in a Neo4j database, where the APOC [154] and GDS [155] plugins provide the graph procedures and algorithms used by the repair process. The code is available in the online repository [156].

Figure 3.15 gives an overview of the architecture, which is divided into three components. The user interface (Figure 3.15(a)) is a ReactJS web application comprising an Admin Panel and a User Panel. The web server (Figure 3.15(b)), implemented in Flask, processes requests, orchestrates the data operations, interacts with the database and maintains the Graph Repair Dependency Graph. The database is a Neo4j instance where the input graph, the violations and the repair history are stored and queried.

Data Model

The application uses a graph data model for everything it stores, whose schema is shown in Figure 3.15(c). **USER** holds the information about the registered users and **REPAIR** records the transformations each of them applied. The **INPUT GRAPH** is the data uploaded by the admin and keeps its own schema. On top of it the system materialises the Graph Repair Dependency Graph of Section 3.5.1, in which every subgraph violating a constraint becomes a **VIOLATION** hypernode. Because the bookkeeping lives in the same store as the data, statistics such as the number of interactions or of repairs per user are obtained with simple graph queries rather than with a separate analytics layer.

Web Server

The web server is the component that constructs and manages the GRDG and, with it, governs the repair process. Given an input graph and a set of constraints, it performs the following operations:

1. it parses the input files, loads the property graph into Neo4j, and evaluates the constraint queries to retrieve the violations (Section 3.3);
2. it constructs the GRDG by computing the dependencies between violations and projecting the resulting hypergraph through the GDS library (Section 3.5.1);
3. it assigns independent violations to users following the policy of Section 3.5.3, so that no two dependent violations are being repaired at the same time;
4. when a user submits a sequence of transformations, it checks whether they resolve the violation and, if safety is required, whether the repair is safe (Definition 3.10) before applying it, propagating the consequences to the neighbouring hypernodes;
5. it performs the CRUD operations that feed the admin dashboard with the number of violations, the list of users and the repair history.

User Interface

The interface has two views, one for each role in the process: the Admin Panel, used by the data owner who wants a graph repaired, and the Repair Panel, used by the people who repair it.

Admin Panel. The admin initialises the repair process by uploading a property graph and a list of constraints in CSV format. Once the upload is complete, the main dashboard shown in Figure 3.16(a) reports in real time the number of violations still to be fixed, the number of users currently engaged, and the total number of interactions, that is, of repairs applied to the graph. Four additional views complete it: a line chart of the violations solved by each user per iteration, the list of users, the schema of the property graph, and the list of repairs. From the sidebar the admin reaches the user page, where further users can be added; each user has a unique username and receives a URL, also encoded as a QR code, that opens a repair session. A dedicated repair history page, Figure 3.16(b), lists every repair performed together with the number of violations

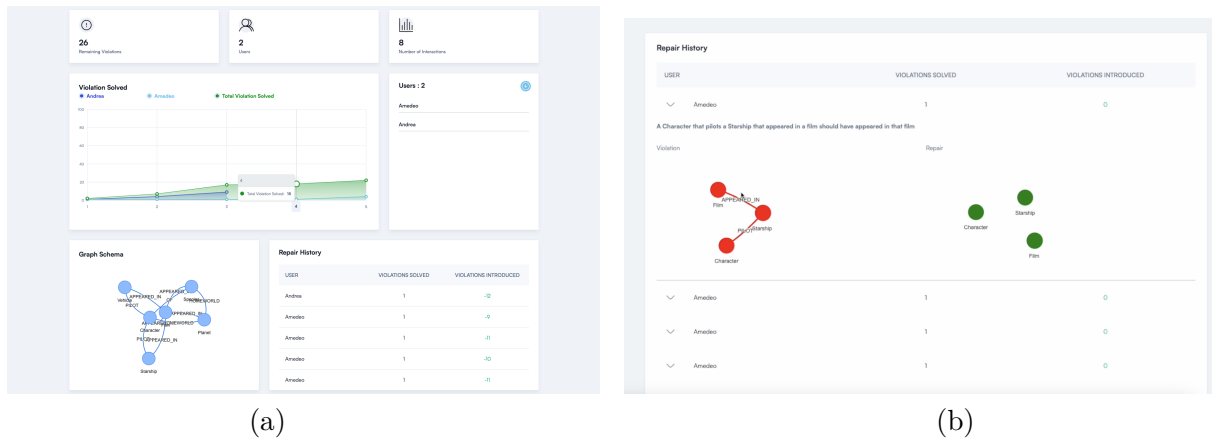


Figure 3.16: **Grafixer** Admin Panel: the main dashboard (a) and the repair history (b).

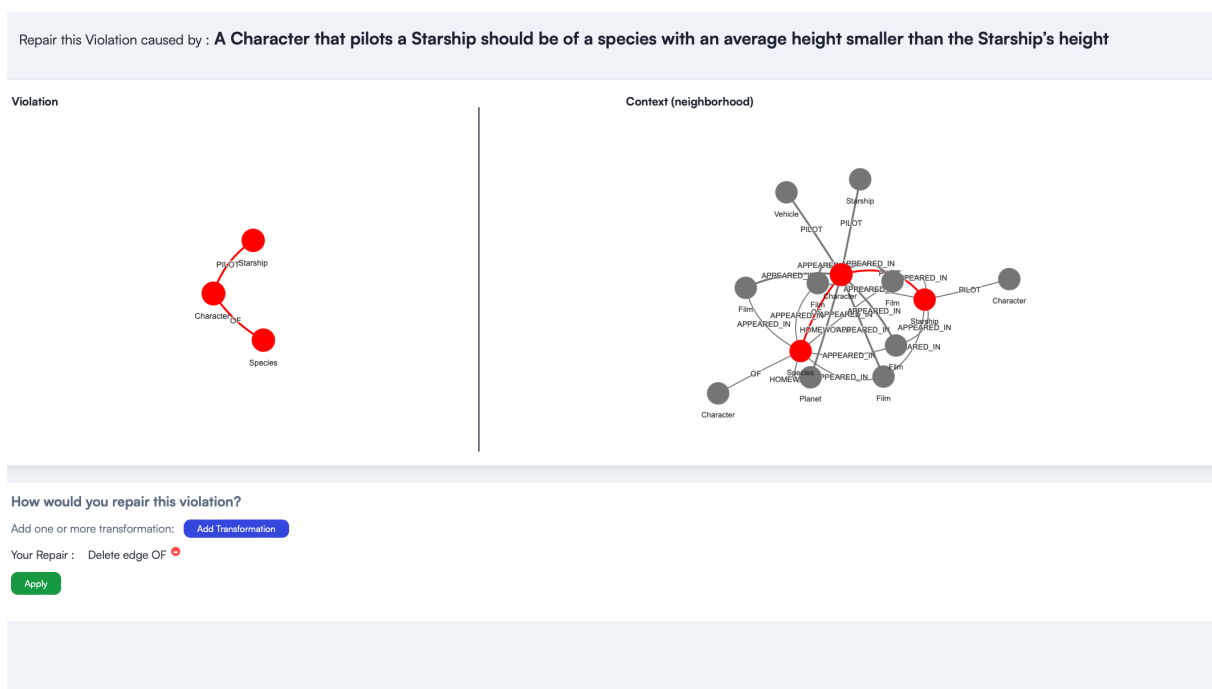


Figure 3.17: **Grafixer** Repair Panel: the violation to be fixed (left), its surrounding context (right), and the menu of atomic transformations.

it resolved or introduced, the original violation, and the subgraph resulting from the repair, so that the effect of the process can be audited after the fact. A settings page exposes the two parameters that the analysis of this chapter identifies as the interesting ones: whether safety is enforced, and which assignment policy is used.

Repair Panel. Figure 3.17 shows the interface used to resolve a violation. It is split in two: the left side displays the violation itself, while the right side presents the context of the inconsistency, including the neighbouring nodes that the user may need in order to decide which of the candidate repairs is the correct one. The user composes a repair by adding one or more atomic transformations, namely edge deletions and property updates, and applies them. If the chosen transformations do not resolve the violation, or if they are not safe when safety is required,

an alert asks the user to revise them. When no independent violation is available, the panel shows a waiting message until the assignment algorithm releases a new one, which is the interface counterpart of the locking mechanism described in Section 3.5.3.

3.8 Conclusion

This chapter studied the problem of user-centric repair of property graphs under denial constraints, together with its complexity. It formalised a pipeline that detects inconsistencies by compiling constraints into queries, materialises the dependencies between violations in the Graph Repair Dependency Graph (Section 3.5.1), and uses that structure to assign violations to users so that several interactive repairs can proceed at once without invalidating one another. The local notion of safety (Definition 3.10) was shown to be necessary and sufficient for convergence (Theorem 3.2), so that the process is guaranteed to terminate within a number of iterations bounded by the number of violations (Theorem 3.3), independently of the expertise of the participating users.

The evaluation used synthetic and real-world datasets with denial and neighbourhood constraints. Simulated users at different expertise levels showed the behaviour of the approach across the whole competence range and outperformed every baseline we could construct, and the same advantage held in a real-world scenario with a group of twenty non-oracle participants. The approach also scaled with respect to the size of the dataset and the number of users, with the running time dominated by human deliberation rather than by maintaining the dependency structure.

The assignment strategy proposed in this Chapter treats every user as equally able to answer any question, and it asks for as many users as independence in the Graph Repair Dependency Graph permits. Chapter 4 lifts both, estimating how difficult each violation is to repair, modelling users as heterogeneous agents with distinct skills, capacities and costs, and recasting the allocation of violations as a budgeted optimisation problem over the same dependency structure. The remaining directions opened by this chapter, from richer repair operations to collaboration among several users on a single violation, are taken up together with those of Chapter 4 in Chapter 5.

Chapter 4

Difficulty-Aware, Budgeted Repair with GNNs

The framework of Chapter 3 makes two assumptions that do not necessarily hold in practice: (i) the users available to perform repairs are interchangeable, and (ii) human effort is unconstrained. This chapter lifts both. We estimate, with a graph neural network, how difficult each violation is to repair; we model users as heterogeneous agents with distinct skills, capacities and costs; and we cast the allocation of violations to users as a budgeted optimisation problem. The chapter therefore addresses the interactive repair problem of Definition 3.5 under realistic assumptions about who does the work and at what cost. Figure 4.1 shows two example violations caused by a denial constraint, which motivate treating violations as unequal in difficulty. This Chapter is the result of the publication titled *Budgeted Interactive Property Graph Repair with GNN* [15] accepted at SIGMOD 2027.

4.1 Context

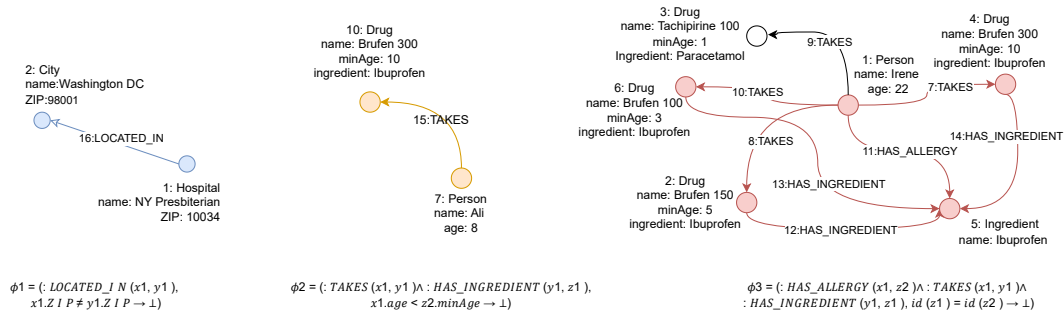


Figure 4.1: Three examples of violations caused by denial constraints.

Previous approaches to graph repair have generally followed two orthogonal types of approaches: (i) fully automated methods that leverage constraints and optimization techniques [3, 44, 45, 4], which assume that rules are known in advance and are sufficient to guide repair decisions; and (ii) human-in-the-loop approaches [121, 119, 120], which engage humans in repair validation or suggestion but often oversimplify task assignment, rely on a single oracle, or do not consider budget constraints. This also include our framework detailed in Chapter 3.4. Both face key limitations: full automation struggles with ambiguous cases requiring domain expertise to make

the right choice, while human-driven solutions lack scalability and often ignore practical concerns such as cost [157], diversity in user expertise, and cognitive load [158].

Figure 4.1 shows a property graph for a healthcare scenario with three denial constraints causing different types of violations.

- a) Constraint ϕ_1 states that a hospital and the city it is located in must have the same ZIP code, producing the violation depicted in blue. This is an instance of a violation that can be solved by an automated method, since the information to fix it can be found in the original graph or learned by inference [132].
- b) Constraint ϕ_2 states that a person cannot take a drug whose ingredient is age-restricted, while being younger than the minimum age required, resulting in the orange violation. Solving this violation requires domain knowledge. No one except Ali’s physician can know whether the source of inconsistency lies in the wrong age or in him taking the Ibuprofen.
- c) Constraint ϕ_3 states that if a person has an allergy to an ingredient, they should not take a drug that contains that ingredient, producing the red violations. Here, a single constraint causes three violations with two overlapping nodes (Person and Ingredient). In this case, not only is domain knowledge required (e.g., knowing Irene’s allergies), but the violations include a complex topological structure with many nodes and edges.

Based on the above observations, one can notice that there are two fundamental problems in graph repair. The first is to identify the *difficulty* of the repair and to factor it into the assignment, second, assigning the repair tasks to either automatic methods based on inference or to users with appropriate expertise. Crowdsourcing approaches assume the difficulty to be a function of the semantics of the elements composing a task [159], while the recent interactive graph repair works assume that the difficulty is a function of the graph topology [13]. By contrast, we argue that there is a need for a *holistic* assessment of the difficulty, considering both topological and semantic aspects of the violation. Previous work in crowdsourcing, while considering human skills and an overall budget, does not consider interconnected tasks like violations in a graph in making assignments. Furthermore, to our knowledge, it does not consider human limitations such as time and fatigue [160, 161, 162, 163].

To address this gap, we propose a novel budget-aware assignment strategy that combines difficulty estimation with user limitations. A key requirement for such an approach is the ability to reason simultaneously about the structural context of a violation (e.g., how it fits into the surrounding topology) and the semantic properties in nodes and edges (e.g., attribute values and types).

Graph Neural Networks (GNNs) naturally meet these requirements, as they are designed to integrate structural and attribute-based information in a unified representation [164], making them particularly suitable for both predicting violation difficulty and for automating less complex repairs (e.g., edges with low existence probabilities). More precisely, we leverage GNNs, which blend topological dependencies and node/edge feature representations, to predict edge existence probabilities and to extract node embeddings via latent feature representations. These predictions are used to compute a *difficulty score* for each violation, reflecting the model’s confidence in

resolving the violation automatically. These scores are integrated into a user model, which estimates the likelihood that a user, given their knowledge, will successfully repair a violation.

The same model can automatically resolve simple violations by deleting the corresponding edge. This creates a trade-off between users, who provide higher-quality but costly repairs, and automated methods, which are cost-free but may produce lower-quality results. To manage this trade-off, we propose a tunable assignment strategy that allows to balance violations between users and automated repair based on quality and cost preferences.

Our approach jointly considers violation difficulty and user expertise and capacity (i.e., the amount of work they can perform), with the goals of *maximizing* the expected repair quality, while staying within a specified *budget*, making sure people are not overloaded with repairs.

In sum, we present a framework for budgeted interactive repair of property graphs that combines a learned difficulty estimator, automated repair of low-difficulty violations and a budgeted assignment algorithm. Our main contributions are the following.

- We introduce a new notion of violation difficulty tailored to graph repair, based on the model’s confidence in automatic resolution of violation. Unlike prior work that models difficulty based solely on topology or semantics, we jointly capture both via a GNN and further tailor the representation to the repair task via a constraint-driven feature filtering mechanism (Section 4.3).
- We propose a unified repair pipeline in which the GNN is used both to estimate difficulty and to automatically resolve easy violations via confidence-based pruning. We formalize this process and show that automated repairs commute with human repairs, ensuring that pruning does not affect the correctness or quality of the final repair outcome.
- We formulate the violation assignment problem as a budget-constrained covering problem over a graph of dependent violations, where tasks are inherently interconnected and prove that it is NP-Hard (Section 4.4.1 and Section 4.4.2). To address this, we design a tailored Lagrangian-relaxation-based algorithm that assigns connected subgraphs to users through a seed-based construction, explicitly accounting for structural dependencies, user expertise, and cognitive capacity (Section 4.4.4). Building on the proposed formulation, we derive a closed-form dual upper bound on the optimal objective, enabling us to quantify the quality of the obtained assignments despite the problem’s NP-hardness.
- We validate our approach with an extensive set of experiments evaluating 1) the validity of our difficulty measure and 2) the overall effectiveness of our approach, considering both the quality of the repairs and cost, against interactive and automated graph repair methods and other task assignment strategies (Section 4.5).

4.2 Related Work

Everything concerning graph dependencies and graph repair, the constraint classes and their relative expressivity, the deletion and update repair models, and the complexity of computing minimal repairs, was surveyed in Section 3.2 and is not repeated here: this chapter works with the same data model, the same class of Graph Denial Constraints and the same repair operations,

and the interactive framework of Chapter 3 is its closest prior work as well as its main baseline. What that body of work does not supply is the other half of the problem addressed here. It offers no way to tell how hard an individual violation is to repair, and no principled way to decide which of several unequally skilled and unequally expensive users should be asked about it. Those two questions have been studied instead in crowdsourcing, and it is that literature, reported below, which completes the background needed for the rest of the chapter.

Modeling task difficulty and assignment. A significant body of work has explored task assignment optimization and task difficulty modeling in crowdsourcing [160]. These approaches typically assume tasks to be independent units, and aim to assign them to workers with suitable skills under constraints such as cost, quality, and group compatibility [161]. To solve the assignment problem, they introduce probabilistic or integer programming models to optimally allocate workers based on skill vectors and cost budgets [160, 161, 162]. Other studies rely on scalable heuristics, like greedy algorithms, to iteratively select workers who provide the highest utility per unit cost [163]. Despite their effectiveness in micro-task environments, these approaches fall short in scenarios such as collaborative repairs of property graphs. In our setting, tasks are not independent: each violation is part of a larger, interconnected structure, and repairing one can have cascading effects, resolving or introducing other violations. Existing models do not capture this violation dependency or the semantic interrelation between graph entities, which is central to our task and contributes to the challenge. On the task difficulty modeling side, prior work has focused on estimating task complexity using statistical models that separate worker reliability from inherent task hardness [159]. However, these methods require an initial seed to train the statistical models. Other works consider the cognitive load (e.g., the number of elements involved in the task) as the difficulty measure [165]. However, modeling the difficulty of repairing a graph requires evaluating the semantics of nodes and edges, the violations structure, and the potential side effects of a repair operation. Unlike previous work, our tasks are interdependent and demand reasoning over the neighborhood of given inconsistencies, rendering prior task assignment and difficulty modeling approaches unsuitable for our setting. The strategy developed in this chapter therefore combines the two threads: it uses a graph neural network to estimate the repair difficulty of each violation, and then pairs users with violations according to skill, capacity and cost. To our knowledge, such a targeted assignment has not been explored in prior work.

4.3 GNN Difficulty Annotation

To estimate the difficulty of repairing a violation, we propose to leverage a GNN trained on the property graph to predict the likelihood of individual graph components (nodes and edges) conforming to the learned structural and semantic patterns of the graph. Rather than designing a handcrafted heuristic, we follow a standard principle in graph anomaly detection: subgraphs that have low likelihood under the model’s learned joint distribution are anomalous and therefore easier to repair [166]. For instance, violation 20 in Figure 4.2 is caused by a person being coworker with another one even if working for a different company (e.g, Irene (node 1) is coworker of Li (node 5), but they work for company A and B respectively). However, Irene has multiple COWORKER_OF outgoing edges, each one consistently paired with a WORK_FOR edge (e.g.,

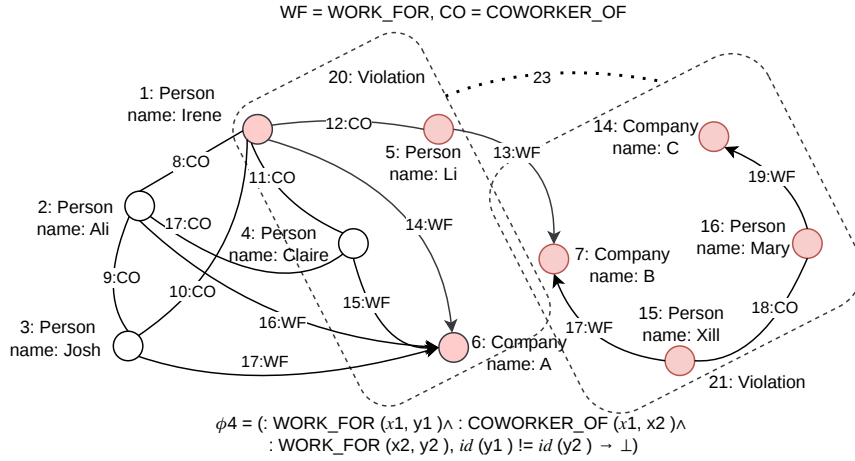


Figure 4.2: Example of a GRDG with two violations.

nodes 2, 3 and 4 work for company A). In this violation, edge 13 deviates from the local pattern and is therefore anomalous (it is the only case where there is a WF edge from a person to a company different from the company that their coworker works for). By contrast, in violation 21, each edge occurs only once, and no clear regularity emerges from the local neighborhood. As a result, violation 21 is less clearly attributable to a specific anomalous component, making it more difficult to repair.

In other words, violations that deviate significantly from learned graph-wide patterns will tend to receive lower confidence scores from the model, indicating lower repair difficulty. On the contrary, the more a subgraph is aligned with learned graph-wide patterns, the harder it is to fix and may require human expertise.

We adopt a spatial GNN architecture, namely the Heterogeneous Graph Attention Network (hetero-GAT) [57], as it naturally handles subgraphs of varying size and irregular topology. Its attention mechanism further emphasizes relevant neighbors, reducing unnecessary computation and highlighting meaningful relationships.

Because directly computing the joint likelihood of a heterogeneous subgraph is intractable for graph neural networks, we follow the common factorization approach used in prior work [167, 168] on GNN-based anomaly scoring and link prediction. In particular, we approximate the subgraph likelihood by independent node and edge factors, in line with standard independence assumptions used to derive edge likelihoods and node reconstruction scores in graph autoencoders [168, 169] and link prediction models [170]. Under this approximation, the confidence of a subgraph decomposes into the mean confidence of its edges (link prediction) and the mean confidence of its nodes (density-based estimates in the learned embedding space).

To obtain the edge confidence, we employ a hetero-GAT trained specifically for link prediction tasks, which produces edge-level scores by computing the probability of existence for each edge type between heterogeneous node types.

Unlike standard GNN-based anomaly detection, we explicitly tailor both the representation and the scoring mechanism to the repair task. In particular, we restrict node features to those

involved in the constraints via a feature filtering step. In the context of property graphs, nodes' features correspond to node property values.

Let $h_v \in \mathbb{R}^d$ denote the vector of property values associated with a node v . Constraints typically involve only a subset $\mathcal{K}' \subseteq \{1, \dots, d\}$ of these properties. To restrict node representations to constrained properties, we apply a linear projection $F \in \mathbb{R}^{|\mathcal{C}| \times d}$, which selects the components of h_v indexed by $\mathcal{K}'$. The resulting representation is then mapped into a shared feature space via a node-type-specific transformation: $h'_v = M_{T_v} F h_v$, where $M_{T_v} \in \mathbb{R}^{d' \times |\mathcal{C}|}$ is the transformation matrix associated with node type T_v .

Then, we compute the embedding z_v^T of a node v for an edge of type T aggregating the neighbor's projected features h'_u :

$$z_v^T = \sigma \left(\sum_{u \in N_v^T} a_{vu}^T \cdot h'_u \right)$$

where σ is the sigmoid activation function, N_v^T is the set of neighbors of node v via edges of type T , and a_{vu}^T is the normalized node-level attention for the edge type T . To be specific, $a_{vu}^T = \text{softmax}(\text{att}_{node}(h'_v, h'_u); T)$, where att_{node} is a deep neural network trained for node-level attention.

The advantage of using type-specific attention is that att_{node} is shared for all the edges of type T . More details about hetero-GAT models can be found in [57].

For an edge $e = (u, v)$ of type T , the **edge confidence** is:

$$c_{edge}(e) = \sigma \left(\text{MLP}_T \left([z_v^T \parallel z_u^T] \right) \right), \quad (4.1)$$

where z_v^T and z_u^T are the latent embeddings of nodes v and u extracted by the hetero-GAT, $[z_v^T \parallel z_u^T]$ denotes their concatenation, $\text{MLP}_T(\cdot)$ is a relation-specific feed-forward network, and $\sigma(\cdot)$ is the sigmoid function.

Moreover, we define the **node confidence** based on the proximity of a node's latent representation to other nodes of the same type in the latent space. Specifically, given a node v of type T_v , we define its node confidence as:

$$c_{node}(v) = \exp \left(- \text{avg}_{u \in N_{T_v} \setminus \{v\}} \left\| z_v^T - z_u^T \right\|_2 \right) \quad (4.2)$$

where N_{T_v} is the set of nodes of type T_v . The intuition is that nodes whose embeddings are closer to similar nodes (i.e., within the same type) in the learned space represent more "typical" or expected cases, while outliers will exhibit larger distances and lower confidence.

To estimate the overall difficulty of a violation, a subgraph of the property graph G , we integrate confidence scores of both nodes and edges belonging to it. The *estimated difficulty* of a violation $v = (N_v, E_v)$ is computed as the average of the two contributions:

$$\tilde{D}(v) = \text{avg} \left(\frac{1}{|N_v|} \sum_{v \in N_v} (c_{node}(v)), \frac{1}{|E_v|} \sum_{e \in E_v} (c_{edge}(e)) \right), \quad (4.3)$$

where N_v and E_v are the node and edge sets of the subgraph v respectively. With this definition, higher confidence for nodes and edges means a higher likelihood of existence for the subgraph and

thus higher difficulty. Finally, we annotate every hypernode of the GRDG (denoting a violation) with the estimated difficulty score.

4.4 Graph Repair Framework

We present our property graph repair framework, where a GNN-based module first estimates the difficulty of repairing each violation in a property graph. Second, a task assignment module assigns portions of the graph to users, leveraging the predicted difficulty scores, leaving easy violations for the GNN to repair first, following which users apply repairs tapping into their expertise in handling ambiguity, domain knowledge, and complex interdependencies between violations.

4.4.1 User Model

Different users vary significantly in their expertise, cognitive ability, and access to resources, which in turn affects their repair quality.

Our user model is motivated by two considerations grounded in prior work. (1) Tasks such as identification and resolution of graph violations impose a cognitive burden on users, which depends on both the structural complexity of the task and the user’s domain expertise, as shown by prior work in cognitive psychology [171] and graph data processing and visualization [172]. They show structural properties of data directly affect cognitive load and user performance. (2) Involving users incurs costs, be it monetary payments or expert’s time or an organization’s limited workforce. While standard models consider only skill and cost, graph repair tasks impose varying cognitive demands depending on both the size and structure of the violations. Hence, we explicitly model cognitive capacity (CC) as the maximum cumulative difficulty a user can handle, capturing workload limitations, not accounted for in prior task assignment models. We adopt a compact but expressive representation of users as a tuple (K, CC, c) , where $K \in [0, 1]$ denotes skill level, $CC \in \mathbb{R}^+$ captures cognitive capacity, and $c \in \mathbb{R}^+$ models cost. This formulation provides a flexible abstraction that reflects real-world dynamics: **Skill Level** (K): a higher skill level increases the likelihood of a user choosing a high-quality repair for a violation. E.g., medical experts can identify clinically meaningful resolutions more accurately than non-experts. W.l.o.g., we model the skill level as a scalar (e.g., assuming that a property graph models a single domain). The model easily extends to multidimensional skill level vectors, where each component of the vector models the skill level on a specific aspect of the domain. This requires a more expensive data collection in order to train a higher dimensional model. Our approach remains largely unchanged with higher dimensions.

Cognitive capacity (CC): This captures the maximum workload a user can sustain before performance degrades. Prior work models this effort via cognitive load functions based on graph structure [158], but such approaches ignore graph size: a large sparse graph may be less demanding than a small, densely connected one. We therefore adopt an operational notion of cognitive capacity, defined as the maximum cumulative difficulty a user can handle across assigned violations. As task number or difficulty increases, performance may deteriorate due to

overload, even for skilled users, making CC a critical constraint in our optimization. We assume that CC is user-provided as in platforms such as Mechanical Turk or Prolific.

Cost (c): Users come with different costs: e.g., crowdsourced workers may be cheaper but less skilled, than domain experts.

With this model, a domain expert (e.g., a medical doctor) may be modeled as ($K = 0.9, CC = 4.0, c = 5.0$), while a data scientist may be modeled as ($K = 0.6, CC = 8.0, c = 2.0$), indicating high domain skills and high cost but limited capacity to handle high cognitive load tasks for the domain expert, compared to the data scientist.

The parameters (K, CC, c) can be obtained following standard practices in crowdsourcing systems. The skill level K can be estimated either through questionnaires or by evaluating user performance on repair tasks having ground-truth answers. In some settings, user profiles may be updated online using behavioral metrics and prior performance [163]. The cost c can be either declared by the user or pre-defined by the platform, as commonly done in crowdsourcing settings. Finally, the cognitive capacity CC reflects the maximum workload a user is willing or able to sustain, and can be specified by the user (e.g., indicating how many tasks they are willing to complete), as in platforms such as Prolific [173].

After collecting users' data, we can train a user model to determine which transformation a user would apply to a given violation. Formally, given a user model (K, CC, c) , the probability of selecting a graph transformation T_i to repair a violation v is modeled as: $P_{T_i} = P(T_i | D_v, K)$, where D_v is the intrinsic difficulty of the violation, and K is the user's skill level. To train the model, it is possible to leverage known information, users' past experiences, or to collect an initial seed with a preliminary experiment (see § 4.5.1). Finally, we define:

Definition 4.1 (Expected Quality). *Given a violation v , a set of candidate transformations $\{T_i\}$ with outcome qualities $\{Q_i\}$, user skill level K , and user-specific selection probabilities $\{P_{T_i}\}$, the expected quality EQ of the user's interaction is defined as:*

$$EQ(D_v, K) = \sum_i P_{T_i} \cdot Q_i. \quad (4.4)$$

This function is component-wise monotonic: for a fixed difficulty D_v , the higher the user skill level, the higher the expected quality; for a fixed skill level K , the higher the difficulty, the lower the expected quality. With this notion of expected quality, we can estimate the performance of users on all the violations. Moreover, we can use the user model to solve conflicts that may manifest when multiple users repair overlapping violations, by applying only the repair done by the user with the highest skill level.

We note that, following common practice in the literature [160, 159], we adopt a static setting in which user capacities are fixed during assignment. Modeling temporal effects such as capacity recovery or user fatigue dynamics is an interesting direction for future work.

4.4.2 Problem Formulation

We formulate the problem of assigning dependent violations to users as a constrained graph covering problem. Consider a GRDG graph $\mathcal{G} = (V, F)$, where each node $v \in V$ represents a violation with an associated difficulty score $D_v \in \mathbb{R}_{\geq 0}$, and each edge $(v_i, v_j) \in F$ indicates

a dependency between violations, i.e., they are preferably handled together. The goal is to cover the graph using a subset of users drawn from a pool $U = \{u_1, u_2, \dots, u_N\}$. Each user u_i has a *cognitive capacity* $CC_i \in \mathbb{R}_{\geq 0}$, representing the total difficulty of violations they can handle, a skill level $K_i \in \mathbb{R}_{\geq 0}$, and a cost $c_i \in \mathbb{R}_{\geq 0}$. A user is assigned a group of violations, a connected subset of $GRDG$, whose total difficulty does not exceed their capacity.¹ To prevent conflicts, we assign only disjoint violation subgraphs to different users. As a result, repairs operate on non-overlapping portions of the graph, ensuring that concurrent updates do not interfere with each other. Combined with the safety property (see Section 3), this guarantees that repairs cannot invalidate or undo each other. Moreover, since all repairs are required to be safe, each applied transformation strictly reduces the number of violations without introducing new ones. Therefore, the repair process is guaranteed to terminate after we assigned all the violations. Finally, we have a budget $B \in \mathbb{R}_{\geq 0}$. The objective is to assign violations to a subset of users, maximizing the expected quality of the repairs while respecting the budget constraint and ensuring that each user is assigned a connected subgraph with total difficulty not exceeding their cognitive capacity.

Definition 4.2 (Budget-Constrained GRDG Covering Problem). *Given a GRDG graph $\mathcal{G} = (V, F)$ with node difficulties D_v and a set of users $U = \{u_1, \dots, u_N\}$, each user u_i having a skill value K_i , cognitive capacity CC_i , and cost c_i , and a total budget $B \in \mathbb{R}_{\geq 0}$, find a subset of users $U^* \subseteq U$ and an assignment of disjoint subgraphs $\{P_i \subseteq V \mid u_i \in U^*\}$ such that:*

$$U^* = \arg \max_{U' \subseteq U} \sum_{u_i \in U'} \sum_{v \in P_i} EQ(D_v, K_i) \quad (4.5)$$

subject to:

$$\text{Each } P_i \text{ is connected,} \quad (4.6)$$

$$\sum_{v \in P_i} D_v \leq CC_i \quad \forall u_i \in U', \quad (4.7)$$

$$P_i \cap P_j = \emptyset \quad \forall u_i \neq u_j, \quad (4.8)$$

$$\bigcup_{u_i \in U'} P_i = V, \quad (4.9)$$

$$\sum_{u_i \in U'} c_i \leq B. \quad (4.10)$$

where $EQ(D_v, K_i)$ is a component-wise monotonic function defined in Section 4.4.1, Equation (4.4), estimating the expected quality of a user u_i with skill level K_i on a violation with difficulty D_v .

The above formulation captures the key aspects of the violation assignment task. Constraint (1) ensures that each user is assigned a connected subgraph of violations, preserving local context and facilitating coherent resolution. Constraint (2) guarantees that the total difficulty of assigned violations does not exceed the user's cognitive capacity, reflecting individual workload limits. Constraint (3) enforces disjointness in the assignment, ensuring each violation is handled by

¹We assume that we have sufficiently many users.

at most one user. Constraint (4) ensures that the assignment covers all necessary violations. Finally, constraint (5) is the budget constraint.

Theorem 4.1. *The Budget-Constrained GRDG Covering Problem (bud-GCP) is NP-hard.*

Proof. We reduce the classical Multiple Knapsack problem (MKP) to bud-GCP. An instance of MKP consists of items $I = \{1, \dots, n\}$, where item i has weight $w_i \geq 0$ and profit $p_i \geq 0$, and knapsacks $M = \{1, \dots, m\}$, where knapsack j has capacity $C_j \geq 0$. In the decision version of MKP, given a threshold P , the question is whether there exists a packing of each item into at most one knapsack respecting the capacity constraints such that the total profit of the packing is at least P .

Given an instance $\mathcal{I}$ of MKP, we create an instance $\mathcal{J}$ of bud-GCP as follows: create a node v_i for each item $i \in I$ and set $V = \{v_1, \dots, v_n\}$. Construct a complete graph $G = (V, F)$.

For each node v_i assign the difficulty $D_{v_i} := w_i$. Create a user u_j for each knapsack $j \in M$ and set its cognitive capacity to $CC_j := C_j$. We also create a dummy user u_0 with very large capacity, say $CC_0 := \sum_{i \in I} w_i$. Set $K_j = 1$ for $j \geq 1$ and $K_0 = \varepsilon \in (0, 1)$. Set the user costs $c_i := 1$, $i \geq 0$ and set the budget to $B := m + 1$.

Define the expected-quality function as

$$EQ(D_{v_i}, K_j) = \begin{cases} p_i, & \text{if } K_j = 1, \\ 0, & \text{if } K_j < 1. \end{cases}$$

It is easy to verify that this definition is component-wise monotone and that this reduction is polynomial time in n and m .

We claim that $\mathcal{I}$ is a YES-instance of MKP, i.e., it admits a valid packing of items into knapsacks with total profit at least P subject to budget B iff $\mathcal{J}$ admits a valid assignment of users to violations that covers the entire graph, whose total quality is at least P and the total cost is at most B .

Let $\pi : I \rightarrow M$ be a partial function denoting the packing of a subset of items I into the m knapsacks for the MKP instance $\mathcal{I}$. We can construct a corresponding solution to the bud-GCP instance $\mathcal{J}$: assign violation node v_i to user u_j whenever $\pi(i) = j$; for each unassigned item i , assign the violation v_i to the dummy user u_0 , i.e., assign a violation v_i to the dummy user u_0 whenever $\pi(i)$ is undefined. Let P_j denote the set of violations assigned to user u_j . Since G is complete, each assigned part P_j is connected, so (4.6) holds. The cognitive-capacity constraints (4.7) hold since $\sum_{v \in P_j} D_v = \sum_{i \in P_j} w_i \leq C_j = CC_j$, $j \in \{1, \dots, m\}$. Disjointness (4.8) follows from the fact that each item is placed in at most one knapsack and thus each violation is assigned to at most one user. By construction, the assignment covers all violations, satisfying (4.9). The budget constraint (4.10) holds by our choice of costs and B . By construction, the bud-GCP objective value equals the MKP profit P .

Conversely, consider a feasible solution for the instance $\mathcal{J}$ of bud-GCP with quality Q and cost $\leq B$. We obtain a feasible MKP item packing as follows: for each user u_j , $j \geq 1$, place item i into knapsack j iff $v_i \in P_j$, i.e., $\pi(i) = j, \forall v_i \in P_j$. Constraints (4.7) and (4.8) guarantee that the knapsack capacities are respected and that no item appears in more than one knapsack. Violations assigned to the dummy user, if any, correspond to items left unassigned in MKP.

Removing the unassigned nodes doesn't reduce the profit of the item packing since the dummy user contributes 0 quality (and thus 0 profit). Thus, the items assigned to non-dummy users form a feasible MKP packing whose total profit is Q . The theorem follows. $\square$

4.4.3 Automated Repairs with GNN

We extend the use of GNNs beyond difficulty estimation by integrating them directly into the repair process. While some violations require human expertise due to ambiguity or domain-specific knowledge, others arise from edges whose existence is highly unlikely given the structure and semantics of the violation subgraph. Involving users in such cases is unnecessary and inefficient. Specifically, we propose a hybrid repair framework in which the model automatically resolves easy violations based on confidence scores, while delegating harder cases to human experts. Our intuition is that we can leverage the GNN to automatically repair some violations, essentially deleting low-confidence edges, thereby reducing the number of violations to be assigned to humans. Concretely, for a violation $v = (N_v, E_v)$ and a threshold $\theta \in [0, 1]$, we remove an edge $e \in E_v$ whenever its uncertainty $1 - c_{\text{edge}}(e)$ exceeds θ . If multiple edges satisfy this condition, we perform a minimal repair by deleting only the edge with the highest uncertainty (breaking ties at random). These GNN Repairs can be applied synchronously or asynchronously w.r.t. user repairs. For any fixed threshold θ , the relative order of GNN and human repairs does not affect the quality outcome.

Lemma 4.2 (Commutativity of GNN and Human Repairs). *Given a violation assignment strategy AS that respects the constraints of problem 4.2, we define V_θ as the set of violations that would be automatically solved using GNN by using $1 - \theta$ as the threshold. Applying the GNN repairs before running AS yields the same result as applying GNN repairs after or during running AS on the GRDG $\mathcal{G} = (V \setminus V_\theta, F)$.*

Proof Sketch. By construction, GNN repairs consist exclusively of deleting edges belonging to violations whose edge confidence falls below the threshold $1 - \theta$ (i.e., they do not affect nodes). Since dependencies in the GRDG are created only from shared nodes, and node sets are unaffected, GNN repairs neither alter violation overlaps nor introduce new violations. Hence, the user assignment as well as the total expected quality of repairs conducted by users on the V_θ violations won't change if the GNN repairs are applied before, after or even during the AS assignment. $\square$

The threshold θ is the central parameter that controls the tradeoff between automated and manual repairs. Intuitively, large values of θ make the system conservative, delegating most violations to human experts, while smaller values increasingly favor automation by removing low-confidence edges, by raising the threshold for what is deemed “low confidence”. As a result, our approach is fully tunable: practitioners can select θ to explicitly balance repair quality, human effort, and computational cost according to application requirements and available budget. Decreasing θ monotonically increases the number of violations resolved automatically by the GNN, and correspondingly reduces the size of the GRDG passed to the assignment strategy. This reduction directly translates into fewer involved users and a smaller optimization problem, yielding significant improvements in runtime and scalability (Section 4.4.4).

4.4.4 Lagrangian-Relaxation Assignment

Given its NP-hardness, we solve the budget-constrained GRDG covering problem (Definition 4.2) with a Lagrangian-relaxation framework [174]. Intuitively, Lagrangian relaxation replaces the hard budget constraint by a soft penalty: a multiplier λ charges a price for each unit of budget consumed. For a fixed λ , the problem decomposes into independent user subproblems, each deciding whether the value of covering a partition outweighs its penalized cost. This relaxation is easier to solve than the original problem, and by adjusting λ with subgradient updates, we balance solution quality and budget usage.

Lagrangian Relaxation of the Budget Let B be the total budget and c_i the cost of user u_i . We introduce a multiplier $\lambda \geq 0$ and relax the budget constraint $\sum_{u_i \in U'} c_i \leq B$:

$$\mathcal{L}(\lambda) = \max_{\substack{\{P_i\} \text{ connected,} \\ \sum_{v \in P_i} D_v \leq CC_i, \\ P_i \cap P_j = \emptyset, \bigcup_i P_i = V}} \sum_{u_i} \left(\sum_{v \in P_i} EQ(D_v, K_i) - \lambda c_i \right) + \lambda B. \quad (4.11)$$

For any $\lambda \geq 0$, $\mathcal{L}(\lambda)$ sets the upper-bound of the optimal value of the objective function of the *primal problem* (e.g. the original budget-constrained covering problem) and any feasible solution to it yields a *lower bound* on the optimal objective. The *dual problem* is to minimize the Lagrangian function (4.11) over $\lambda \geq 0$, whose value provides an *upper bound* since relaxing the constraint enlarges the feasible set. The gap between these primal and dual bounds quantifies how close the relaxation is to the true optimum.

For a fixed λ , user u_i contributes with a gain

$$\Phi_i(P_i; \lambda) =_{\text{def}} \sum_{v \in P_i} EQ(D_v, K_i) - \lambda c_i,$$

with P_i being a connected subgraph assigned to u_i and $\sum_{v \in P_i} D_v \leq CC_i$. Since users' partitions must be disjoint and cover V , we work with a set of candidate subgraph assignments per user and then perform a final selection among the candidates.

Per user assignment Unlike classical assignment problems where tasks are independent, our setting requires assigning connected subgraphs under structural dependencies. This fundamentally changes the optimization problem and motivates our seed-based candidate generation. Algorithm 3 shows the procedure generating the candidate assignments. For each user u_i , we build a small set $\mathcal{S}_i$ of connected subgraphs $P \subseteq V$, with $\sum_{v \in P} D_v \leq CC_i$. We first identify the violation v that produces the highest gain $g_i(v) \leftarrow EQ(D_v, K_i)$ for the user u_i , then we greedily grow the subgraphs by including nodes that maximize the gain (lines 2-6). Finally, we perform t steps of local improvement by swapping neighboring nodes, if the user's capacity allows it (line 7). We note that in practice, the algorithm uses *estimated* difficulty $\tilde{D}$, but just write D for simplicity.

Assignment under a budget Algorithm 4 shows how to select the candidate subgraphs for each user. Given the set of candidate subgraphs $\{\mathcal{S}_i\}$ for each user u_i , we now select which users to activate and which single assignment each of them will take. We proceed greedily. Let

Algorithm 3 BuildCandidatesSet(i): candidate subgraphs for u_i

Require: GRDG $\mathcal{G} = (V, F)$, $g_i(v) \leftarrow EQ(D_v, K_i)$, capacities CC_i , difficulties D_v
Ensure: A set $\mathcal{S}_i$ of candidate subgraphs.

```

1:  $\mathcal{S}_i \leftarrow \emptyset$ 
2: for each violation  $v$  in Top- $V$  vertices by  $p_i(v)$  do
3:    $P \leftarrow \{v\}$ 
4:   while there exists  $u \in \text{Neighbors}(P)$  with  $\sum_{v \in P} D_v + D_u \leq CC_i$  and  $EQ(D_u, K_i) > 0$ 
     do
5:     add to  $P$  the  $u$  with max  $g_i(u)$ 
6:   end while
7:   Apply  $t$  steps of local improvement (swap/add/drop neighbors if feasible)
8:    $\mathcal{S}_i \leftarrow \mathcal{S}_i \cup \{P\}$ 
9: end for
10: return  $\mathcal{S}_i$ 

```

A be the set of violations already assigned (empty in the beginning). For any pair (i, P) with $P \in \mathcal{S}_i$, we compute its current marginal gain as $\Delta g(i, P) = \sum_{v \in P \setminus A} EQ(D_v, K_i) - \lambda c_i$, i.e., the expected quality contributed by solving violations in P that are not yet covered (line 4). At each step we choose the feasible pair with the largest ratio $\Delta g(i, P)/c_i$ (line 6). We add P to the assignment, update $A \leftarrow A \cup P$, and remove (i) all other candidates of user i (at most one per user) and (ii) any pair (j, P') with $P' \cap A \neq \emptyset$ (to avoid overlaps). We stop when the budget $\sum_{i \in U'} c_i$ would be exceeded or when no positive-gain pair remains.

Algorithm 4 BudgetAssignment($\{\mathcal{S}_i\}, B, \lambda$)

Require: Sets of candidates assignments $\mathcal{S}_i$, Budget B , multiplier λ .

Ensure: The best subgraph assigned to each user.

```

1:  $U' \leftarrow \emptyset$ ,  $A \leftarrow \emptyset$ ,  $\mathcal{I} \leftarrow \{(i, P) : i \in U, P \in \mathcal{S}_i\}$ 
2: while  $\exists (i, P) \in \mathcal{I}$  with  $c_i + \sum_{j \in U'} c_j \leq B$  do
3:   for each feasible  $(i, P)$  with  $P \cap A = \emptyset$  do
4:     compute  $\Delta g(i, P) = \sum_{v \in P} g_i(v) - \lambda c_i$ 
5:   end for
6:   Select  $(i^*, P^*) = \arg \max_{(i, P)} \frac{\Delta g(i, P)}{c_i}$  with  $\Delta g(i, P) > 0$ 
7:   if no positive-gain item exists then
8:     break
9:   end if
10:   $U' \leftarrow U' \cup \{i^*\}$ 
11:  assign  $P_{i^*} \leftarrow P^*$ ,  $A \leftarrow A \cup P^*$ 
12:  Remove all items  $(i^*, \cdot)$  and all items overlapping  $A$  from  $\mathcal{I}$ 
13: end while
14: return  $(U', \{P_i\}_{i \in U'})$ 

```

Full algorithm A key challenge is that the inner solver (Algorithm 4) is a greedy priced selector (i.e. $\Delta g(i, P) = \sum_{v \in P} g_i(v) - \lambda c_i$). This solver does not come with a worst-case guarantee on the Lagrangian objective $\mathcal{L}(\lambda)$. We cannot guarantee an upper bound of the optimal solution because we are optimizing the total priced gain $\sum_i \left(\sum_{v \in P_i} EQ(D_v, K_i) - \lambda c_i \right)$, and not a pure monotone sub-modular one (like in a sub-modular knapsack problem [175]).

We therefore certify solution quality with a decoupled dual upper bound. Conceptually, we relax all the constraints on the assignment (overlaps, “one-per-user”, and coverage) and let each user contribute at most their best candidate value in isolation. For each user i , let $\text{Val}_i = \max_{P \in \mathcal{C}_i} \sum_{v \in P} EQ(D_v, K_i)$ be the best value they could achieve in isolation on a candidate subgraph. The decoupled upper bound is:

$$UB^{\text{dec}}(\lambda) = \sum_{i=1}^N \max\{0, \text{Val}_i - \lambda c_i\} + \lambda B, \quad \lambda \geq 0,$$

This relaxation is guaranteed to sit above the true Lagrangian and therefore above the optimal objective, so it provides a valid certificate regardless of how the priced greedy selector behaves. A key practical advantage is that this bound admits a closed-form minimizer via a simple breakpoint computation after sorting users by value-per-cost; it is inexpensive, deterministic, and independent of implementation details (e.g., tie-breaking or step sizes).

To compute the certified upper bound, we first take, for each user i , their best candidate P_i with value Val_i and compute the ratio $r_i = \text{Val}_i / c_i$ where c_i is the cost of the user. We sort users by r_i in decreasing order, carrying along (Val_i, c_i) , and define the prefix sums $C_k = \sum_{j \leq k} c_{(j)}$ and $Sg_k = \sum_{j \leq k} g(j, P_j)$. If the budget covers all users ($B \geq C_N$), the minimizer is $\lambda^* = 0$ and the certificate is $UB_{\text{cert}} = \sum_i g(i, P_i)$. Otherwise, letting k^* be the first index with $C_{k^*} \geq B$, we set $\lambda^* = r_{(k^*)}$ obtaining $UB_{\text{cert}} = Sg_{k^*-1} - \lambda^* C_{k^*-1} + \lambda^* B$.

The decoupled bound can be written as $UB^{\text{dec}}(\lambda) = \sum_{i: r_i > \lambda} (\text{Val}_i - \lambda c_i) + \lambda B$, which is convex and piecewise linear in λ with kinks exactly at the ratios $\{r_i\}$. Between two consecutive ratios, the slope equals $B - \sum_{i: r_i > \lambda} c_i$, i.e., the budget minus the total cost of users whose ratio exceeds the current threshold. As λ increases, this cumulative cost drops in steps when we pass a ratio, so the minimum occurs when the slope first changes sign—precisely at the smallest prefix whose cost reaches the budget. Plugging $\lambda^* = r_{(k^*)}$ and separating the top $(k^* - 1)$ users yields the closed form above; the case $B \geq C_N$ corresponds to $\lambda^* = 0$.

Algorithm 5 shows the LARA, the full LAgrangian Relaxation Assignment algorithm. We first run `BUILDCANDIDATESSET` to obtain the assignment candidates for each user (lines 2-4). We then compute the certified upper bound with the method described above before running the outer loop for T iterations. In the loop, we call `BUDGETASSIGNMENT` at price λ obtaining a feasible assignment $U'(\lambda)$ with value Z (the primal) and cost $\text{cost}(\lambda) = \sum_{i \in U'(\lambda)} c_i$. We then compute the dual upper bound UB_t at iteration t (lines 7-9). We maintain the best feasible solution $(U^*, \{P_i^*\})$ with value Z^* seen so far, and a running dual upper bound $UB = \min_t UB_t$ (lines 10-13). We then update the Lagrange multiplier by a projected subgradient step using the difference between the cost and the budget: $\lambda \leftarrow \max\{0, \lambda + \alpha_t((\sum_{i \in U'} c_i - B))\}$, so λ increases when the assignment exceeds the budget and decreases otherwise (typical choices are diminishing steps $\alpha_t = \alpha_0 / \sqrt{t}$).

We track the best feasible value Z_t returned by the greedy assignment and stop once the certified gap $1 - \frac{Z_t}{UB_{\text{cert}}} \leq \varepsilon$.

The outer loop stops if the multiplier stabilizes ($|\lambda_t - \lambda_{t-1}| \leq \varepsilon_\lambda$ for k iterations) or if the maximum T is reached.

Algorithm 5 LARA($B, \{EQ, CC_i, c_i\}$)

```

1: Initialize  $\lambda \geq 0$ , step sizes  $\{\alpha_t\}$ , best feasible solution  $(U^*, \{P_i^*\}) \leftarrow \emptyset$ , best primal value  $Z^* \leftarrow -\infty$ , best dual upper bound  $UB \leftarrow +\infty$ 
2: for each user  $i$  do
3:    $\mathcal{C}_i \leftarrow \text{BuildCandidatesSet}(i)$ 
4: end for
5:  $(UB_{\text{cert}}, \hat{\lambda}) \leftarrow \text{BreakpointUB}(\{\text{Val}_i\}, \{c_i\}, B)$ 
6: for  $t = 1, 2, \dots, T$  do
7:    $(U', \{P_i\}) \leftarrow \text{BudgetAssignment}(\{\mathcal{C}_i\}, B, \lambda)$ 
8:    $Z \leftarrow \sum_{i \in U'} \sum_{v \in P_i} EQ(D_v, K_i)$ 
9:    $UB_t \leftarrow \left( \sum_{i \in U'} \sum_{v \in P_i} EQ(D_v, K_i) - \lambda \sum_{i \in U'} c_i \right) + \lambda B$ 
10:  if  $Z > Z^*$  then
11:     $(U^*, \{P_i^*\}, Z^*) \leftarrow (U', \{P_i\}, Z)$ 
12:  end if
13:   $UB \leftarrow \min\{UB, UB_t\}, \lambda_{\text{old}} \leftarrow \lambda$ 
14:   $\lambda \leftarrow \lambda + \alpha_t \left( \sum_{i \in U'} c_i - B \right)$ 
15:   $s \leftarrow \begin{cases} s + 1, & \text{if } |\lambda - \lambda_{\text{old}}| \leq \varepsilon_\lambda \\ 0, & \text{otherwise} \end{cases}$ 
16:  if  $\frac{UB_{\text{cert}} - Z^*}{UB_{\text{cert}}} \leq \varepsilon_{\text{gap}}$  or  $s \geq k$  then
17:    break
18:  end if
19: end for
20: return  $(U^*, \{P_i^*\})$ , LowerBound =  $Z^*$ , UpperBound =  $UB$ 

```

Guarantees

To assess the quality of the solutions returned by LARA, we need two ingredients. First, we show that the decoupled relaxation introduced in the previous section provides a valid dual upper bound on the optimal objective, regardless of the greedy solver behavior. Second, we use this bound to certify that whenever the algorithm terminates by the certified stopping rule, the returned solution has a relative optimality gap at most ε with respect to this dual upper bound. That is, the algorithm certifies near-optimality with respect to the Lagrangian relaxation, though not necessarily w.r.t. the original NP-hard problem. The following theorem formalizes the validity of the certified upper bound.

Theorem 4.3 (Certified Upper bound). *For all $\lambda \geq 0$, $UB^{\text{dec}}(\lambda) \geq \mathcal{L}(\lambda) \geq \text{OPT}$, hence $UB_{\text{cert}} \equiv \min_{\lambda \geq 0} UB^{\text{dec}}(\lambda)$ is a valid dual upper bound on the optimal value of the original problem.*

Proof sketch. $UB^{\text{dec}}(\lambda)$ drops all the constraints for the assignments, enlarging the feasible set of the inner maximization that defines the Lagrangian. This can only increase the value, so $UB^{\text{dec}}(\lambda) \geq \mathcal{L}(\lambda)$. The weak duality gives $\mathcal{L}(\lambda) \geq \text{OPT}$. $\square$

Given the certified dual upper bound UB_{cert} from Theorem 4.3, we monitor the best feasible primal value Z_t . If $1 - \frac{Z_t}{UB_{\text{cert}}} \leq \varepsilon$, then Z_t is guaranteed to be within a relative gap ε of this dual bound, i.e., $Z_t \geq (1 - \varepsilon) \cdot UB_{\text{cert}}$. This provides a computable certificate of near-optimality with respect to the dual bound. In § 4.5.3 we show how different numbers of GNN repairs affect the achieved value and the certified gap with respect to UB_{cert} .

Time complexity.

Let $V := |V|$ be the number of nodes in the GRDG, $U := |U|$ the number of users, and T the number of outer Lagrangian iterations actually executed. The dominant cost is BUILD CANDI-

Table 4.1: Dataset statistics.

Dataset	# Nodes	# Edges	# Viol.	# Dep.	Density	Avg Diff.
FAERS	~ 11K	~ 62K	~ 2.4K	~ 127K	0.0220	0.7595 (0.1139)
Finbench	~ 56K	~ 311K	~ 9K	~ 31K	0.0003	0.7666 (0.1395)
ICIJ	~163K	~364K	~ 33K	~ 24M	0.0220	0.8100 (0.1135)
SNB	~ 358K	~728K	~ 70K	~ 5.9M	0.0120	0.7241 (0.0955)

DATESSET, which, for each user, grows seed candidate sets and performs a few local-improvement steps. Up to constants, the complexity depends on how many neighbors each candidate set can see and how many seed sets and local steps we allow. Formally, the hidden constants in our bounds are controlled by: (i) the graph’s average degree Δ , (ii) the typical candidate size X , (iii) the number of seeds per user S , and (iv) the local-search steps t . Larger Δ , S , t , or X make candidate construction heavier, while the assignment phase remains roughly linear in the total size of all candidates and is not the bottleneck.

Worst case. In dense graphs ($\Delta = \Theta(|V|)$) with many seeds and long local search ($S, t = \Theta(|V|)$) and large candidate sets ($X = \Theta(|V|)$), the per-user time is $\Theta(|V|^4)$, hence $T_{\text{total}} = O(T |U| |V|^4)$.

In practice. GRDGs are sparse, as shown in Table 4.1. The number of violations is strictly lower than the number of nodes [116, 45], and hence also the number of overlaps (i.e., number of dependency edges). With sparse GRDGs ($\Delta = O(1)$), capped seeds and local steps ($S, t = O(1)$) and candidate subgraphs much smaller than the graph ($X \ll |V|$), the per-user cost is near-linear, yielding $T_{\text{total}} \approx O(T |U| |V| \log |V|)$, with small constant factors. Intuitively: bigger or denser graphs, more seeds, and longer local search increase the constant; sparse graphs and capped neighborhoods keep the runtime close to linear in V per user per outer iteration. Early stopping simply replaces T by the observed $T' \leq T$. For a more detailed complexity analysis, refer to the Appendix B.1. Notice that this corresponds to the most common scenarios as we show in Table 4.1 and it is also reflected in the results of our experimental study (see § 4.5.3).

4.5 Experimental Evaluation

In this section, we experimentally evaluate the effectiveness of our approach over different datasets and settings. We used Neo4j [140] (version 5.12) as the underlying graph database, while the GNN was implemented using PyTorch. All materials needed to reproduce the results and train the GNN are publicly available [176].

4.5.1 Settings

Datasets

The datasets used in our experiments are illustrated in Table 4.1 and include **FAERS** [177] which contains real world data from a food and drug adverse event reporting system, **Finbench** [178], a synthetic dataset that simulates real-world financial transactions, **SNB** [179], a synthetic dataset that simulates a realistic social network scenario, with users, posts, comments and likes. Additionally, we used **ICIJ** [148], a real-world dataset with the investigation on Paradise Papers led by the International Consortium of Investigative Journalists. We manually derived real world

denial constraints for each dataset using different pattern lengths. As previously done in the literature [116, 45, 151, 13] we introduced $\sim 20\%|V|$ violations in each dataset, where $|V|$ is the number of nodes in the graph. The list of constraints is reported in Appendix B.2.

Behavior of the Simulated Users

We model user behavior as described in Section 4.4.1, where each user is represented as a tuple (K, CC, c) , denoting their skill, cognitive capacity, and cost respectively. We created the users by drawing from a uniform probability their value of skill $K \in [0, 1]$, cognitive capacity $CC \in [5, 20]$, and the cost of each user $c \in [1, CC - 1]$ is computed as a linear function of the skill: the more skilled a user is, the more it costs.

To validate the model and ensure it captures real-world behavior, we conducted a user study with 20 users repairing the *Star Wars* dataset [143]. Participants first self-assessed their domain knowledge (familiarity with Star Wars) and technical skill (experience with graph data structures) on a Likert scale from 1 (none) to 5 (expert). These values were normalized and mapped to the user-model parameter K . Then, participants were presented with 6 distinct graph violations extracted from the dataset. For each one, they were asked to select a repair from a predefined list of candidate repairs. The details are in Appendix B.2.

We collected success/failure outcomes for each user–violation pair. We then applied a 1-Parameter Logistic Item Response Theory model [180] to estimate the latent difficulty of each violation. These ground-truth difficulty values were then used to compute the users’ perceived difficulties for each dataset, as a linear combination of node and edge confidences. These perceived difficulties are used as input to our user model in order to simulate user behavior.

Since training a user model to predict the exact repair a user would choose requires a large amount of data, we decided instead to predict the *class* of a repair according to their quality (F1 score). Different repairs indeed lead to the same F1 score (e.g. all the repairs corresponding to the ground truth lead to $F1=1$ or all the repairs that delete the wrong edge lead to $F1=0$, etc.). We trained a Naive Bayes classifier to predict the most likely repair based on user skill and the violation’s perceived difficulty. While assignments rely on *estimated* difficulty, user behavior is driven by *perceived* difficulty. Lastly, to ensure all the violations are repaired, even in the worst case (i.e., each user can repair only one violation), we consider the initial number of users equal to the number of violations. This choice is an experimental control to ensure that all compared methods are evaluated under identical conditions. It does not imply that all users are used by the repair process: the assignment algorithm selects only a subset of users subject to the budget and capacity constraints.

4.5.2 Validating the difficulty estimation

This experiment aims to validate our GNN confidence based difficulty estimate (**ConfGNN**), obtained using the method described in § 4.3, by comparing it with alternative heuristics.

Specifically, we consider two families of baselines: structural heuristics such as node **Degree** and **PageRank**, assuming the GRDG topology influences task difficulty as in [13], and cognitive load (**CL**), computed as a simple function of the number of entities and relations involved in a violation, following the approach used in crowdsourcing [159]. For the **PageRank** and **Degree**

baselines, we replace the **ConfGNN** difficulty score with a topology-based score computed from the underlying graph. Specifically, we aggregate node-level PageRank (resp. degree) scores over the nodes involved in each violation to obtain a scalar difficulty value, which is then used unchanged as input to the assignment algorithm. The second family belongs to the anomaly detection score-based methods. **Radar** [181] is residual analysis based anomaly detection method for attributed networks. **HO-GAT** [182] is a GAT based method that combines attributes information with structural properties of the graph. **SmoothGNN** [183] is a smoothing aware unsupervised GNN method for graph anomaly scoring. For this experiment, all other components of the pipeline, including the user model, assignment strategy, and repair process, remain unchanged.

First, we evaluated the accuracy of each estimate by comparing it to the real difficulty scores, using the Mean Squared Error (MSE) and the Kullback-Leibler Divergence (which measures the relative entropy between distributions). We use KL-divergence to check that the predicted difficulty distribution matches the true one, not just pointwise accuracy. This matters because rare high-difficulty violations disproportionately affect assignment quality, so preserving them in the predicted distribution is critical. As shown in Table 4.2, our method yields the most accurate estimates, indicating that indeed the **GNN** approach blends topology and the semantics of the graph to better estimate the difficulty of the violations. While **SmoothGNN** achieves lower MSE on two datasets, **ConfGNN** attains lower KL-Divergence, better capturing outliers. Second, we ran the full graph repair pipeline to assess how each difficulty estimate affects the assignment of violations and the resulting repair quality. To isolate the effect of the difficulty estimate, we disabled the GNN’s edge pruning capability by setting the pruning threshold to 1.0, ensuring that no edges would be removed (see § 4.4.3). Each difficulty function was integrated into our assignment algorithm, and we evaluated the resulting repairs using the **F1 Score**, and Mean Reciprocal Rank (MRR). While the **F1 Score** measures the correctness of the final repairs, the MRR evaluates how highly the correct repair action is ranked for each violation. Table 4.2 shows that our estimator consistently achieves the best overall performance across datasets and metrics. In particular, **ConfGNN** obtains the highest MRR scores in nearly all cases, indicating that it not only improves the final repair quality, but also produces substantially better repair rankings during assignment. **CL** is the second-best method for **FAERS**, but it fails to deliver good results on the larger datasets, suggesting that estimations based solely on the size of the violations are insufficient to capture their difficulty. **Degree** and **PageRank** fail to achieve satisfactory results, indicating that the local topology alone does not correctly capture the complexity of repairing a graph. Moreover, **Radar** and **HO-GAT** do not scale to larger datasets due to prohibitive matrix operations and feature expansion, respectively.

Table 4.2: Comparison of our method with different difficulty estimations. ME = Memory Error. Bold indicates the best. FinB.=Finbench

Metric	Mean Square Error				KL-divergence				F1 Score				MRR			
	FAERS	FinB.	ICIJ	SNB	FAERS	FinB.	ICIJ	SNB	FAERS	FinB.	ICIJ	SNB	FAERS	FinB.	ICIJ	SNB
CL	0.059	0.146	0.118	0.178	0.229	2.777	0.252	8.313	0.923	0.944	0.596	0.515	0.820	0.805	0.917	0.346
Degree	0.180	0.216	0.210	0.151	4.044	6.059	3.955	2.975	0.677	0.511	0.592	0.533	0.820	0.466	0.516	0.374
PageRank	0.057	0.116	0.086	0.038	3.201	5.865	1.772	1.423	0.678	0.437	0.406	0.475	0.820	0.718	0.754	0.918
Radar	0.098	0.213	ME	ME	0.068	0.336	ME	ME	0.570	0.814	ME	ME	0.607	0.482	ME	ME
HO-GAT	0.029	0.033	ME	ME	0.063	0.037	ME	ME	0.844	0.648	ME	ME	0.829	0.942	ME	ME
SmoothGNN	0.064	0.027	0.016	0.030	0.080	0.016	0.028	0.037	0.571	0.995	0.972	0.944	0.782	0.942	0.988	0.511
ConfGNN	0.024	0.093	0.063	0.019	0.003	0.004	0.003	0.002	1	0.9996	0.9996	0.9994	0.962	0.996	0.9991	0.9994

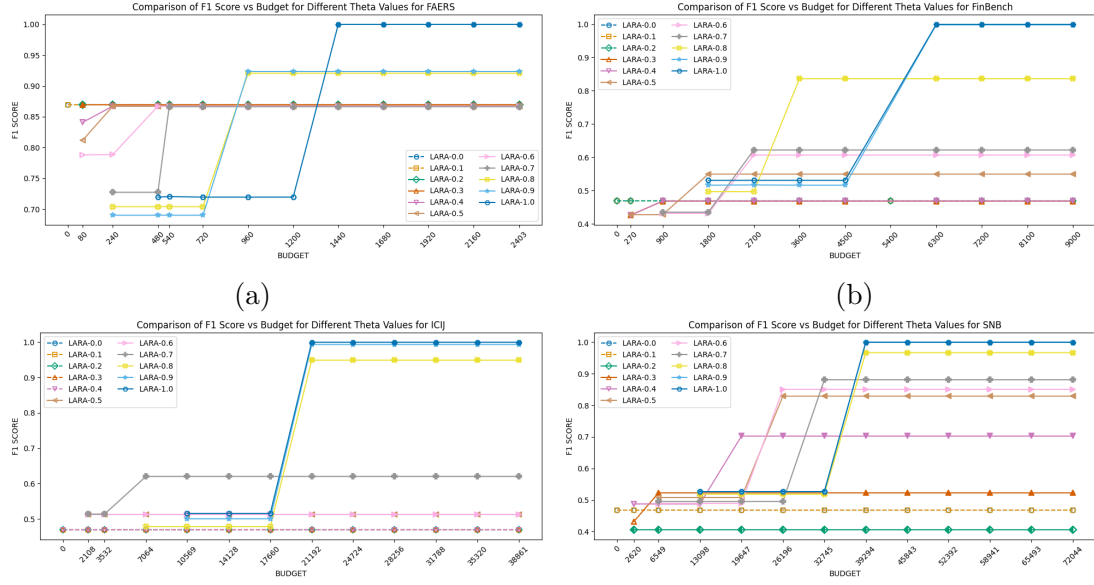


Figure 4.3: F1-Score of our approach using different θ across different datasets at different budget values.

4.5.3 Balancing human involvement

In this experiment, we analyzed the influence of GNN involvement and the available budget on the final repair quality.

We vary θ from 0 (the GNN performs all the repairs) to 1 (only humans repair the graph). The budget depends on the dataset, and it varies from 0 (no human can be involved), to the amount necessary to involve all the humans in the repair of all the violations.

Figure 4.3 illustrates the results. Each graph shows the F1 score as a function of the budget, with different lines corresponding to the employed θ values (e.g., LARA-0.1 corresponds to $\theta = 0.1$). The reported F1 score is computed as the average between the scores obtained by the GNN and the users. The dashed lines represent the values of θ where the GNN repairs all the violations. We vary the budget from 0 to maximum to cover all the users. Each point thus represents a run of the assignment with that budget.

We can observe a common behavior across the different datasets where the charts are divided into four regions. The plot exhibits four regions. The infeasible area contains missing points, corresponding to settings where the budget and GNN involvement are insufficient to assign all violations. This occurs rarely and only at higher θ values requiring more human involvement (e.g., the first two or three steps for LARA-0.8). The feasible region is where all violations are assigned within the available budget, but the more expert (and costly) users cannot be involved. The inflection point marks the budget needed to achieve the highest-quality assignment, where high-skilled users become affordable. Finally, the plateau area is where additional budget no longer improves repair quality; solutions here always fall within the optimality gap (shown as filled dots on the line, versus empty dots for solutions outside the gap).

For instance, let us consider the case of LARA-0.8 applied to FinBench (Figure 4.3). When the budget is set to 0 or 900, the algorithm is unable to allocate a sufficient number of users to repair all the violations; this situation corresponds to the *infeasible* region. When the budget

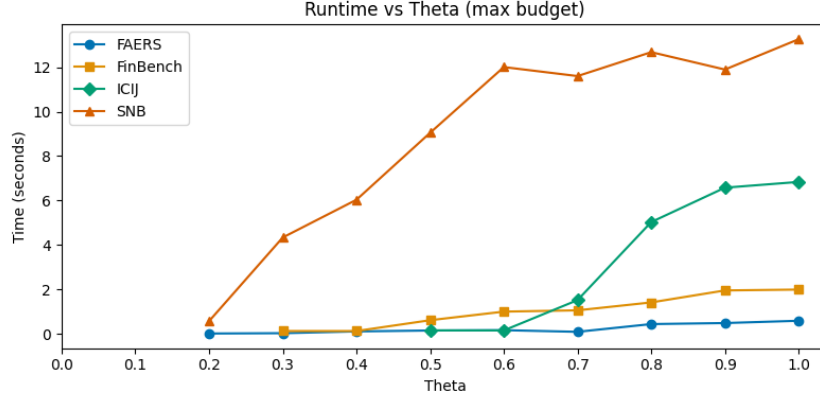


Figure 4.4: Runtime of LARA at different θ . Missing points correspond to run where only the GNN repaired the graphs.

increases to values between 1800 and 2700, the graph can be repaired; however, not all highly skilled users are involved in the process. As a consequence, the F1 score, remains around 0.5, which characterizes the *feasible* region. The *inflection point* is reached at a budget of 3600, where the involvement of additional skilled users results in an improvement of the repair quality, leading to an F1 score of 0.92. Beyond it, increasing the budget does not produce significant gains (e.g. the budget is sufficient to assign the enough expert users). It is interesting to observe that for all the datasets, the plateau is reached while under 70% of the total maximum budget is consumed. In fact, for F1 scores > 0.8 the plateau is reached within three to four successive budget increments prior to the maximum budget.

Notice that some values of θ result in a flat line. This occurs for lower values (e.g., LARA-0.2 on FinBench in Figure 4.3(a)), where the violations predominantly repaired with GNN-Repairs. Because the GNN already fixes the majority, assigning the remaining violations to experts has little impact on overall quality.

Finally, Figure 4.4 reports the runtime of LARA for each θ under the maximum budget. Being sparse, our graphs lead to near log-linear complexity in the number of nodes. Runtime increases with θ due to the larger number of assigned violations, but remains moderate. In contrast, the F1-score, discussed next, increases monotonically with θ .

4.5.4 Comparison with baselines

We compared our method with several baselines from the literature, covering crowdsourcing-based assignment strategies, fully automated approaches, and interactive graph repair techniques.

Crowdsourcing-based assignment methods aim to allocate tasks to human users based on models of skill, cost, and expected task success. Although not specifically designed for graph data, these approaches are well-established in the crowdsourcing literature. The similarity-based assignment strategy (SIM) embeds users and violations in a shared feature space and assigns tasks based on vector similarity [184]. We use violation difficulty and user skill as features, with normalized difference as the similarity metric. Another baseline is heterogeneous GNN-based label inference (LI) [165]. We adapt it to graph repair by predicting the most suitable repair using at most 10% of the available users.

We also compared **LARA** with the interactive graph repair method **uQAR** [13], which assigns violations based on structural characteristics (e.g., node degree). It uses a dependency graph with a locking mechanism to avoid conflicting repairs. Finally, we evaluated automated graph repair methods based on preferred repairs [117]. These approaches make repair decisions heuristically, without user involvement. **Pref-D** (Preferred Delete): always deletes edges or nodes to resolve violations first. **Pref-U** (Preferred Update): preferentially modifies node or edge properties instead of deletions. **Pref-S** (Preferred Schema): selects transformations according to a schema, favoring safer operations.

We report the F1 score of all methods, cost as the sum of the costs of all involved users, and number of users involved for the interactive repair methods. While Table 4.2 evaluates the quality of the difficulty estimators in isolation, the results in this section assess their impact on the end-to-end repair pipeline when used within the assignment strategy. As shown in Table 4.3, **LARA** outperforms all baselines across all datasets, achieving the highest F1 scores consistently. In particular, the row corresponding to **LARA-x** in Table 4.3 reflects the full pipeline performance using the difficulty estimates evaluated in Table 4.2, allowing us to assess how estimation quality translates into downstream repair effectiveness (in particular, the f1 score reported for **LARA-1**) coincide with the ones for the **ConfGNN** estimation in Table 4.2. The **SIM** strategy performs reasonably well on smaller datasets but fails to scale to **ICIJ** and **SNB** due to computational overhead. **LI** also suffers from “memory exceeded” (ME) errors on larger datasets. Among the automated methods, **Pref-S** yields the best results, suggesting schema-based heuristics can serve as a viable fallback in the absence of human expertise. Still, all automated methods lag significantly behind **LARA**.

Table 4.3: Comparison of F1 score, total cost, and total number of users for each dataset across all methods. TO = Timeout, ME = Memory Exceeded. Bold values indicate where **LARA** beats the baselines.

Method	F1 Score				Total Cost				Total Users			
	FAERS	FinBench	ICIJ	SNB	FAERS	FinBench	ICIJ	SNB	FAERS	FinBench	ICIJ	SNB
SIM	0.8061	0.7926	TO	TO	1326	4931	TO	TO	2403 (100%)	8998 (100%)	TO	TO
LI	0.4172	ME	ME	ME	264.3	ME	ME	ME	480 (20%)	ME	ME	ME
uQAR	0.8345	0.7926	0.7678	0.7783	1326	4931	18246	39362	2403 (100%)	8998 (100%)	32948 (100%)	71443 (100%)
Pref-D	0.5000	0.3833	0.1996	0.2984	-	-	-	-	-	-	-	-
Pref-U	0	0.6908	0.6002	0.4104	-	-	-	-	-	-	-	-
Pref-S	0.6253	0.7912	0.6409	0.7874	-	-	-	-	-	-	-	-
LARA-0.0	0.8691	0.4689	0.5127	0.3964	0	0	0	0	0	0	0	0
LARA-0.1	0.8691	0.4689	0.5127	0.3964	0	0	0	0	0	0	0	0
LARA-0.2	0.8691	0.4689	0.5127	0.4052	2.102	0	0	477.4	6 (0.24%)	0	0	253 (0.35%)
LARA-0.3	0.8691	0.4681	0.5127	0.4065	23.72	395.9	0	6236	29 (1.2%)	4 (0.04%)	0	925 (1.29%)
LARA-0.4	0.8691	0.4686	0.5127	0.7026	135.7	402.9	0	14692	65 (2.7%)	14 (0.15%)	0	1455 (2.04%)
LARA-0.5	0.8691	0.5494	0.5127	0.8291	273.1	1238	0.506	22590	114 (4.7%)	425 (4.7%)	1 (0.003%)	1952 (2.73%)
LARA-0.6	0.8691	0.6066	0.5126	0.8383	367.7	1844	5.073	25675	150 (6.2%)	774 (8.6%)	2 (0.006%)	2373 (3.32%)
LARA-0.7	0.8691	0.6219	0.6202	0.8858	612.3	1998	5312	27772	205 (8.5%)	856 (9.5%)	263 (0.7%)	2830 (3.96%)
LARA-0.8	0.9237	0.8321	0.9489	0.9669	913.0	3258	17840	33401	278 (12%)	918 (10%)	932 (2.83%)	3444 (4.82%)
LARA-0.9	0.9266	0.9982	0.9933	0.9996	1014	4849	18701	35977	329 (14%)	1261 (14%)	941 (2.85%)	3863 (5.41%)
LARA-1.0	1	0.9996	0.9996	0.9994	1477	5093	20090	37437	446 (19%)	1640 (18%)	1162 (3.5%)	4014 (5.62%)

uQAR ranks second among interactive methods but lacks adaptation to heterogeneous user skills and violation complexity, resulting in suboptimal performance compared to **LARA**.

LARA achieves high-quality repairs with lower cost and less human effort. At 0.8–0.9, it consistently outperforms baselines in quality and budget, The only exceptions are **LI** on **FAERS** and one **uQAR** case on **ICIJ**, both however resulting in lower F1 scores.

Furthermore, Table 4.3 also reports the number of users involved in each baseline, both as an absolute count and as a fraction of the initial pool (equal to the number of violations). Our method not only outperforms the others, but also consistently reduces the number of involved

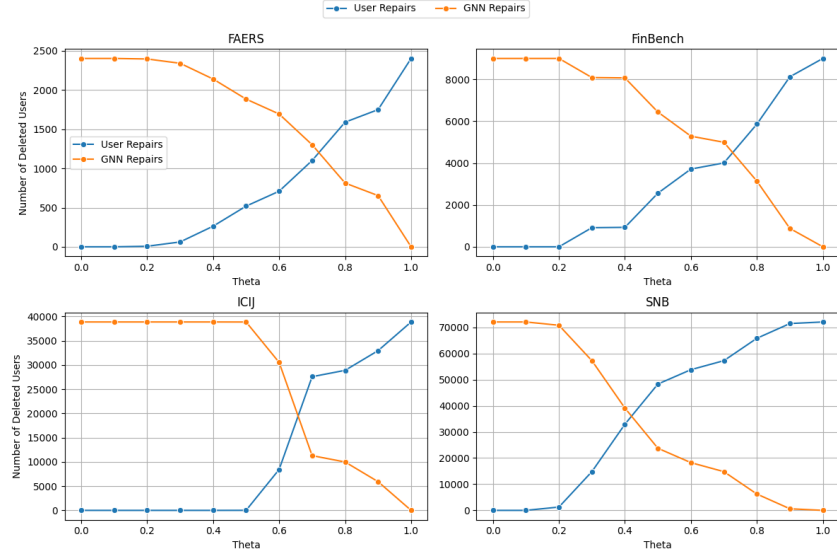


Figure 4.5: Number of repairs done by the GNN and by the users at different θ .

users by more than 80%. This suggests that the framework remains applicable when the number of detected violations is large but the effective number of available users is limited.

Finally, Figure 4.5 reports the number of repairs performed by users and the GNN, showing how the tunable parameter θ impacts the trade-off between quality and costs highlighted in Table 4.3. Notice that, to achieve a good quality score and outperform the baselines, it is enough to choose the smallest θ that results in more violations assigned to users than to the GNN. In general, for all the datasets, $\theta \in \{0.8, 0.9\}$ represents the best trade-off between automated repairs and user intervention, suggesting that selecting a θ close to the average difficulty value results in a good quality/cost balance.

4.5.5 LLMs for Data Repairs

To further investigate the repair process, we evaluate whether Large Language Models (LLMs) can partially or fully replace human workers for repairing violations, analyzing the trade-off between repair quality and cost. We consider three values of $\theta \in \{0.5, 0.8, 1.0\}$, corresponding to **LARA- θ** . For each θ , we evaluate two repair strategies: (i) assigning all remaining violations to the LLM (G-L), and (ii) splitting in half the remaining violations between the LLM and human users according to the violation difficulty score, assigning the easiest to the LLM and the hardest to the users (G-L-U). The LLM used in our experiments is Llama-3-8B [185]. Table 4.4 reports the resulting F1-score and total repair cost for each dataset and configuration. We additionally report the relative increase ($\uparrow$) or decrease ($\downarrow$) in F1-score w.r.t. **LARA** without LLM-based repairs. We kept LLM cost as 0 (as for the GNN), while for users we maintained the same settings described in § 4.5.1. Overall, the results show that fully replacing users with LLMs substantially reduces the repair cost, but at the expense of repair quality. In almost all configurations, the F1-score drops compared to **LARA** without LLM-based repairs, with drops ranging from moderate to severe depending on the dataset and the value of θ . Only a few configurations with $\theta = 0.5$ achieve improvements over the original approach, i.e., on **FinBench** and **ICIJ**, suggesting LLMs can sometimes effectively repair medium-difficulty violations when the GNN already filters out

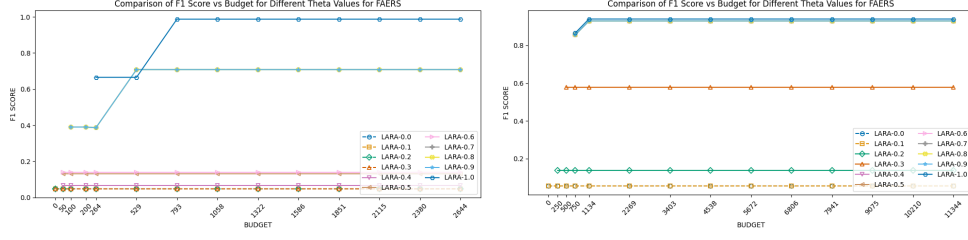
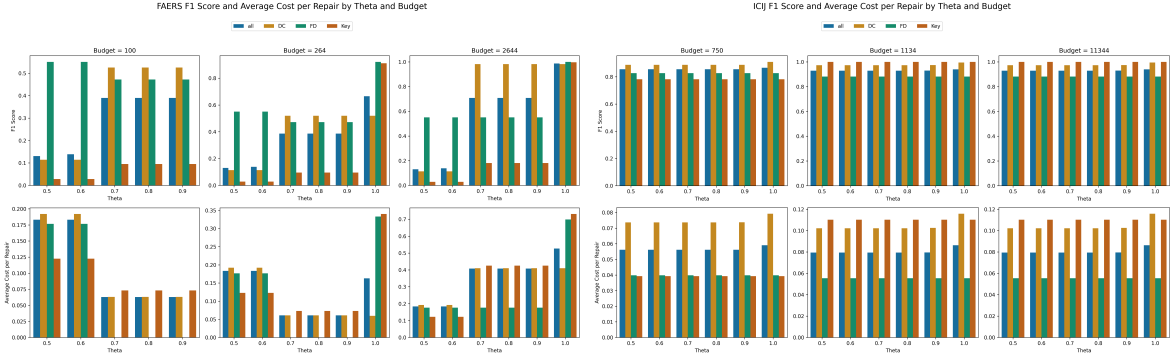
Table 4.4: Comparison of F1 score and total cost for each dataset with different values of θ and LLM involvement. FinB.=Finbench.

Mode	θ	F1 Score			Total Cost		
		FinB.	ICIJ	SNB	FinB.	ICIJ	SNB
G-L	0.5	0.245 ↓ 55%	0.629 ↑ 23%	0.445 ↓ 47%	0	0	0
G-L	0.8	0.277 ↓ 67%	0.613 ↓ 36%	0.439 ↓ 55%	0	0	0
G-L	1	0.262 ↓ 74%	0.605 ↓ 39%	0.445 ↓ 55%	0	0	0
G-L-U	0.5	0.785 ↑ 42%	0.900 ↑ 75%	0.807 ↓ 3%	866.7	0.385	18229
G-L-U	0.8	0.787 ↓ 5%	0.867 ↓ 9%	0.835 ↓ 14%	2313	12844	23714
G-L-U	1	0.762 ↓ 24%	0.897 ↓ 10%	0.857 ↓ 15%	3514	14464	26580

the easier cases. Hybrid strategies combining LLMs and users consistently outperform fully LLM-based repairs, but are largely dominated by the original fully human-assisted configuration. We additionally conducted experiments using GPT-5 on **FAERS** and **FinBench** only, due to budget constraints. Compared to Llama 3 8B, GPT-5 consistently improved the repair quality across all tested θ values, yielding gains of approximately 0.05-0.10 in F1-score. For instance, with $\theta = 1.0$, the fully LLM-based configuration achieved F1-scores of 0.406 and 0.605 on **FAERS** and **FinBench**, respectively, while the hybrid configuration with 50% user involvement reached 0.608 and 0.895. Nevertheless, even with GPT-5, the repair quality generally remained below the original **LARA** approach, confirming that current LLMs are still less reliable than human users for complex data repair tasks.

4.5.6 Testing Different Constraint Types

To evaluate the generality of our approach, we conducted an additional study using different classes of constraints. We considered three representative categories: a general denial constraint (DC), a non-key functional dependency (FD), and a key constraint (Key). These constraints were instantiated on the two real-world datasets **FAERS** and **ICIJ**. The complete list of constraints is reported in Appendix B.3. We reran the full end-to-end repair pipeline while varying both θ and budget. Figure 4.6 shows the F1-score on **FAERS** and **ICIJ**. Overall, the results confirm the trends observed in § 4.5.3, with a plateau reached for higher values of θ (> 0.7) and without using the full available budget. Figure 4.7 instead, breaks down the results for each type of constraint reporting the F1-score for $\theta > 0.5$ and budget values corresponding to the plateau regions observed in Figure 4.6. For **FAERS**, denial and key constraints benefit most from larger budgets, achieving near-perfect repair quality with sufficient resources. FDs show more stable behavior across parameter settings, with moderate F1-scores and limited sensitivity to θ . Higher-quality repairs generally incur higher repair costs, particularly for key constraints at large budgets, reflecting the complexity of resolving identity inconsistencies. For **ICIJ**, performance is even more stable across parameter configurations. Denial and key constraints achieve very high F1-scores for medium and large budgets, while FDs maintain slightly lower but highly consistent performance. Unlike **FAERS**, repair costs remain relatively stable across θ , suggesting that the assignment strategy adapts effectively to different constraint semantics without substantial additional effort. Overall, key constraints appear harder to solve, since high F1-scores are reached only with the full user budget ($\theta = 1$), and they are also the most expensive. FDs require smaller budgets but

Figure 4.6: F1-Score of our approach using different θ for FAERS and ICIJFigure 4.7: F1-Scores and total costs of our approach using $\theta > 0.5$ across FAERS and ICIJ at different budget values.

converge to lower accuracy, suggesting that GNNs may underestimate their difficulty, leading to lower-quality repairs. Overall, these results indicate that the proposed framework is not limited to a single type of integrity constraint. Although the experiments use GDCs as a unifying formalism, the assignment and repair pipeline remains robust across keys, FDs, and general DCs.

4.5.7 Illustrative Examples

Figure 4.8 illustrates the practical utility of our framework with three representative violations, one per constraint type, together with the difficulty scores assigned by our method (ConfGNN) and by Degree and PageRank. For readability, constraints are shown as Cypher-like queries.

Example 4.1. *[DC (FAERS)] A case whose recorded age (11) falls outside the range of its assigned age group (Adolescent, 13–18). Repairing this violation requires knowing either the true age or the correct age group of the specific case, information that cannot be reliably inferred from*

DC: (a:Age)←[r:HAS_AGE]-(c:Case)-[r1:FALLS_UNDER]→(ag:AgeGroup) WHERE a.value < ag.minAge OR a.value > ag.maxAge Inst.:(a:Age 'value': '11') ←[r:HAS_AGE]-(c:Case 'id': 43956) -[r1:FALLS_UNDER]→(ag:AgeGroup 'ageGroup': 'Adolescent', 'maxAge': 18, 'minAge': 13) ConfGNN = 0.987 Degree = 0.082 PageRank = 0.706	FD: (c1:Country)←[r1:REGISTERED_IN]-(n:Entity)-[r2:ADDRESS]→(a:Address), (a)-[l:LOCATED_IN]→(c:Country) WHERE c1.code ≠ c.code Inst.:(c1:Country 'code': 'BMU;HKG')←[r1:REGISTERED_IN]-(e:Entity 'id': 68202.0,...) (e)-[r2:ADDRESS]→(a:Address 'value': '83 Front Street') -[l:LOCATED_IN]→(c:Country 'code': 'BMU') ConfGNN = 0.345 Degree = 0.8 PageRank = 0.437	Key: (e:Entity)-[r1:JURIS]→(j:Jurisdiction)←[r2:JURIS]-(e1:Entity) WHERE e ≠ e1 AND e.inc_date = e1.inc_date AND e.name = e1.name Inst.:(e:Entity 'id': 81155,'inc_date': '01-03-14','name': 'Salamander Inc',...), (e)-[r1:JURIS]→(j:Jurisdiction 'code': 'KY', 'description': 'Cayman Islands'), (j)←[r2:JURIS]-(e1:Entity 'id': 78094, 'inc_date': '01-03-14', 'name': 'Salamander Inc',...) ConfGNN = 0.87 Degree = 0.11 PageRank = 0.3
--	--	--

Figure 4.8: Three examples of constraints with violations and difficulty estimations.

the graph. Accordingly, our method estimates a high difficulty (0.987) and, for $\theta = 0.8$, routes the violation to a user. The degree-based estimator assigns a very low score (0.082) and would incorrectly delegate the repair to the GNN. The Pref-* repair approaches in this case would fail, because we cannot solve specific violations with a globally tuned algorithm.

Example 4.2 (FD (ICIJ)). *An entity has a registered country (BMU;HKG) different from the country of its registered address (BMU). Such inconsistencies over country codes are well-studied in the database literature and can typically be repaired with high reliability [186, 187]. Our method estimates a low difficulty (0.345), correctly routing the violation to the GNN for $\theta = 0.5$. PageRank yields a comparable score (0.437), but the degree-based estimator assigns a much higher value (0.8), unnecessarily favoring human repair. Both the Pref-* methods and LARA would choose the correct repair.*

Example 4.3 (Key (ICIJ)). *Two entities are sharing the same name and incorporation date in the same jurisdiction. Key-constraint violations are typically hard, since deciding whether the records refer to the same real-world object or to distinct objects with identical attributes generally requires external knowledge (e.g., access to registries). Consistent with this, our method assigns a high difficulty (0.87), while PageRank and degree return much lower scores (0.3 and 0.11). Pref-* strategies could be tuned to always delete one of the two conflicting records, but such a fixed policy cannot determine whether the apparent duplicate is actually erroneous.*

These examples illustrate that topological estimators alone are insufficient for capturing repair difficulty: violations that appear structurally simple may require substantial semantic knowledge, while others can be repaired automatically despite involving larger graph structures. They also highlight the limitations of preferred-repair approaches, which rely on a fixed strategy that cannot adapt to the semantics of individual violations. Our framework adapts automatically through the difficulty estimator and threshold θ , delegating to users only the cases where automatic repair is unreliable.

4.6 Conclusion

This chapter addressed property graph repair by combining automated inference with human expertise. It introduced a GNN-based difficulty estimator that fuses topological and semantic cues to predict whether a violation can be fixed automatically (Section 4.3); that prediction feeds a user model of skill, cognitive capacity and cost (Section 4.4.1) and drives LARA, a difficulty- and capacity-aware assignment algorithm that batches violations and allocates them so as to maximise expected repair quality under a budget. The allocation problem was shown to be NP-hard by reduction from Multiple Knapsack (Theorem 4.1) and solved by Lagrangian relaxation with a closed-form dual certificate (Theorem 4.3), while Theorem 4.2 established that the repairs performed by the network commute with those performed by users, which turns the automation threshold into a free dial rather than a scheduling constraint.

Across four datasets, two of them real, the approach outperformed fully interactive and fully automated baselines in both accuracy and cost effectiveness, and matched or exceeded the repair quality of Chapter 3 while involving more than 80% fewer users. With this chapter the two

designs of Part I are complete: the first shows that interactive repair can be coordinated with guarantees, the second that it can be afforded. Chapter 5 draws the conclusion of the two previous chapters, states what they leave unsettled, and sets out the directions they open.

Chapter 5

Conclusion of Part I

Part I studied the repair of property graphs under the assumption that the knowledge needed to choose among candidate repairs lies with users. Chapter 3 showed that multiple, fallible users can be coordinated over an inconsistent property graph with formal guarantees on termination. Chapter 4 showed that the same process can be run under realistic conditions: users have unequal skills, bounded capacity and a cost. This chapter summarises what the contributions of the two chapters (Section 5.1), states the observations that generalise beyond the specific systems (Section 5.2), lists the limitations of Part I (Section 5.3) and the directions it leaves open (Section 5.4), and introduces the question addressed in Part II (Section 5.5).

5.1 Summary of Part I

The two chapters begin from Graph Denial Constraints, the most expressive of the property graph constraint families [4]; both compile constraints into constraint queries and materialise violations as subgraphs; and both reason not over violations in isolation but over the Graph Repair Dependency Graph, the hypervertex overlay that records where violations share nodes and therefore where repairs interfere. Everything that distinguishes Chapter 4 from Chapter 3 is a different answer to one question asked of that same structure: *who should be given which part of it, and why*.

Chapter 3 answers with independence. Violations whose GRDG neighbourhoods are disjoint are handed to different users, so that concurrent repairs cannot invalidate one another, and the local property of *safety* is imposed on every accepted repair. Two results follow. First, edge deletion is unconditionally safe; hence, the safety check is cheap in the common case. Second, safety is necessary and sufficient for convergence: the process terminates within a number of iterations bounded by the number of violations, without an empirical stopping rule. This guarantee is independent of the expertise of the users. Incorrect answers cannot make the process diverge; they can only make it converge on a graph farther from the ground truth. Empirically, uQAR outperformed four preferred-repair variants, two graph-repairing-rule baselines and both automated and interactive neighbourhood-constraint methods by roughly 30% in average repair quality, without oracle users. The user study with twenty participants confirmed the premise of the design: users who rated their own domain knowledge at 5 out of 10 still outperformed every

automated baseline, and the running time was dominated by human deliberation rather than by GRDG maintenance.

Chapter 4 answers the same question with difficulty and price. It abandons two conveniences of the first design, e.g. that users are interchangeable, and that their effort is free. A heterogeneous graph attention network scores each violation by the confidence with which its nodes and edges are predicted by the rest of the graph, and this confidence is read as difficulty: a violation that stands out against learned regularities is localisable and therefore easy, while one that conforms is ambiguous and therefore hard. Users become tuples of skill, cognitive capacity, and cost, and the allocation of violations becomes a budget-constrained covering problem over the GRDG that is NP-hard by reduction from Multiple Knapsack, solved by Lagrangian relaxation with a closed-form dual certificate. The same network that scores difficulty also repairs the easiest violations itself, and Theorem 4.2 shows that these automated repairs commute with human ones, so the automation threshold θ is a tunable parameter and not a constraint. On four datasets, two of them real, LARA achieved higher repair quality than crowdsourcing assignment strategies, automated preferred-repair methods and uQAR, while involving more than 80% fewer users.

Taken together, the two chapters establish that interactive repair can be organised as an allocation problem with guarantees on termination and bounds on optimality, and that its repair quality exceeds that of the automated baselines even when the users involved are not experts.

5.2 Lessons Learned

Seven observations recur across both chapters and do not depend on the specific algorithms.

Detection by query, repair by decision. Finding violations in a property graph is, formally and practically, pattern matching: a GDC compiles to a constraint query and the violations come back as subgraphs, at a cost polynomial in the size of the graph and linear in the number of constraints (Section 3.5.4). A single violation admits an unbounded, sometimes infinite, set of repairs, and selecting among them requires information the database does not contain. This explains the ceiling reported for automated methods on standard benchmarks [7], which cannot be removed by a better search over repairs. We proved that approximating a repair with an automated process is not sufficient and therefore involve the human for the judgement needed to choose between repairs.

Safety as a local invariant. Safety is checkable on a bounded neighbourhood of the edit, yet it yields monotone progress and therefore termination for the entire repair process (Theorem 3.2), regardless of user competence. Safety is the least expensive guarantee in Part I. However, it has a cost: restricting the repair space to edits that introduce no violation can exclude the only path to the ground truth. The *Fincen* experiments show this case, where the safe variant converges but does not reach the correct graph, while the unsafe variant reaches it and forfeits the convergence guarantee. Since neither choice is correct for every deployment, the framework exposes safety as a configuration parameter (Section 3.7).

The dependency structure is the unit of coordination. The most instructive difference between the two chapters is that they read the GRDG in opposite directions. Chapter 3 uses it to keep users *apart*: assignment requires disjoint neighbourhoods, because the scarce resource being protected is safe concurrency. Chapter 4 uses it to keep violations *together*: each user receives a connected subgraph, because the scarce resource being protected is human context, and a user who has understood one violation is cheap to ask about its neighbours. Both policies are correct and optimise different quantities. Once violation dependencies are materialised in the GRDG, the coordination policy becomes an explicit design choice, instead of a consequence of the order in which violations are enumerated.

Difficulty is both topological and semantic. Crowdsourcing models difficulty from task semantics: the interactive-repair literature, including our own first design, leans on graph topology. Neither suffices. Degree and PageRank estimators were the weakest performers in Section 4.5.2, and cognitive-load counts of the entities involved worked on the smallest dataset and failed on the rest. The illustrative cases in Chapter 4 show why: a two-node violation in which a recorded age contradicts its age group is structurally trivial and epistemically hopeless without the patient record, while a country-code discrepancy spanning a larger subgraph is a well-studied and reliably automatable fix. A confidence-based estimator that jointly encodes structure and property values ranked violations best on both pointwise error and distributional agreement. The latter metric matters for assignment: rare, hard violations are the ones an assignment policy must not misplace.

A correct assignment can repair the graph without a cohort full of experts. Across both chapters, the performance curve in user competence is flat at the low end. Simulated users who select the ground-truth repair a quarter of the time already beat the automated baselines on most datasets; half the time suffices everywhere. In the budgeted setting the same effect appears as an inflection point in the budget curve: below it, quality sits near 0.5; at it, the most skilled users become affordable and quality jumps; beyond it, additional budget doesn't improve the accuracy (Section 4.5.3). For every dataset the plateau was reached while consuming under 70% of the maximum budget. Expertise is therefore not a prerequisite for interactive repair: non-expert users already outperform the automated baselines, and skilled users add a further increment in quality (Section 4.5.3). This makes the approach applicable in domains without a cohort of experts.

Users are the bottleneck. The runtime measurements in both chapters point the same way: GRDG construction dominates the first iteration and then falls to seconds, while the total duration of the real user study tracked human deliberation almost exactly. Once the algorithmic cost is that far below the human cost, the useful optimisation target changes. Increasing throughput therefore yields little. The remaining gains lie in reducing the number of questions asked and in asking each of them of the right person, which motivates Chapter 4. The result confirms this: the same or better repair quality with over 80% fewer users involved.

Commutativity of automated and human repairs. Because GNN repairs delete edges only, and GRDG dependencies arise only from shared nodes, automated and human repairs commute (Theorem 4.2). The practical consequence is that automation can be applied without re-deriving the assignment, and that θ can be tuned as a quality/cost parameter; $\theta \in \{0.8, 0.9\}$ gave the best trade-off across all datasets.

More flexible automated methods, such as LLMs, do not remove these limitations. In Section 4.5.5, replacing users with a mid-sized open-weight LLM [185] reduced cost to near zero and reduced quality in almost every configuration; hybrid human/LLM allocation outperformed LLM-only allocation but remained below the human-assisted configuration. Two preliminary studies report the same trend at the two ends of the pipeline that Part I takes as given. At the *constraint* end, an exploratory study of LLM-based mining of graph consistency rules finds that the models recover largely schema-level constraints (e.g. keys, uniqueness, label enforcement) while failing to distinguish graph functional from graph entity dependencies, and that 20–40% of the generated Cypher is wrong through hallucinated properties, reversed relationship directions, or plain syntax errors; the authors conclude that the pipeline is not fully automated and that expert refinement is required [188]. At the *repair* end, an empirical study of six open-source LLMs on property graph repair reports the same shape of failure under different names : eager generation, indecision, and hallucination, with models over-generating edits and not than minimal ones, proposing repairs that satisfy the constraint while diverging from the ground truth, and deleting edges that carry information the graph cannot afford to lose [189].

The three sets of measurements are consistent. LLMs perform well where the answer is recoverable from the data and its schema, i.e., the regime that the GNN already covers at lower cost and with a calibrated confidence score. They perform poorly where a violation is ambiguous because the information is not in the graph. In the latter case an LLM returns a well-formed repair that removes the violation and diverges from the ground truth, which in a repair loop is more damaging than an abstention. Generative models can plausibly cover the intermediate tier of difficulty once the confidence model has removed the easy cases, and can support users by explaining a violation in prose or ranking candidate repairs.

5.3 Limitations

Several limitations bound the claims of Part I and delimit the future work.

Scope of the repair space. Repairs are edge deletions, relabelings, and property updates (Section 3.4). Insertions, node merges, and node splits are outside the model, which is a real restriction: entity resolution naturally produces errors that are best undone by splitting a wrongly merged entity. Deletion, meanwhile, is safe but lossy, since removing an edge can disconnect parts of the graph that no subsequent repair restores.

Convergence versus correctness. The guarantees of Part I concern termination and, in the budgeted setting, near-optimality of the *assignment*. Neither implies that the repaired graph is the true one. Safety can exclude the ground truth; the dual certificate of Theorem 4.3 bounds

the gap to the relaxation’s dual value, not to the optimum of the NP-hard problem itself; and the expected-quality objective is only as good as the user model that defines it.

Simplifications in the user model. Simulated users in Chapter 3 pick the ground truth with a fixed probability; in Chapter 4 they are a Naive Bayes classifier over skill and an item-response-theory estimate of perceived difficulty [180], calibrated on a single study of twenty participants on one small, general-knowledge dataset. Skill is scalar and single-domain, cost is a linear function of skill, and capacities are static: fatigue, learning, recovery, and the possibility that a user’s confidence is uncorrelated with their accuracy are all outside the model. A larger and more domain-realistic calibration study is the single most valuable piece of missing evidence in Part I.

Training data of the difficulty estimator. The confidence model is trained on the graph that contains the errors, so systematic errors risk being learned as regularities and therefore scored as hard. The feature-filtering step ties the representation to the active constraint set, which sharpens the estimate but means the model must be re-derived when constraints change. And the estimator is not uniformly good across constraint classes: difficulty for functional dependencies is systematically underestimated relative to keys and general denial constraints (Section 4.5.6), which sends FD violations to automation more often than it should.

5.4 Future Work

The limitations above translate into future work.

Richer repair operations. The reachable repair space is deletions, relabelings and property updates. Adding insertions, node merges and node splits would let a repair undo the entity resolution error that produced the violation in the first place. The obstacle is not the assignment machinery, which is indifferent to the kind of edit being proposed, but the difficulty estimator: the confidence signal of Section 4.3 is defined over graph elements that exist, and an insertion has no such element to score. Extending the network so that it can rank candidate additions, and calibrating that score against the existing one, is a prerequisite for widening the repair space without losing the filtering that makes it affordable.

Collaboration on a single violation. Both chapters assign a violation to exactly one user, in the first case to protect concurrency and in the second to control cost. Letting several users work on the same hard violation would raise the accuracy on precisely the cases that matter most. The challenge becomes how to aggregate answers that disagree. Crowdsourcing offers an established body of work on that problem [190, 90, 191] which has not been transposed to graph repair, where the candidate answers are structured edits rather than labels and where an aggregate must itself remain safe. Knowledge sharing between users, system-guided partial suggestions and the interface work required to make the loop usable by non-experts belong to the same direction.

Learning the policy instead of choosing it. Several decisions are currently made by hand. Which assignment algorithm suits a given dependency structure is chosen by the operator, and could instead be selected by a heuristic or a learned policy that reads the Graph Repair Dependency Graph directly. The automation threshold θ is fixed by hand and could be optimised from dataset and budget signals. Further out, the difficulty estimator, the user model and the assignment policy are three learned or specified components that are currently trained and tuned in isolation, and could be optimised end to end against the quantity the repair quality per unit of human effort spent.

5.5 From Cleaning to Reasoning

A violation with more than one candidate repair is a graph with more than one possible continuation. Each repair candidate yields a graph that satisfies the constraint set, so choosing between them means choosing between incompatible readings of the same data. In the running example of Chapter 3, either Markus was never allergic to paracetamol, or he was never given a drug containing it. Part I resolves that choice by asking a person who knows which reading corresponds to what happened, and the dependency graph, the safety condition and the budgeted assignment exist to make that question cheap enough to ask at the scale of a real graph.

The same operation appears in a different role in Part II. Every repair proposed in Part I is first evaluated as a hypothetical update: the safety check of Definition 3.10 computes the violations that a candidate would resolve and introduce before the candidate is written to the graph, and discards the hypothesis if the outcome is not acceptable. In Part I this evaluation is a step in the service of choosing one graph. In Part II it becomes the object of study: the question is no longer which graph is correct, but what an analysis would return over a graph that has been modified and is not committed. One can notice that hypothetical reasoning is, in this sense, an alternative to repair, since the candidate graphs can be queried and compared instead of being reduced to one. However, the two chapters of Part II do not study it as a repair technique. They study it on its own terms, where the modification is proposed by an analyst and the quantity of interest is the outcome of a decision: Chapter 6 asks what a trained model would predict under a hypothetical update, and Chapter 7 asks what would happen under an intervention on a causal structure stored with the data. The user correspondingly moves from the one who decides what the data should say to the one who plans and analyses over it.

Part II

Reasoning and Causal Analysis over Graphs

Chapter 6

What-If Queries over GNN Predictions

Part II turns from repairing graphs to reasoning over them. This chapter addresses the case in which a query is evaluated against a hypothetical version of a graph and the query does not read the data directly but the predictions of a trained graph neural network. This is the regime in which hypothetical reasoning is hardest, because the quantity being queried is a learned, non-linear function of both node features and topology and admits no symbolic delta rules of the kind incremental view maintenance relies on. Here the query does not read the data directly but the predictions of a trained graph neural network, which is the regime in which hypothetical reasoning is hardest, because the quantity being queried is a learned, non-linear function of both node features and topology and admits no symbolic delta rules of the kind incremental view maintenance relies on. Throughout the chapter the model is held fixed and the question concerns its prediction under a modified input; in the terminology of Section 2.4.2 this is an associational question, and the interventional and counterfactual questions of the higher rungs are addressed in Chapter 7.

The chapter formalises this as the *What-if query*: given a trained model, a data graph and a hypothetical update, evaluate an aggregate over the model’s predictions on the updated graph, without ever modifying the graph itself. The update is treated as an overlay on the cached forward state of the model: feature updates are applied as a low-rank correction that propagates exactly through the fixed aggregation weights, structural updates are handled by recomputing only the neighborhood they can reach, and in both cases the result is provably identical to a full recomputation. Since an evaluated state is itself a valid base for the next update, a session of chained, branching and backtracked hypotheses pays only for the increment between successive candidates. This is what makes interactive exploration feasible. We validate the approach on four datasets and two architectures against streaming baselines, and on a healthcare scenario in which the updates are candidate public-health programmes.

6.1 Context

Graph-structured data management has gained popularity, with *graph databases* such as Neo4j, TigerGraph, and Amazon Neptune adopted across healthcare, finance, cybersecurity, and e-commerce to store, query, and update highly interconnected data at scale [5]. As these systems

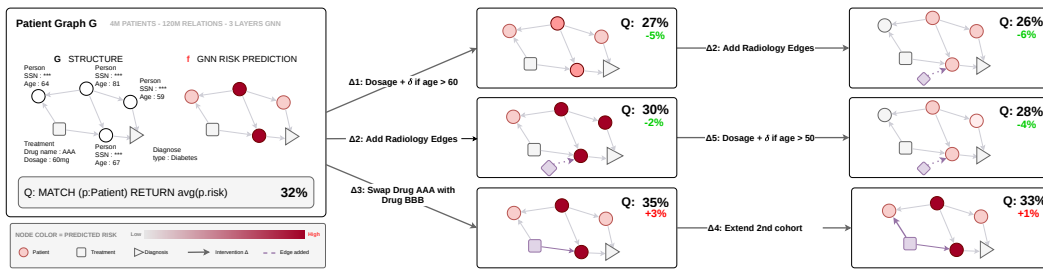


Figure 6.1: A what-if analysis over a trained GNN. The base graph G carries a per-node risk prediction (node color) and answers a query (average risk factor). Each arrow is a single hypothetical update Δ evaluated on G .

mature, organizations increasingly want not only to *manage* graph data but to *reason* about it predictively [192].

Graph Neural Networks (GNNs) have become the de facto standard for learning over graphs, producing predictions that jointly depend on node features and relational structure [9, 8]. They are now deployed in high-stakes settings such as disease-risk modeling [193], financial risk assessment [194, 195], and recommendation [196], where predictions directly affect individuals and institutions. In these settings predictive accuracy alone is insufficient: stakeholders must also be able to understand a prediction [197, 198] and to act on it, e.g., deciding whether to carry out an update. For instance, consider a clinician contemplating rolling out a screening program to a cohort of patients. Before committing such a change, the decision-maker (clinician or authority) wants to know its likely effect on the model prediction: *how would the model’s predictions over the affected population change if the change were applied?*

Hypothetical reasoning addresses these needs.

A hypothetical query asks how a prediction would change under a hypothetical modification of the input [199]. It underpins several uses of a trained model: (i) explaining a prediction by identifying the changes that would alter it [200, 197], and (ii) deciding whether to act, by evaluating the effect of a contemplated change before committing it. The last is the *what-if* question the database community has long studied, as in hypothetical OLAP [201] and provenance-based hypothetical reasoning [202]. However, this line of work does not directly carry over to model-based *predictions*. Prior what-if formulations reason over relational *queries* whose semantics are transparent and compositional: the effect of a hypothetical change propagates through known operators such as joins, selections, and aggregations, and these changes (“deltas”) can be readily derived from the definition of the operators. In contrast, the prediction of a trained GNN is a learned, non-linear function of both node features and graph topology, with no closed-form expression of the said function, so the effect of a hypothetical update cannot be derived symbolically and can only be assessed by evaluating the model. We therefore ask the what-if question directly over the model: given a trained GNN, a data graph, and a hypothetical update, evaluate an aggregate quantity of interest based on the model’s prediction over the modified graph. We call this a *What-if query*.

Example 6.1 (Running Example). *Consider a regional health system whose graph G contains patient nodes alongside treatment and diagnosis nodes, with edges encoding patient–treatment, patient–diagnosis, and patient–patient similarity relations. A trained heterogeneous GNN f*

predicts, for each patient p , a risk score $\hat{r}_p$ for developing cardiovascular disease. Figure 6.1 shows the data graph G with its per-node GNN risk prediction and the aggregate query answer ($\text{avg}(p.\text{risk}) = 32\%$). $\square$

Answering a what-if query given a hypothetical update Δ requires hypothetical reasoning [201]: conceptually, first construct the graph $G^\Delta = G \oplus \Delta$ induced by the hypothetical update Δ , then evaluate f on G^Δ . A naive implementation recomputes f from scratch, performing a full adjacency matrix update, full message passing over every node, and full inference. This incurs considerable computational overhead. When the update is local, e.g., a single edge added to a node influences only its L -hop neighborhood, *can we avoid recomputing [203] every node in the graph?*

We show that we can avoid this redundant compute by maintaining the predictions incrementally [204, 205]. The key observation is that when the aggregation weights of the GNN are fixed independently of the node features, as in a Graph Convolutional Network (GCN), a feature intervention perturbs the cached intermediate representations by a low-rank term that propagates exactly through the fixed aggregation. We apply this term as a low-rank correction to the cached states and recompute in full only the entries the non-linearity invalidates, which keeps the output exact.

Example 6.2 (Running example, continued). *As an example, suppose the system contains about 4M patient nodes¹ joined by 120M relations, with risk scores produced by a trained 3-layer GNN. A public-health analyst evaluates a new program that raises the dosage of a cardiac medication for elderly patients. The intervention consists of increasing the dosage feature x_p^{dose} by an amount δ for every patient over 60, $\Delta 1 = \{x_p^{\text{dose}} \leftarrow x_p^{\text{dose}} + \delta : p \in D_{60}\}$, where D_{60} denotes the over-60 cohort. Suppose these patients account for about 30% of the population, i.e., $\sim 1.2M$ patient nodes. A from-scratch evaluation re-runs f over all 4M patients. Since the dosage change is a single-feature update, incremental maintenance instead applies one low-rank correction to the cached representations, even though the cohort’s neighborhood reaches most of the graph.* $\square$

Moreover, an analyst rarely stops at one hypothesis. As Figure 6.1 illustrates, hypothetical updates are typically evaluated alongside variants, a different cohort, an added edge type, a swapped treatment, with each candidate chosen after seeing the previous answer. A from-scratch strategy pays the full 4M-patient inference at every step of this exploration; incremental maintenance pays only for the delta between candidates, which is what makes an interactive session, not just a single query, tractable.

Challenges: Answering a what-if query efficiently raises two significant challenges, each a familiar data-management problem made harder by the trained model in the loop. (1) We aim to maintain the model prediction, a view derived from the graph, under an update, by applying a delta rather than recomputing. Classical incremental view maintenance in relational databases deals with views defined by operators (e.g., joins, selections, aggregates) through which a delta propagates via known rules [208, 204]. In contrast, a trained GNN is a learned, non-linear function with no closed form, which does not admit symbolic delta rules. (2) A

¹According to the European Society of Cardiology and American Heart Association, there are over 62M cardiovascular patients in Europe [206] and about 82M in the US [207].

hypothetical update, by definition, should not effect persistent changes to the base data graph. Answering a what-if query by changing the data graph, reading off the answer, and rolling back the update tightly couples every (what-if) query with the persistent state; partial or interleaved rollbacks could leave the graph and its cached tensors inconsistent for subsequent queries. What-if evaluation needs snapshot semantics: each hypothetical update is assessed against a fixed base graph which stays invariant. **Our Approach:** We tackle all these challenges by never modifying the graph. We keep the base graph G and the cached forward state of f fixed, and treat each hypothetical update as a *hypothetical overlay*: we compute the change it induces on the cached state, propagate it only through the part of the computation it affects, read the query answer off the model’s prediction on the modified state, and discard the overlay. The overlay, our “delta rule” for the opaque view, is exact despite the non-linearity, and is local to the perturbation rather than a full-scale recomputation; it is applied and rolled off upon completion, so the base graph is never changed.

To summarize, we make the following contributions:

- We formalize the *What-If query*, i.e., evaluate a query over a trained GNN under a hypothetical graph update, and cast it as a data-management problem in the what-if querying tradition [209, 201, 202] (Section 6.3).
- We give an *incremental maintenance algorithm* (Section 6.4), named INQ-GNN, that applies the intervention as a low-rank correction (feature updates) or a locality-bounded recomputation (topological updates) to the cached intermediate state, recomputing only the parts the non-linearity invalidates, and prove that it matches full recomputation exactly (Theorems 6.1 and 6.2).
- We show the maintenance composes (Section 6.4.3): an evaluated state is a valid base for the next update, so a chained, branching, backtracking sequence of what-if queries pays only for the increment between updates (Corollary 6.3).
- We evaluate our approach over 4 datasets (2 synthetic + 2 real) and two architectures (RGCN, hGAT) and compare it with streaming baselines. Finally, we present the application of What-if queries on a use-case for a healthcare scenario.

Section 2 presents the background needed for the rest of the paper. Section 6.2 discusses related work. Finally Section 6.6 concludes the Chapter.

6.2 Related Work

Incremental computation of GNNs. Maintaining a derived quantity efficiently and correctly under updates to its inputs is a classic data-management problem. Incremental view maintenance keeps a materialized view consistent by applying deltas to the view instead of recomputing it from scratch [208, 204, 210, 211]. A parallel line of work on incremental graph processing reuses converged state across iterations of fixpoint analytics [212, 213, 214, 215]. These techniques rely on the views being transparent functions of the data, such as relational operators or monotone iterative computations. Unlike these, the non-linearity present in a trained GNN poses a fundamental challenge for incremental computation.

Recent work studies incremental maintenance of GNN inference on streaming graphs. InkStream [216] prunes the propagation tree for monotone aggregators (min/max), exploiting unchanged vertices after the updated input; however, it cannot work with non-linear activation functions. Ripple [217] handles the accumulative linear aggregators (sum, mean, weighted sum) that the above pruning cannot, maintaining exact embeddings with a look-forward delta model; and NeutronRT [218] extends incremental computation to neighbor-dependent weights and heterogeneous architectures, including GCN and GAT, by decoupling and reordering message-passing operators, with guaranteed equivalence to full recomputation. All three reuse cached intermediate state, touch only the affected subgraph, and target serving latency. These systems maintain a prediction as the graph evolves along a single, forward-moving timeline: each update is committed, and the next builds on it. A what-if query inverts this. The update is hypothetical: the graph is not evolving, and the same base graph is queried repeatedly under different candidate deltas, each evaluated and then discarded. Maintenance must therefore leave the base graph and its cache untouched so the operation can be repeated across candidates, turning incremental maintenance into the inner loop of a branch-evaluate-rollback operation rather than a streaming feed. Furthermore, as our experiments show, our approach dominates these streaming variants on one or both of accuracy and latency.

What-if querying in databases. Hypothetical, or *what-if*, querying has a long history in data management: a what-if query asks what a query answer would become under a hypothetical update to the input, without actually applying the update. Early support targeted OLAP and analytical settings, from hypothetical queries in OLAP [201] and what-if OLAP under changing dimensions [219] to what-if analysis for cost-based tuning of data-parallel jobs [220] and what-if exploration under conflicting goals [221]. A complementary line explains query answers through causality and responsibility: Scorpion [222] finds predicates explaining outliers in aggregates, Roy and Suciu give a formal account of explanations for query answers [223], and Meliou et al. establish causality and responsibility for answers and non answers [199, 224]. Most of this work measures the direct effect of an update on a view and reasons about it symbolically. HypeR [225] goes further: it extends SQL with what-if operators and, instead of provenance, uses a (learned or given) probabilistic causal model over attributes to capture the indirect, collateral effects an update induces on other attributes and tuples.

These formulations share a defining assumption: the map from an intervention to the answer is transparent, given either by the query semantics or, as in HypeR, by a causal DAG over attributes. None of them places a trained predictive model inside the query. Our what-if query instead embeds a fixed, non-linear GNN: the effect of an intervention has no closed form and can be established only by evaluating the model on the perturbed graph, making efficient incremental evaluation more challenging.

6.3 What-if Queries

This section defines what-if queries and their answers. How to answer them efficiently is the subject of Section 6.4. We first fix the space of admissible updates as schema-valid changes over the property graph, and identify the two families, attribute and structure updates (Section 6.3.1). We

then define the query itself and its semantics (Section 6.3.2). Finally, we state the computational problem the query poses (Section 6.3.3).

6.3.1 Interventions

We model a hypothetical scenario as a bulk hypothetical update over the property graph. In the remainder of this paper, the term updates refers to hypothetical updates, unless specified otherwise. Following the notation of intervention [58] where the population remains static, we model a bulk update as $\Delta = (\Delta E^+, \Delta E^-, \Delta \kappa)$, where ΔE^+ and ΔE^- are sets of edge additions and deletions and $\Delta \kappa$ is a set of property updates on nodes, including change of label and change of attribute values (node relabeling can be modeled using node properties). In Section 6.6 we discuss possible extension to node additions and deletions. We let $G^\Delta := G \oplus \Delta$ denote the resulting data graph after G is updated by Δ . We assume that all updates respect the schema: inserted edges follow admissible label patterns and property assignments conform to declared domains and ranges, so $G \models \Sigma$ implies $G^\Delta \models \Sigma$. The model captures the update classes commonly expressed in graph query languages, each expressible in GQL/Cypher and illustrated in Figure 6.1.

With the general update definition, we can define:

Attribute updates. A bulk attribute update assigns to a predicate-selected set of nodes a value given by an expression over their current properties, $\Delta \kappa = \{x.p \leftarrow \text{Expr}(x) \mid x \in V, \phi(x)\}$, where ϕ is a predicate over labels, properties, or topology and $\text{Expr}(x)$ may read x 's properties. Constant assignment $x.p \leftarrow c$, additive shift $x.p \leftarrow x.p + \delta$, and percentage change $x.p \leftarrow (1 + \alpha)x.p$ are all instances; in Figure 6.1, Δ_1 (Δ_2 resp.) raises the dosage of every patient over 60 (50 resp.) by δ , i.e. $x.\text{dosage} \leftarrow x.\text{dosage} + \delta$.

Edge updates. Bulk insertions and deletions are $\Delta E^+ = \{(u, v) \mid \phi(u, v)\}$ and $\Delta E^- = \{(u, v) \mid \psi(u, v)\}$, where ϕ and ψ are arbitrary predicates over the schema vocabulary. Validity is a condition on the result rather than on the predicates: we require the updated graph to satisfy the schema, $G^\Delta \models \Sigma$. This is the right formulation for both directions, an insertion may introduce an ill-typed edge, and a deletion may violate a structural or cardinality constraint, neither of which is captured by restricting ϕ, ψ alone. In Figure 6.1, Δ_2 inserts `Amedeo: treated_with` edges to a radiology diagnosis for a cohort, and Δ_3 swaps a treatment by combining a deletion and an insertion.

Conditional updates. More generally both attribute and edge updates may be restricted by predicates, expressing conditional, population-level interventions such as “*assign a radiology diagnosis to high-risk patients.*”

These mirror declarative graph updates (e.g., `MATCH-SET` or `CREATE`) and apply across single (or set of) nodes and edges.

In the GNN realm, an attribute update corresponds to a modification of a feature, which modifies a single feature coordinate k across nodes. Given the nodes' feature matrix X , the attribute update is $X' = X + \Delta X$, $\Delta X = u e_k^\top$, where $u \in \mathbb{R}^n$ collects the per-node deltas applied to feature k and $e_k \in \mathbb{R}^F$ is the k -th standard basis vector. The perturbation ΔX is rank one: it has a single nonzero column, which may be dense over nodes. For instance, to capture the bulk update “set/shift attribute k for a cohort of patients”: the support of u is the cohort, so Δ_1

applied to the over-60 cohort D_{60} in Figure 6.1 is $u = \delta \mathbb{1}_{D_{60}}$. A multi-feature edit is a sum of such rank-one terms, handled by superposition.

Edge updates correspond to structural modifications, inserting or deleting a set of edges ΔE . We first fix what an aggregator does with the graph structure. An aggregator is normalized when its operator entries depend on node degrees, so the weight it places on an edge is a function of the surrounding structure; the GCN operator $\hat{A} = \tilde{D}^{-1/2}(A + I)\tilde{D}^{-1/2}$ is the canonical example (see Section 2.3), with $\hat{A}_{uv}$ determined by the degrees of u and v . We write $\hat{A}$ for this structure-derived operator, as opposed to the raw adjacency A . A topological intervention edits A directly. Inserting or deleting the edges in ΔE gives $A' = A \pm \sum_{(v_1, v_2) \in \Delta E^\pm} e_{v_1} e_{v_2}^\top$. For a normalized aggregator this raw edit propagates into the operator, $\hat{A}' = \hat{A} + \Delta \hat{A}$, and the change is not confined to the edited entries: because an insertion or deletion alters the degree of each edited endpoint, it shifts the normalization of every edge incident to those endpoints. This is why a topological update, unlike an attribute update, cannot be treated as a purely local edit to $\hat{A}$, a point we return to in the maintenance procedure of Section 6.4.2. In our running example, Δ_2 (adding `treated_with` edges) and Δ_3 (swapping a treatment) in Figure 6.1 are both structural.

The two families expose different handles. A feature update has dense support but low rank: it touches every node yet is a single outer product. A structural update has sparse support but no exploitable low-rank structure, since degree normalization couples it to a whole neighborhood. Rank and locality are the two properties our incremental maintenance strategy exploits, depending on the type of update.

6.3.2 What-if Query

A what-if query asks how a model’s prediction would change under a hypothetical update on its input. Let Q be a readout over the model output: a query that extracts a quantity of interest from the predictions $\{\hat{y}_x(G) : x \in V \text{ (or } V \times V)\}$. A readout may concern a single item, one patient’s risk, one candidate edge, or summarize a set of them; our method maintains the full corrected output exactly (§ 6.4), so it answers any such Q . Typical queries select a subset $S \subseteq V$ (or $S \subseteq V \times V$) using graph predicates over types, attributes, or edges, possibly aggregating over it, e.g., using sum, average, or a threshold count $\sum_{x \in S} \mathbb{1}[\hat{y}_x(G) \geq \tau]$; in Figure 6.1, the query is the average predicted risk over the patient cohort, `MATCH (p:Patient) RETURN avg(p.risk)`. After the update Δ_1 , the analyst may use `MATCH (p:Patient) WHERE p.age > 60 RETURN avg(p.risk)` to check the predicted risk for over-60 patients.

Definition 6.1 (What-if Query). *Let f_θ be a trained GNN with fixed parameters θ , producing output $Y = f_\theta(G)$ on input G . Given a hypothetical update Δ producing the perturbed input G^Δ , and a query Q over the model output, the what-if query is*

$$\text{WHATIF}(G, \Delta, Q) = Q(f_\theta(G^\Delta)).$$

It answers: “what would the answer to Q over the model output have been, had the input been modified by Δ , with the model held fixed?”, the prediction in a counterfactual world where G is replaced by G^Δ .

Example 6.3 (Running example). *In Figure 6.1, the query $Q = \text{avg}(p.\text{risk})$ evaluates to 32% on G . Applying Δ_1 (raise dosage for the over-60 cohort) yields $\text{WHATIF}(G, \Delta_1, Q) = 27\%$ (H_1); applying Δ_2 (add radiology edges) yields 30% (H_2); and applying the combined update $\{\Delta_1, \Delta_2\}$ yields 26% (H_4). Each answer is read from f_θ evaluated on the corresponding hypothetical graph.* $\square$

Since f_θ is deterministic and fixed after training, the what-if query is well-defined: it is the value of the trained model on the hypothetically updated graph, $Q(f_\theta(G^\Delta))$. An update Δ changes only the input, from G to G^Δ , and leaves the model unchanged. This distinguishes the what-if query from its relational counterpart. Hypothetical querying over databases operates on transparent query semantics: the effect of an update can be propagated symbolically through the relational algebra and reasoned about in closed form [201]. Our query instead embeds a trained GNN, whose dependence on the input has no closed form; the effect of an update is therefore accessible only by evaluating f_θ on G^Δ . Efficient evaluation is thus not an optimization detail but the core of the problem, which is what the rest of the paper addresses.

6.3.3 The Incremental Evaluation Problem

Answering the what-if query as defined in Definition 6.1, would require a two-phase strategy: first, *materialize* G^Δ in full, then *infer* by recomputing f on it, a complete adjacency update, a full message-passing sweep over every node, and full inference. However, an update is almost always local or low-rank: Δ_1 in Figure 6.1, for instance, perturbs only the affected patients and their L -hop neighborhoods, leaving the rest of the graph untouched, so a from-scratch pass repeats at full cost an inference whose result is identical almost everywhere. The overhead compounds under an interactive analysis: an analyst rarely asks a single what-if, but tries updates in succession, each chosen after seeing the previous answer, forming a sequence $\Delta_1, \Delta_2, \dots$ against the same base graph. Since later choices depend on earlier answers the sequence cannot be batched, and recomputing f from scratch for each, over a graph that barely changes between steps, is computationally expensive at every step. We therefore define an incremental strategy that does not apply Δ to the graph. We keep G and the cached forward state of f fixed and treat Δ as a *hypothetical overlay*: we compute the change Δ induces on the cached state, propagate it only through the fragment of the computation it disturbs, read the query off the corrected output, and drop the overlay, leaving G and the cache exactly as they were. Each query thus becomes an incremental maintenance step whose cost depends on the size of the perturbation.

Definition 6.2 (Incremental What-if Evaluation). *a trained L -layer GNN f_θ together with its cached parameters and layers computations of a forward pass on G , the Incremental What-if Evaluation problem is to evaluate $Q(f(G^\Delta))$ exactly by updating the cached states in place, without re-running f on the untouched region of G or mutating G .*

Since Q is applied to the model output, evaluating $Q(f(G^\Delta))$ incrementally reduces to maintaining the GNN’s forward pass under Δ . The difficulty therefore lies entirely in the model, not in the query. Incremental what-if evaluation is thus incremental maintenance of $f(\cdot)$ itself. The requirement that the incremental evaluation must exactly match $f(G^\Delta)$ makes Definition 6.2 nontrivial.

Table 6.1: Symbols legend.

Symbol	Meaning
L	number of layers
$V, E; n, m$	node/edge sets; $n = V $, $m = E $
$A \in \{0, 1\}^{n \times n}$	adjacency matrix
$X \in \mathbb{R}^{n \times F}$	input node features, $X = H^{(0)}$
σ	element-wise activation (ReLU)
$W^{(k)} \in \mathbb{R}^{F_k \times F_{k+1}}$	layer- k weight matrix
$F_k; F; F_L$	width at layer k ; input width; output width
$h_v^{(k)} \in \mathbb{R}^{F_k}, H^{(k)} \in \mathbb{R}^{n \times F_k}$	node/matrix state at layer k
$Z^{(k)} \in \mathbb{R}^{n \times F_{k+1}}$	projected features $H^{(k)} W^{(k)}$
$M^{(k)} \in \mathbb{R}^{n \times F_{k+1}}$	aggregated features $\hat{A} Z^{(k)}$
$\hat{A} \in \mathbb{R}^{n \times n}$	fixed linear aggregation operator
$\hat{A}(Z^{(k)})$	data-dependent operator (GAT)

6.4 Answering What-if Queries Incrementally

We now describe INQ-GNN, our solution to the Incremental What-if Evaluation problem defined in Definition 6.2. A feature update is rank one; we maintain it with a low-rank correction (Section 6.4.1). A structure update has no low-rank structure; we maintain it by locality, recomputing the bounded neighborhood the edit reaches (Section 6.4.2). Both are exact and restore the cache on completion. As we show, both procedures are *exact*, i.e., they return precisely what full recomputation on G^Δ would, and both leave G and the cache restored on completion, so they are safe to invoke repeatedly. We first fix the computational model.

We recall the L -layer message-passing GNN. A layer maps node states $H^{(k)} \in \mathbb{R}^{n \times F_k}$ to $H^{(k+1)} \in \mathbb{R}^{n \times F_{k+1}}$, with $H^{(0)} = X$ the input graph features, $n = |V|$, and $m = |E|$. In compact form, a layer is

$$h_v^{(k+1)} = \sigma \left(\text{AGGREGATE}^{(k)}(\{h_u^{(k)} : u \in \mathcal{N}(v)\}) W^{(k)} \right), \quad (6.1)$$

with σ an elementwise activation and $W^{(k)}$ a learnable weight matrix. It consists of three stages: a linear projection $H^{(k)} W^{(k)}$, an aggregation over the neighborhood $\text{AGGREGATE}^{(k)}$, and an elementwise activation σ . The aggregator $\text{AGGREGATE}^{(k)}$ is a function of the graph topology and possibly of the node states themselves, applied row-wise to the projected features. The forward pass computes $H^{(L)}$ from $H^{(0)} = X$ by repeated application of Eq. (6.1). We execute the forward pass once on G and cache the intermediate tensors: for a layer k the input features $H^{(k)}$, the projection $Z^{(k)} = W^{(k)} H^{(k)}$, and the aggregation $M^{(k)}$. Aggregators are in general of two types. An aggregator is *fixed* if it acts as a linear operator $\hat{A} \in \mathbb{R}^{n \times n}$ independent of the node states, i.e., $M^{(k)} = \hat{A} Z^{(k)}$ with $\hat{A}$ a function of the graph topology alone (i.e., GCN, mean/sum pooling). In this case, the order of the two linear stages (projection and aggregation) is immaterial because AGGREGATE is feature independent, since linear maps commute with such aggregation. An aggregator is *data-dependent* if the operator itself is a function of the states, $M^{(k)} = \hat{A}(Z^{(k)}) Z^{(k)}$ (i.e., GAT and other attention models, where edge weights are computed from the features they aggregate). Our incremental evaluation updates these tensors in place. Table 6.1 lists the symbols used in the following explanations. The low-rank maintenance in Section 6.4.1 is applicable for fixed aggregators; locality maintenance (Section 6.4.2) is applicable for both fixed and data-dependent aggregators.

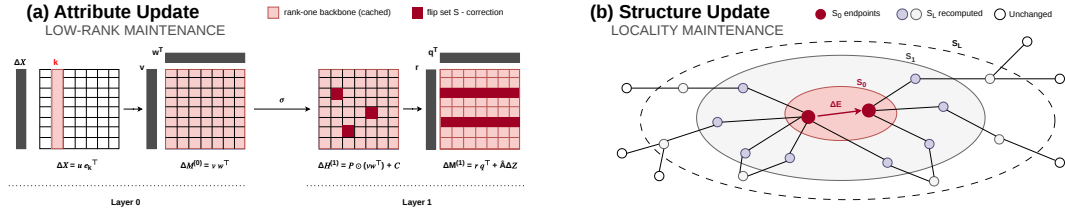


Figure 6.2: Incremental maintenance of message-passing models. a) Under attribute update b) Under structure update.

6.4.1 Attribute Update: Low-Rank Maintenance

Under a fixed aggregator $M^{(k)} = \hat{A}Z^{(k)}$, a rank-one attribute update propagates as a rank-one update through every linear stage; the only obstacle to global rank-one maintenance is the elementwise activation (i.e., ReLU function). Specifically, ReLU zeroes the coordinates whose pre-activation is non-positive and clips those whose argument flips sign, and we absorb both effects exactly, the first by a mask and the second by a sparse correction supported on the entries whose sign flips.

Let $\Delta X = u e_k^\top$ and let $w^\top := e_k^\top W^{(0)} = W_{k,:}^{(0)}$ be the k -th row of the first weight matrix. The first-layer projection and aggregation deltas are the rank-one outer products

$$\Delta Z^{(0)} = u w^\top, \quad \Delta M^{(0)} = (\hat{A}u) w^\top = v w^\top, \quad v := \hat{A}u, \quad (6.2)$$

since $\Delta Z^{(0)} = \Delta X W^{(0)} = u (e_k^\top W^{(0)})$, and aggregation is the fixed linear map $\hat{A}$, so $\hat{A}(u w^\top) = (\hat{A}u) w^\top$ by associativity. Forming $u w^\top$ costs $O(nF_1)$ rather than the $O(nFF_1)$ of a dense projection (e.g., if we were considering the whole graph), because only the single row $w^\top$ of $W^{(0)}$ is touched. Recall that F and F_1 are the width of the input matrices $H^{(0)}$ and $H^{(1)}$ respectively. Forming $v = \hat{A}u$ is a sparse matrix-vector product at $O(m)$, after which $v w^\top$ is again an $O(nF_1)$ outer product, versus the $O(mF_1)$ of a dense sparse-matrix/matrix multiply. The factors u, v, w are stored, never materialized into a dense $\Delta M^{(0)}$. Figure 6.2 illustrates the low-rank multiplication to obtain $\Delta M^{(0)}$.

The element-wise activation is where naive low-rank reasoning fails: $\sigma(M + \Delta M) - \sigma(M)$ is generally not low rank, because σ acts coordinate-wise and destroys the outer-product structure. The following decomposition repairs it exactly. Let $m_{i,d} = M_{i,d}^{(0)}$ and $\delta_{i,d} = \Delta M_{i,d}^{(0)} = v_i w_d$ for a ReLU activation $\sigma = \max(0, \cdot)$. There are two no-flip cases: stable-positive, where $m_{i,d} > 0 \wedge m_{i,d} + \delta_{i,d} > 0$ and stable-negative, where $m_{i,d} \leq 0 \wedge m_{i,d} + \delta_{i,d} \leq 0$. Let

$$P_{i,d} = \mathbb{1}[m_{i,d} > 0 \wedge m_{i,d} + \delta_{i,d} > 0]$$

be the *stable-positive* mask (entries that stay in the linear regime), let $N = \{(i,d) : m_{i,d} \leq 0 \wedge m_{i,d} + \delta_{i,d} \leq 0\}$ be the *stable-negative* set (entries that ReLU zeroes before and after the update), and let $\Gamma = \{(i,d) : \text{sign}(\max(0, m_{i,d})) \neq \text{sign}(\max(0, m_{i,d} + \delta_{i,d}))\}$ be the *flip* set (entries that cross the kink, in either direction). These correspond to the sparse red cells in Figure 6.2. Writing $O_i = \{d : P_{i,d} = 1\}$, $N_i = \{d : (i,d) \in N\}$ and $\Gamma_i = \{d : (i,d) \in \Gamma\}$, the three

sets partition the coordinates of row i . The activation delta splits exactly as

$$\Delta H^{(1)} = \underbrace{P \odot (vw^\top)}_{\text{masked rank-one}} + \underbrace{C}_{\text{sparse correction}}, \quad (6.3)$$

where $\odot$ is the element-wise product and C is supported only on Γ . In particular, $C_{i,d} = -m_{i,d}$ when the values' sign was positive and turns negative, $C_{i,d} = m_{i,d} + \delta_{i,d}$ when the value was negative and turns positive and $C_{i,d} = 0$ otherwise. Equation (6.3) is an entrywise identity: on stable-positive entries ReLU is linear and $\Delta H_{i,d}^{(1)} = \delta_{i,d} = (P \odot vw^\top)_{i,d}$; on stable-nonpositive entries both terms vanish; and the only residual is on the two sign-flip cases, which C carries exactly. Moreover, the identity holds for any ΔM , not only for the rank-one case. When no entry flips ($\Gamma = \emptyset$) the correction vanishes and $\Delta H^{(1)} = P \odot (vw^\top)$, which is the masked rank-one term alone; it coincides with $vw^\top$, and the forward pass collapses to a sequence of outer products, only when in addition no coordinate is stable-negative ($N = \emptyset$), since the mask zeroes exactly the coordinates in $\Gamma_i \cup N_i$. The same decomposition holds for any piecewise-linear activation, modulo changing the definition of sign-flip.

The structure survives the next linear layer. Substituting Eq. (6.3) into $\Delta Z^{(1)} = \Delta H^{(1)} W^{(1)}$ and letting $q^\top := w^\top W^{(1)}$ and $r := \hat{A}v$, row i reads

$$\Delta Z_{i,:}^{(1)} = v_i \sum_d P_{i,d} w_d W_{d,:}^{(1)} + \sum_{d \in \Gamma_i} C_{i,d} W_{d,:}^{(1)}. \quad (6.4)$$

The masked sum is the full sum $q^\top = \sum_d w_d W_{d,:}^{(1)}$ minus the coordinates the mask removes, and those are the coordinates outside O_i , that is $\Gamma_i \cup N_i$ and not Γ_i alone: a row carrying a stable-negative coordinate is not in the linear regime on that coordinate either. Hence

$$\Delta Z_{i,:}^{(1)} = v_i q^\top - v_i \sum_{d \in \Gamma_i \cup N_i} w_d W_{d,:}^{(1)} + \sum_{d \in \Gamma_i} C_{i,d} W_{d,:}^{(1)}, \quad (6.5)$$

$$\Delta Z^{(1)} = \underbrace{v q^\top}_{\text{rank-one backbone}} + \Delta Z_{\text{corr}}^{(1)}, \quad (6.6)$$

$$\Delta M^{(1)} = \hat{A} \Delta Z^{(1)} = \underbrace{r q^\top}_{\text{rank-one backbone}} + \hat{A} \Delta Z_{\text{corr}}^{(1)}, \quad (6.7)$$

where $\Delta Z_{\text{corr}}^{(1)}$ collects the last two terms of Eq. (6.5) and is nonzero only on the *affected rows* $I = \{i : \Gamma_i \cup N_i \neq \emptyset\}$, that is the rows carrying a sign flip or a stable-negative coordinate. The backbone propagates as a single outer product $r q^\top$; the correction propagates by sparse message passing over the affected rows only, $(\hat{A} \Delta Z_{\text{corr}}^{(1)})_{i,:} = \sum_{j \in I} \hat{A}_{ij} \Delta Z_{\text{corr},j,:}^{(1)}$, at cost $O(\sum_{j \in I} \deg(j) F_l)$ where F_l is the feature width at layer l . Algorithm 6 states INQ-GNN, the procedure to incrementally evaluate the query Q ; the model's state after the update Δ , e.g. $\hat{f}(G^\Delta)$, is incrementally updated at every layer with a rank-one backbone (lines 2-7) plus a correction confined to the affected rows (line 5), and Eq. (6.3) is reapplied to this sum at each activation (line 7).

Theorem 6.1 (Exactness of attribute update maintenance). *Let f be an L -layer message-passing GNN with fixed aggregators over a graph G , and let Δ be an attribute update. Given the cached*

Algorithm 6 INQ-GNN Attribute maintenance

Require: cached $\{M^{(\ell)}, H^{(\ell)}\}$ on G ; attribute update $\Delta X = ue_k^\top$; query Q

```

1:  $w^\top \leftarrow W_{k,:}^{(0)}$ ;  $v \leftarrow \hat{A}u$ 
2:  $\Delta M^{(0)} \leftarrow v w^\top$  ▷ stored as factors, not materialized
3: for  $\ell = 0$  to  $L - 1$  do
4:    $\Gamma \leftarrow \{(i, d) : \text{sign}(\sigma(M_{i,d}^{(\ell)})) \neq \text{sign}(\sigma(M_{i,d}^{(\ell)} + \Delta M_{i,d}^{(\ell)}))\}$ 
5:    $\Delta H^{(\ell+1)} \leftarrow P \odot \Delta M^{(\ell)} + C$  ▷ Eq. (6.3), backbone + correction
6:   if  $\ell < L - 1$  then
7:      $\Delta Z^{(\ell+1)} \leftarrow \Delta H^{(\ell+1)} W^{(\ell+1)}$ ;  $\Delta M^{(\ell+1)} \leftarrow \hat{A} \Delta Z^{(\ell+1)}$  ▷ backbone  $rq^\top$  + sparse over  $I$ 
8:   end if
9: end for
10:  $r \leftarrow Q(H^{(L)} + \Delta H^{(L)})$ ; discard all  $\Delta$  tensors ▷ cache restored
11: return  $r$ 

```

states of a forward pass on G , Algorithm 6 returns for every query Q the exact what-if answer, identical to full recomputation on G^Δ , i.e. $Q(\hat{f}(G^\Delta)) = Q(f(G^\Delta))$.

Proof. It suffices to show the maintained states satisfy $\hat{H}^{(\ell)} = H^{(\ell)}(G^\Delta)$ for all ℓ , where $\hat{H}^{(\ell)}$ denotes the layer- ℓ state produced by the maintenance procedure (Algorithm 6); then $\hat{f}(G^\Delta) := \hat{H}^{(L)} = H^{(L)}(G^\Delta) = f(G^\Delta)$ and the claim follows for every Q . We induct on ℓ . *Base:* $\hat{H}^{(0)} = X + \Delta X = H^{(0)}(G^\Delta)$ by construction.

Induction: assume $\hat{H}^{(\ell)} = H^{(\ell)}(G^\Delta)$. Projection and aggregation are linear, so the maintained delta $\Delta M^{(\ell)} = \hat{A}(\hat{H}^{(\ell)} W^{(\ell)} - H^{(\ell)}(G) W^{(\ell)})$ equals the true delta $M^{(\ell)}(G^\Delta) - M^{(\ell)}(G)$ exactly, whatever its rank. The activation step applies the decomposition (6.3), which is an entry-wise identity valid for arbitrary $\Delta M^{(\ell)}$; hence $\hat{H}^{(\ell+1)} = \sigma(M^{(\ell)}(G) + \Delta M^{(\ell)}) = \sigma(M^{(\ell)}(G^\Delta)) = H^{(\ell+1)}(G^\Delta)$. The induction closes at $\ell = L$. Exactness therefore does not rely on the perturbation remaining rank one; the rank-one backbone plus affected-row correction of Eqs. (6.2) and (6.5)–(6.7) is the representation in which $\Delta M^{(\ell)}$ is stored and propagated, which leaves the value exact. $\square$

Cost. When no coordinate flips and none is stable-negative ($\Gamma = \emptyset$ and $N = \emptyset$), every stage is an outer product or a sparse matrix-vector product, and a full two-layer pass costs

$$O(m) + O(nF_1) + O(F_1 F_L) + O(nF_L), \quad (6.8)$$

while the full from-scratch computation costs $O(nFF_1) + O(mF_1) + O(nF_1 F_L) + O(mF_L)$. Note that in Eq. (6.8), the dependence on the input width F disappears and the two dense sparse-matrix/matrix products become sparse matrix-vector products. Outside that regime the overhead is carried by the affected rows: a row $i \in I$ costs $O(|\Gamma_i \cup N_i| F_L)$ to form its correction and $O(\deg(i) F_L)$ to propagate it, so the overhead is $O(\sum_{i \in I} (|\Gamma_i \cup N_i| + \deg(i)) F_L)$ ($\deg(i)$ is the degree of a node i); the update reduces to a full pass only when a constant fraction of the rows is affected.

6.4.2 Structure Update: Locality Maintenance

A structure update changes the aggregation operator itself, so the rank-one calculus of Section 6.4.1 does not apply. What replaces it is *locality*: with endpoint set $S_0 = \{v_1, v_2 : (v_1, v_2) \in \Delta E\}$, in an L -layer model an edit confined to S_0 can reach only the nodes within

Algorithm 7 INQ-GNN Locality maintenance

Require: cached $\{H^{(\ell)}\}$ on G ; structure update ΔE ; query Q

- 1: $S_0 \leftarrow \{v_1, v_2 : (v_1, v_2) \in \Delta E\}$
- 2: **for** $\ell = 0$ **to** $L - 1$ **do** ▷ expand receptive field
- 3: $S_{\ell+1} \leftarrow S_\ell \cup \mathcal{N}^\cup(S_\ell)$; $E_\ell \leftarrow \{(i, j) \in E \cup \Delta E : i \in S_\ell\}$
- 4: **end for**
- 5: refresh $\hat{A}$ on edges incident to S_0 ▷ normalized aggregators only
- 6: **for** $\ell = 0$ **to** $L - 1$ **do**
- 7: recompute projection on S_ℓ and aggregation on E_ℓ to obtain $\hat{H}^{(\ell+1)}$ on $S_{\ell+1}$
- 8: **end for**
- 9: $r \leftarrow Q(\text{cached } H^{(L)} \text{ with } S_L \text{ overwritten by } \hat{H}^{(L)})$; **restore** S_L entries
- 10: **return** r

L hops of S_0 , and every node outside that ball is provably unchanged. This holds for any message-passing aggregator, fixed or data-dependent.

Let $\mathcal{N}^\cup(\cdot)$ denote the neighborhood in $G \cup G^\Delta$, the union of pre- and post-update edges, so that both deletions and insertions are covered, and define the receptive-field (L-hop) expansion

$$S_{\ell+1} = S_\ell \cup \mathcal{N}^\cup(S_\ell), \quad \ell = 0, 1, \dots, L - 1, \quad (6.9)$$

together with the edges touched at layer ℓ , $E_\ell = \{(i, j) \in E \cup \Delta E : i \in S_\ell\}$. Figure 6.2(b) illustrates the expansion of the updated endpoints through the receptive field. The red nodes are the ones affected by the added edge, while the gray nodes are the ones affected by the locality maintenance.

The maintenance rests on the standard receptive-field property: in an L -layer MP-GNN, $h_v^{(L)}$ is a function only of the input features and edges contained in the L -hop neighborhood of v . This is immediate by induction on layers: $h_v^{(1)}$ depends through Eq. (6.1) only on $\{h_u^{(0)} : u \in \mathcal{N}(v)\}$ and the edges incident to v , and since $h_v^{(\ell)}$ depends only on the ℓ -hop neighborhood, $h_v^{(\ell+1)}$ depends on $\{h_u^{(\ell)} : u \in \mathcal{N}(v)\}$, the union of whose dependencies equals the $(\ell + 1)$ -hop neighborhood of v . This bounds the affected region of Eq. (6.9) under the computation. Algorithm 7 states INQ-GNN, the full procedure to incrementally evaluate the query Q ; the model's state after the update Δ , e.g. $\hat{f}(G^\Delta)$, is incrementally updated by computing the L-hop expansion of Eq. (6.9) (lines 2-4) and recomputing the aggregation and projection locally (lines 5-8).

Theorem 6.2 (Locality and exactness of structure maintenance). *Let f be an L -layer message-passing GNN and Δ a structure update with endpoint set S_0 , and let S_L be the receptive field. Given the cached states of a forward pass on G , Algorithm 7 recomputes only the nodes in S_L and, for every query Q , returns the exact what-if answer, i.e. $Q(\hat{f}(G^\Delta)) = Q(f(G^\Delta))$.*

Proof. G and G^Δ differ only on edges incident to S_0 . By the receptive-field property, $h_v^{(L)}$ depends only on its L -hop neighborhood. A node v whose L -hop neighborhood meets S_0 is, by symmetry of the hop relation, within L hops of S_0 , i.e. $v \in S_L$. If the L -hop neighborhood of a node v contains no edge of ΔE , the two computations read identical inputs and produce identical outputs. Hence $v \notin S_L \Rightarrow h_v^{(L)}(G^\Delta) = h_v^{(L)}(G)$, establishing the unchanged region. For $v \in S_L$ the algorithm recomputes projection and activation on the affected nodes and edges using exact cached inputs at the frontier (unchanged, by the first part), reproducing the from-scratch values on S_L . Combining both regions gives $\hat{H}^{(L)} = H^{(L)}(G^\Delta)$, so $\hat{f}(G^\Delta) = f(G^\Delta)$ and hence $Q(\hat{f}(G^\Delta)) = Q(f(G^\Delta))$ for every Q . The overwritten S_L entries are restored on completion. $\square$

Cost. The exact incremental work has the layerwise form

$$\sum_{\ell=0}^{L-1} \left(\underbrace{\text{aggregation on } E_\ell}_{O(|E_\ell| F_{\ell+1})} + \underbrace{\text{projection on } S_\ell}_{O(|S_\ell| F_\ell F_{\ell+1})} \right), \quad (6.10)$$

with projected features cached, and recomputed only where a hidden state actually changed, in which case the aggregation term typically dominates. Writing $\bar{d}$ for the average degree, the affected sets grow as $|S_\ell| \approx O(|\Delta E| \bar{d}^\ell)$ and $|E_\ell| \approx O(|\Delta E| \bar{d}^{\ell+1})$, so the cost is proportional to the size of the L -hop closure of ΔE . *Best case*, edits at low-degree nodes with small closures, gives $C_{\text{inc}} \ll C_{\text{full}}$. *Worst case*, edits at hubs, or strong graph expansion with $S_L \approx V$, degenerates to the order of a full pass. For normalized aggregators, one adds a one-time normalization-maintenance cost $O(\sum_{(u,v) \in \Delta E} (\deg(u) + \deg(v)))$ to refresh degree-dependent edge weights around the edited endpoints.

6.4.3 Composability of the Maintenance

A typical use-case for What-if queries consists of applying updates in sequence: an analyst applies an update, reads the answer, and chooses the next one accordingly, extending some hypotheticals and over others. Such a session evaluates a sequence of updates against the same base graph. We show that the maintenance of Section 6.4.1-6.4.2 supports this workload directly: its results compose, so a state, once evaluated, is a valid base for the next update, and no update is ever replayed.

A *hypothetical state* is the graph produced by a sequence of updates $D = (\Delta_1, \dots, \Delta_t)$ applied in order, $G^D := (((G \oplus \Delta_1) \oplus \Delta_2) \dots \oplus \Delta_t)$, $G^0 = G$, where each $\oplus$ applies a single update to the graph on its left. Because the order is fixed by the sequence, G^D is well defined for any updates.

Corollary 6.3 (State composition). *Let G^D be a hypothetical state whose intermediate representations $H^{(\cdot)}(G^D)$ are cached, let Δ be a further update, and write $D' = D \parallel \Delta$ for the extended sequence. For any query Q , evaluating Δ incrementally on the cached state (Algorithm 6 or 7) is exact, i.e. $Q(\hat{f}(G^{D'})) = Q(f(G^{D'}))$.*

Proof sketch. Theorems 6.1 and 6.2 require only that the input cache be the true forward state of the graph the algorithm is invoked on, nothing ties that graph to the base G . Applying the maintenance step to the cache $H^{(\cdot)}(G^D)$, which is true by hypothesis, yields $\hat{H}^{(\cdot)} = H^{(\cdot)}(G^D \oplus \Delta) = H^{(\cdot)}(G^{D'})$, so $\hat{f}(G^{D'}) = f(G^{D'})$ and Q agrees. Hence any state reached by a sequence of updates is itself a legitimate base for the next, and the argument iterates. $\square$

Composition determines exactly what an interactive session can reuse. Reaching $G^{D'}$ from a cached prefix G^D costs only the incremental work of the appended update, independent of the length of D ; backtracking to a cached prefix is free; and two hypotheticals that share a prefix share its work. Reuse is therefore at the granularity of cached states, each identified by the sequence that produced it. The value of a state depends only on its graph, but the cost of reaching it is path-dependent: the flip set of a feature update (Eq. (6.3)) is taken relative to the pre-activation values it is applied to, so distinct sequences to the same graph traverse

Table 6.2: Dataset statistics.

	FinBench	FAERS	eICU	LDBC SNB
Nodes $ V $	55,604	25,934	6,322	367,342
Edges $ E $	309,577	5,507,867	137,110	1,435,066
Node types	5	7	5	8
Relation types	13	7	4	20
Avg. deg.	11	425	43	8
Feature dims	2-24	1-28	1-14	1-7

different incremental recomputations. What cannot be reused is the sum of separate effects. Composition is not additivity: the joint effect of two updates on a query generally differs from the sum of their individual effects, because they interact wherever their receptive fields overlap and a flip induced by one shifts the operating point of the other. Each single effect remains exact; only their superposition fails. A combined state must therefore be evaluated by composition rather than by summing single-update deltas, which is exactly what Corollary 6.3 makes cheap. Section 6.5.4 exercises these properties directly with a use-case on an healthcare scenario, running an interactive what-if session, a branching sequence of composed updates with a backtrack, over a live graph database, and measuring both the exactness preserved under composition and the recomputation avoided by resuming from cached states.

6.5 Experimental Evaluation

Our evaluation focuses on three questions. **(Q1)** What is the efficiency of the incremental evaluation over recomputing from scratch and how does it depend on the intervention type and size? **(Q2)** How does the speedup change with respect to the model’s depth? **(Q3)** How does our method compare with other exact and non-exact baselines? Section 6.5.2 addresses (Q1) and (Q2); Section 6.5.3 addresses (Q3).

6.5.1 Setup

Datasets. We use four heterogeneous graphs (Table 6.2) chosen to span three orders of magnitude in size and, more importantly, in *density*, the property that governs incremental cost, since the work of an incremental update tracks the size of the perturbation’s L -hop closure rather than the size of the graph. **SNB** [179] is the largest and most schema-rich (367K nodes, 8 node types, 20 relations) but sparse (avg. degree 8). It is a synthetic dataset modeling a social network. **FinBench** [178] is mid-size and sparse (avg. degree 11), with rich relational structure (13 relations) but bounded neighborhood growth. It is also synthetic and models a banking system with accounts and loans. **eICU** [226] has the smallest node set (6.3K) at moderate density (degree 43). It contains deidentified critical-care patient records (vital signs, diagnoses, treatments) from $\sim 2,500$ ICU stays across 20 US hospitals, a subset of the eICU Collaborative Research Database. **FAERS**, the densest by far (degree 425, 5.5M edges over just 26K nodes). It contains adverse drug event reports collected by the FDA Adverse Event Reporting System, each linking a patient case to the drugs taken and the adverse reactions observed. Table 6.2 summarizes the datasets statistics.

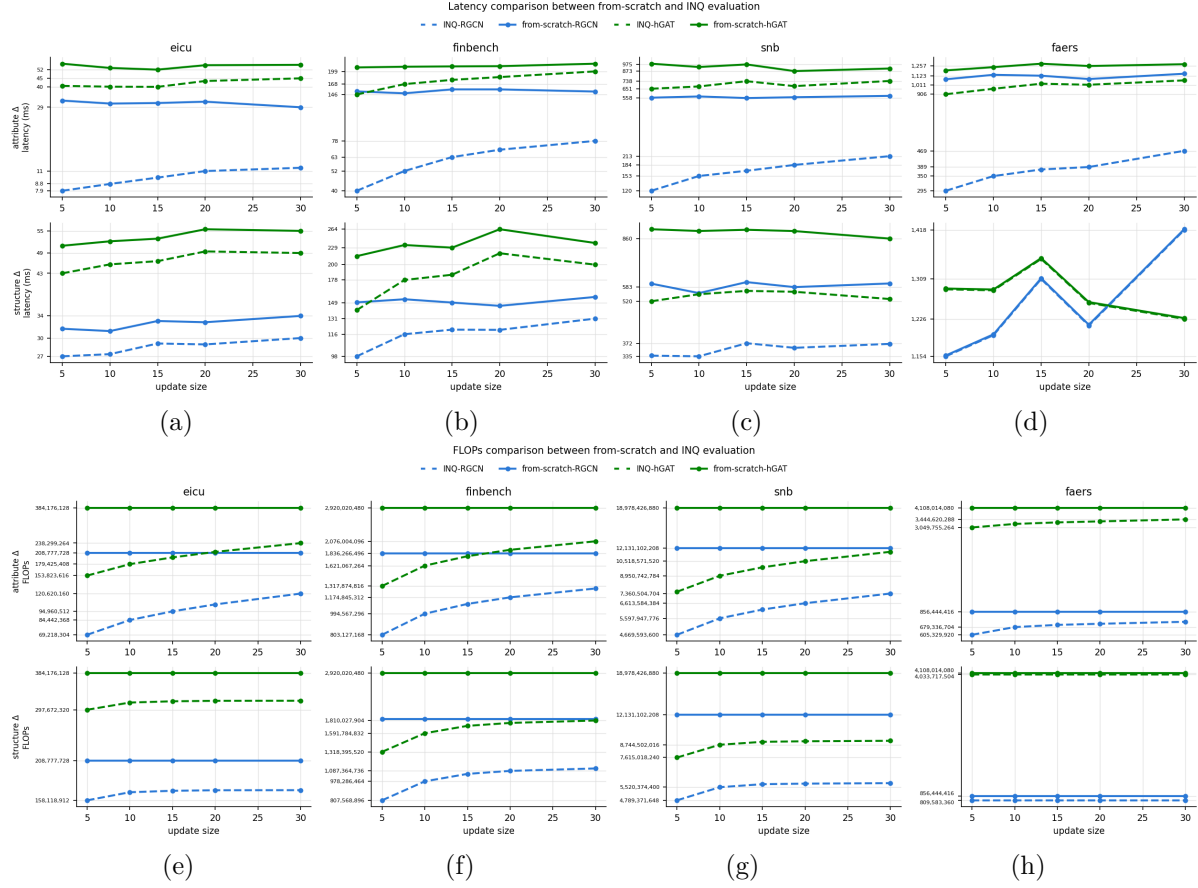


Figure 6.3: Latency (a-d) and num ops (e-h) vs. update size for incremental vs. from-scratch, at $L=2$.

GNN architectures and depth. We evaluate two message-passing architectures instantiated for heterogeneous graphs: **RGCN** (fixed, per-relation normalized aggregation) and **heterogeneous GAT (hGAT)** (data-dependent, per-relation attention). RGCN exercises the fixed-aggregator regime in which feature interventions admit exact low-rank maintenance; hGAT exercises the data-dependent regime, where the aggregation operator itself is a function of the states and only the locality guarantee applies. We choose these two architectures because they are among the most widely adopted message-passing GNNs [227], and because they instantiate the two aggregator regimes of Section 6.4; our maintenance applies unchanged to any message-passing GNN following the form of Eq. ((6.1)). We sweep across the depth range $L = 1-4$, which covers the practical range (2-3 layers is typical; over-smoothing discourages deeper models [228]); we add $L = 4$ to probe the closure-saturation boundary. We evaluate all three canonical GNN tasks, node classification, link prediction, and link regression. Since INQ-GNN maintains the forward pass exactly and is agnostic to the final prediction head, results are consistent across tasks; we report node classification in the main text and defer link prediction and regression to Appendix C.1.

Hypothetical Updates. While our framework supports two fundamental types of updates, in our evaluation we subdivide them into the following four kinds: feature shifts (bulk attribute updates over a cohort), edge additions, edge deletions, and swaps (a paired delete+add). The

list of updates and queries for each dataset is reported in Appendix C.2. The single asymmetry worth calling out is that additions are at least as expensive as deletions on normalized aggregators, because a deletion that removes a neighbor still forces a refresh of the degree-dependent normalization around the edited endpoints, and an addition does the same while also propagating a new message.

Baselines. We compare our approach INQ-GNN re-running the entire GNN forward on each hypothetical G^Δ (**from-scratch**), as our central baseline: it is the exact answer every method must reproduce (our exactness ground truth) and the true cost of ignoring incrementality, which we use as the reference our speedups. We then also include among baselines two methods from the streaming-GNN literature, **Ripple** [217] (an approximated linear look-forward delta with frozen degree/attention) and **NeutronRT** [218] (decoupled degree/attention operators with constrained neighborhood recompute), as the closest prior art for maintaining GNN outputs under graph changes. We re-implement both on our heterogeneous graphs and drive all methods through one shared op-counter to measure directly comparable counts.

Metrics. We compare INQ-GNN and **from-scratch** on four metrics. (i) *Latency*: time to answer a single what-if query, reported as the median over 10 repeated runs after warm-up, with the base cache materialized once beforehand. (ii) *Operations*: the number of Floating-Point Operations Per Second (FLOPs); being hardware-independent, it helps verify that latency gains reflect saved work rather than implementation artifacts. Latency and operations together quantify the savings of INQ-GNN over **from-scratch**. (iii) *Sign flips*: the size of the flip set $|\Gamma|$, that is the number of activation entries the update invalidates; this is the cost parameter of the low-rank maintenance, and we use it to explain how incremental cost scales with model depth. (iv) *Exactness*: agreement between the incrementally maintained answer and full re-evaluation, $Q(\hat{f}(G^\Delta)) = Q(f(G^\Delta))$, verified within floating-point tolerance (absolute 10^{-5} , relative 10^{-4}) at both the GNN output and the query level.

Implementation. All methods share one pipeline: a single cached forward pass on the base graph, a common update API, and shared instrumentation, so **from-scratch**, every incremental variant, and the external baselines (**Ripple**, **NeutronRT**) increment the same node/edge counters at the same granularity. Operation counts are therefore comparable across methods and reflect the algorithm, not the implementation. Our implementation uses PyTorch / PyTorch Geometric, single-machine, in-memory, CPU-only (no GPU); seeds, versions, and dataset cards are logged per run. The implementation details are reported in the code repository [176].

6.5.2 Incremental vs. From-Scratch Evaluation

We first measure what the incremental maintenance of Section 6.4 gains over **from-scratch** (materialize G^Δ and re-run f in full). Both evaluation strategies answer every query identically, so the comparison is purely on cost, which we measure as both latency and FLOPs.

Figure 6.3 fixes the depth at $L=2$ layers, the most common deployment depth [228], and varies the intervention size from 5% to 30% of the dataset; Figure 6.3(a)-(d) (resp. (e)-(h)) shows

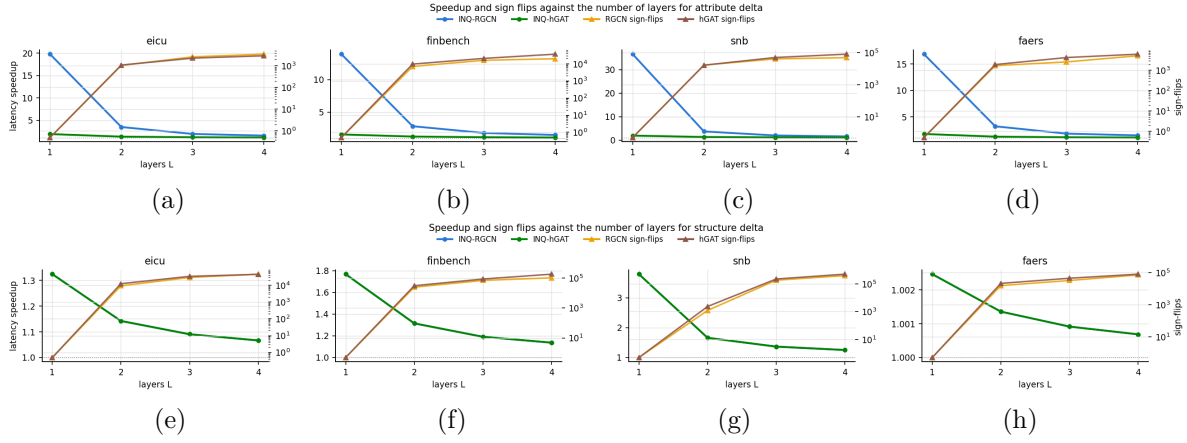


Figure 6.4: Speedup vs. depth L for the attribute updates (a-d) and structure updates (e-h) at 10% of the dataset.

latency (resp. number of operations) on bulk attribute update (top) and bulk structure update (bottom), on RGCN and hGAT over all four datasets.

Attribute updates on RGCN show the largest gains: incremental evaluation answers the query $1.9\text{--}4.7\times$ faster than a full re-run across all four datasets, with the maximum ($4.69\times$, on **snb**) at 5% intervention. The plots with number of operations (Figure 6.3(e)-(h)) show why: in this case, the low-rank maintenance of Section 6.4.1 applies in full, the update propagates as outer products plus sign-flip corrections, and cuts FLOPs by up to $3.0\times$ (**eICU** at 5%), tapering to $1.2\text{--}1.7\times$ by 30% as the L -hop expansion grows. On hGAT the same updates save a comparable fraction of FLOPs (up to $2.5\times$) yet yield only $1.1\text{--}1.3\times$ in latency: the data-dependent aggregator forces attention to be recomputed over every affected neighborhood, and this attention work, memory-bound and data-dependent, dominates latency even when its FLOP count does not, so the realized speedup falls short of the FLOPs avoided.

Structure updates use the locality-based maintenance, which is architecture-independent: both RGCN and hGAT avoid nearly the same fraction of FLOPs per dataset (up to $2.5\times$ on **snb**), gaining up to $1.7\text{--}1.8\times$ in latency.

Because FLOPs count compute independently of hardware, the two panels answer different questions: (e)-(h) report the work removed, (a)-(d) the time actually saved. For RGCN the two agree in direction, incremental is both cheaper and faster everywhere, but not as a fixed ratio; for hGAT they diverge, since attention takes more time per FLOP than RGCN’s linear operations. This trend doesn’t hold on **faers**, whose exceptional density places it at the boundary of the method; we return to it below. Figure 6.3 reports structure updates as edge additions; the trend is the same for edge deletions, reported in Appendix C.1.

Two patterns are visible across all cases. First, the gap shrinks as the intervention grows: 5% is uniformly the best case (maximum gap) and 30% the worst, because a larger update induces a larger L -hop closure, and at these fractions the closure already covers much of the graph. Second, the advantage degrades to parity but never below it. On **faers**, the densest graph ($|E| \approx 5.5M$, average degree 425), the 2-hop closure of even a 5% edge edit reaches essentially every node: incremental evaluation removes almost no FLOPs (Figure 6.3(h)), and with almost nothing to reuse the latency curves nearly coincide. This is why **faers** shows only a little gain

in latency, because it is evaluated at its limit, where the avoidable work is itself near zero. Even there INQ-GNN never costs more than **from-scratch**.

Scalability with Model Depth

Depth is the stress parameter for incremental evaluation: each GNN layer widens the affected region and adds an activation whose corrections add a cost. Figure 6.4 fixes the intervention at 10% of the dataset and sweeps L from 1 to 4, plotting the latency speedup over **from-scratch** (left y -axis) together with the size of the flip set $|\Gamma|$ (Eq. (6.3), right y -axis, log scale), the number of activation entries the intervention invalidates. Trends at the other intervention sizes are identical (See Appendix C.1.1).

With a single layer no internal activation separates the intervention from the output: the flip count is exactly zero on every dataset, and the speedup peaks, $14\text{--}37\times$ for attribute updates on RGCN (e.g. $36.6\times$ on **snb**), up to $3.8\times$ for structure updates. This point realizes the flip-free cost: maintenance is a pure linear delta, a sequence of outer products and sparse matrix-vector products, with no dependence on the graph beyond a single sparse pass.

The two curves of each panel mirror each other: the first internal activation, at $L=2$, introduces the bulk of the flips at once ($10^3\text{--}10^4$, depending on dataset), and the speedup drops correspondingly; beyond $L=2$ flips grow only mildly and the speedup decays gently. On **snb**, RGCN attribute updates fall $36.6\times \rightarrow 3.7\times \rightarrow 2.0\times \rightarrow 1.6\times$ as the flip count climbs $0 \rightarrow 17\text{k} \rightarrow 41\text{k} \rightarrow 49\text{k}$; the other datasets trace the same pattern at their own scales. The mechanism is the one identified in Section 6.4.1: each flip is an entry whose activation state changed and must be corrected and propagated, converting a would-be-free linear update into local recomputation. From-scratch cost grows only linearly in L , it re-evaluates everything at any depth, so the gap narrows monotonically. It narrows but does not invert: even at $L=4$, RGCN attribute updates retain $1.5\text{--}1.6\times$ on all four datasets, and incremental evaluation never drops below parity (dotted line).

On hGAT, attribute updates are bounded by attention, not by flips: hGAT accumulates flip counts comparable to RGCN’s, yet its speedup sits at $1.2\text{--}1.4\times$ almost independently of L , because recomputing attention over the affected neighborhoods dominates whatever the linear stages save. On **faers**, structure updates are bounded by density: the speedup is pinned at $1.0\times$ at every depth while the flip count still climbs to $\sim 70\text{k}$, flips measure invalidation, which keeps growing with depth even when the saturated closure leaves no work to avoid. In sum, the incremental advantage is largest at the depths at which GNNs are commonly deployed, and decreases along the two axes of the cost model: the flip set for attribute updates, the closure for structure updates.

6.5.3 Comparison with other baselines

The maintenance of Section 6.4 is exact. The streaming GNN-inference systems closest to our setting are exact as well, but built for a different workload and under different restrictions. Ripple [217] maintains embeddings under committed updates for linear aggregators, but attention-based weights (hGAT) fall outside its exactness guarantee. NeutronRT [218] extends incremental maintenance with small batches to neighbor-dependent weights, handling GCN and RGCN

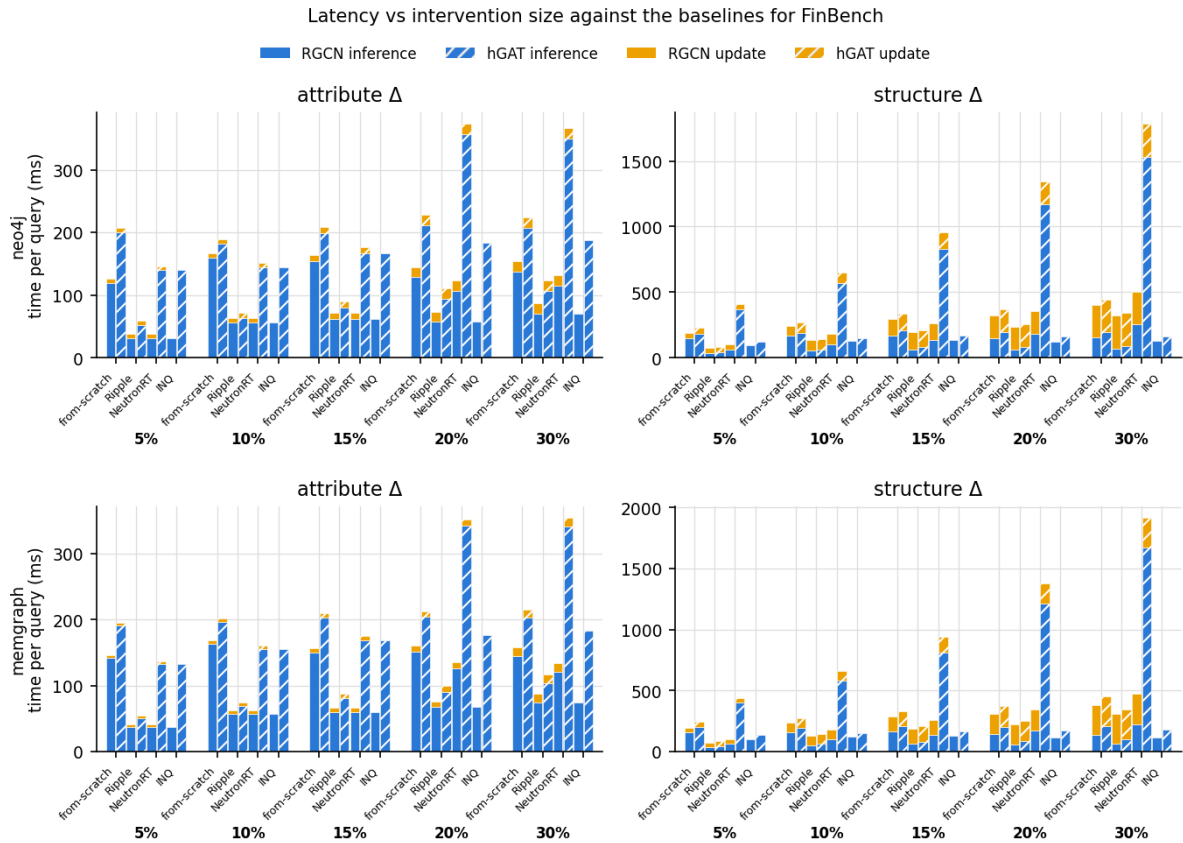


Figure 6.5: Latency comparison vs. the baselines for FinBench.

with a genuine incremental algorithm and falling back to full-neighborhood recomputation for models whose messages depend on the destination embedding (e.g., GAT). Neither, however, targets hypothetical workloads: both maintain a single committed timeline, whereas a what-if query applies a disposable overlay and must leave the base state intact so that it can be repeated across candidates. We therefore adapt both to the apply-evaluate-rollback pattern of Section 6.3.3 and compare on a store-backed deployment: the graph resides in a property-graph database (Neo4j [140] or Memgraph [229]), as it would in the scenario of Example 6.1. Figure 6.5 reports per-query time on FinBench at $L=2$, decomposed into store-update time (applying and rolling back Δ in the database) and inference time; the other datasets show similar trends (See Appendix C.3). On inference alone, the three incremental systems are close at these intervention sizes (5-30% of the graph): Ripple, thanks to its approximation, is at times the fastest of all, a point we return to below. NeutronRT is close to INQ-GNN on RGCN for small updates (5%-15%) but slows down for higher sizes due to its batching. For hGAT instead, its attention message depends on the destination embedding, so any node update invalidates all incident edges and forces full-neighborhood recomputation. **From-scratch** remains constant since recomputation is independent of the update size. INQ-GNN is the fastest exact maintainer across all interventions and both architectures, staying below **from-scratch** everywhere. The advantage of INQ-GNN is highlighted when considering the overhead due to interacting with the database for applying the updates (orange bar). Because the streaming systems maintain a committed timeline, every hypothetical must be applied to the store and rolled back after evaluation. This costs tens to

Table 6.3: Maximum elementwise deviation from the from-scratch output on **FinBench**.

Arch	Δ type	INQ-GNN	NeutronRT	Ripple
RGCN	attribute	1.2×10^{-6}	1.2×10^{-6}	1.2×10^{-6}
RGCN	structure	0	1.4×10^{-6}	6.0×10^{15}
hGAT	attribute	0	0	1.1×10^{-1}
hGAT	structure	0	0	2.8×10^{12}

hundreds of milliseconds and the cost grows linearly with the edit size (~ 10 ms per 1k edge writes on both stores neo4j and memgraph), incurred on every query. Our overlay evaluates the hypothetical against the in-memory cache and never writes to the database, so it carries no update time overhead (i.e., orange bar) at any size. The impact of the interaction with the database increases further when the baselines’ synchronization cost is accurately incorporated (not displayed in Figure 6.5 for clarity reason): a committed-timeline system must re-extract the graph after each roll-back scenario (or maintain its own second copy), which costs ~ 6 -7 s per extraction on **FinBench**, the cost of an entire from-scratch query; by contrast, INQ-GNN incurs extraction cost just once per session; the break-even occurs for a single query, and every subsequent hypothetical is served from the overlay at ~ 180 ms as opposed to ~ 7 s end-to-end.

However, the speed of Ripple comes at the price that it is exact only when its assumptions hold (e.g., feature updates under weights that do not change) and produces incorrect results otherwise. Table 6.3 reports the maximum elementwise deviation of each system’s maintained output from the **from-scratch** forward pass on **finbench**, over each architecture and intervention type, against the tolerance gate of 10^{-5} . Our maintenance is exact everywhere: on RGCN attribute updates, the max deviation of 1.2×10^{-6} is due to float32 rounding on single coordinates, but it is still $4 \times$ smaller than the tolerance window. In all other cases, INQ-GNN has 0 deviation. NeutronRT is likewise exact ($\leq 1.4 \times 10^{-6}$; it accumulates rounding but stays well inside tolerance). Ripple, however, matches recomputation only on RGCN attribute updates, and diverges elsewhere: by 1.1×10^{-1} on hGAT and by $10^{12} - 10^{15}$ on structure updates, far outside any tolerance. The failure is structural, not numerical: Ripple’s guarantee holds only when the aggregation operator is fixed under the update, but a structure update changes RGCN’s degree normalization and any update changes hGAT’s attention coefficients, exactly what our maintenance refreshes. The comparison situates our contribution precisely: we match the throughput of streaming systems while retaining the exactness that hypothetical query answering, unlike best-effort serving, requires; and we do this without ever mutating the data store.

6.5.4 Use Case: An Interactive What-If Session

We now illustrate a typical scenario of an analyst exploring a data graph interactively with hypothetical updates and empirically evaluate how well different approaches support it. Specifically, the exactness of INQ-GNN is critical to keeping interactive exploration reliable. Over the eICU clinical graph, backed by Neo4j, a clinician composes the attribute update on dosage with the structure update of drug-program intervention on an over-60 cohort (1,550 cases) and reads back the high-risk fraction $Q(G) = |\{i \in C : r_i(G) \geq \tau\}|/|C|$, where r_i is the model predicted mortality risk. The session consists of a five-state DAG with one branch and one

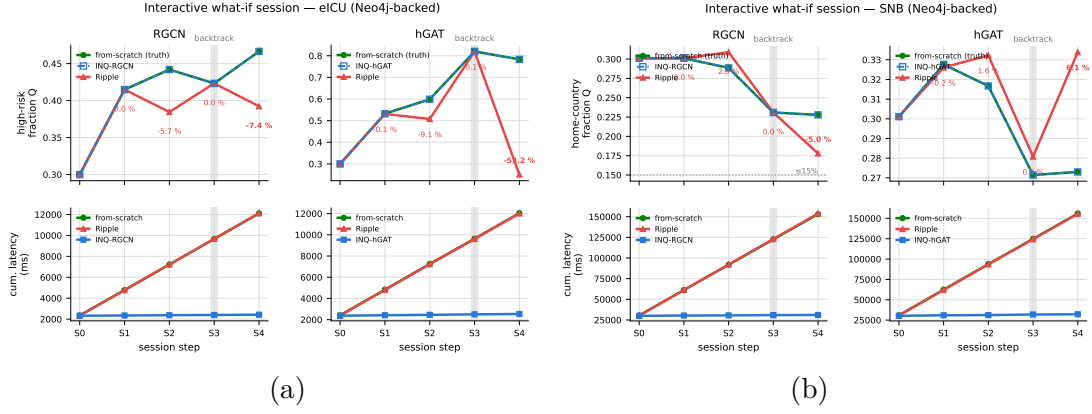


Figure 6.6: Interactive what-if session on eICU.

backtrack:

$$S_0\{\} \xrightarrow{\Delta_1} S_1\{\Delta_1\} \xrightarrow{\Delta_2} S_2\{\Delta_1\Delta_2\} \xrightarrow{\Delta_2'} S_3\{\Delta_1\Delta_2'\} \xrightarrow{\Delta_3} S_4\{\Delta_1\Delta_2', \Delta_3\}$$

where Δ_1 (resp. Δ_2') raises the medication dosage (attribute) by $\delta=1.0$ (resp. $\delta = 2.5$), and Δ_2 (resp. Δ_3) adds a second-drug edge to sub-cohorts. The analyst first explores the path $S_0 \rightarrow S_1 \rightarrow S_2$; then backtracks to S_1 and explores $S_1 \rightarrow S_3 \rightarrow S_4$.

The SNB session instantiates the same five-state DAG where the cohort is made by young users (641 persons) and Δ_1/Δ_2' shift the (z-scored) age coordinate of the cohort by $\delta = 1.0 / 2.5$; Δ_2/Δ_3 add **knows** edges from sub-cohorts (320 and 321 persons) to the most-connected person in the graph (a “befriend the hub” what-if). The query asks for predicted-positive fraction for the home-country class over the cohort. In both sessions the graph resides in Neo4j; the committed-timeline baselines apply each state to the store and re-fetch the graph, while ours reads the graph once at S_0 and evaluates every state on the resident cache.

From-scratch and **Ripple** both apply each state to Neo4j and re-fetch the full graph; **INQ-GNN** reads the graph once at S_0 (incurs 2.33s), then composes each residual delta on the nearest retained cache, with no write, no re-fetch, and makes a free return to S_1 at the backtrack.

Figure 6.6(a) illustrates the results of our evaluation. In both architectures (**RGCN** on the left and **hGAT** on the right), the query value obtained from **INQ-GNN** exactly matches **from-scratch** at every state. On **RGCN**, **Ripple** is exact at the attribute-only states (S_1, S_3) and deviates on structural edits (5.7% at S_2 , 7.4% at S_4). On **hGAT**, it treats the attention coefficients as fixed and is therefore wrong almost everywhere: $0.1 \rightarrow 9.1\%$ ($S_1 \rightarrow S_2$), back to 0.1% at the attribute-only backtrack S_3 , then 53.2% at S_4 , reporting $Q = 0.25$ against the true 0.78.

As for latency, **from-scratch** and **Ripple** overlap, each paying a full Neo4j apply+re-fetch (~ 2.4 s) per state and climbing linearly to ~ 12 s by S_4 ; **INQ-GNN** is flat at ~ 2.5 s (one load, then 22-47ms/step). The session total drops $\sim 12\text{s} \rightarrow 2.5\text{s}$ ($4.8\times$), and the gap widens without bound with session length. In sum: **INQ-GNN** is exact across a branching, backtracking session; **Ripple** is neither exact nor cheaper (full per-state round-trip); and **INQ-GNN** wins by reading the graph once and by removing the per-state DB cost entirely.

Figure 6.6(b) repeats the interactive what-if session on the largest dataset, **SNB** (367K nodes, 1.4M edges).

Exactness mirrors the eICU session: ours coincides with from-scratch at every state for both architectures (maximum query error 0.0, while Ripple’s frozen-aggregation maintenance drifts, reaching 5.0 pp (RGCN) and 6.1 pp (hGAT) at S_4 , on hGAT its error grows $39\times$ between S_1 and the final state. On RGCN, Ripple is exact precisely at the attribute-only states (S_1, S_3), confirming that its frozen-degree assumption fails exactly on structural edits.

The efficiency gap is set by the graph size. The one-time load costs ≈ 30 s; after it, ours answers each state in 170–716 ms from the resident cache, whereas from-scratch and Ripple pay a full Neo4j apply+re-fetch of ≈ 31 s per state. The session total drops from ≈ 156 s to ≈ 32 s ($4.9\times$ for both architectures, with $3.4\times$ / $2.1\times$ of the edge messages avoided on RGCN / hGAT), and, as in the eICU session, the gap keeps widening with session length since every additional state costs milliseconds instead of a full store round-trip.

6.6 Conclusion

In this Chapter we introduced the *what-if query* over the predictions of trained GNNs under a hypothetical, disposable graph update, and proposed two exact maintenance procedures. Attribute updates propagate as a rank-one correction to the cached forward state; structure updates exploit the receptive-field property of message passing to bound recomputation to a local neighborhood. Both match full recomputation exactly, leave the base graph untouched, and compose across an exploration session, so a chained sequence of queries pays only for the increment between retained states. Experiments across heterogeneous graphs and architectures show significant speedups and against streaming-GNN baselines, our method avoids the db-synchronization cost and, unlike the fastest of them (Ripple), remains exact. Our method is restricted to message-passing models: future work includes extending the approach to different architectures; developing efficient approaches for supporting how-to queries is another interesting direction. Finally, handling hypothetical addition/deletion of nodes, updates that change the dimensions of the GNN require a different maintenance strategy.

Chapter 7

Causal Analysis with Graph Databases

Chapter 6 evaluates hypothetical updates against a trained model. This chapter changes what the alternative worlds are computed over: instead of a predictor whose behaviour under an update has to be maintained, the graph database itself holds the causal structure, and interventional and counterfactual questions become queries over data that never leaves the store. The motivation is that causal analysis, as it is practised, is largely manual work that resembles data management: defining a causal DAG is graph data modelling, and asking a causal question is querying observational data together with that DAG. Property graphs can hold both in a single artefact, which relational models cannot, and this chapter asks what follows from taking that possibility seriously.

Three things are developed. A causal property graph model in which causal variables are hypernodes over their instances and conditional, interventional and structural information is carried as meta-properties; the query-language constructs this model requires, covering the extraction of confounding, mediator and collider paths as well as $do(\cdot)$ interventions and counterfactuals, and the extensions to Cypher, GQL and SQL/PGQ that current languages lack; and a proof-of-concept implementation whose measurements show that the cost of these queries is driven by the size of the causal DAG rather than by the volume of observational data. The chapter then extends the same construction: Section 7.6 describes a causal world ecosystem, in which many independently governed sources publish local causal views that a mediator reconciles into a global causal structure, served at once to analysts, learning systems and autonomous agents. This Chapter is the result of the vision papers *What-If: Causal Analysis with Graph Databases*[16] published at VLDB2025 and *Toward a Causal Data Management Ecosystem for Decision Making and Agentic AI*[17] published at ACM AI Leadership Summit 2026.

7.1 Context

Causal analysis [58] focuses on identifying, representing, and quantifying cause–effect relationships. Its application is crucial in various domains [230], as it goes beyond correlation and association to extract causal relationships between variables. Causal directed acyclic graphs [63] are a cornerstone of causal analysis, providing a rigorous framework for representing and analyzing causal relationships. By modeling variables as nodes and causal connections as directed edges,

causal DAGs facilitate the visualization of complex systems, the identification of confounders, mediators, and colliders, and the extraction of causal paths.

However, conducting causal analysis remains a significant challenge, as it involves a large amount of manual labor [71, 231, 232]. This process is not only time-consuming and costly but also prone to errors, limiting the applicability of causal analysis, particularly in complex domains with evolving or heterogeneous data.

We argue that causal analysis, to some extent, shares commonalities with data management. Defining a causal DAG is akin to graph data modeling, as both involve identifying relevant entities and relationships. Additionally, causal analysis can be viewed as performing queries on graph observational data and causal DAGs.

The data management community already explored how to integrate data management methodologies and causal analysis. Existing research has focused on relational databases and has explored how to facilitate causal analysis by treating it as relational queries [233, 234]. Further extensions have been considered with counterfactual queries [225, 235], using provenance to determine the effect of interventions and counterfactuals in SQL queries. Other works utilize causality to explain the causes of query results [199, 236]. Additionally, some studies have established a connection between these explanations and database repairs [237, 238, 239], reducing the problem of causal query explanation to a repair problem. Another work [240] has considered the problem of data completeness of causal DAGs while considering information coming from different tuples in a relational table. To the best of our knowledge, the idea of leveraging a graph database to carry out causal inference over interlinked data has not been explored before.

Graph data models and graph query languages are becoming increasingly widespread across various domains [241]. They serve as the underlying data models for numerous open-source and commercial database tools (e.g., Neo4j [140], Amazon Neptune [242], Oracle PGX [243], SAP Hana Graph [244], RedisGraph [245], Sparksee [246], Kuzu [247], etc.) and have recently been standardized with the introduction of graph query languages such as GQL and SQL/PGQ.

For this reason, we propose leveraging property graphs (PGs) [248, 20] as a foundational model for causal analysis. The expressive power and flexibility of PG allow the encoding of multi-valued nodes and edges, along with edge and node properties represented as key-value pairs, enabling a natural mapping of causal variables and relationships to nodes and edges. This representation can be further utilized to extract causal paths and align disparate data sources with greater consistency and efficiency. Another advantage of property graphs is their compatibility with concrete graph query languages, such as openCypher [249], GQL, or SQL/PGQ [250]. These languages enable graph pattern matching operations, retrieving and outputting results in the form of tuples. Additionally, recent advancements in high-level abstractions, such as path-based algebraic foundations for graph query languages, enhance the compositionality of graph queries, allowing property graphs to be returned as results instead of plain relational tuples [251].

In our vision, we leverage the underlying property graph data models and declarative formalisms to allow the precise extraction of all causal paths, identification of bias-inducing relationships, and inference on the extracted graph. By using graph queries, we can automate

the discovery of confounder, mediator, and collider paths, as well as explore the causal structure across different datasets.

Furthermore, using PGs allows us to embed the causal DAG directly within the data, whereas, in a relational model, it must reside outside the data model itself. This enables us to leverage the expressive power of graph data models and query languages to manipulate the observational data and the causal DAG seamlessly.

However, the property graph model and query languages are currently not expressive enough for causal analysis, particularly when entire paths and subgraphs need to be inspected as a whole. For example, as we show in Section 7.3, simple analyses, such as identifying mediators or confounders, can be performed using existing Cypher/GQL constructs. However, more complex causal tasks that require navigating between the causal DAG and the observational data require additional constructs that are not present in current languages, e.g., a path algebra [251].

In summary, the research vision that we investigate with this line of work centers around the following fundamental question: *How can we empower graph databases, property graph models and queries with causal analysis capabilities?*

To address the above question, in this Chapter we make the following contributions:

- We propose and formalize a causal property graph model augmented with hypernodes representing possibly related causes and with meta-properties encoding conditional and interventional probabilities along with structural equations (Section 7.2).
- We study the extensions needed in current graph query languages to support *path-based semantics* [251] and *do-calculus* constructs (Section 7.3). The former are meant to extract from property graph observational data different variants of causal paths, namely confounding paths, bias paths, mediator paths and collider paths.
- In a proof-of-concept implementation, we show the performances of causal path queries on causal DAGs of various sizes, showing that queries are executed efficiently on the causal DAGs and guarantee constant time access to the property graph data (Section 7.5).

The direction we propose is to represent causal knowledge in the graph database itself, so that the causal DAG and the observational data it abstracts are stored and queried through the same model and language. Embedding causal constructs into the data model and the query language would allow (i) interventions, counterfactuals and structural analysis to be expressed declaratively, and (ii) query results to be accompanied by the causal paths that explain them. A further consequence is interoperability with machine learning pipelines, e.g., querying causal effects post hoc or using the causal graph in model interpretation. Sections 7.2 and 7.3 develop the model and the constructs, and Section 7.5 assesses their feasibility.

7.2 Graph Model

We use property graphs to represent SCMs together with the observational data they abstract, so that their extraction, maintenance and inspection can be expressed as graph queries. Many of the tasks that currently require ad-hoc scripting (e.g., enumerating the confounding paths

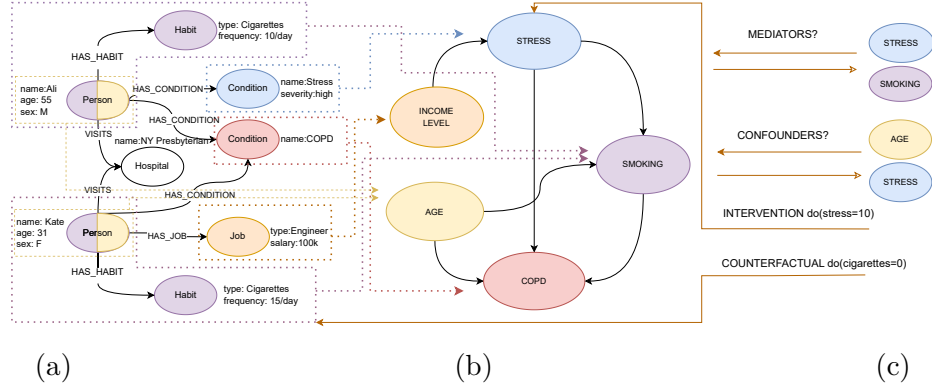


Figure 7.1: Example of a Causal Directed Acyclic Hypergraph model including the observational data (a), their mapping to the causal variables in the causal DAG (b), and the results of different causal queries (c).

between two variables) then become declarative. Property graphs carry multiple labels and key-value properties on both nodes and edges [20]. Hence, they can hold, in one artefact, the causal variables, the metadata attached to variables and relationships (e.g., conditional distributions and structural equations), and the instances that realise each variable, and they can be updated as the data evolves.

Figure 7.1(a) shows an example of a property graph. It models a person named Ali who smokes and drinks some alcohol and has two diseases (COPD and Stress), and a person named Kate suffering from the same COPD condition, but having also a job as an engineer.

To perform causal analysis, we need to extract causal variables from the PG and the relationships between them. Moreover, in order to apply interventions and perform causal analysis, we need to maintain a mapping between the PG instances and the causal variables in the causal DAG. It is seldom the case that one variable in a causal DAG is mapped to a single vertex in a property graph. In several cases, this mapping is not only one-to-many but also compositional. For this reason, we introduce a novel extension of the PG data model to cover hypergraphs. In particular, a hypervertex is a subgraph containing nodes and edges in the original property graph but can be considered a new node and can be linked to other existing vertices and new hypervertices. Increasingly expressive property graph models have already been discussed in the past [20, 252], motivating their need in several application domains.

Causal Directed Acyclic Hypergraph (cDAH): Given a property graph $G = (V, E)$, a cDAH is a structure $(H, F, X, S, P, I, \gamma, \lambda, \eta, \mu, \nu)$, where H is a finite set of hypervertices, F is a set of edges disjoint from E , X is a set of causal variables, S is a set of structural equations, and P and I are sets of conditional and interventional probability distributions, respectively. The function $\gamma : H \rightarrow \mathcal{P}(V) \times \mathcal{P}(E)$ assigns to each hypervertex a subset of nodes and edges from G . An injective function $\lambda : H \rightarrow X$ maps each hypervertex to a causal variable, and $\eta : S \rightarrow H$ assigns structural equations to hypervertices. The bijections $\mu : F \rightarrow P$ and $\nu : F \rightarrow I$ assign conditional and interventional distributions to edges in F .

Representing causal DAGs as property graphs offers several key advantages. First, it enables the causal DAG and the associated observational data to coexist within a unified data artifact, allowing them to be queried and analyzed jointly. Second, this representation makes it possible

to leverage well-established techniques from graph data management—such as PG-Schema [2], PG-keys [32], and graph views [253, 254]—as well as powerful declarative query languages like Cypher, GQL, or SQL/PGQ for expressive and efficient analysis. We argue that causal analysis is fundamentally navigational, as causal relationships are naturally encoded as edges in a graph. The property graph model is therefore particularly well-suited since graph query languages are designed precisely to support such navigational reasoning. From the above definition, it is clear that hypernodes need to be extracted from the underlying property graphs. A solution would be to adopt tuple generating dependencies [253] or graph transformations [254]. An example of graph transformation merging *Smoking* and *COPD* in a causal DAG is the following Cypher query, where a new relationship is generated when a condition is met (the matched pattern):

```
MATCH (p:Person)-[:HAS_HABIT]->(h:Habit),
      (p)-[:HAS_CONDITION]->(c:Condition)
WHERE c.name = "COPD" AND h.type="Cigarettes" WITH h,c
MERGE (h)-[:BELONGS]->(x:SMOKING)-->(y:COPD)<-[:BELONGS]-(c)
```

However, this query produces a node (variable) for each matched path. In Figure 7.1(b), for example, a variable (SMOKING) is extracted twice from the property graph instance. To deal with duplicates, we should merge the generated nodes that have the same label. This is possible in concrete graph query languages but requires multiple complex queries. The first step of our vision consists of extending the GQL syntax with a new operator **EXTRACT**, that allows us to easily express the causal variable extraction by abstracting out the details of hypernodes and graph transformations:

```
MATCH (p:Person)-[:HAS_HABIT]->(h:Habit),
      (p)-[:HAS_CONDITION]->(c:Condition)
WHERE c.name = "COPD" AND h.type="Cigarettes"
EXTRACT (x:SMOKING)-->(y:COPD)
```

After extracting the variables from the property graph instance, the causal DAG is not yet complete as it needs to be enriched with the probability distributions to the edges as properties.

Figure 7.1(b) shows an example of causal DAG extracted from a property graph (the colors indicate the mapping between the observational data in the property graph and the causal variables). We can see how different paths in the property graph are mapped to the causal DAG, specifically the query listed above created the path (SMOKING)-->(COPD) from the path including *Person* and *Habit*.

7.3 Causal Property Graph Queries

Causal investigation consists of a three-level hierarchy of analysis. First, association, which serves as the starting point, identifies relationships between variables. While this alone cannot answer causal queries, it might guide hypothesis generation. It consists of posing questions like “*What if I see..?*” or “*Is the value of X relevant for estimating the value of Y?*”. Second, intervention, which is used for interventional queries, addresses questions like “*What happens if we do X? Why?*”. Finally, counterfactual, which is essential for providing insights into hypothetical

alternate realities, handles questions like “*What if I had done...? Why?*”. These investigations need to be paired with the structural analysis of causal models, which plays a crucial role in the broader process of causal discovery and causal inference. During the causal discovery phase, identifying confounder paths (and blocking them) prevents a biased interventional analysis because confounders can create spurious associations or hide true causal effects. The same occurs with colliders path because they can create spurious associations if incorrectly conditioned on.

In this section, we propose our vision about **conducting causal analysis using a declarative approach relying on graph queries and their path semantics**, highlighting the advantages of relying on property graph models. In particular, we identified a list of queries that correspond to a specific type of analysis and we show how to write them in a standard graph query language. In our examples, we use **bold** for the part of the queries that refer to hypernodes (the causal variables in Figure 7.1(b)) and the **green** color to indicate our proposed extensions to the query language. All the following graph queries are expressed in GQL [24], but they can be similarly encoded in Cypher or SQL/PGQ [255].

7.3.1 Causal Path Queries

Causal mediation analysis examines how an exposure influences an outcome through intermediary variables, known as mediators. This approach decomposes the total effect of an exposure into a direct effect, the impact of the exposure on the outcome not mediated by the intermediary, and an indirect effect, the portion of the effect that operates through the mediator [256]. Correct mediation analysis has to be conducted after checking for existing confounding paths between the investigated variables.

Identifying the mediator nodes can be done using graph queries supported by the current standards. For instance, identifying the mediators between two variables X and Y checking the absence of confounding paths, can be achieved via this simple query:

```
MATCH (a:X)-->(m)-->(b:Y)
WHERE NOT EXISTS { MATCH (a)<--(c)-->(b) }
AND NOT EXISTS { MATCH (a)<--(c)-->(m) }
AND NOT EXISTS { MATCH (m)<--(c)-->(b) }
RETURN a as Exposure, m as Mediator, b as Outcome
```

Current graph query languages (GQL and SQL/PGQ 1.0) cannot retrieve all relevant paths needed for statistical adjustment. To address this, path operators must be added, as in [251]. This becomes especially important for multiple mediators, e.g., matching $(a:X) \rightarrow (m) \rightarrow (b:Y)$ where $*$ denotes the Kleene star for paths with zero or more intermediates [257].

The advantages of relying on PGs come from the possibility of accessing directly the observational data for further analysis since the DAG and the instances belong to the same data model. For example, to know which graph instance corresponds to the mediator between the variable INCOME LEVEL and COPD in the case of Kate in the graph in Figure 7.1, we can use the following query:

```
MATCH (p)-(j:Job)-[:BELONGS]->(a:INCOME LEVEL)-->(m)-->(c:COPD)
WHERE p.name="Kate" WITH m
```

```

MATCH ALL instance=(m)<-[ :BELONGS ]-(n1)
-[]-[*}(n2)-[ :BELONGS ]->(m)
RETURN instance

```

Where **MATCH ALL** is the path operator that lists all the path that matches the given pattern. This query first reaches the causal variable **INCOME LEVEL** from the node **Person**. Then it identifies the mediator via pattern matching. Finally, it leverages the path algebra to return the subgraph that corresponds to the mediator (e.g. the path expressing that Kate is smoking 15 cigarettes a day).

The same type of query applies to retrieve collider paths. Colliders block the path between two variables. By conditioning on the collider, we can open the path and study the association between the variables induced by the selection bias. Retrieving the colliders can be easily done by matching the path $(a:X) \dashrightarrow (c) \dashleftarrow (b:Y)$ where c is the collider. Regular path queries [20] allow us to list the descendants of the collider, since conditioning on one of them also opens the path [258]. Moreover, conditioning on a variable means identifying it with a value. Similarly to what we discussed with confounder path queries, we can leverage path algebra to directly access the instances of a collider path and select accordingly the right value to identify the collider random variable.

7.3.2 Interventional Query

Interventional queries focus on predicting the effects of specific interventions or manipulations by involving do-calculus [65] to simulate interventions. An example consists of investigating what will be the risk of contracting the **COPD** from smoking 10 cigarettes a day. In our model, this can be answered by traversing the path between the variables (**SMOKING** and **COPD** in Figure 7.1(b)) to obtain the structural equations from the node properties:

```

MATCH SHORTEST 1 (a:SMOKING) -->[*}(m) -->(b:COPD)
RETURN a.s_eq as s_s, m.s_eq as s_m, b.s_eq as s_l

```

where **SHORTEST 1** is the path algebra operator that matches the shortest path between two nodes [251]. Then, we need to apply the do-operator $do(cigarettes = 10)$ to the variable **SMOKING** and to evaluate the impact of this intervention on the variable **COPD**. To obtain the values for the other variables in the equations we can leverage the path operators to access the data instances in the property graph as described in Section 7.3.1. Linear equations can be solved in GQL since they do not require Turing completeness. However, GQL does not have built-in matrix operations, so solving systems with multiple variables is inefficient or impractical. Current solutions consist of relying on procedural languages (such as custom APOC procedure [154]). However, this obliges to go beyond the declarativeness of the query language. We propose to extend the GQL standard with an operator **DO-CALCULUS([values],[equations])** that evaluates the structural equations for an identified value.

7.3.3 Counterfactual Query

Counterfactual queries focus on exploring “what if” scenarios, both on individual-level, for example asking “*If patient A had not smoked, would they still have COPD?*”, or on a population-

level, for example asking “*If smoking rates had been reduced by 20%, how much would lung-related mortality have decreased?*” [259].

Answering counterfactual queries implies forcing a variable to take a specific value regardless of its natural causes. Interventional queries involve directly manipulating a variable ($do(X = x)$) to observe its effect on an outcome, while counterfactual queries extend this by conditioning on observed data to hypothesize alternate scenarios. The process of solving counterfactual queries simulates the intervention within a causal model, integrated with additional steps to account for observed evidence. This translates into computing the noise for the specific instance using the observed evidence.

For example, if we want to find out what the COPD severity of *Ali* would have been if he had not smoked, we can proceed by computing the noise of the variable COPD from the observed data. This means that knowing that he smokes 10 cigarettes a day and the severity level of COPD is moderate, we can use the structural equations to derive the noise in his case. Then we apply the do-operator $do(cigarettes = 0)$ to obtain the result. With our model, this translates into:

```
MATCH (h:Habit)<-[HAS_HABIT]-(p:Person)-[HAS_CONDITION]->(c:Condition)
WHERE p.name="Ali" AND h.type="Cigarettes" AND c.name="COPD"
RETURN h.frequency as f, c.severity as s
WITH f,s
MATCH SHORTEST 1 path=(a:SMOKING)-->(b:COPD)
RETURN DO-CALCULUS([SMOKING=f,COPD=s],[b.seq]) as noise
WITH noise
MATCH SHORTEST 1 path=(a:SMOKING)-->(b:COPD)
RETURN DO-CALCULUS([SMOKING=0,εCOPD=noise],[b.seq])
```

7.3.4 Causal DAG Maintenance

Maintaining the causal DAG according to the changes in the underlying data is a very tedious process that entails manually rerunning all causal analysis on the new and updated data, since existing methods disregard the case where the underlying data change [58, 260, 64]. This is important because it avoids recomputing the causal DAG from scratch each time the underlying data change. Our model allows us to use established data management techniques such as incremental view maintenance [253, 254, 261, 262, 204] and reactive graph data management [263, 264]. We envision utilizing triggers [264] to maintain the causal DAG by reacting to changes on edge patterns corresponding to causal variables, and running queries to compute the new probability and interventional distributions. Another option is to adapt works on view maintenance [253, 254, 261, 262]. In this case, the causal DAG will be implemented as one or more graph views and the incremental maintenance can be done by either extending recent proposals on graph views and transformations [262, 254], or by adapting the well-known DRed algorithm [204] for the graph case.

7.4 Related Approaches

In this section, we discuss the features of our approach compared with causal analysis frameworks - such as PyWhy [265] and Salesforce CausalAI [266] - and graph DBMSs. We distinguish tasks related to causal path finding and causal DAG management tasks, as well as causal graph discovery, and interventional and counterfactual analysis as foundations of causal analysis [258].

Causal analysis frameworks are empowered with imperative programming interfaces, requiring familiarity with both causal analysis and the APIs of the respective libraries. Whereas they support causality-oriented operations, such as causal discovery leading to learn a causal DAG from the data, along with API methods for interventions and counterfactuals, they are not able to natively support graph-oriented operations, such as causal pathfinding, leading to retrieve all collider, mediator, and confounder paths between two variables (as shown by the first two columns of Table 7.1).

In contrast, our framework (last column) adopts a different perspective grounded in data management, allowing DAG manipulation operations based on declarative update languages, efficient causal path traversals through graph queries, along with DAG mining utilizing the topology and semantics of the underlying observational property graph data. A causality-driven approach in a graph database can also facilitate one of the hardest tasks in the theory of causality, namely DAG transportability (i.e., applying causal DAGs, from one context to another [267]). It elevates path-based management to a first-class operation where users can express complex causal queries, directly interacting with causal paths and their properties to enumerate, filter, transform, and compare causal paths using a declarative approach without external libraries, as causal structures are encoded in the graph itself.

The third column of Table 7.1 finally shows the contrast between what a current graph DBMS supports and what our approach supports. As also shown in the experiments, a graph DBMS does not handle interventions and counterfactuals natively but with the support of APOC procedures in an external library. We envision extensions of graph query languages with DSL operators.

Our framework is not meant to replace existing tools but to complement them. It can, for example, retrieve valid adjustment sets or causal paths to be used by PyWhy for estimation, with results then annotated in the graph for further querying and analysis.

Table 7.1: Comparison of tools on causal inference tasks.

Task	PyWhy	CausalAI	Graph DBMS	WhatIf
Causal Discovery	✓	✓	✗	✓
Causal DAG Mining	✗	✗	✗	✓
Causal Path Finding	~	~	✓	✓
Causal DAG Manipulation	✗	✗	✗	✓
Causal DAG Maintenance	✗	✗	✗	✓
Causal DAG Transportability	✗	✗	✗	✓
Intervention	✓	✓	~	✓
Counterfactual	✓	✓	~	✓

Table 7.2: Runtimes (s) by query type and DAG size; bracketed values show runtimes without the causal model, and best runtimes are in **bold**.

#Nodes (#C.Var.)	Extract	Mediators	Confound.	Colliders	Interv.	Counterf.
5k (5)	0.780	0.003 (0.648)	0.004 (0.656)	0.003 (2.218)	0.194 (0.113)	0.016 (0.017)
25k (25)	1.236	0.005 (3.660)	0.004 (8.246)	0.012 (24.003)	0.801 (0.104)	0.055 (0.020)
50k (50)	4.074	0.009 (25.855)	0.013 (14.095)	0.004 (39.596)	10.01 (2.954)	0.926 (0.289)
50k (5)	1.032	0.010 (2.917)	0.008 (4.891)	0.004 (11.358)	0.109 (0.757)	0.023 (0.048)
250k (25)	1.277	0.009 (37.431)	0.009 (82.466)	0.012 (230.949)	0.838 (1.161)	0.056 (0.044)
500k (50)	3.974	0.011 (367.681)	0.011 (771.086)	0.004 (2274.338)	9.951 (3.262)	0.935 (0.293)
500k (5)	1.028	0.008 (20.081)	0.007 (33.072)	0.004 (64.904)	0.106 (4.233)	0.020 (0.472)
2500k (25)	1.207	0.006 (93.703)	0.006 (537.99)	0.003 (961.93)	0.834 (0.314)	0.053 (0.042)
5000k (50)	3.989	0.007 (162.502)	0.005 (1242.618)	0.004 (1842.14)	9.831 (0.653)	0.892 (0.104)

7.5 Prototype and Experimental study

To assess the feasibility of our vision, we implemented a proof-of-concept causal graph database model and conducted preliminary experiments to evaluate the performance of our proposed queries. Since hypernodes aren't currently supported in any graph DBMS, in our prototype we emulate their behavior by creating new nodes and linking them to their respective instances. Using DoWhy [268], we generated synthetic datasets based on causal DAGs that accurately capture realistic dependencies between variables. We constructed three DAGs with 5, 25, and 50 causal variables, ensuring a balanced proportion of mediators, colliders, and confounders. Additionally, we varied the number of instances per causal variable (1000, 10k, and 100k), representing each instance as a node in the graph. We ran experiments on Neo4j 5.12 [269], using a MacBook M3 Pro Max (128GB RAM, 32GB allocated to Neo4j). Code is available at [176].

We tested different types of queries to evaluate the performance. The *Extract Queries* extract causal variables as hypernodes, as described in Section 7.2. The *Mediator*, *Confounder*, and *Collider Queries* retrieve all mediators, confounders, and colliders from the graph. The *Intervention Queries* were implemented by setting the property of each instance to a specified input value and using Cypher aggregators and mathematical operators to solve linear equations. The *Counterfactual Queries* followed the same implementation but were limited to a single instance at a time.

Table 7.2 reports the performances of various queries with our solution integrating property graphs and causal models in the same artifact with hypernodes and comparing it with the case when they are kept separate. We can observe that for the majority of the queries, the comparative solution is significantly less efficient than our integrated model. Specifically, for interventional and counterfactual queries our solution is slightly less efficient as the property graphs grow in size. This can be explained by the fact that, since the hypernodes are not supported by current graph DBMS, there is no efficient way to query both the DAG and the data at the same time while guaranteeing scalability. The results show that runtimes depend on the size of the causal DAGs while the size of observational property graph data has a negligible impact on the performances.

7.6 Extension: Towards a Causal World Ecosystem

The model developed so far assumes a single setting: one property graph, one causal DAG embedded in it, and one analyst asking interventional and counterfactual queries against that pair. The same construction, read at a larger scale, describes something else. An organisation or a research lab does not hold one graph but dozens of independently governed sources, tables, logs, documents, images, audio streams and application stores, over which classical predictors, large language models, retrieval pipelines and agents are trained and run side by side, each consuming the outputs of the others. The causal knowledge that matters there is not contained in any single source: it is latent in the relationships between them, a campaign in a marketing platform, an order in a commerce database, a complaint in a support system and a churn event in the CRM. Making that knowledge available is therefore a data management problem before it is a learning problem, and it is the natural extension of what-if queries from one graph to an ecosystem.

An application can observe that revenue fell after a price change or that a model regressed after fine-tuning, and cannot tell whether the change produced the outcome or what would have happened otherwise, so models trained without an explicit notion of intervention conflate correlation with causation [58, 270] and inherit spurious shortcuts [271]. The gap becomes critical once systems are deployed to act and not only to predict: an agent that changes the system it observes must anticipate the consequences of its own actions, and what would have followed from a different one instead of extrapolate from what has co-occurred before [272, 273]. We therefore propose treating cause and effect as shared infrastructure, and call the resulting artefact the **Causal World System (CWS)**: a persistent, explicit and queryable causal model that overlays the systems and processes of an organisation and is served to analysts, models and agents at once.

Causal machine learning formalises data generation as structural causal models and studies interventions and counterfactuals across supervised, generative and reinforcement-learning settings [274, 58, 271, 270]; we keep that foundation and move the unit of analysis from a single model to a whole ecosystem. World models and JEPA-style architectures learn predictive latent dynamics for perception and control [272, 275, 276, 277, 273]; we replace the implicit, single-model substrate with an explicit and shared one. Data integration contributes the mediated schema and the answering-queries-using-views machinery [278, 279] that we lift from data to causal structure, while data fusion [280] and causal abstraction [281] supply the tools for combining heterogeneous evidence and for reasoning across levels of granularity.

7.6.1 Architecture.

Figure 7.2 shows the architecture, which lifts the mediated-schema duality of classical data integration from data to causal structure. In Global-As-View the global schema is defined as views over the sources, which makes querying easy and the schema fragile when a source changes; in Local-As-View each source is defined as a view over the global schema, so sources evolve independently at the price of turning query answering into the harder problem of answering queries using views [278, 279]. The CWS applies the same duality to causal claims: sources publish local causal views, a mediator reconciles them into a global structural causal model, and

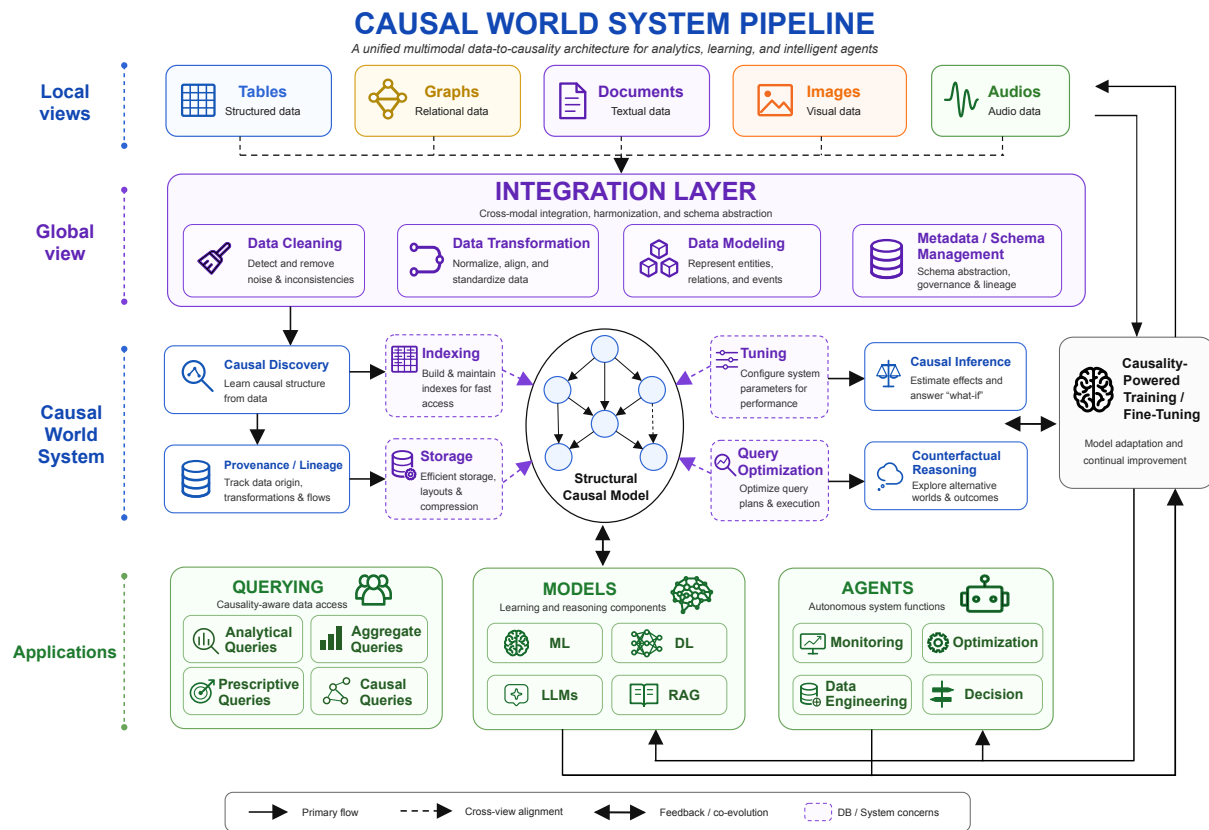


Figure 7.2: Overview of the Causal World System: local views over heterogeneous sources, a mediator that integrates them into a global causal view, and the querying, training and acting systems served by it.

causal queries are answered against that global view by rewriting them over the local ones. A defining commitment is that the result is white box. It is an explicit causal DAG over named variables instead of a latent representation, so every variable, edge and estimate is inspectable and carries its provenance, and an answer arrives with the mechanisms and assumptions it is modeled on.

Local views. Each source is exposed as a local view, the to layer of Figure 7.2. A view declares which global causal variables the source observes, together with the entities, timestamps, granularity, instrumentation, access constraints, provenance and local assumptions of the evidence it provides. Structured sources expose their variables directly; unstructured ones enter through learned encoders that map raw signals onto named ecosystem variables, so that a return spike in a table, its explanation in a support ticket and the product photo that misled the buyer are recognised as one causal story told in three modalities. An encoder is a local view over an opaque source: it does not replace the causal model but populates it, which keeps even opaque modalities subject to the same provenance and identifiability machinery as the rest.

The mediator and the global view. The mediator turns the local views into a global causal view over ecosystem variables. This is where ordinary integration work becomes causal infrastructure: it cleans noisy sources, transforms heterogeneous formats into comparable representations,

aligns entities and differently instrumented fields onto shared causal variables, and maintains metadata, schema mappings and lineage. The result is not a union of source records but a mediated representation in which local evidence can be compared, joined and traced back to its origin, and it plays for causality the role a mediated schema plays for data: it fixes the variables and relationships over which $do(\cdot)$ and counterfactual queries are answered [58].

The causal machinery. On top of the global view the system runs discovery, arbitration, identifiability checking and inference. Constraint- and score-based methods [64, 282], latent-variable variants [283] and continuous-optimisation approaches [284] propose structural relations, and provenance decides how much trust each proposal earns, since an A/B test or a staged rollout licenses a stronger claim than an observational log. Identifiability is tracked per query with ID/IDC and do-calculus [285], and effects that must cross populations or instruments invoke data-fusion and transportability machinery [280]. Because queries repeat and sources change, the system also needs the ordinary database apparatus of storage, indexing and query optimisation, together with incremental maintenance that revalidates only the affected edges and estimands instead of rebuilding the global graph.

Querying at several levels. Views exist at several granularities, so the same system can be interrogated at the altitude a question requires: a source-local view answers questions about one mechanism, a subsystem view answers team-level diagnostics, and the global view answers ecosystem-level questions. Causal abstraction is what keeps these consistent, since coarse and fine views are τ -abstractions of one another [281, 286] and a coarse query can be answered by marginalising over mechanisms it does not depend on with a guarantee that the answer agrees with the fine-grained model. The mediator can then choose the cheapest level that still identifies the effect, and because the graph is explicit at every level, each answer doubles as an explanation: the path of mechanisms that produced the estimate and the assumptions under which it holds can be returned with it.

7.6.2 Using the system

Analysts. A decision maker wants to know which lever to pull, not only what will happen. The same substrate serves a graded spectrum of questions: aggregate and analytical queries about how a quantity evolved, prescriptive queries about which intervention minimises an outcome under a budget, and explicitly interventional and counterfactual ones. Instead of reporting that support tickets will rise next week, the system reports that raising the price by 5% raises tickets by 12% and churn by 3%, and that discounting shipping offsets two thirds of that. These are actionable predictions, that is, estimates of the effect of an intervention expressed in terms of the outcomes a person actually controls.

Agents. An agent embedded in the ecosystem can consult the CWS before acting, simulating the interventional distribution of its candidate actions and comparing their counterfactual outcomes, which turns a reactive predictor into a deliberative actor able to ask whether a different action would have produced a better outcome. Predicting the next state [287] is not

enough for this: choosing between actions requires comparing states that were never observed, and optimising the choice requires attaching a cost to each of them.

Models. The structure that answers human and agent queries is also an inductive bias for the learning systems in the CWS. Conditioning training and fine-tuning on causal structure rather than on raw correlations suppresses spurious shortcuts, improves robustness under distribution shift and lowers the number of samples needed to reach a given level of performance [274, 271]. The explicit skeleton can be injected as invariance constraints [288, 289], as a structural prior on attention or message passing, or as a generator of counterfactually augmented data, and it also sharpens data selection by prioritising samples that are informative about contested mechanisms [290]. Since the constraints are available at every level of the view hierarchy, models can be trained at the granularity of their scope: a source-local predictor against its local view, a subsystem model against an intermediate view, and an ecosystem model against the global one. A mechanism certified once is reused downstream rather than relearned, which is where the saving in samples and compute comes from, and because the skeleton is explicit, the constraints a model was required to respect remain inspectable [291, 292, 293, 294, 295, 296, 297].

7.7 Conclusion

This chapter set out a research direction: making graph databases aware of causal knowledge, so that causal analysis becomes a matter of querying data that never leaves the store. Two steps were taken towards it. The first is a causal property graph model in which the causal DAG is embedded in the data itself through hypernodes and meta-properties (Section 7.2), together with the query-language constructs needed to extract causal paths and to express interventions and counterfactuals (Section 7.3). The proof of concept of Section 7.5 shows that these queries are not only expressible but efficient, with runtimes governed by the size of the causal DAG. The second is the extension of Section 7.6, which reads the same construction at the scale of an ecosystem: many independently sources, published as local causal views and reconciled by a mediator into a global causal structure that analysts, learning systems and agents can query at the level of granularity each of them needs.

Chapter 8

Conclusion of Part II

Part II studied reasoning over alternative versions of a property graph. Chapter 6 addressed the case in which the alternative is a hypothetical update and the quantity of interest is the output of a trained graph neural network, under the requirement that the answer be exact and cheap enough to be asked repeatedly. Chapter 7 addressed the case in which the alternative is generated by an intervention on a causal structure stored in the graph database itself. This chapter summarises the contributions of the two chapters (Section 8.1), states the observations that generalise (Section 8.2), lists the limitations and open directions (Sections 8.3 and 8.4), and introduces Part III (Section 8.5).

8.1 Summary of Part II

Chapter 6 defines the *What-If query*, e.g. the evaluation of a query over a fixed trained GNN’s predictions on a hypothetically updated graph, and places it in the what-if querying tradition of hypothetical OLAP [201], provenance-based hypothetical reasoning [202] and causal SQL extensions [225]. Prior what-if work reasons over transparent query semantics, where an update’s effect propagates symbolically through known operators, whereas a trained GNN is a learned non-linear function with no closed form, so the effect of an update is accessible only by evaluating the model. The chapter therefore poses *incremental what-if evaluation* as the problem of finding a delta rule for a view with no closed form, and answers it with two maintenance strategies, one per type of update. An attribute update is dense over nodes but has rank one: it propagates as an outer-product term plus a correction confined to the rows whose activation changes. A structure update has no low-rank structure, since degree normalization couples it to a whole neighborhood. However, it is bounded by locality: only the L -hop closure of the edited endpoints can change. Both strategies are exact. Both are applied as a *hypothetical overlay* that leaves the base graph and its cache untouched, so that the same hypothetical can be evaluated repeatedly against the same base. We then show the maintenance composes: an evaluated state is a legitimate base for the next update, so a branching, backtracking session pays only for the increment between candidates.

At the depth where GNNs are actually deployed, attribute updates on a fixed aggregator answer $1.9\text{--}4.7\times$ faster than full recomputation, and with no internal activation to invalidate ($L=1$) the maintenance reduces to a pure linear delta, at $14\text{--}37\times$. The advantage decreases

with density: on FAERS, at average degree 425, the two-hop closure of even a 5% edit reaches essentially the whole graph and the curves coincide at parity. The comparison with streaming GNN maintenance is where the strategy pays off. Against Ripple [217] and NeutronRT [218], adapted to the apply-evaluate-rollback pattern over a live graph database, a committed-timeline system must write each hypothetical to the database and roll it back, and re-extract the graph afterwards at seconds per scenario, while an overlay pays extraction once per session and serves each subsequent hypothetical from cache. In the interactive clinical session, with five states, one branch and one backtrack, this is ~ 12 s reduced to ~ 2.5 s end to end, with per-step cost of tens of milliseconds against a full re-run.

Chapter 7 develops a research programme with a proof of concept [16]. Its premise is that causal analysis and data management overlap more than the literature admits: defining a causal DAG is graph data modelling, and causal analysis is largely querying, and specifically path finding, since confounders, mediators and colliders are structural patterns. The chapter’s model contribution is the *causal directed acyclic hypergraph* (cDAH), which extends the property graph with hypervertices so that a causal variable can be a subgraph of the observational data, and annotates edges with conditional and interventional distributions and hypervertices with structural equations. Embedding the DAG in the data model allows a single query to navigate between a causal variable and the instances that realise it. On the language side, mediator and collider identification are already expressible in GQL [24] today, while enumerating the paths needed for statistical adjustment requires the path-based semantics of [251], and extraction and do-calculus require new constructs, for which the chapter proposes EXTRACT and DO-CALCULUS. The prototype over synthetic DAGs [268] of 5, 25 and 50 variables reports that query time tracks the size of the causal DAG and is almost insensitive to the size of the observational data. The chapter closes with the proposal of the Causal World System, in which a mediated *causal* schema, built on data-integration foundations [278, 280], is made explicit and queryable.

8.2 Lessons Learned

A trained GNN as a view. Incremental view maintenance keeps a derived quantity current by applying a delta instead of recomputing [208, 204], and it works because the view is a transparent composition of operators through which a delta propagates by known rules. A trained GNN is a view over the graph except that it is derived, expensive, and it has no symbolic delta rule. Once the problem is posed as an incremental evaluation problem, the technical content of Chapter 6 is the construction of the missing delta rule: exact, because the activation decomposition of Equation (6.3) is an entrywise identity, and cheap, because the perturbation is stored and propagated as a low-rank backbone plus a sparse correction.

Accumulation of approximation errors. Approximate maintenance is attractive because a single answer is usually close enough, and the evaluation of Chapter 6 shows why that reasoning does not survive an analysis session. Ripple’s frozen-weight assumption holds only while the aggregation operator is unaffected by the update, and outside that regime the error is not small: deviations reach 10^{12} to 10^{15} on structure updates, and in the interactive session the reported high-risk fraction is 0.25 against a true value of 0.78. The reason is structural, since a

structure update changes degree normalisation and any update changes attention coefficients, so no tolerance bounds it. It also compounds: because an evaluated state is the base of the next hypothetical (Theorem 6.3), an error introduced at one step is inherited by every state derived from it, and the analyst sees only the final number. Since a what-if query informs a decision not yet taken, an approximate answer that crosses a decision threshold is worse than a slower exact one. The maintenance of Chapter 6 is therefore exact (Theorems 6.1 and 6.2).

Causal analysis as data management. Specifying a causal DAG is graph data modelling: it means deciding which entities and relationships are relevant and how they are represented. Identifying confounders, mediators and colliders, and enumerating adjustment sets, are path queries, and an imperative framework has to reimplement traversal in order to do what a graph database does natively (Table 7.1). Keeping a causal DAG current as its observational data changes is view maintenance, and carrying an effect from one population to another is a mapping problem between local and global schemas. This reading underlies the model of Section 7.2: the DAG is held in the same artefact as the data it abstracts, and one query can move between a causal variable and the instances that realise it. It also explains what the prototype measures: cost tracks the size of the causal DAG and is nearly insensitive to the volume of observational data. The same reading extends upward, where an ecosystem of sources turns the problem into data integration proper, with a mediated schema over local views (Section 7.6).

Declarative causal queries and the user. The knowledge needed to pose a causal question belongs to a domain expert, and current tooling asks that expert to be fluent in causal inference and in a particular library’s interface before they can ask anything at all. Expressing interventions and counterfactuals in a query language removes the second requirement without removing the first, which is the same move that Part I makes when it puts a person at the point where a repair is chosen: in both cases the system supplies the machinery and asks the user only for the judgement that the data does not contain.

8.3 Limitations

The maintenance is restricted to message-passing models. Both procedures of Chapter 6 are derived from the layer form of a message-passing network, in which each layer projects node states, aggregates over neighbours and applies an elementwise activation. The exactness arguments (Theorems 6.1 and 6.2) depend on that form, so architectures that do not compute predictions this way are outside the scope of the method and would require a different maintenance strategy.

The rank-one route assumes an aggregator that the update does not change. An attribute update propagates as a rank-one correction only when the aggregation weights are fixed independently of the node states. On architectures whose weights are data dependent, such as attention, an update changes the coefficients themselves, and what survives is the weaker locality guarantee: the recomputation is bounded by the L -hop closure of the edited nodes, and the speedups measured there are correspondingly smaller because attention has to be recomputed over the affected neighbourhoods.

The update language is narrower than the questions one may want to ask. Interventions are edge insertions and deletions and property updates over an existing population (Section 6.3.1), and updates are assumed to respect the schema. Hypothetical additions and deletions of nodes, and any update that changes the dimensions of the network, fall outside the formulation and need a different maintenance strategy. The chapter also answers what-if questions only: how-to questions, which ask for an update achieving a desired answer rather than for the answer to a given update, are not supported.

Extraction of the causal DAG is done by experts. The model of Chapter 7 starts from a causal DAG that already exists. In practice such DAGs are labelled manually by domain experts or learned by causal discovery algorithms [260, 64].

No graph DBMS supports the model or the queries it requires. Hypernodes have no native support in current systems, so the prototype of Section 7.5 emulates them with ordinary nodes linked to their instances, which is also why interventional and counterfactual queries lose part of their advantage as the observational data grows: there is no efficient way to query the DAG and the data together. The language constructs proposed in Section 7.3 are proposals, and integrating them into a system has to be done without degrading ordinary query performance.

8.4 Future Work

The limitations above define an agenda of future research.

Constructing the causal structure. Causal DAGs are traditionally labelled by domain experts or learned by causal discovery algorithms [260, 64]. Embedding the DAG in a property graph suggests methods that exploit both the topology of the graph and the semantics of the data values, which neither family of techniques currently does. At ecosystem scale the same task becomes a causal generalisation of schema mapping: structure has to be discovered and aligned across many evolving sources, combining observational logs with the natural experiments an organisation already runs, such as A/B tests, staged rollouts and outages, and partial or conflicting local views must be merged into a sound global one with identifiability tracked throughout [64, 280].

Maintenance and transportability. Keeping a causal DAG consistent as the observational data changes is open in the single-graph setting, and it can be addressed by extending work on reactive data management and view maintenance, and by Global-as-View methods that map local causal graphs into a global representation [278]. What is needed is incremental maintenance over causal mechanisms, able to detect stale edges, invalidated estimates and sources that require re-certification [253, 261, 262, 204, 13], together with transportability machinery for carrying an effect from one population or instrument to another [267].

Language and query processing support. No current graph DBMS supports causal queries or the data structures the model requires, and any integration has to be designed so that it does

not degrade ordinary query performance. Work is needed on extending Cypher, and possibly on a domain-specific language for causal graph analytics, so that non-expert users can express causal paths, interventions and counterfactuals declaratively.

Multimodal causal alignment. Tables and logs expose causal variables directly, whereas text, images, audio and video do not. Mapping the latent encodings of JEPA-style models [275] onto shared causal variables with calibrated uncertainty remains open, and requires encoders that declare which causal variables they implicate rather than returning dense embeddings alone.

8.5 Towards the Human Dimension

Both chapters of Part II end, in different ways, at an interface. The maintenance of Chapter 6 is valuable because it makes an interactive session viable, but only if an analyst can compose the sequence of hypotheticals in the first place, which means writing a graph query, reading an aggregate off a model’s output, and deciding what to try next. The language extensions of Chapter 7 are motivated explicitly by the wish that a domain expert without technical knowledge could refine and pose causal queries directly. In both cases the contribution is only as good as the user’s ability to express what they want. Chapter 9 reports what happens when people formulate graph queries: which constructs they misunderstand, where the distinction between schema and data breaks down, and which errors persist after instruction. Its findings serve two purposes in this thesis. First, they provide evidence for the premise of Parts I and II that the user is fallible about the formalism, and they characterise that fallibility by construct. Second, they are a design input for the interfaces through which the repairs of Part I are accepted and the hypotheses of Part II are posed.

Part III

The Human Dimension

Chapter 9

Understanding How Users Query Graphs

The repair framework of Chapter 3 assigns violations to users who are assumed to be fallible, and the difficulty model of Chapter 4 prices their answers accordingly; the language extensions of Chapter 7 are motivated by the wish that a domain expert could pose a causal question without first becoming fluent in a library’s interface. In both cases the claim is about what a non-specialist can and cannot express when interrogating a graph. This chapter measures it.

The study reported here asks in which aspects of the Cypher language users struggle the most, and what causes their errors. Its participants are students of two international master’s programmes who had followed a course on graph processing, and the design has four phases: they write Cypher queries of varying complexity for schema discovery, analytical and aggregation tasks over a genomic dataset, and then answer a post-study survey and, for part of the cohort, a live interview in which they reason about their own mistakes. The analysis proceeds in three steps. A taxonomy of graph query errors organised around Cypher constructs gives the coding scheme; a quantitative analysis relates errors to the type of task, the clauses involved and the participants’ prior knowledge; and a qualitative analysis of the handwritten queries identifies the constructs that go wrong most often, namely `WITH`, `COLLECT`, the definition of graph patterns, and the distinction between the schema and the data. The interviews supply the causes, among them the observation that previous experience with SQL and with programming languages does not transfer cleanly and is itself a source of error. The chapter closes with a set of best practices for database educators. To the best of our knowledge this is the first study of its kind for graph query languages. The chapter is based on *Understanding Student Errors in Graph Query Formulation* [18], published in ACM Transactions on Computing Education.

9.1 Introduction

Graph databases are a novel type of NoSQL store in which data is natively represented as a graph. In particular, the most widely used model is the *property graph* one [298], i.e., a multi-labeled multi-graph with key-value properties attached to both nodes and edges. Graph databases are becoming increasingly popular and are being adopted in different domains, such as the life sciences, finance, security, logistics, and planning, to name a few [241]. Hence, it is paramount to teach students how to use this technology and, in particular, to understand where they have the most difficulties.

Many works have investigated the errors that students make when formulating relational (SQL) queries, as relational databases have been around since the 70s. In particular, these works focus on classifying these errors [299, 300, 301, 302] and investigating their root causes [303, 304] by integrating user studies with the insights from experts [305]. However, students' struggles with NoSQL and especially with graph databases remain under-researched. Works on this end focus on large-scale quantitative studies to investigate the query language features with which students struggle the most [306, 307], ignoring however the reasons why these errors occur. In addition, we argue that replicating the studies done on SQL is not sufficient to understand the difficulties students might have. Indeed, graph database query languages (such as Cypher) differ from SQL-like query languages for multiple reasons; for instance, they have a completely different syntax and semantics. Data stored in graph databases lacks an a priori schema (undefined or unknown) contrary to SQL databases. Moreover, the fundamental construct of graph query languages relies on graph pattern matching as opposed to join-based predicates and thus may need to be approached differently.

To exemplify these differences, let us consider the queries in Figure 9.1. To answer the following question: "Which are the names of all the actors who played in the movie The Matrix?", the user needs to approach the problem differently when using a SQL database (e.g. Figure 9.1(a)) or a graph database (e.g. Figure 9.1(b)). For instance, when writing the SQL query - Figure 9.1(c) - one needs to know how the data is structured (e.g., the fact that there are tables storing information regarding actors and movies) and think about the join conditions (e.g. when joining `Movie`, `Acted_in`, and `Actor` tables). Conversely, when writing the Cypher queries, one has to leverage navigational patterns on the graph (e.g. the edge pattern `-[:ACTED_IN]->` as in Figure 9.1(d)). The latter figure shows a variant of the query where the structure of the data is known (e.g., the node labels `Actor`, `Movie`, and the edge label `ACTED_IN`). One can also express the same query without the labels, as in Figure 9.1(e), obtaining the same results ¹.

Given the above challenges associated with graph query languages, it is crucial to identify the primary barriers to learning. Some of them are shared with SQL (as they are linked to learning how to query a database in general), while others rise specifically from the graph data model or from the particular syntax of graph query languages. In particular, we are interested in answering the following research questions:

- RQ1: In which aspects of the Cypher language do the students struggle the most? Here we are interested in identifying specific constructs that are more difficult for the students - like highlighted in [306] - or if there are other learning barriers.
- RQ2: What are the main causes of student errors? Here we are interested in understanding what are the reasons behind the errors. Previous work highlighted conflicts with previous knowledge and lack of practice [305], or unfamiliarity with new elements introduced by the language [307, 306, 308].

To this end, we designed a four-phase user study with students enrolled in an international master's program in France. In the study, the students had to write Cypher queries of varying

¹Notice that the exact result set of the query depends on the data stored in the database. In general, removing labels from a graph query could return more results.

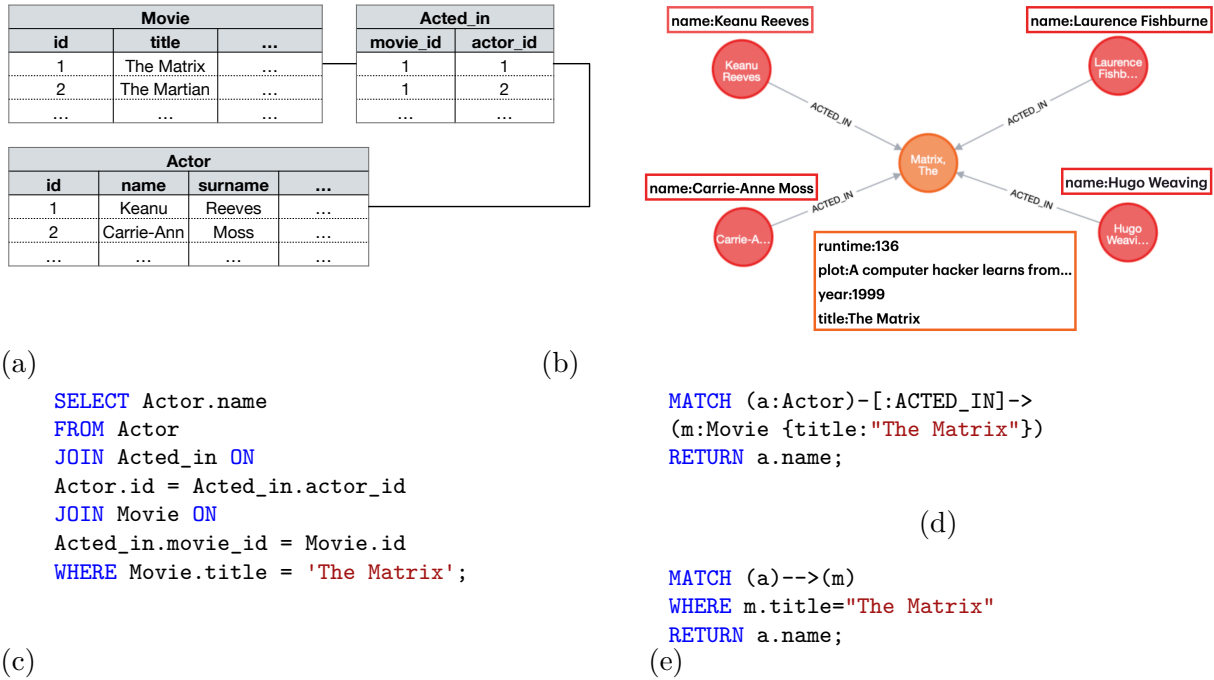


Figure 9.1: Example of a relational (a) and graph (b) data model storing the same information and three queries answering the same information need (e.g., list the names of the actors who played in the movie "The Matrix"). The query (c) is written in SQL while (d) the equivalent Cypher formulation. The last query (e) is the same as (d) without labels.

complexity to solve different tasks such as schema discovery, analysis, and aggregation. Next, to better understand the causes of the participants' errors, we designed an additional post-study survey, and we performed live interviews where we asked the participants to reason about their mistakes. More details on the user study are presented in Section 9.4.

In this paper, we focus on Neo4j's [140] Cypher [23] language, due to its popularity [309] and support in numerous other databases, e.g., SAP HANA [310], Amazon Neptune [242], Memgraph [229], RedisGraph [311], and AgensGraph [312]. Although our study is centered around Cypher, we argue that some of the results would apply to graph queries in recent standard query languages and, specifically, to GQL queries, due to the inherent similarities between the two languages. The latter standard query language has recently appeared [24] and we believe that teaching standard graph query languages will be a compelling challenge for database education in the upcoming years [313].

In our work, we make the following main contributions.

- We build a new taxonomy of graph query errors centered on constructs of the Cypher graph query language.
- We carry out a quantitative analysis of the mistakes. We provide statistics in relation to the type of errors, the affected clauses of the query language, and previous knowledge.
- We perform a qualitative analysis focusing on the hand-written queries and highlight the error-prone constructs (WITH, COLLECT, pattern definition, and schema misunderstanding).

- We investigate the causes of the errors with an additional survey and live interviews with the students.
- We synthesize a list of best practices for database educators to improve the teaching processes and address the issues around learning graph query languages.

To the best of our knowledge, this is the *first study of this kind for graph query languages*. This work contributes to a better understanding of where educators should focus their efforts and, ideally, also of how the design of the graph query languages themselves can be improved.

The remainder of the chapter is structured as follows. Section 9.3 gives a brief introduction to the property graph model and Cypher. Section 9.2 describes the related works. Section 9.4 explains the research procedure and the setting, Section 9.5 presents the results of the study, while Section 9.6 discusses the contextual factors of the errors. Section 9.7 delves deeper into the causes of the errors with semi-structured interviews, and discusses the implications for the education of graph query languages. Finally, Section 9.9 concludes.

9.2 Related Work

Many works studied what are the most common errors students make when querying relational databases using SQL, in order to understand how educators should teach this query language and on which parts they should focus. Taipalus et al. [314] were the first to develop a unified classification of error types in SQL queries. They developed this categorization by conducting a user study with students to expand concepts related to errors previously defined by the literature [299, 300, 301, 302] identifying the following error categories: syntactical [300], logical [301], semantic [299] errors, and complications [302] (i.e., a correct query written with unnecessary constructs). The study covered both common concepts (e.g., single-table expression, grouping, nesting, etc. [315]) and more ambiguous ones (e.g., correlation names and aliases [314]).

Miedema et al. [303] went beyond error classification investigating what are the common student misconceptions while learning SQL that can lead to errors. With a think-aloud study involving 21 students, they analyzed the students' thought processes while writing SQL queries, starting from a schema and a natural language prompt. They found students made errors mainly because of conflicting previous knowledge (of SQL or other languages), wrong or incomplete mental models, and the language itself. These findings were confirmed and expanded in a later study [305] where they integrated the perspective of experts on the reasons why errors occur. They found that most errors stemmed from prior misconceptions and insufficient mastery of the language and its use. They also found that errors come from students mismanaging the complexity of the queries they have to write. In another study [304], they highlight how students typically fail on the first try and arrive at a final solution by iterating over their previous attempts, which often leads to over-complicated or incorrect results. Similarly, Migler et al. [316] studied the learning process of 114 students over 5 weeks in designing SQL queries (introduced by a textual description and an indication of the language construct to use). They analyzed quantitative metrics, e.g., success rate, number of attempts, and time spent on a query, and used them to examine which parts of SQL are the most difficult (e.g., set operations, correlated subqueries, full outer join, and relational division).

There are not many works investigating students' struggles with NoSQL query languages, probably due to the relative novelty of the topic and inclusion in teaching curricula. Alkhabaz et al. [307] investigated which are the most common errors of students in MongoDB. By analyzing ~500 homeworks, they highlighted that students struggled more with the new constructs that are introduced by the language. Chen et al. [306] performed a large-scale quantitative study to investigate which are the most common errors made by students while learning graph databases (in particular the Cypher query language). They analyzed over 40k homeworks where students had to write Cypher queries given the schema of the data and a textual description of the query. They found that students struggle in particular with complex pattern matching, as well as with the `WITH` clause. Similar findings were reported by Alkhabaz et al. [308] in a recent study comparing students' errors in SQL, MongoDB, and Cypher.

Li et al. [317] studied if and how the knowledge of different query languages (SQL, MongoDB, and Cypher) transfers. They divided the students into two groups and taught them the query languages in different sequences, always presenting Cypher last. They found that syntax errors decreased over time, but on the other hand, not many students were able to transfer skills between languages. However, they found that teaching SQL first seems to reduce the number of semantic errors in MongoDB. Similarly, Tshukudu and Cutts [318] found that learners switching between programming languages often carry over semantic misconceptions from familiar ones, particularly during the early stages of learning.

While these studies engaged with several students and highlighted where they struggled the most, they did not explore the reasons for these errors, which is the main objective of our work.

9.3 Property Graphs and the Cypher Graph Query Language

The Property Graph model is an expressive formalism in which nodes and edges are rich objects characterized by multiple labels and a list of key-value properties (a more formal definition can be found in [319]). The schema of a property graph consists of node and edge types, formed by node and edge labels along with the list of property types, respectively. For example, Figure 9.1(b) shows a property graph whose schema consists of two node types, with labels `MOVIE` (in orange) and `ACTOR` (in red) between which there is an edge type with label `ACTED_IN`. Also, one can notice that the `ACTOR` nodes have one property, i.e., `name`, while the `MOVIE` node has a set of properties, i.e., `runtime`, `plot`, `year` and `title`. The graph itself contains the data of four actors who acted in the movie "The Matrix". Cypher [23] is a popular declarative graph query language for writing queries on property graphs. While it was created within the Neo4j company [140] for their database, it was made open source with the openCypher project [249], and it is adopted by a large variety of companies - as mentioned in Section 9.1. The structure of a Cypher query is :

```
MATCH <graph pattern>
WHERE <conditions>
RETURN <result>
```

The core concept of Cypher - and the main difference from SQL - is the *graph pattern matching*, meaning that data is retrieved by specifying navigational patterns within the graph. The `<graph pattern>` could be a single node (e.g., `(m:Movie)`), for retrieving all the nodes of type `Movie`),

a relationship (e.g., `(a:Actor)-[p:ACTED_IN]->(m:Movie)`, for retrieving the triples (`actor`, `ACTED_IN`, `movie`)) or a path (e.g., `(a:Actor)-[p]1,3->(m:Movie)`, for retrieving all the paths of length three from a node of type `Actor` to a node of type `Movie`). Another key difference between Cypher and SQL is its treatment of edge directions. While SQL deals with relationships between tables without considering directional constraints, Cypher explicitly takes edge direction into account when querying relationships between nodes. This allows Cypher to differentiate between directed and undirected relationships, enabling more precise traversal and querying of graph data. Moreover, Cypher adheres to trail semantics [23], meaning that in a given path, relationships (edges) cannot be traversed more than once. This ensures that each edge in a path is unique, which is particularly important for certain types of graph queries, such as those involving traversal without revisiting the same relationship, thus avoiding potential cycles or redundant results. The second part of the query (i.e., the `WHERE` clause) consists of filtering and the last part consists of projecting and returning the results. For example, in Figure 9.1(d), to retrieve all the actors who played in the movie "The Matrix" we need to navigate all the `ACTED_IN` paths between an `ACTOR` and a `MOVIE` node. Then, Cypher also provides other constructs to compute aggregation (e.g., `COUNT`, `AVG`, etc.), and to perform advanced graph operations (e.g., compute the shortest path between two nodes). Other frequent Cypher constructs are `WITH`, `COLLECT`, and `UNWIND`. The `WITH` clause is used to chain multiple query parts together, allowing intermediate results to be passed from one part of the query to the next. It functions similarly to SQL's `WITH` or to subqueries, but in Cypher, it is also used to manage query scope, ensuring that certain aggregations or filtering operations are applied before continuing with the rest of the query. Additionally, `WITH` is essential for defining variables that are needed later in a query while controlling which variables are visible at each step. The `COLLECT` function, on the other hand, is an aggregation function that gathers multiple values into a list. This is particularly useful when querying relationships or nodes and one wants to group related data into a single collection. For instance, `COLLECT` can be used to accumulate all outgoing relationships from a node into a list, which can then be processed or returned as a single unit. When used alongside `WITH`, `COLLECT` enables powerful data transformations and groupings within Cypher queries. The `UNWIND` clause in Cypher is used to transform a list of values into individual rows. This is the inverse of the `COLLECT` function, which aggregates values into a list. With `UNWIND`, one can take a collection (such as a list of nodes or relationships) and break it down into separate, distinct elements. Each item in the list is processed as an individual row, which can then be used in subsequent parts of the query. `UNWIND` is particularly useful when working with collections of data where one needs to treat each element separately. For example, if a query returns a list of nodes and one needs to perform further operations on each node individually, `UNWIND` allows one to efficiently iterate over each element.

9.4 Methods

As shown in Figure 9.2, the entire study was conducted over a period of six weeks. Specifically, the experiment took place in the context of the 2023/2024 master's course on graph processing at two universities in France, the Lyon 1 University and at the ENSIIE Paris engineering school. The course was the same in both institutions, and the lectures covered the property graph model,

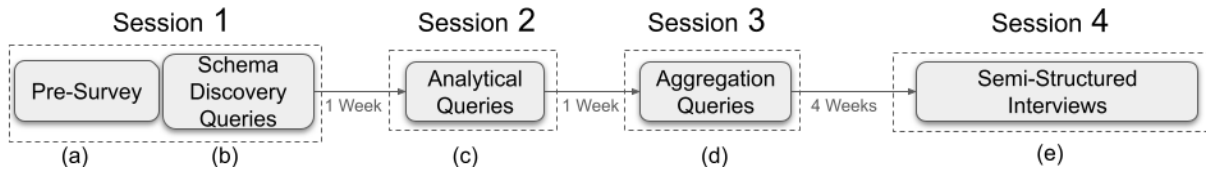


Figure 9.2: Timeline of the study

Category	Question	Involved Concepts
Schema Discovery Queries	Q1: Write a Cypher query that returns the set of all distinct node labels. Δ Q2: Write a Cypher query that returns the set of all distinct relationship types. Δ Q3: Write a Cypher query that returns each distinct node label together with its corresponding set of properties. Δ Q4: Write a Cypher query that returns each distinct relationship type together with its corresponding set of properties. Δ Q5: Write at least three additional Cypher queries that are needed for you to understand the underlying schema structure of the dataset. You can also write more than three queries in case. For each of them, Δ	MATCH, RETURN, DISTINCT MATCH, RETURN, DISTINCT MATCH, RETURN, DISTINCT MATCH, RETURN, DISTINCT MATCH, RETURN, DISTINCT, Difference between schema and data
Analytical Queries	Q6: Write a Cypher query to return each pangenome, together with its associated genes. Δ Q7: Write a Cypher query to return each pangenome, together with its associated spots. Δ Q8: Write a Cypher query to return each pangenome and its associated modules that contain at least a pair of gene families that are more than 80% identical. Δ Q9: Write a Cypher query that returns pairs of inter-pangenome families that both contain annotations and that are more than 80% identical. Δ Q10: Write a Cypher query that returns, for each pangenome, the neighborhood of its gene families containing annotations. Δ	MATCH, Pattern matching, RETURN, DISTINCT MATCH, Pattern matching, edge direction, RETURN, DISTINCT MATCH, Pattern matching, edge direction, WHERE, RETURN, DISTINCT, COUNT MATCH, Pattern matching, WHERE, RETURN
Aggregate Queries	Q11: For every pangenome, write a Cypher query that return its name, together with its number of families that contain annotations. Δ Q12: Write a Cypher query that returns all modules together with the average similarity of the gene families they are comprised of, in descending order. Δ Q13: Write a Cypher query that returns the names of the modules that contain the pairs of gene families that have the minimum and, respectively, the maximum similarity (as indicated by the value of the "identity" property of the "IS_SIMILAR" edge connecting them). Δ Q14: Write a Cypher query that returns the top 10 gene families with the highest number of associated modules. Q15: Return the names of the top 10 pairs of similar families (as defined in Q13), whose associated identity and coverage metrics surpass 0.8, together with the names of their partitions (as indicated by the value of the "subpartition" property associated to the nodes of type "Partition").	MATCH, Pattern matching, WHERE, RETURN, COUNT MATCH, Pattern matching, edge direction, WHERE, RETURN, COLLECT, ORDER BY MATCH, Pattern matching, edge direction, UNION, WHERE, RETURN, COLLECT, ORDER BY, LIMIT MATCH, Pattern matching, WITH, RETURN, COUNT, ORDER BY, LIMIT MATCH, Pattern matching, edge direction, UNION, WHERE, RETURN, ORDER BY

Table 9.1: List of questions together with the related language concepts being tested. Δ = "State also how many answers it returns."

graph schemas, and the formulation of graph queries, both simple (involving basic graph patterns) and complex (composed queries, with correlated clauses, navigational patterns involving more than three edges, and aggregates). Each of these topics was illustrated with practical examples that the students were encouraged to reproduce on their machines. At the end of the course, for three consecutive weeks, the students performed the tasks we assigned during the lab activities (Sessions 1-3). Each session lasted 3 hours. We then analyzed and further examined the results for four weeks and prepared the questions for the semi-structured interviews that we conducted during the last lab session (Session 4). The students submitted the answers using a Google Form.

9.4.1 Dataset and Questions

The questions were designed based on the real-world scenario of analyzing highly interconnected, complex genomic data. The reason behind this choice is to expose students to a realistic, specialized use case with which they did not have first-hand experience and hence which was unlikely to generate biases (i.e., some students knew about the use case more than others).

The questions were designed around the fundamental concepts needed to learn graph queries. As shown in Table 9.1, the questions encompass concepts such as graph pattern matching and aggregation and gradually expose students to increasingly difficult complex query formulation

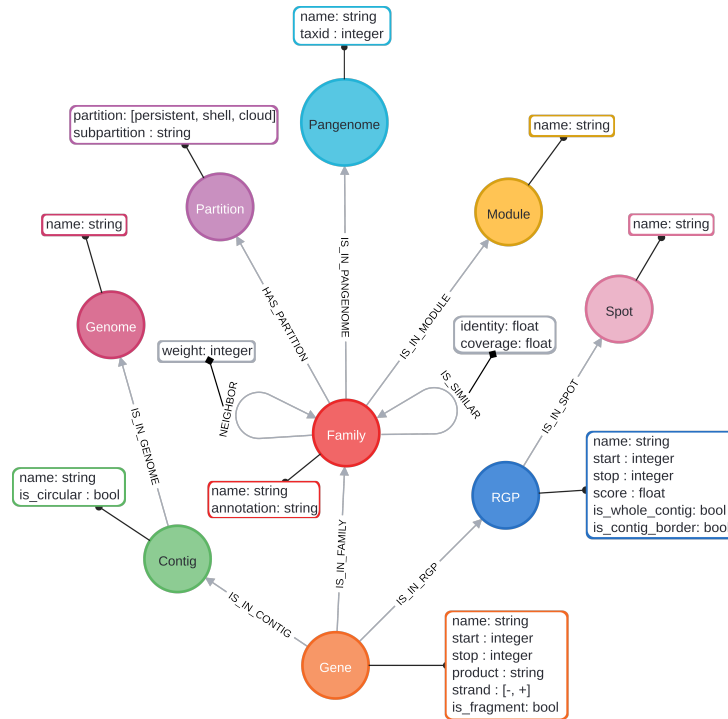


Figure 9.3: Graph schema structure of the pangenome dataset.

tasks. In addition, they target information required in real-world bio-informatics analyses, as conducted by domain experts with whom we consulted [320].

In addition, for each question, we handcrafted the corresponding correct query (i.e., the ground truth) to have an efficient and objective way to annotate the responses submitted by the participants. Given a query, there could be multiple ways to rewrite it and still obtain the same result (for example by reflecting the pattern in the **MATCH** clause). For this reason, to evaluate students' queries we also compared the results.

The considered dataset [320] consists of 10 graphs comprising genome and gene sets shared across different organisms. These are called *pangenome graphs* in the bioinformatics literature [321] and provide a unifying representation amenable to cross-species comparative genomics. Figure 9.3 captures the schema structure of the dataset as follows. **Pangenome** nodes are connected to their genomic **Family** nodes and neighboring and similar nodes of the same label in **Modules**. **Family** nodes contain antibiotic resistance annotations. These nodes are classified as belonging to persistent, shell, and cloud **Partitions**, depending on whether their **Genes** are prevalent in all, some, or only a few corresponding genomes. Finally, **Genes** are part of **Contigs**, associated with particular **Genomes**, and can be part of **RGP**s, located in **Spots**.

9.4.2 Research Procedure

The user study was composed of four sessions and it was approved by the university ethics review board. In the first one, the participants were first presented with survey questions that collected data about their previous knowledge of relational and non-relational databases (including graph-oriented ones), as well as their proficiency with programming and query languages. Then, in the same session, we asked the participants to write **schema discovery queries**. Without

showing Figure 9.3 we asked them to write queries that would help them infer the underlying dataset schema. Among these, there were queries to retrieve the set of node and relation labels (e.g., **Gene** and **IS_IN_FAMILY**) and their properties (e.g., the **name** of a **Gene**). While we specifically gave them the description of four such queries, they also had to provide a minimum of three additional queries to help them grasp the schema structure. Finally, we asked the participants to draw the schema, as they understood it, from the results of the query they wrote. Even though the schema is not mandatory in a graph, understanding it is helpful to write complex queries. For this reason, we first tested students' ability to infer it.

In the second session, we showed the participants the schema of Figure 9.3 and asked them to write five **analytical queries**. These include queries about the genomic data itself, e.g., what **Genes** belong to a **Family**?

We also asked the students, on a voluntary basis, to participate in a third session consisting of writing five additional **aggregate queries**. These include queries that gather statistical information, e.g., counting gene families, retrieving average similarities, and ranking associations. The full list of queries asked is reported in Table 9.1.

After analyzing the responses from the first three parts, we categorized the errors found and designed the semi-structured interview with specific questions to understand the nature and cause of these errors. In the fourth session, we conducted live interviews with the participants to further investigate their misunderstandings and areas of confusion. The interview questions are listed in Appendix D.4, while the summarized answers are in Appendix D.4.

To write and test the queries, the participants used Neo4j Desktop Edition version 4.4.14 [140] on their laptop and submitted their responses in a Google Form. They could make any number of attempts before submitting the queries via the form. Finally, they drew the schema using the Arrow tool[154]. All the questionnaires, together with the responses, are part of the Appendix D.4.

9.4.3 Participants

In total, there were 68 participants. All of them were enrolled in a Master's program, in computer science or related topics, at the universities. Note that one participant was excluded because they did not consent to be involved in the study. We have also excluded another eight participants because they did not submit the first part of the study (that included the consent form). A total of 59 completed the first part of the study. Of them, 45 participants submitted answers to the second part, while the volunteers that attended the third part were 13. Finally, 14 students participated in the interviews.

The participants' ages were between 18-24 (N=42) and 25-34 (N=14) years old (three participants did not disclose their age). Generally, they claimed good knowledge of relational databases, with an average of 3.63 (± 0.83) on a five-point Likert scale, and less experience with NoSQL (2.68 ± 0.97) and graph databases (2.36 ± 1.05). Most of them knew Python (N=56) and Java (N=41), and several also stated knowing some graph query languages, mainly SQL/PGQ [322](N=32) and SPARQL [25](N=21). The informed consent and the knowledge assessment questions are reported in Appendix D.1.

Category	Type	Explanation
Correct		The result is correct
Complication	Added UNWIND Added COLLECT Added WITH Additional information	Student used UNWIND when it was not necessary Student used COLLECT when it was not necessary Student used WITH when it was not necessary Student returned information that were not requested (e.g. results count)
Semantic Pattern	Wrong Pattern w.r.t. Schema Wrong Pattern Wrong Edge Direction Wrong Edge Label Additional Node Missing Node	The pattern does not exist in the graph schema The pattern is valid but it is not the correct one to answer the question The pattern is valid but the edge directions lead to a wrong result The pattern is valid but it contains a wrong edge (in case of multiple edges between nodes) The pattern is valid but it contains an additional node w.r.t. the correct one for the answer The pattern is valid but it misses a node w.r.t. the correct one for the answer
Semantic Where	Wrong Property Wrong Operator Wrong Condition Missing Condition Additional Condition	An operator is applied to the wrong property (e.g. <i>Node.A</i> > 0 instead of <i>Node.B</i> > 0) The wrong operator is applied to a property (e.g. <i>Node.A</i> > 0 instead of <i>Node.A</i> >= 0) The entire condition is wrong (e.g. <i>Node.A</i> > 0 instead of <i>Node.B</i> IS NOT NULL) A condition is missing leading to more results An additional condition is applied leading to fewer results
Semantic Return	Missing Information Missing LIMIT Missing ORDER BY Missing DISTINCT Wrong Information Wrong LIMIT Wrong ORDER BY Wrong DISTINCT	Some information are missing in the result (e.g. results count) LIMIT is missing leading to a wrong result ORDER BY is missing leading to a wrong result DISTINCT is missing in the return clause leading to a wrong result The result contains the wrong information wrt the question LIMIT is used incorrectly (e.g. LIMIT 1 instead of LIMIT 10) The ordering is applied on the wrong objects/values DISTINCT is used incorrectly (e.g. DISTINCT(node.name,id) instead of DISTINCT(node.name))
Other		The query was formulated using Neo4j-specific procedures (i.e., via the CALL statement) rather than standard Cypher syntax.
SyntaxError		The query does not run

Table 9.2: Error taxonomy.

9.4.4 Error Taxonomy and Annotation of the Responses

To better understand the challenges students face when learning Cypher, we developed a tailored error taxonomy for conducting both qualitative and quantitative analyses. In contrast to the approach taken by previous studies [306], our method does not involve analyzing query output logs for syntax error classification. Instead, we are interested in analyzing the students' queries to investigate their correctness in obtaining the desired results. As explained in Section 9.4.2, indeed, students could try their queries multiple times and upload their final versions. Thus, to analyze the errors made by the participants, we extend the work by Taipalus et al. [314]. In particular, we adopt *Complication*, *Semantic*, and *Syntax* errors from their taxonomy, but further extend it to account for specific elements of the graph query language. First, we propose a specialization of the *Semantic* category: *Semantic Pattern* if the error is caused by a mistake in the pattern matching, *Semantic Where* if it is caused by the conditions defined (or missing) in the **WHERE** clause, and *Semantic Return* if the error occurs in the **RETURN** clause. We also propose a fine-grained annotation related to the cause of the error to better investigate the nature of the mistakes. Table 9.2 lists our taxonomy together with an explanation for each label. We define the additional labels as *Missing+Object/Construct* when a missing construct (COUNT, DISTINCT, etc.) or object (node, variable, condition, etc.) leads to a wrong result, *Wrong+construct* when the construct is misused, and *Added/Additional+Object/Construct* when an additional construct or object leads to a wrong result or a complication. In case of syntax errors, the query does not run, while with a semantic one, the query is executed but returns the wrong results. Finally, *Complication* are correct queries that use unnecessary constructs. Specifically, if a query returns the same results but the pattern in the **MATCH** clause is written differently, it is labeled as *Correct*, while a query that needlessly uses additional constructs (e.g. WITH, COLLECT, or UNWIND) is labeled as *Complication*.

This taxonomy helps us differentiate error causes. For instance, *Semantic Pattern* errors may highlight problems due to the graph data model, while *Semantic Where* errors may be related to

general query languages (including SQL). Finally, complications or errors in the `RETURN` clause may be connected to a wrong use of Cypher syntax.

Two of the authors independently examined and annotated the queries by executing them on the same database used by the participants, any disagreement was solved with a discussion with a third author. If the query did not run and the database returned an error, the query was annotated as having a *Syntax* error. If the query did not terminate or if it returned a result set different from the correct one (w.r.t the ground-truth query) it was annotated as a *Semantic* error. Then, the query was compared with the ground truth to fine-grain the annotation. In the majority of the cases, the error was evident (e.g. if only `DISTINCT` was missing in the `RETURN` clause), for the others the queries were re-run iteratively removing constructs to investigate the cause of the error (e.g. removing conditions in the `WHERE` clause or changing the graph pattern). A query can have multiple labels if it contains multiple independent errors (i.e., errors that pertain to different parts of the query), otherwise, only the one that causes the incorrect result is labeled. For instance, if a query contains both a wrong pattern and misses `DISTINCT` in the `RETURN` clause, it is labeled with *Semantic Pattern* and *Semantic Return*. On the contrary, if a query contains an incomplete pattern (*Semantic Pattern*, *Missing Node*) and it misses a condition on the missing node, it is labeled only with *Semantic Pattern*. If the query returned the correct results but contained additional useless constructs (w.r.t the ground-truth query) it was annotated as a *Complication*. Finally, if the query corresponded to the ground truth, it was annotated as *Correct*. In addition, we use the *Other* annotation in case the participants used database procedures instead of writing Cypher queries (i.e. if they used the `CALL` command). We chose not to focus on syntax errors, but rather on the fundamental query language principles and core construct usage that are expected to persist. The main reason is that the syntax of graph query languages will be evolving into the recent standards. Also, the students had access to Cypher’s debugger and a principled analysis of such syntax errors would require real-time logging. In the case of Q5, where students were asked to write additional queries to gather information about the schema, we counted the number of queries that retrieved information about the schema. To evaluate the schemas produced by the students, we used the F1 score. Specifically, we calculated precision and recall for two aspects: node and edge labels, and properties. This distinction was made because not all students included properties in the schemas they generated. In particular, we consider as *true positive* a correctly inserted label/property, as *false positive* an incorrect label or an extra node/edge/property added compared to the correct schema, and as *false negative* a missing node/edge/property.

9.5 Quantitative Analysis (RQ1)

In this section, we present the quantitative results for schema discovery, analytical, and aggregation queries.

9.5.1 Schema Discovery Queries

Figure 9.4(a) shows the results of four queries (Q1-Q4) on schema discovery. Among the first set of queries, most of them are *Correct* (or have *Complications*) and there are only eight semantic

errors. Some students (18) used Neo4j specific procedures, and thus we labeled the answers as *Other*. Finally, only four answers contain syntax errors. Overall, the results highlight that Q1 and Q2 were more straightforward for students, with a higher number of correct responses. In contrast, Q3 and Q4 proved to be more challenging, reflected by the higher number of *Complications* and *Semantic Errors*. Table 9.3 summarizes the in-depth analysis of the *Complications* and the *Semantic Errors*. The majority of *Complications* consist of the additional use of `COLLECT` and `WITH`, where they account together for almost 80% of the *Complications* - while all the semantic errors occurred in the return clause of the queries, mainly due to a missing `DISTINCT` statement. From Figure 9.5, instead, we can see that the most problematic queries were Q3 and Q4. The first one was often over-complicated by the participants, whereas the second one was more susceptible to *Semantic Errors*.

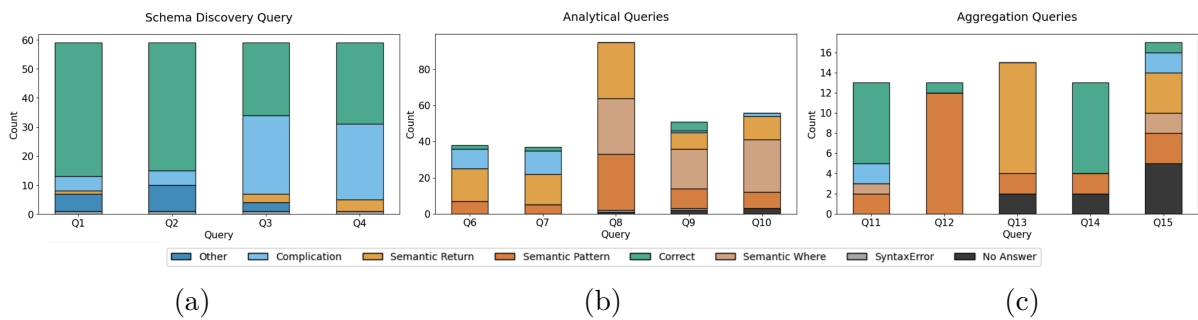


Figure 9.4: Amount of the different types of errors performed by the participants in the schema-discovery queries (a) in the analytical queries (b) and in the aggregation queries (c).

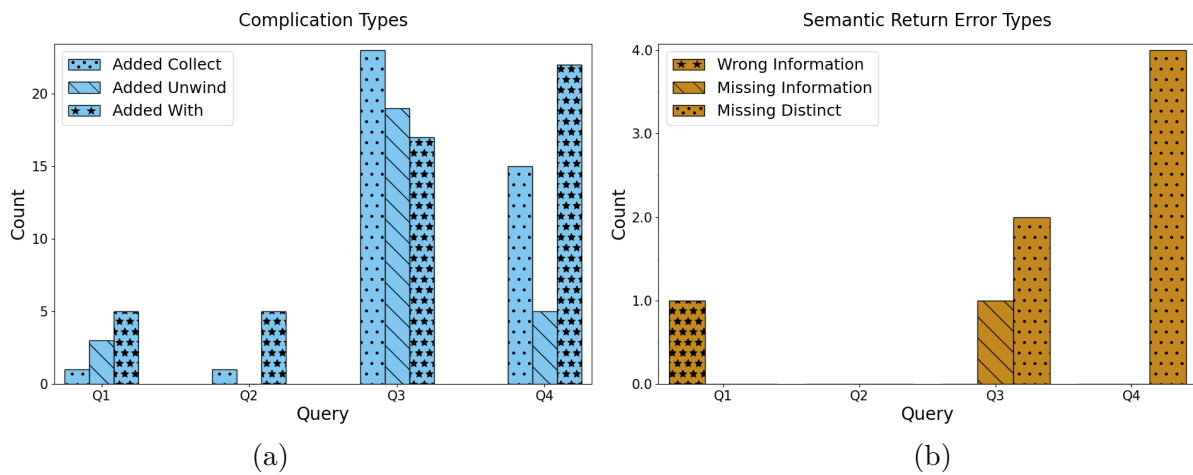


Figure 9.5: Amount of the different types of complications (a) and semantic return errors (b) performed by the participants in the schema-discovery queries.

Additional Queries and Schema Derivation

As described in Section 9.4.4, we labeled the responses to Q5 according to how many useful queries the participants wrote. The results were: 29 responses had zero useful queries; 19 responses had one, indicating some degree of usefulness; seven responses had two, showing a moderate level of usefulness; and four responses had three, indicating that they were highly

Category	Type	Count	Count / Category Total	Count / Total
Complications	Added COLLECT	49	0.422	0.395
	Added WITH	40	0.345	0.323
	Added UNWIND	27	0.233	0.218
	Total	116	1	0.935
Semantic Return Errors	Missing DISTINCT	6	0.75	0.048
	Missing Information	1	0.125	0.008
	Wrong Information	1	0.125	0.008
	Total	8	1	0.065
Total		124	1	1

Table 9.3: Number of error categories and type occurred writing schema discovery queries. We report the absolute number, the percentage of occurrence both within the category and with respect to the whole set of errors.

useful. The high number of responses with zero useful queries may indicate a fundamental misunderstanding of how to utilize the schema to gather relevant information.

We also asked the participants to draw the graph schema using the knowledge gathered during the schema discovery queries. We computed the F1 score of the schema labels and the schema properties. Concerning the schema labels, the average F1 score is 0.929 ± 0.164 while for the schema properties, the average F1 score is 0.453 ± 0.485 . Some students (26) did not write the properties in the schemas, lowering the average. Finally, to investigate if there was a relation between writing a correct query and the quality of the drawn schemas, we computed the Pearson correlation between the results of the schema discovery queries (number of correct answers and semantic errors as well as the results of Q5) and the F1 scores, but we did not find any statistically significant correlation ($p > 0.05$).

9.5.2 Analytical Queries

Figure 9.4(b) shows the results for the first part of the analytical queries (Q6-Q10), highlighting the types of errors and correct responses. This section of our study focused on more advanced queries, requiring students to analyze the provided graph schema.

Correct responses were notably few, with only 9 occurrences, and *Complications* were present 27 times, as shown in Table 9.4. These correct responses and complications were mainly concentrated in the first two queries, underscoring the increased complexity that students faced as they progressed through the more advanced queries. The over-use of COLLECT was still frequent, particularly in Q7 (13 cases) and Q6 (11 cases) as shown in Figure 9.6(a). Notably, there were no complications involving WITH and UNWIND in this phase. For completeness, an extended version of Table 9.4, including these clauses and the other from the taxonomy, is in the supplementary materials. Overall, the results from this part of the study show that while students may grasp the basics of Cypher queries, they face significant challenges when it comes to more advanced and nuanced aspects of graph querying.

One of the most significant findings was the prevalence of *Semantic Where Errors*, which were the most common (82/233). This indicates that students struggled significantly with formulating correct conditional statements within their queries. The most frequent mistake was due to a

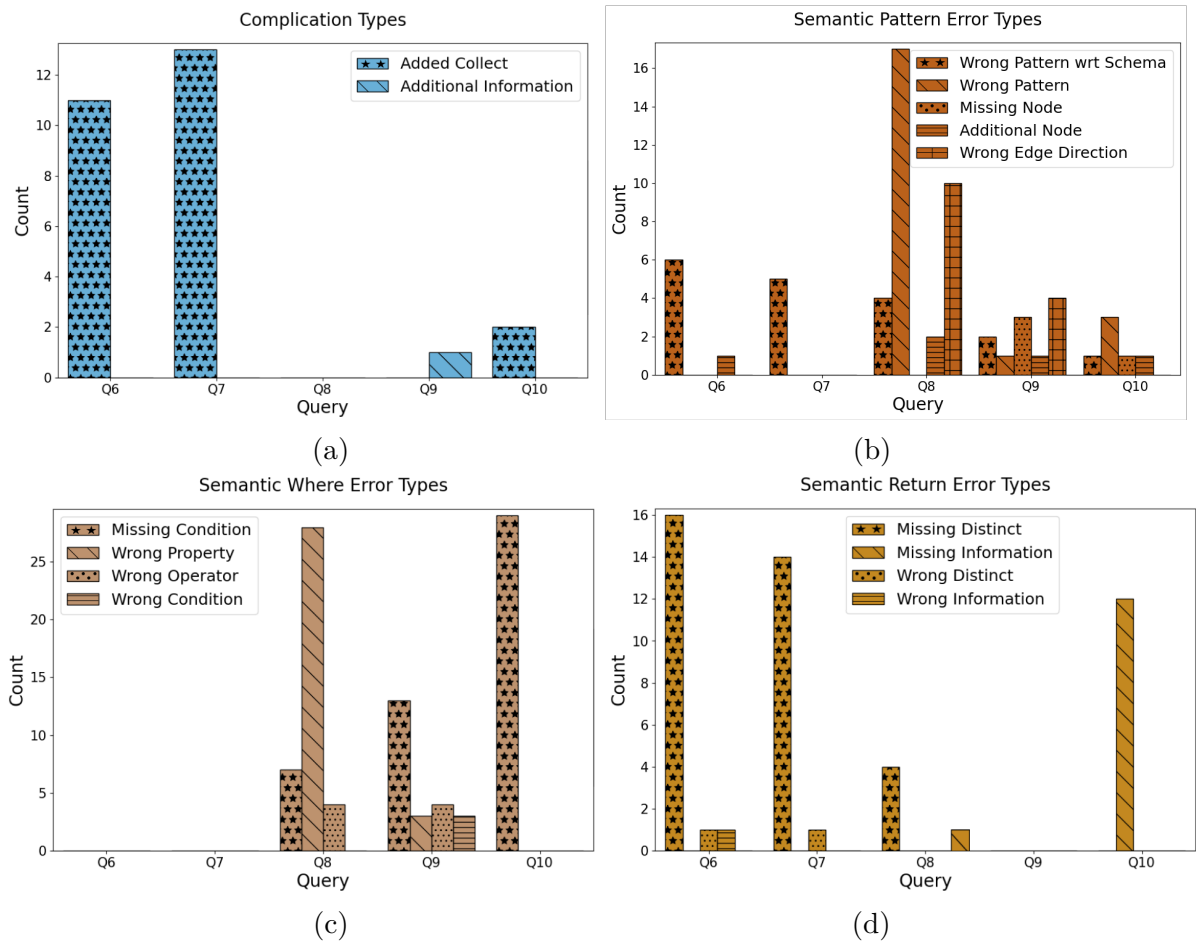


Figure 9.6: Amount of the different types of complications (a), semantic pattern errors (b), semantic where errors (c) and semantic return errors (d) performed by the participants in writing analytical queries

Category	Type	Count	Count/ Category Total	Count / Total
Complications	Added COLLECT	26	0.963	0,096
	Additional Information	1	0.037	0,004
	Total	27	1	0.1
Semantic Pattern Errors	Wrong Pattern w.r.t. Schema	18	0.29	0,067
	Wrong Pattern	21	0.339	0,078
	Missing Node	4	0.065	0,015
	Additional Node	5	0.081	0,019
	Wrong Edge Direction	14	0.226	0,052
	Total	62	1	0.230
Semantic Where Errors	Wrong Property	31	0.34	0,115
	Missing Condition	49	0.538	0,181
	Wrong Condition	3	0.033	0,011
	Wrong Operator	8	0.088	0,030
	Total	91	1	0.337
Semantic Return Errors	Missing DISTINCT	34	0.378	0,126
	Missing Information	13	0.144	0,048
	Wrong Information	41	0.456	0,152
	Wrong DISTINCT	2	0.022	0,007
	Total	90	1	0.333
Total		270	1	1

Table 9.4: Number of error categories and type occurred writing analytical queries. We report the absolute number, the percentage of occurrence both within the category and with respect to the whole set of errors.

missing condition in the **WHERE** clause (49/82), followed closely by conditions on wrong properties (31/82). This suggests a fundamental difficulty in understanding how to accurately filter and retrieve data based on specific criteria within the graph. Additionally, *Semantic Return Errors* were notably high (88/233). These errors involved issues with specifying the correct data to be returned from a query. A frequent problem was the absence of the **DISTINCT** keyword, with 16 such instances in Q6 and Q14 in Q7, as shown in Figure 9.6(d). This highlights a challenge in comprehending how to effectively extract and present the desired results from the graph database. *Semantic Pattern Errors* were also prominent (63/233). These errors involved mistakes in identifying and utilizing the correct patterns to match and retrieve data within the graph. For instance, in Q8, there were 17 instances of *wrong patterns* and 10 instances of *wrong edge direction* as shown in Figure 9.6(b). Even with the schema available, some students tried to use non-existent patterns, while others struggled with defining the pattern correctly. This indicates that as the complexity of the tasks increased, students found it increasingly difficult to construct and apply the appropriate graph patterns.

9.5.3 Aggregate Queries

Figure 9.4(c) presents the outcomes of aggregate queries (Q11-Q15), categorizing the results into correct responses and various types of errors. As specified in Section 9.4, 13 students participated in the final part of the study, as their involvement was voluntary. To ensure a fair analysis of this part of the study, we isolated the results of the 13 students in the previous queries to ensure we did not encounter *survivor bias* - i.e., verifying the students who did the

Category	Type	Count	Count/ Category Total	Count / Total
Complications	Added WITH	2	0.5	0,038
	Additional Information	2	0.5	0,038
	Total	4	1	0.075
Semantic Pattern Errors	Wrong Pattern w.r.t Schema	3	0.143	0,057
	Wrong Pattern	1	0.048	0,019
	Missing Node	1	0.048	0,019
	Additional Node	3	0.143	0,057
	Wrong Edge Direction	13	0.619	0,245
	Total	21	1	0.396
Semantic Where Errors	Missing Condition	2	0.667	0,038
	Wrong Operator	1	0.333	0,019
	Total	3	1	0.057
Semantic Return Errors	Missing Information	14	0.560	0,264
	Wrong Information	1	0.040	0,019
	Missing LIMIT	10	0.400	0,189
	Total	25	1	0.472
Total		53	1	1

Table 9.5: Number of error categories and type occurred in writing aggregate queries. We report the absolute number, the percentage of occurrence both within the category and with respect to the whole set of errors.

final part were not exceptionally different from the rest. We verified that the distribution of correct responses and semantic errors was not statistically different from the distribution of the entire cohort. The tests for the survivor bias are in the additional material. This part focused on aggregate queries, which required students to construct intricate patterns and use aggregating clauses. Table 9.5 summarizes the results. The *Semantic Pattern Errors* were the most prevalent, with students committing 21 such errors out of 53. Despite the complexity of the tasks, there were 19 *Correct* responses, suggesting that some students were able to successfully write advanced queries. Then, 15 out of 53 errors were *Semantic Return Errors*, indicating difficulties in specifying the correct data to be returned from the queries. The majority of them correspond to missing information in the results, suggesting a possible misunderstanding of the question. A considerable number of errors (12) is given by the incorrect use of LIMIT. Interestingly, the incidence of *Semantic Where Errors* decreased to three, compared to earlier parts of the study, while the number of *Correct* responses and *Complications* both increased. Although it is difficult to directly compare these results with those from previous sections due to the smaller sample size, this trend might suggest that the primary difficulties students face are related to the WHERE clause of the queries.

From Figure 9.7 we can see that Q12 and Q13 were the most difficult queries. In Q12, the participants had difficulties in writing the graph pattern, while in Q13, the participants made mistakes on what the query should return.

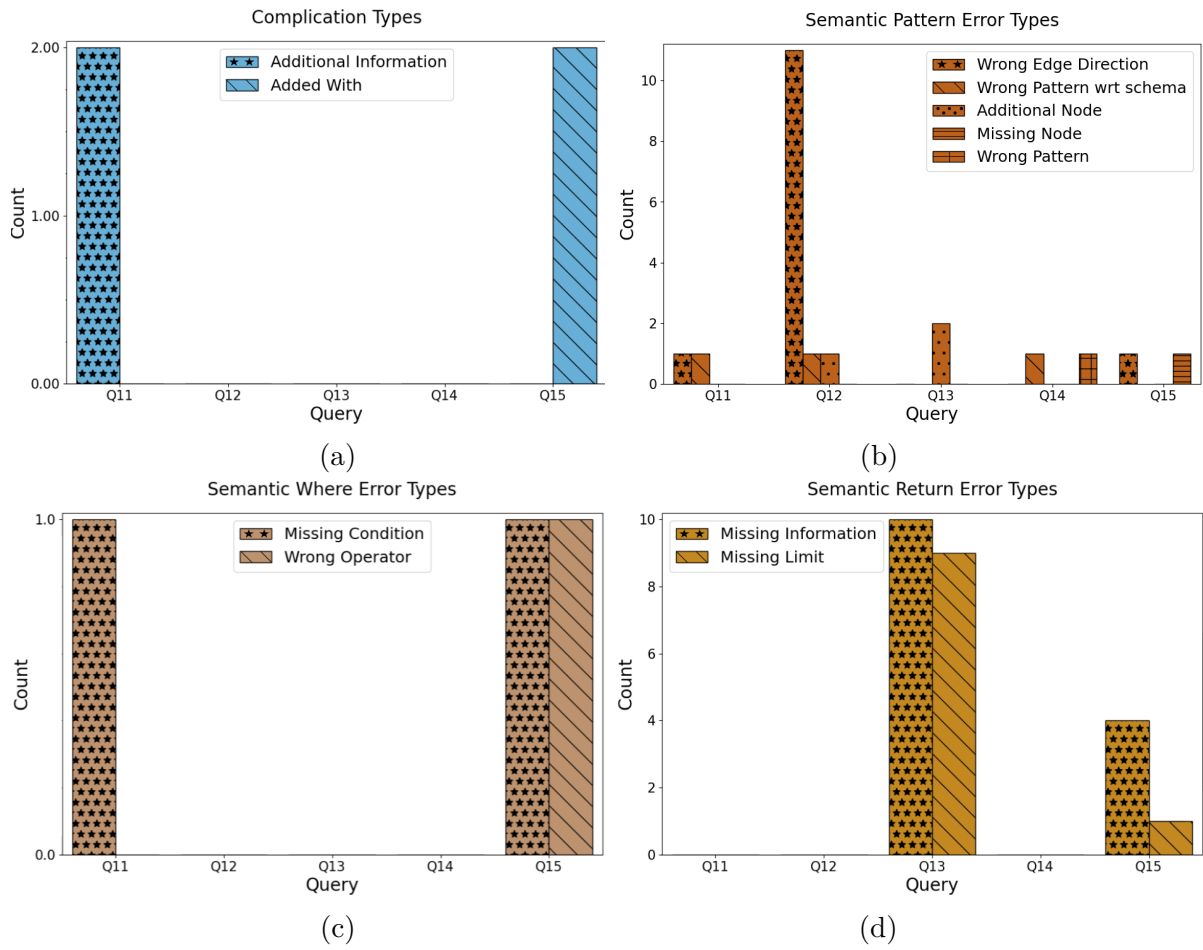


Figure 9.7: Amount of the different types of complications (a), semantic pattern errors (b), semantic where errors (c) and semantic return errors (d) performed by the participants in writing aggregate queries.

9.6 Qualitative analysis (RQ1)

In this section, we dive into a qualitative analysis of the errors and complications identified in students' Cypher queries. While the quantitative data provided an overview of the frequency and types of errors, a qualitative approach allows us to explore the underlying reasons and contextual factors contributing to these mistakes. By examining specific examples from the students' work, we aimed to uncover patterns and insights that may inform better instructional strategies and targeted interventions. We provide detailed explanations of these errors, supported by representative student query examples (errors marked in red). This approach helps highlight the specific challenges students face and the common misconceptions that lead to these errors.

9.6.1 The use of COLLECT

The misuse of the COLLECT keyword was the cause of many *Complication* (75/147). Interestingly, we noticed that students tend to over-engineer simple queries. For instance, Q3 and Q4 ask to list the node and edge labels together with the associated properties. An example of a recurrent *Complication* we detected is:

```
MATCH (n)
RETURN labels(n) AS NodeLabel, COLLECT(DISTINCT keys(n)) AS Properties
```

Then, we often found that the incorrect use of COLLECT in *Semantic Return* errors was often coupled with a missing DISTINCT. For example, in the following query (Q6) DISTINCT was required to return unique genes, but 16 out of 37 students used only COLLECT:

```
MATCH (p:Pangenome)<-[:IS_IN_PANGENOME]-(f:Family)<-[:IS_IN_FAMILY]-(g:Gene)
RETURN p, COLLECT(g) as genes
```

9.6.2 The use of WITH

The second most popular *Complication* - in agreement with previous works [306, 308] - is due to the use of WITH, often paired with UNWIND (27/147). An example of a recurrent *Complication* we detected is:

```
MATCH (n)
WITH DISTINCT labels(n) AS NodeLabels
RETURN NodeLabels, count(NodeLabels) AS NumberOfDistinctLabels
```

The WITH clause allows chaining query parts and affects variables in scope. In this example, though, there is no need to use WITH to carry the scope of the variable *n* into the final part of the query. We found the same application of WITH in combination with UNWIND as in the following example:

```
MATCH (n)
WITH LABELS(n) AS labels
UNWIND labels AS label
RETURN DISTINCT label, COUNT(DISTINCT label) AS num_labels
```

9.6.3 Missing and Wrong Conditions

The majority of *Semantic Where* errors were caused by a missing condition (51/94) in the WHERE clause. Analyzing the answers given by the students, we found two main types of missing condition errors: the first one is related to dealing with complex patterns, and the second one concerns filtering. When querying a graph, it could happen that patterns are unintentionally mapped to cycles.

Given the following pattern :

```
(p1:Pangenome)-[]-(f1:Family)-[]-(p2:Pangenome)
```

the trail semantics of Cypher [23] ensures that an edge is traversed only once during the pattern matching, and thus nodes `p1` and `p2` are different. This is no longer true when dealing with longer or more complex patterns. An example consists of the following student's answer to Q9:

```
MATCH (p1:Pangenome)-[:IS_IN_PANGENOME]-(f1:Family)
-[r1:IS_SIMILAR]-(f2:Family)-[:IS_IN_PANGENOME]-(p2:Pangenome)
WHERE r1.identity >= 0.8
AND f1.annotation IS NOT NULL
AND f2.annotation IS NOT NULL
RETURN f1.name, f2.name
```

In this example, nodes `p1` and `p2` could be the same, and so, it is important to specify `p1.taxid <> p2.taxid` (the nodes are distinct) in the WHERE clause to get the correct number of results.

As for the second case of *Missing Condition*, an example consists of this answer to Q10:

```
MATCH (p:Pangenome)-->(f:Family)-[ss:NEIGHBOR]-(g:Family)<--(p:Pangenome)
WHERE g.annotation IS NOT NULL
RETURN DISTINCT p.name, f.name
```

In this case, the students did not add the condition `f.annotation IS NOT NULL`. The question was "Write a Cypher query that returns, for each pangenome, the **neighborhood of its gene families containing annotations**" and thus we traced back the cause of this error to a language misunderstanding or a question incomprehension.

We also found mistakes due to *Wrong Condition* in the WHERE clause: caused by applying it on the wrong property of a node, or by applying the wrong logical operator in a condition. We found respectively 31 and 9 of these errors. An example consists of this answer to Q8:

```
MATCH (p:Pangenome)<--(f1:Family)-->(m:Module)
MATCH (m:Module)<--(f1:Family)-[sim]-(f2:Family)
WHERE sim.identity>0.8
WITH p, collect(DISTINCT m.name) as modules
RETURN p.name as pangerome_name, modules
```

In this case, the right query would have `WHERE sim.coverage>=0.8`; thus, both the property and operator were wrong.

9.6.4 Pattern Definition

Wrong Pattern definition was the cause of 83 over 399 errors. In particular, 21 consisted of a pattern that did not exist in the dataset. An example is this answer to Q6, in which they used `:ASSOCIATED_WITH` which is not present in the graph:

```
MATCH (p:Pangenome)-[:ASSOCIATED_WITH]->(g:Gene)
RETURN p, collect(g) AS Genes
```

Even if the students were given the correct schema before answering the analytical queries, some of the participants (6 out of 54) still tried to use non-existent patterns with the `OPTIONAL MATCH` clause. For example:

```
MATCH (p:Pangenome) OPTIONAL MATCH (p)-[:IS_IN_PANGENOME]->(s:Spot)
RETURN p, COLLECT(s) AS associated_spots
```

Moreover, 27 of the *Semantic Pattern Errors* consist of *Wrong Edge Direction*. For instance, in Q12 some students wrote:

```
MATCH (m:Module)-[:IS_IN_MODULE]-(f:Family)-[s:IS_SIMILAR]-(f2:Family)
RETURN m.name AS Module_Name, AVG(s.identity) AS avg_similarity
ORDER BY avg_similarity DESC
```

To correctly answer this question, the right pattern to use is :

```
(m:Module)<-[:IS_IN_MODULE]-(f:Family)<-[s:IS_SIMILAR]-(f2:Family)
```

This is a fundamental difference with respect to SQL, where there is not the concept of direction of relationships. When defining a graph pattern instead, expliciting edge directions reduce the number of matched subgraphs. Furthermore, 13 of the errors consist of a correct pattern but incomplete (*Missing Node*) or too specific (*Additional Node*). For example, this answer to Q9 :

```
MATCH (f:Family)-[ss:IS_SIMILAR]->(g:Family)
WHERE ss.coverage >= 0.8
AND f.annotation IS NOT NULL
AND g.annotation IS NOT NULL
RETURN distinct f.name, g.name
```

is missing `:Pangenome` in the pattern:

```
(p1:Pangenome)-(f1:Family)-[r1:IS_SIMILAR]-(f2:Family)-(p2:Pangenome)
```

and thus producing more results, while the following answer to Q12 has a too specific pattern, producing fewer results:

```
MATCH (m:Module)-[:IS_IN_MODULE]-(f:Family)-[r:IS_SIMILAR]-(f2:Family)-[:IS_IN_MODULE]-(m)
WITH m, AVG(r.identity) AS avg_similarity
RETURN m.name AS module_name, avg_similarity
ORDER BY avg_similarity DESC;
```

Finally, we detected 22 errors in which the students used a wrong pattern to write the query. To illustrate, consider the below student answer to Q10.

```
MATCH (p:Pangenome)<-[:IS_IN_PANGENOME]-(f:Family)
WHERE f.annotation IS NOT NULL
WITH p, f MATCH (f)-[]-(g:Gene)-[]-(f2:Family)
WITH p, f, collect(f2) AS neighbors
RETURN p, f.annotation AS annotation, neighbors
```

In this case, while the pattern is valid (i.e., exists in the database), it is not the correct one to answer the question.

9.6.5 Schema Misunderstanding

In our analysis of additional queries aimed at discovering the schema (Q5), we observed a diverse range of approaches and varying levels of misunderstanding. Out of 59 students, 18 focused on counting nodes, properties, and frequencies of edges and patterns within the graph. This approach suggests a gap in comprehension regarding the purpose of schema discovery, or an inability to distinguish between the different levels of queries required. Moreover, 23 students misunderstood the question and repeated their previous queries. The remaining students focused on understanding the edge directions and the connectivity between a pair of nodes. Only five of the students asked queries related to disconnected nodes. This indicates a lower recognition of the importance of understanding isolated elements within the graph, which can be crucial for identifying schema completeness and data integrity issues. On the other hand, analyzing the schemas drawn by the students, concerning the labels (and properties, when specified) the students generally performed well. Concerning the errors, we found three particular cases, illustrated in Figure 9.8. The first example in Figure 9.8(a) shows an isolated error where a student duplicated the node `:Family` instead of using self-relationships, while Figure 9.8(b) shows a case in which the student put two labels on the same edge instead of drawing two different edges (this happened with three participants). The last isolated case corresponds to Figure 9.8(c): in this case, the student did not investigate the edge directions in Q5 and drew two edges for every relationship, one for each orientation.

9.6.6 Syntax Errors

As expected, since the students could retry their queries as much as they wanted, we found only 12 *Syntax Errors*. The majority of them consist of typos, while two of them are noteworthy. Namely, *Invalid input Count(*)*, caused by using `*` inside the `COUNT` operator like in SQL, while in Cypher the universal match is done by using unlabeled nodes, and *PatternExpressions are not allowed to introduce new variables*, caused by trying to create new variables in a pattern, like in the following example (an answer to Q9):

```
MATCH (f1:Family), (f2:Family)
WHERE f1.pangenomeId <> f2.pangenomeId
AND f1.annotation <> "" AND f2.annotation <> ""
```

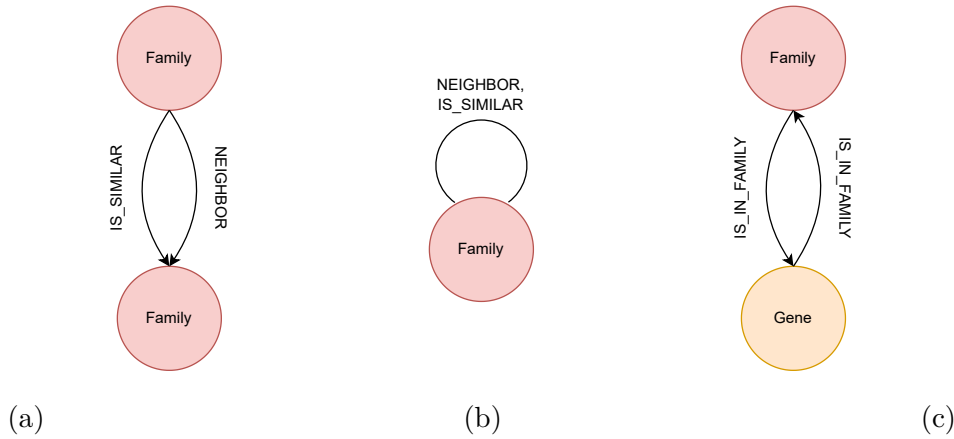


Figure 9.8: Examples of errors in schema drawn by the students, duplicating the node to represent a loop (a), multiple labels on the edge (b), and double edges (c).

```

AND size((f1)-[:CONTAINS]->(g1:Gene)) * 0.80
< size((f2)-[:CONTAINS]->(g2:Gene))
RETURN f1 AS family1, f2 AS family2, COUNT(*) AS total_results

```

9.7 Investigation on the cause of the errors (RQ2)

After conducting a comprehensive analysis, both quantitatively and qualitatively, we investigated the causes of the errors committed by the students. To do so, first we carried out a correlation study between the errors made by the students and their self-reported prior knowledge collected in the first phase of the user study to see whether we could observe an effect as in the previous works related to SQL databases [305]. Second, we conducted additional semi-structured interviews on specific concepts that caused most problems in the previous phases. In particular, we identified four main aspects to investigate: **WITH**, **COLLECT**, the graph pattern formulation, and the schema. After conducting a comprehensive analysis, both quantitatively and qualitatively, we investigated the causes of the errors committed by the students. This interview included specific questions aimed at uncovering the underlying reasons behind the errors. More precisely, we posed questions about the role of Cypher operators, as well as examples of queries with and without errors to check if students were able to recognize and understand them. We asked about the **WITH** and **COLLECT** statements, which represent the main causes of complications during the entire study. We also focused on the role of the schema and pattern composability. The precise questions are reported in the supplementary material. Participation was on a voluntary basis and open to anyone who did at least one session of the study, and 14 students opted to participate. While the questions were in English (and an English proficiency certification is required to be enrolled in the masters), the participants could answer either in French or in English.

To ensure a fair analysis of this part of the study, we isolated the results of the students in the previous queries to ensure we did not encounter survivor bias - i.e., verifying the students who did the final part were not exceptionally different from the rest. We verified that the distribution of correct responses and semantic errors was not statistically different from the distribution of the entire cohort. Finally, we analyzed the results of the interviews, identifying three main

reasons for errors: confusion with previous knowledge, challenges with understanding the role of the schema, and misunderstanding of graph query elements.

9.7.1 Correlation Analysis

We used the Spearman correlation coefficient to compare the students' errors with their self-reported knowledge of Relational Databases, NoSQL Databases, and Graph Databases. Comparing the categories of errors made in the schema discovery queries, we found a positive correlation between *Complications* and NoSQL knowledge (0.250) and a negative correlation with *Correct* responses (-0.274). No significant correlations were found with the other types of database knowledge. This indicates that students with more NoSQL knowledge write more over-complicated queries. Interestingly, a further correlation analysis showed that these complications include **COLLECT** and **WITH** statements. For the analytical queries, we observed a positive correlation between relational database knowledge and *Semantic Return Errors* (0.303) as well as *Semantic Where Errors* (0.295). Relational DB knowledge was also correlated with the increase of *Semantic Pattern Errors* in aggregation queries (0.290). This could indicate that relational knowledge conflicts with some aspects related to graph query (as discussed in more detail in Section 9.7.5).

Interestingly, the number of *Correct* was also negatively correlated to previous graph database knowledge (-0.268). However, 20 out of the 59 students who made such errors claimed previous exposure to the Semantic Web Stack, in particular to RDF and SPARQL. While similar in functionality to property graphs and Cypher, these do not provide the same level of support for defining attributes on properties and handling n-ary relations.

Schema Discovery Queries					
	Correct	Complication	Semantic Pattern	Semantic Where	Semantic Return
Relational DB	-0.105 (0.431)	0.019 (0.886)	-	-	-0.078 (0.558)
NoSQL DB	-0.274 (0.036)	0.250 (0.048)	-	-	0.015 (0.908)
Graph DB	-0.224 (0.088)	0.215 (0.102)	-	-	-0.120 (0.366)
Analytical Queries					
	Correct	Complication	Semantic Pattern	Semantic Where	Semantic Return
Relational DB	-0.080 (0.547)	-0.122 (0.356)	-0.067 (0.617)	0.295 (0.023)	0.303 (0.020)
NoSQL DB	-0.192 (0.143)	-0.176 (0.183)	-0.013 (0.922)	0.239 (0.069)	0.151 (0.255)
Graph DB	-0.268 (0.040)	-0.136 (0.306)	0.127 (0.336)	0.213 (0.105)	0.145 (0.272)
Aggregation Queries					
	Correct	Complication	Semantic Pattern	Semantic Where	Semantic Return
Relational DB	0.120 (0.365)	0.115 (0.385)	0.290 (0.026)	-0.135 (0.306)	0.083 (0.533)
NoSQL DB	-0.026 (0.843)	0.015 (0.913)	0.041 (0.758)	-0.109 (0.409)	-0.069 (0.604)
Graph DB	-0.100 (0.452)	-0.015 (0.913)	0.004 (0.977)	-0.052 (0.693)	-0.099 (0.454)

Table 9.6: Spearman Correlation between the declared previous knowledge about Relational Databases, NoSQL Databases and Graph Databases and the type of errors. Between brackets are the corresponding p values. Bold values indicate a significant correlation ($p < 0.05$).

9.7.2 Understanding the overuse of WITH

Concerning the use of `WITH`, in the interviews we propose two queries asking the students to select which one is correct and to explain the difference between them:

```
// both queries are correct because they return the same elements.
```

1. `MATCH (n)`
`RETURN DISTINCT labels(n), collect(keys(n)) AS properties`
2. `MATCH (n)`
`WITH DISTINCT labels(n) AS labels, n`
`RETURN labels, collect(keys(n)) AS properties`

Even if both queries are correct, six students chose the second one saying that using `WITH DISTINCT` is the only way to reduce the redundancy in the result. Four students opted for the first, considering the use of `WITH` "useless", while only four students correctly selected both queries. This disagreement highlights confusion in their understanding of the construct. The answer we expected is that both queries are correct, but the use of `WITH` in the second one is not necessary. As explained in Section 9.3, the `WITH` operator chains subqueries. To better investigate how this was understood by the students, we asked them to explain its use in Cypher. The two main answers were: to chain subqueries (five students) and to reuse the variables (nine students), showing that this construct indeed creates confusion among the students. To understand the root of this misconception, we asked them where they find something similar to `WITH` in other programming languages or tools. Their answers include: `AS` (7) or `WITH` in SQL (3), "is reminiscent of programming languages in general" (2), the Pandas Python library (2), `LINQ` in C# (1) and promises in JavaScript (1). Since several students mentioned concepts related to other programming languages and specifically the `AS` statement in SQL (used to rename variables) this explains the confusing answers obtained in the interview and may suggest that previous knowledge indeed influences participants' understanding.

9.7.3 Understanding the overuse of COLLECT

As we described in Section 9.6, the misconceptions that involve the use of `COLLECT` are the most frequent among all the complications. To better investigate the student understanding of this operator, we asked which of the following queries is correct to return each distinct node label together with its corresponding set of properties:

- ```
// correct
```
1. `MATCH (n)`  
`RETURN DISTINCT labels(n) AS labels, keys(n) AS properties`
- ```
// correct
```
2. `MATCH (n)`
`RETURN DISTINCT labels(n) AS labels, collect(distinct keys(n))`
`AS properties`

```
// wrong
3. MATCH (n)
   RETURN DISTINCT labels(n) AS labels, collect(keys(n))
   AS properties
```

Even though the first two queries are correct, six students chose the first, four students chose the second, three students chose the second and the third, and one student chose the third. When asked to motivate the answer, we noticed that the students who selected query number three thought that `COLLECT` turns a list into a set, removing the duplicates. To investigate this misconception, we also asked whether the following queries, both intended to retrieve each pangenome and its associated spots, were correct.

```
// wrong
1. MATCH (s:Spot)<-[:IS_IN_SPOT]-(:RGP)<-[:IS_IN_RGP]-(g:Gene)
   -[:IS_IN_FAMILY]->(:Family)-[:IS_IN_PANGENOME]->(p:Pangenome)
   RETURN p.name as pangenome, COLLECT(s.name) AS spots

// correct
2. MATCH (sp:Spot)<-[:IS_IN_SPOT]-(g:Gene)<-[:IS_IN_RGP]-(g:Gene)
   -[:IS_IN_FAMILY]->(:Family)-[:IS_IN_PANGENOME]->(p:Pangenome)
   RETURN distinct p.name, sp.name
```

Nine out of 14 students correctly selected the second query as the correct one, while the others stated that `COLLECT` was needed to remove the duplicates.

To understand why this misconception occurred, we asked the students to explain the use of `COLLECT` in Cypher. In total, 10 out of 14 students reported that `COLLECT` aggregates the results in a single list, and thus it makes the return more readable. The remaining four students instead misunderstood the function of `COLLECT` as shown previously.

Finally, we asked the students which other programming languages or tools have a similar construct to `COLLECT`. The students mentioned several SQL aggregators with similar behaviors as `COLLECT`. This suggests the majority of the students understood what `COLLECT` does - it is indeed similar to the SQL constructs mentioned - but they used it when it was not necessary to improve the readability of the results.

We found one last case of cause for the complications with `COLLECT` due to text incomprehension. During the interview, seven students stated that they used `COLLECT` because the questions asked, "...return the *list* of ...".

9.7.4 Understanding the comprehension of the use of the schema

As we described in Section 9.6, the results revealed that students had difficulty understanding the role of the graph schema. Specifically, the students struggled to grasp how the schema should be derived. This was evident from the formulated queries, which frequently revolved around simple data retrieval operations, such as counting and frequencies, rather than investigating edge directions and node connectivity.

However, the results from the interviews showed a notable improvement. We asked students what the role of the schema is and why it is important. Students replied that "It is a structure defining the organization of data and the relationships between different entities in a database", "It's used to ensure data consistency and integrity, and to facilitate their management and retrieval", and similar explanations. Indeed, all of the 14 students demonstrated a clear grasp of its role, by stating that it shows the different types of nodes and relations, along with their attributes, allowing users to better formulate queries. These answers suggest that, over the course of the experiment, students were able to bridge the gap in their understanding.

9.7.5 Understanding the misconceptions when working with complex patterns

One of the critical challenges identified in this study was the significant number of misconceptions students had when working with complex patterns. As reported in Section 9.6.4, we recorded 84 *Semantic Pattern Errors*, indicating widespread difficulty in accurately defining and utilizing graph patterns. Moreover, a substantial portion of these errors was due to *Missing Conditions*, which were often tied to the formulation of the pattern itself.

To further investigate this issue, in the interview we asked the students if they believed it was necessary to explicitly define the inequality between nodes in a pattern. We asked whether, given the following patterns, the two nodes with the same label `f1` and `f2` could be the same.

1. `MATCH (p:Pangenome)<--()-->(m:Module)<--(f1:Family)<-[r:IS_SIMILAR]-(f2:Family)`
2. `MATCH (p:Pangenome)<-[:IS_IN_PANGENOME]-(f1:Family)-[:IS_IN_MODULE]->(m:Module), (p:Pangenome)<-[:IS_IN_PANGENOME]-(f1:Family)-[sim:IS_SIMILAR]->(f2:Family)`

Two students indicated that the nodes are different, while three others suggested adding the condition `f1<>f2`. Five students responded that the answer depends on the domain, noting that "a family cannot be similar to itself" but that, in general, it is possible.

Only one student said that looking at the schema, there is only one edge between two families and thus, the nodes are different. To answer this question correctly, students need to understand the trail semantics of Cypher described in Section 9.3, which stipulates that an edge can be traversed only once in a single pattern. This rule is crucial for constructing valid and efficient queries. However, the survey responses revealed that many students were unaware of this principle, leading to errors in their pattern definitions.

9.7.6 Discussion (RQ2)

The user study identified several common errors made by students when formulating Cypher queries. To address RQ2, the findings can be categorized into three key areas: errors related to general query languages, errors related to the graph data model, and errors specifically related to Cypher.

Errors related to learning a query language. As reported in previous works [299, 314, 303], a source of errors shared between every query language is related to the `WHERE` clause, where students struggle with logical expressions. The Cypher syntax for filtering is indeed similar to

the one of SQL, and the *Semantic Where* error is the most prominent one - as shown in Table 9.4. Moreover, consistent with previous works in [303, 318], we found that the student's programming experience influences the understanding of query languages. In our case, it manifests mainly as a tendency to overuse `WITH` in order to chain query operations, a pattern more common in procedural SQL and imperative programming paradigms. Indeed, in SQL, `WITH` is used for defining Common Table Expressions, which allow complex queries to be broken down into reusable subqueries, chaining operations across multiple query steps. In Python, the `WITH` statement is commonly used for resource management (e.g., file handling), ensuring that operations, e.g., opening/closing files, are executed within a defined scope.

Errors related to the graph data model. A distinct class of errors was directly tied to the novelty of the graph data model for most students. Unlike relational databases, where schemas are explicit and often dictate query structure and guide its formation, graph databases emphasize flexible, navigational access patterns. This conceptual shift proved challenging. Many students initially struggled to recognize how the schema could influence their queries, which resulted in incorrect pattern formulations and over-complicated queries. This misunderstanding was rooted in their unfamiliarity with graph query languages and their lack of experience working with graph structures. Another area of difficulty revolved around graph pattern matching, particularly when students were required to define complex patterns. This issue was again attributed to a lack of understanding of the graph-based logic. This is also reflected in the correlation with Relational Database Knowledge shown in Table 9.6, highlighting that many students faced difficulties in transitioning from traditional database paradigms to a graph-focused approach, leading to errors rooted in both prior knowledge interference and misunderstandings of the new language.

Errors related to Cypher language features Several errors were linked to students' difficulties in understanding the specific features and syntax of the Cypher language, confirming previous works [308, 306]. However, unlike Chen et al., [306] we were able to identify the specific constructs, such as the `WITH` clause and aggregation functions (e.g., `COLLECT`) that are frequent sources of complications. The usage of `COLLECT` to remove duplicates from lists was a direct result of students' incorrect grasp of the function's purpose. Additionally, students overused `COLLECT` attempting to enhance readability, even if unnecessary. This particular misuse did not have a clear underlying cause but suggests a potential misunderstanding of when `COLLECT` is required and a possible desire to align Cypher queries with perceived norms of clarity from other programming languages.

9.7.7 Best Practices for Teaching (Future) Graph Query Languages

The distribution of errors and the scarcity of correct answers showcased in our study further emphasize the need for targeted instructional support to help students comprehend the complexities of graph queries and overcome the aspects that proved to be challenging.

To address these issues related to Cypher we recommend the following best practices.

Emphasize schema understanding early on. A key takeaway is the importance of understanding the schema from the results of the schema discovery queries (see Section 9.5.1) and the students’ difficulties in using complex patterns (see Section 9.5.2 and Section 9.7.4). To address this, instructors should highlight the role of the schema from the beginning of the course, teach techniques for deriving and understanding schemas, and provide examples and exercises that require students to derive schemas before writing queries. Understanding the schema can also help overcome language comprehension issues, as it provides a clearer context for query construction and expected results.

Clarify the use of `COLLECT` and `WITH` clauses. From Section 9.5.1, we observed that many complications arise from the additional use of `COLLECT` and `WITH`. To address this, one should provide detailed explanations and examples of how these constructs are used, illustrate their output (e.g., lists of nodes, relationships), and their efficient processing. Practical exercises would help reinforce these concepts and ensure students understand how to aggregate and transform data within queries.

Enhance instruction on complex pattern matching. Complex patterns pose a significant challenge, as seen in Sections 9.5.2 and 9.5.3. The scarcity of correct answers and the distribution of errors highlight the need for more targeted instructional support. Our findings suggest a fundamental gap in students’ understanding of constructing and manipulating complex patterns within graph databases. To bridge this gap, it would be useful to extensively illustrate the semantics of Cypher’s pattern matching, highlighting the differences between SQL and relational algebra, as well as to provide exercises of increasing complexity, using real-world scenarios to explain how complex queries are formulated and executed. Incorporating insights from this error analysis could help to develop in-depth examples targeting the main aspects students struggled with. Given that the recent GQL standard for native graph querying is rooted in Cypher, we argue that ensuring a firm understanding of Cypher’s fundamental pattern-matching principles is a crucial first step. In light of the language’s convergence to supporting full GQL expressiveness, future pedagogical efforts should also highlight the practical needs that have shaped graph query language evolution through dedicated case studies. By implementing these best practices, educators can provide more effective instructional support, helping students to better understand and master advanced graph query languages. This approach not only addresses current challenges but also prepares students for the complexities they will encounter in real-world applications of graph databases.

9.8 Limitations

We identified the following limitations in our study. The participants of our cohort included students of two international master programs in France universities that followed a specific course in graph processing. While this may not be a representative sample of the people learning Cypher, our results confirm and corroborate some of the findings in a few available studies on graph query languages [306, 308].

Also, not all the students participated to the interviews, and, while we did not find any significant differences in the responses of the previous part of the study between the interviewees and the rest of the participants, we acknowledge that we could have missed some other insights.

We also recognize the genomic domain is complex and that it could have further impacted the student's ability to write the queries. However, in our qualitative analysis, this aspect didn't emerge, and in the interviews, none of the students highlighted this as a problem. Another limitation is related to the fact that none of the students were native English speakers, and this could lead to language incomprehension. We observed some errors of this type that lead to using wrong operators (e.g., `>` instead of `>=`), returning an incomplete result, or overcomplicating the query. However, these represent a small fragment of the total cases and, in the interview, this issue was not mentioned. We noticed that the other reasons reported in Section 9.7 were more predominant.

9.9 Conclusion

In this chapter, we presented the first, to the best of our knowledge, comprehensive study on student errors in graph query formulation. To investigate which aspects of the Cypher language the students struggle with the most and what are the main causes of the errors, we performed a user study asking them to write schema discovery, analytical, and aggregation Cypher queries, and we completed the study with a post-study survey and face-to-face interviews to understand the cause of the misconceptions. The results show that errors concentrate on a small set of constructs: the `WITH` and `COLLECT` clauses, the definition of complex graph patterns, and the distinction between the schema and the data. The interviews attribute a large part of these errors to previous experience with SQL and with imperative programming languages, which does not transfer to graph pattern matching and is itself a source of misconceptions.

For this thesis, the study provides the empirical basis for one of the two assumptions made in Part I and Part II, namely that the user in the loop is fallible about the formalism, and it characterises this fallibility construct by construct. The participants are students rather than the domain experts of the previous parts. However, the errors we observed concern the language and not the domain, and the interviews trace them back to prior knowledge that domain experts share (e.g., familiarity with SQL). Three correspondences with the previous chapters follow. First, the confusion between schema and data affects the reading of a violation, which Chapter 3 presents to the user as a subgraph whose labels and property values have to be told apart. Second, the misuse of `WITH` and `COLLECT` affects the aggregate queries over which the what-if queries of Chapter 6 are defined. Third, the errors in pattern definition affect the causal path queries of Chapter 7, which are graph patterns over the causal DAG. Formulating a graph query therefore remains a skill that the user has to acquire, and the interfaces of the previous chapters have to be designed under this assumption. Whether language models and agents, which can produce the query on the user's behalf, remove this obstacle or move it from writing a query to verifying one is discussed in Section 10.1.

The study is also a snapshot of a language that is still changing. GQL has only recently been standardised [24], and open-source and commercial databases now using Cypher are gradually adopting its features, e.g., path conditions, path variables, subqueries, disjunctions, and optional

matching. The errors documented in this chapter concern the stable core of the language, which these extensions build upon, while the new constructs are untested with users. Each of them is an opportunity to ask whether its design, and not only its documentation, is what makes it hard to use.

To address the limitations presented in Section 9.8, we plan to package the study as an instrument that can be run in different institutions, with participants having more diverse backgrounds and levels of education, and with datasets from domains closer to the experience of the participants. Adding the logging of query refinements and think-aloud sessions alongside the interviews would show how a query is arrived at and not only where it ends. A second direction is to extend the study to the GQL features listed above and to schema and constraint languages, such as PG-Schema [2] and PG-Keys [32], which would extend the schema discovery part of the study from exploration by query to the reading of a declared schema. Finally, the taxonomy of this chapter can serve as a diagnostic for text-to-query models: scoring the output of a model against the same error families, on the same schema and questions given to the participants, would yield directly comparable error profiles, and would answer a question a scalar accuracy cannot, i.e., whether models and novices fail on the same instances or only in the same categories.

Chapter 10

Conclusion and Future Perspectives

This thesis studied two problems that recur wherever property graphs are deployed: repairing the graph so that it satisfies its integrity constraints, and reasoning over it so that its analyses support decisions. Neither can be solved from the content of the database alone. In both, we treated the user as a component of the problem formulation, with (i) knowledge that the data does not contain, (ii) a cost and a bounded capacity, (iii) an exploratory way of working, and (iv) a documented set of misconceptions about the formalism. The chapters consider this user in three roles: deciding what the data should say (Part I), planning and analysing over it (Part II), and being the object of study (Part III).

Part I treats the user as the one who decides. Chapter 3 starts from the observation that a violation usually admits several repairs, all of which restore consistency and only one of which corresponds to the ground truth. The choice between them requires knowledge the graph does not contain, and the framework asks the user for it. The remaining components make asking practical: violations are detected by compiling the constraint into a query, their interactions are materialised in a dependency graph so that several users can work at once without invalidating each other, and the safety condition on an accepted repair yields convergence whatever the competence of the users involved (Theorem 3.2). Section 3.7 shows the resulting interaction as an actual interface. Chapter 4 then takes the two conveniences that the first design allows itself, that users are interchangeable and their effort is free, and removes them. Difficulty becomes a parameter the system estimates, users become agents with skills, capacities and prices, the easy cases are handled automatically, and the question of who is asked what becomes an explicit optimisation problem.

Part II keeps the user and changes their role from deciding to planning and analysing. Chapter 6 is built around the way an analyst works: they propose a change, read its effect, propose a variant, abandon a branch and return to an earlier state. A hypothetical update is applied as an overlay that never touches the base graph, the maintenance is exact so that a number shown to a decision maker is the number the model would produce, and it composes into an interactive session. Chapter 7 moves from what a model would predict to what would happen, and the user reappears at the point where the question is posed. Causal analysis is, in practice, expert labour carried out with scripts; embedding the causal structure in the graph and giving the query language constructs for extraction, intervention and counterfactuals is what allows a domain expert to ask the question without also being a specialist in the machinery,

and the answer returned is the effect of a change the user can decide to make. The ecosystem extension of Section 7.6 carries the same principle further, with one explicit and inspectable causal structure serving analysts, learning systems and agents at once.

Part III stops assuming the user and studies them. Chapter 9 reports what happens when people who have been taught the language sit down to query a property graph, and the findings are specific enough to design against: a taxonomy of the errors, the constructs that go wrong most often, the difficulty of separating the schema from the data, and the finding that previous experience with other query languages does not transfer cleanly and is itself a source of error. These findings support the premise of Parts I and II that the user in the loop is fallible about the formalism, and characterise that fallibility by construct; the participants of the study are students, so the premise that the user is knowledgeable about the domain is not tested there.

In the following Section 10.1 we analyze what changes when language models and agents enter the same loop. We argue that they widen the set of people who can participate without removing the need for the guarantees that made participation safe, since the difficulty moves from writing a query to deciding whether to accept one.

Taken together, the chapters support one claim about design: treating the user as a component with properties changes the formulation of the problems and not only their interface. In Part I, it turns repair into an assignment problem with a convergence guarantee. In Part II, it makes exactness and isolation requirements of the maintenance, and motivates declarative constructs over library calls. In Part III, it makes an error taxonomy a design input. The systems presented in this thesis are instances of this approach.

10.1 Language Models and Agents as Interfaces to Graph Data

Every contribution of this thesis involves a user who holds knowledge that the system does not, and who has to express it in a formalism they have not necessarily mastered: the user who selects a repair in Part I, the analyst who composes a sequence of hypothetical updates or a causal query in Part II, and the students whose errors are measured in Part III. Since this work started, large language models and agents have become common interfaces to data [323, 324]: a user can describe a question in natural language and obtain a query, and increasingly can delegate the whole task to an agent that inspects the schema, drafts a query, executes it, reads the error and tries again. This section discusses what this development changes for the results of the thesis and what it does not. Part of the discussion draws on Co-Data [19], a workshop on human and LLM collaboration for data processing that we co-organised at CHI 2026, whose framing of collaboration we adopt below.

Query formulation. Text-to-query systems moved from grammar-driven interfaces to instruction-following models and, more recently, to pipelines that generate several candidate queries and select among them [325]. Reported accuracy on the classical benchmarks is now high enough that these benchmarks no longer discriminate between systems. However, performance drops on realistic databases: on Spider 2.0, whose tasks are drawn from enterprise warehouses with schemas of over a thousand columns, the best evaluated model solved 21.3% of the tasks, against 91.2% on the original Spider [326], and a recent audit found annotation error rates above 50% in two widely used benchmarks [327]. The graph counterpart of this line of work is smaller

and more recent, partly because the target language was fragmented until the standardisation of GQL [328], and its results are more modest: execution accuracy around 31% over full-scale property graphs [329], a degradation of roughly 37% between simple and complex aggregation queries, and two dominant failure modes, i.e., mapping a relationship incorrectly and inventing an operation the database does not support [330]. These failure modes correspond to families of the taxonomy of Chapter 9, namely pattern definition and schema misunderstanding. One can notice that the users and the models fail on the same constructs, and that a user who cannot read a generated query cannot detect these failures. The language therefore remains a prerequisite for using such a system rather than a skill that the system replaces, which is the conclusion Chapter 9 reaches from the human side.

From asking to acting. The change that affects this thesis most is not that models translate questions but that agents execute operations [331, 332]. A translator that produces a wrong `MATCH` wastes a query. An agent that produces a wrong `SET` or `DELETE` damages the graph, and an agent that produces a plausible wrong repair damages it in a way that is hard to detect later, since the result still satisfies the constraints. The autonomy that the agent literature envisions therefore presupposes guarantees that the same literature lists as open problems: safety and governance, rollback, human-in-the-loop confirmation, and evaluation of an action before it is committed [331]. These translate into four system requirements, none of which is specific to language models, and each of which corresponds to a primitive developed in this thesis. The first is a checkable condition on a proposed edit, evaluable locally, so that an edit can be rejected before it is applied; this is the safety condition of Definition 3.10, whose consequence for termination is Theorem 3.2. The second is isolation between concurrent writers, since a population of agents editing one graph can invalidate each other’s work as concurrent human editors can; this is the assignment of independent sets over the Graph Repair Dependency Graph (Section 3.5.1). The third is allocation, i.e., deciding which participant handles a task, at what cost and within what capacity, once the population contains humans, models and agents with different accuracies and prices; this is the budgeted assignment of Section 4.4.4, in which an agent is one more user with its own skill, capacity and cost. The fourth is evaluation without commitment, so that a candidate action can be measured against a fixed base state instead of being applied and rolled back; this is the hypothetical overlay of Chapter 6, whose exactness is established by Theorems 6.1 and 6.2. The guarantees of Parts I and II were designed for human participants. They apply unchanged to an agent, since they constrain the effect of an edit on the graph and not the nature of whoever proposes it.

Replacing or assisting the user. The measurements of this thesis and of independent studies locate the boundary of what a model can take over. In Section 4.5.5, replacing users with a mid-sized open-weight model [185] reduced cost to near zero and reduced repair quality in almost every configuration, while a hybrid human/LLM allocation outperformed the model alone but remained below the human-assisted configuration. Studies of LLM-based constraint mining [188] and of LLM-based graph repair [189] report the same pattern: models perform well where the answer is recoverable from the data and its schema, and poorly where a violation is ambiguous because the information is not in the graph. The second regime is the one this thesis addresses. In it, a model does not abstain but returns a well-formed repair that removes the violation and

diverges from the ground truth, and in a repair loop a confident wrong answer is more damaging than an abstention, since the purpose of the loop is to route the ambiguous cases to a person. Along the same line, we note that questions that are not expressible in the query language (e.g., whether a complaint describes a safety problem) require a judgement that neither the database nor its translation into a query can provide [333]. What the models can plausibly cover is the intermediate tier of difficulty once the confidence model of Chapter 4 has removed the easy cases, together with explaining a violation in prose or ranking candidate repairs for the user.

Collaboration rather than allocation. The framing used so far, i.e., who performs a task, at what cost and with what guarantee, treats a person and a model as interchangeable units of work that differ in price and accuracy. The discussion at Co-Data [19] suggests that the interaction between the two has properties of its own: *interdependence*, since a model that drafts an edit has already framed the user’s decision and a user who corrects one output has changed the context for the next; *mutual responsiveness*, which requires the model to express uncertainty in a form the user can act on and the user’s corrections to reach the model instead of the database only; and *shared outcomes*, since both are accountable for a joint result. Two consequences follow for the systems of this thesis. First, a suggestion shapes the judgement of the person who receives it, so a plausible wrong repair presented with an explanation is more likely to be accepted than one presented without; a system that shows a proposal before eliciting a judgement has chosen to accept this effect. Second, the evaluation of such a system cannot be the execution accuracy of a single turn, which is how text-to-query systems are currently measured: it has to measure the quality of the final result under a budget, with participants of differing expertise and a task whose correctness can be checked. Neither the education literature nor the text-to-query literature evaluates this configuration, and building such an instrument is a prerequisite for claiming that any interface improves the loop.

In summary, language models and agents change who can ask a question to a graph database and what can be done to it without a person on the critical path. They do not change the premise of this thesis. The graph has to be repaired, since a hallucinated relationship and a wrong edge produce the same wrong answer; the reasoning over it has to be exact and inspectable, since an approximate answer that crosses a decision threshold is worse than a slower one; and the participant in the loop, whether a person, a model or an agent acting on their behalf, is fallible and has to be treated as such. What changes is the shape of the interface problem, which moves from writing a query to specifying an intent, verifying a generated query one may not be able to read, and authorising an action one did not perform. Safety, isolation, evaluation without commitment and difficulty-aware allocation, developed in the previous chapters, are the primitives this new arrangement requires.

10.2 Directions for Future Research

Each part of the thesis states the work it leaves open, and those statements are not repeated here. Section 5.4 collects the directions of Part I: richer repair operations, collaboration of several users on one violation, learning the assignment policy and the automation threshold instead of fixing them by hand, dynamic and open user pools, and a larger calibration study of the user model. Section 8.4 collects those of Part II: joining the maintained model with the causal

structure that would license an interventional reading of its output, constructing and maintaining causal DAGs at the scale of many evolving sources, transportability, the language and query processing support a real system would need, multimodal alignment, and the governance of a causal layer that decisions depend on. Section 9.9 collects those of Part III: extending the study to other institutions and to the constructs GQL is adding, and using the error taxonomy as a diagnostic for text-to-query models. Section 10.1 adds the primitives an agent writing to a graph would require and the evaluation of human-model collaboration.

The remainder of this section states three directions that follow from the thesis as a whole.

Graph database technology is still evolving. It is worth closing on an observation that runs underneath much of the thesis. Relational systems have had five decades in which their constraint languages, schema facilities, optimisers and standards were worked out; property graph systems are recent by comparison, and GQL was standardised only after most of the work reported here was carried out. Constraints have no standard surface syntax, so they are compiled to queries by hand. Property graphs frequently carry no declared schema, and the formalisms that would supply one are still maturing, which is why schema discovery is a problem for people, for models and for agents alike. Hypervertices have no native support, so the prototype of Chapter 7 emulates them with ordinary nodes and pays for it on the queries that touch both the causal structure and the data. The constructs needed to express interventions declaratively do not exist in any language. The dialects that do exist are fragmented, which is visible even in the corpora that models are trained on, and the storage, indexing and optimisation techniques for these workloads are largely unexplored.

Reliable systems as the precondition for agentic data management. Agents and tool ecosystems are being assembled faster than the guarantees they would need in order to be trusted with a database. The engineering effort has gone into connecting a model to a store, while the properties that make such a connection safe, namely a checkable condition on a proposed write, isolation between concurrent writers, evaluation of a candidate action without committing it, and a record of what was changed and on whose authority, are still listed as open problems. The research direction is therefore not more capable agents but systems in which an agent can be integrated without weakening what the store guarantees, and in which integration is a matter of declaring capabilities and limits. The causal world ecosystem sketched in Section 7.6 is one proposal of this kind: an explicit, queryable and inspectable layer that many consumers share.

User-centred design, privacy and trust. The user remains central, and in our view building around it has to become more prominent. When a person can obtain a draft query, a candidate repair or an explanation on demand, the difficulty is no longer producing an artifact but deciding whether to accept it, and that decision is shaped by what the system chooses to show. Designing for it means treating calibrated uncertainty, provenance and the ability to inspect and contest an output as first-class requirements of the data management layer. Privacy belongs to the same direction, since the questions this thesis studies are most valuable on exactly the graphs whose contents cannot be sent to a hosted model. Governance and compliance are conventionally treated as legal matters, but a system that cannot say which participant changed what, under

whose authority, and on what basis, cannot support an obligation of that kind, so the requirement has to enter the design of the data management layer.

Chapter A

Appendix for Ch. 3

A.1 Survey for the user study

To investigate how the findings from the synthetic experiments translate to a real use case, we tested our approach with real users. For this qualitative experiment, we focus on the **Star Wars**¹ dataset, which is sufficiently small and understandable by real users. We created a small web application using Flask² that shows the users the violations and asks them to choose an appropriate repair. We recruited 20 users involving people from our laboratory (including Master, PhD students, and postdocs) and from our connections, aiming for a mix of people with and without a database background. Before the experiment, they were briefly instructed on what their task was. We collected their expertise on databases, graphs, and Star Wars (since it's the domain of the dataset we used) using a 10-level Likert scale. The average knowledge about databases is 5.65, on constraints is 4.35, on graphs is 4.65, and on Star Wars is 5. The questions with their relative answers were:

- How would you rate your knowledge about Databases?

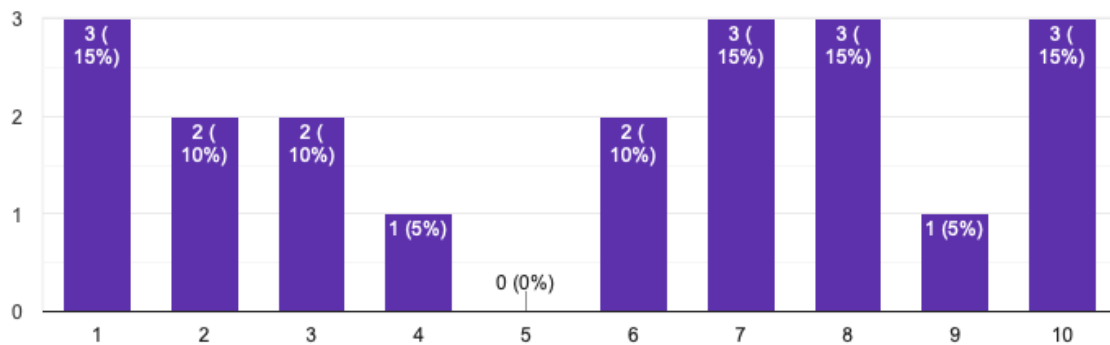


Figure A.1: Histogram for the 10-level Likert scale: 10 means expert.

- How would you rate your knowledge about Database constraints?

¹<https://github.com/neo4j-graph-examples/star-wars>

²<https://flask.palletsprojects.com/en/3.0.x/>

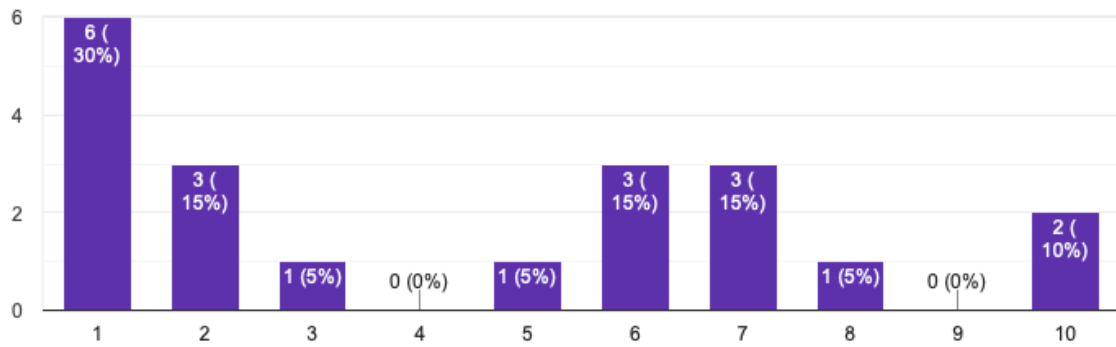


Figure A.2: Histogram for the 10-level Likert scale: 10 means expert.

- How would you rate your knowledge about Graphs?

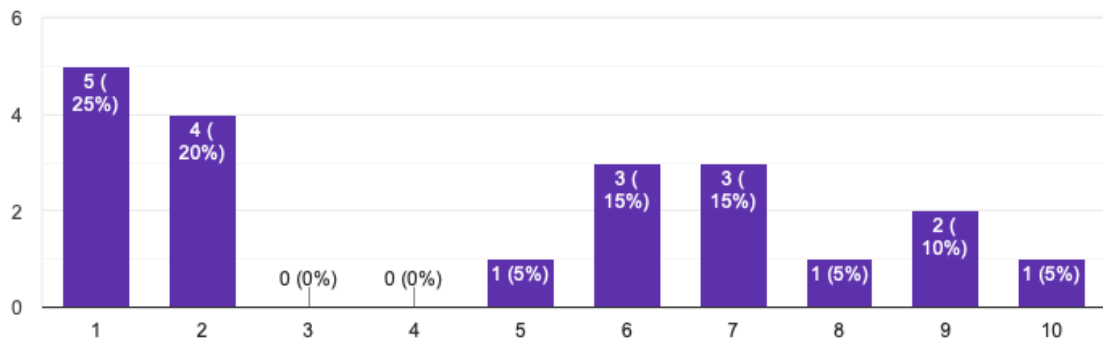


Figure A.3: Histogram for the 10-level Likert scale: 10 means expert.

- How would you rate your the Star Wars universe?

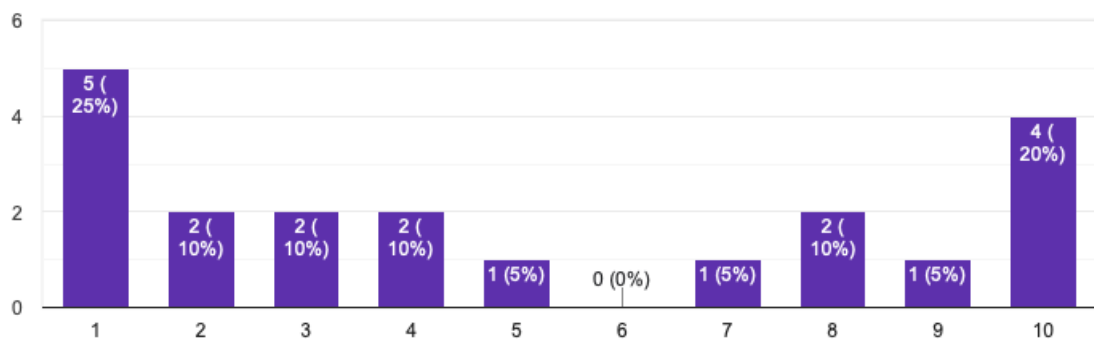


Figure A.4: Histogram for the 10-level Likert scale: 10 means expert.

- Which of the following Star Wars movies have you seen?

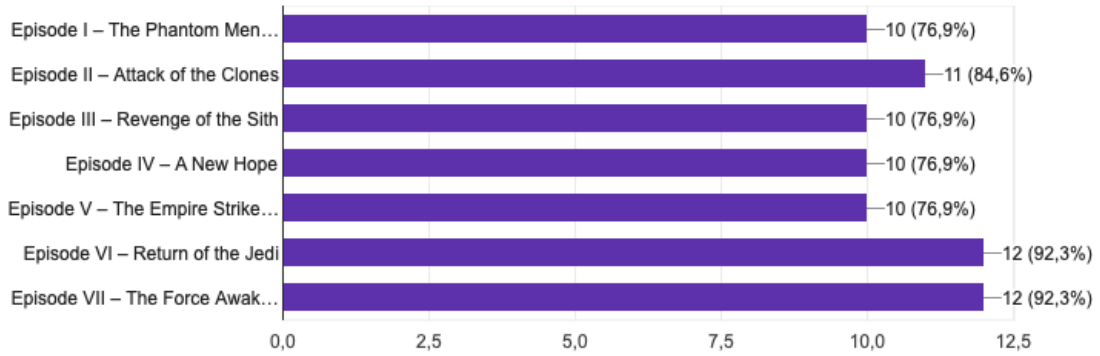


Figure A.5: Barplot for the movies count.

A.2 Table shapes

Table A.1 reports the information about the number of graph pattern shapes that we use in the experiments and affects the construction of the Graph Repair Dependency Graph. For instance, let us consider a star pattern having a central node linked with multiple other nodes with outgoing edges, like *Star 3* shown in Figure 3.9. An inconsistency in the central node could potentially be shared with all the nodes of the star, and as a consequence, the effect of its repair can impact all the neighbors. Notice also that, as opposed to the sizes of the datasets, the number of patterns - that is the element on which the constraints are built - grows exponentially with respect to the size of the graph, exploding to the order of billions of elements.

Dataset	Chain 2	Chain 3	Star 2	Star 3	Flower	Cycle
StarWars	646	160	2156	10122	0	0
StackOverflow	869	869	15192	~1.5M	0	0
wwc_2019	43548	43548	55520	~499k	0	0
Fincen	85158	~116k	~12M	~40B	5113	0
Synthea	36504	51681	~263k	~2.5B	439	153
Synthea50	60974	83035	~2.5M	~1.8B	1452	415
Synthea75	~123K	~172K	~6.7M	~3.3B	2489	665
Synthea100	~144K	~199K	~7M	~4.3B	2639	757
Synthea200	~294K	~403K	~15B	~36B	4431	1264

Table A.1: Distribution of the number of patterns present in the different datasets

A.3 Evaluation of different assignment policies with Unsafe Repairs

To evaluate the effectiveness of the assignment algorithm, we compare it with two alternative policies: Density-Based Assignment (DBA) and Relevance-Based Assignment (RBA). The DBA policy prioritizes violations that are more interconnected in the graph, assigning those with the highest degree first, ensuring that violations affecting many nodes are addressed early. The RBA policy assigns violations based on their importance within the GRDG, with those having the highest PageRank score prioritized. The PageRank reflects the relevance of a node, so addressing more influential violations first may increase the propagation of the repair early on. As shown

in in Figure A.6 in our experiment with the unsafe configuration we didn't find any significant difference in term of quality of the results.

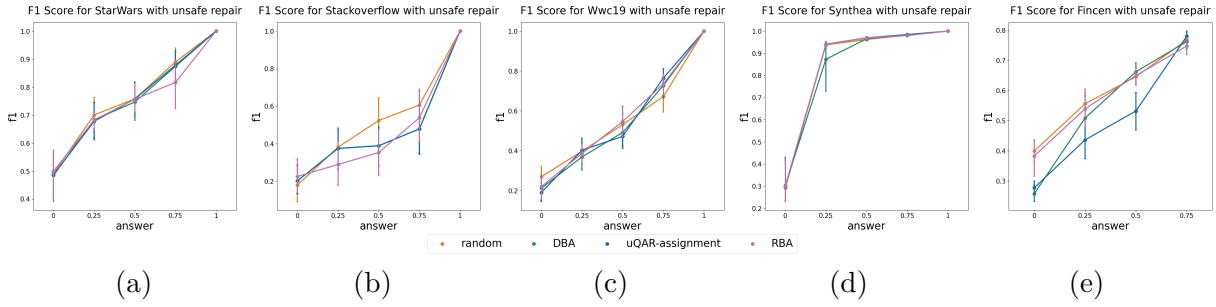


Figure A.6: Average F1-measure of our approach using different assignment policies. The error bars represent the standard deviations.

A.4 Number of Waitings with Unsafe repairs

To assess the scalability of our approach, we tested our repair framework in its safe version (because we wanted to ensure the termination property) on the Synthea dataset generated with an increasing number of patients (15, 50, 75, 100, and 200). Additionally, we considered different numbers of (oracle) users, from 5 to 100. Figure A.7 shows that once a plateau is reached, adding more users does not reduce the iterations required, but it increases the number of times the users need to wait. We also compared our algorithm with assigning the violations at random and with the policies described in Section 3.6.2. As shown in Figure A.7, when the size of the dataset increases, as well as the number of users (when the plateau is not reached), our assignment algorithm takes fewer iterations and produces fewer waits.

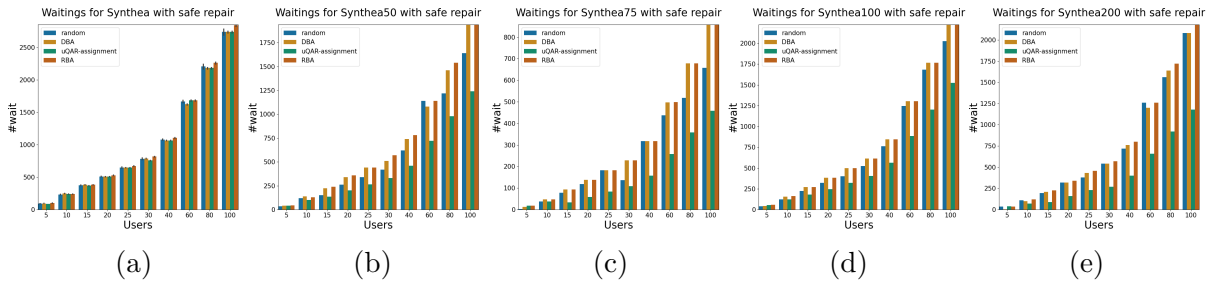


Figure A.7: Number of waitings for each dataset vs. users number and the function used to rank the nodes in the GDRG. The y axes have different scale.

Chapter B

Appendix for Ch. 4

B.1 Time Complexity of LARA

Let $|V|$ be the number of nodes of the GRDG, $|F|$ the number of edges, $|U|$ the number of users, $C = |\mathcal{C}_i|$ the size of candidates subgraphs per user and T the number of iterations of the algorithm.

Complexity of BuildCandidatesSet Let Δ be the maximum degree of the GRDG. For user u_i , denote by CC_i the capacity and by D_v the node difficulties. Let $X_i := \min\left\{|V|, \left\lfloor \frac{CC_i}{\min_{v \in V} D_v} \right\rfloor\right\}$ be an upper bound on the number of nodes a candidate subgraphs for u_i can contain. Assume we start from S_i seeds of violations per user and run t_i iterations of 1-exchange local search (add/drop/swap one node) per seed, keeping at most $C_i \leq S_i$ candidate subgraphs. The greedy growth step performs at most X_i insertions, checking at most $O(X_i \Delta)$ neighboring nodes to select the one with maximal gain (cost $O(\log n)$), giving $T_{\text{greedy}}(i) = O(X_i \Delta \log n)$. One step of local search scans a 1-exchange neighborhood of size $O(X_i \Delta)$ with cached deltas; over t_i iterations this is $T(i) = O(t_i X_i \Delta)$. Hence the per-seed cost is $T_{\text{seed}}(i) = O(X_i \Delta (\log n + t_i))$. With S_i seeds, the complexity for finding the candidate subgraphs for a user is $T_{\text{user}}(i) = S_i \cdot T_{\text{seed}}(i) = O(S_i X_i \Delta (\log n + t_i))$,

Complexity of SubModularAssignment. For each user i , let $\mathcal{C}_i$ be the set of candidate subgraphs produced by BUILD_CANDIDATESSET. The total candidates for all the users are $M = \sum_{i=1}^N |\mathcal{C}_i|$. and the total nodes accross the candidates are $Y = \sum_{i=1}^N \sum_{P \in \mathcal{C}_i} |P|$. We can leverage an inverted index from each node v for the list of candidate items (i, P) that contain v (of size $O(M)$). Let $c_{\min} = \min_i c_i$ be the minimum cost among the users, and let X_{sel} be the number of selected candidates; clearly $X_{\text{sel}} \leq \min\{M, |U|, \lfloor B/c_{\min} \rfloor\}$.

Computing initial gains $\sum_{v \in P} EQ(D_v, K_i)$ for all (i, P) and building the inverted index costs $O(S)$.

At each iteration of the greedy loop, we extract the current best feasible item in $O(\log M)$ time. Upon choosing (i^*, P^*) we (i) remove all other patches of user i^* (at most $|\mathcal{C}_{i^*}|$ items overall across the run), and (ii) invalidate all candidates that overlap P^* . Using the inverted index, step (ii) takes $O(\sum_{v \in P^*} \Gamma_v)$ time, where Γ_v is the number of candidate items containing node v . Because subgraphs are enforced disjoint, each node is invalidated at most once; hence

across the whole run $\sum_{\text{selected}} P^* \sum_{v \in P^*} \Gamma_v = \sum_v \Gamma_v = Y$. The total number of operations on the lists of candidates is $O(M + X_{\text{sel}})$, so heap work is $O((M + X_{\text{sel}}) \log M)$.

Summing up, $T_{\text{assignment}} = O(Y + (M + X_{\text{sel}}) \log M)$. Under uniform parameters ($|\mathcal{C}_i| = L$ candidates per user, each of size at most X), we have $M = |U|L$ and $X \leq |U|LX$, giving

$$T_{\text{assignment}} = O(|U|LX + (|U|L + X_{\text{sel}}) \log(|U|L)).$$

Complexity of the Final Outer Loop. Let T be the maximum number of Lagrangian iterations. The remaining outer-loop work per iteration is lightweight: (i) computing the current primal value Z is $O(\sum_{i \in U'} |P_i|) \leq O(|V|)$, (ii) the dual quantity UB_t is $O(1)$ once Z and $\sum_{i \in U'} c_i$ are known, (iii) the subgradient update and projection of λ are $O(1)$, and (iv) updating the incumbent $(U^*, \{P_i^*\})$ is $O(1)$.

Hence the worst-case total time (without early stopping) is $T_{\text{total}} = T \cdot (|U| \cdot T_{\text{user}} + T_{\text{assignment}} + O(|V|))$. Therefore, T_{total} is

$$O\left(T \cdot (|U| S X \Delta (\log |V| + t) + |U|LX + (|U|L + X_{\text{sel}}) \log(|U|L) + |V|)\right).$$

In practice the loop halts at some $T' \leq T$ when either the dual-primal gap $UB - Z^*$ drops below tolerance or the price λ stabilizes; then T above can be replaced by T' .

B.2 Constraints

In this Section we list the constraints used to introduce the violations for each dataset.

B.2.1 LDBC Finbench

An account cannot transfer money if it is blocked

$$\phi_1 := (\text{Transfer}(X, Y), X.\text{isBlocked} = \text{True} \rightarrow \perp) \quad (\text{B.1})$$

A guarantor needs to have more than 5M in the bank account

$$\phi_2 := (\text{Guarantee}(X, Y) \wedge \text{Own}(Y, Z), \quad (\text{B.2})$$

$$Z.\text{balance} < 500000000 \wedge \text{id}(X) \neq \text{id}(Y) \rightarrow \perp) \quad (\text{B.3})$$

A person cannot be guarantor of themselves

$$\phi_3 := (\text{Guarantee}(X, Y), \text{id}(X) = \text{id}(Y) \rightarrow \perp) \quad (\text{B.4})$$

Interest Rates for company need to be greater than 0.2

$$\phi_4 := (\text{Apply}(X, Y), Y.\text{interestRate} < 0.2 \rightarrow \perp) \quad (\text{B.5})$$

B.2.2 LDBC SNB

A person cannot know themselves

$$\phi_1 := (Knows(X, Y), id(X) = id(Y) \rightarrow \perp) \quad (B.6)$$

Two people that know each other need to live in the same city

$$\phi_2 := (Knows(X, Y), id(X) \neq id(Y) \rightarrow \perp) \quad (B.7)$$

$$\wedge X.LocationCityId \neq Y.LocationCityId \rightarrow \perp) \quad (B.8)$$

A user cannot like their own post

$$\phi_3 := (CREATED(po, p) \wedge LIKES(po, p) \rightarrow \perp) \quad (B.9)$$

A person has to live in the same country as the organization they work for.

$$\phi_4 := (LIVES_IN(pl1, p) \wedge WORK_AT(p, o) \wedge LOCATED_IN(o, pl) \rightarrow \perp) \quad (B.10)$$

$$id(pl) \neq id(pl1) \rightarrow \perp) \quad (B.11)$$

B.2.3 FAERS (Real)

A drug cannot be primary suspect and secondary suspect of the same case

$$\phi_1 := (IS_PRIMARY_SUSPECT(X, Y) \wedge IS_SECONDARY_SUSPECT(Y, Z), \quad (B.12)$$

$$id(X) = id(Y) \rightarrow \perp) \quad (B.13)$$

$$id(X) = id(Y) \rightarrow \perp) \quad (B.14)$$

A drug cannot be primary suspect of itself

$$\phi_1 := (IS_PRIMARY_SUSPECT(X, Y) \wedge id(X) = id(Y) \rightarrow \perp) \quad (B.15)$$

$$id(X) = id(Y) \rightarrow \perp) \quad (B.16)$$

A case cannot fall under two different age groups

$$\phi_3 := (FALLS_UNDER(X, Y) \wedge FALLS_UNDER(X, Z), \quad (B.17)$$

$$id(Z) \neq id(Y) \rightarrow \perp) \quad (B.18)$$

A drug cannot be prescribed to a child

$$\phi_3 := (PRESCRIBED(X, Y) \wedge RECEIVED(Y, Z) \wedge FALLS_UNDER(Z, W) \rightarrow \perp) \quad (B.19)$$

$$W.ageGroup = "Child" \rightarrow \perp) \quad (B.20)$$

$$W.ageGroup = "Child" \rightarrow \perp) \quad (B.21)$$

B.2.4 ICIJ (Real)

The registered address of an entity must be the same as the one of the address

$$\phi_1 := (\text{registered_address}(X, Y), \quad (\text{B.22})$$

$$X.\text{country_codes} \neq Y.\text{country_codes} \rightarrow \perp) \quad (\text{B.23})$$

An address cannot be registered both by an office and an entity

$$\phi_2 := (\text{OFFICER_OF}(o, e) \wedge \text{REGISTERED_ADDRESS}(e, a) \quad (\text{B.24})$$

$$\wedge \text{REGISTERED_ADDRESS}(o, a) \rightarrow \perp) \quad (\text{B.25})$$

A shareholder needs to have a minimum amount of money

$$\phi_3 := \text{SHAREHOLDER_OF}(o, e) \quad (\text{B.26})$$

$$\text{WHERE } o.\text{networth} < 17000000, \rightarrow \perp) \quad (\text{B.27})$$

An officer cannot be both intermediary and shareholder for the same entity

$$\phi_4 := (\text{INTERMEDIARY_OF}(i, e) \quad (\text{B.28})$$

$$\wedge \text{SHAREHOLDER_OF}(i, e) \rightarrow \perp) \quad (\text{B.29})$$

B.3 Different Type of Constraints

B.3.1 ICIJ

Denial Constraint: An officer relation cannot end after the entity closed date

$$\phi_5 := (\text{OFFICER_OF}(o, e) \wedge \text{HAS_CLOSED_DATE}(e, d), \quad (\text{B.30})$$

$$o.\text{end_date} > d.\text{value} \rightarrow \perp) \quad (\text{B.31})$$

Key Constraint: Two different entities in the same jurisdiction cannot have the same name and incorporation date

$$\phi_6 := (\text{HAS_JURISDICTION}(e, j) \wedge \text{HAS_JURISDICTION}(e_1, j), \quad (\text{B.32})$$

$$id(e) \neq id(e_1) \wedge e.\text{incorporation_date} = e_1.\text{incorporation_date} \quad (\text{B.33})$$

$$\wedge e.\text{name} = e_1.\text{name} \rightarrow \perp) \quad (\text{B.34})$$

Functional Dependency: The country associated with an entity must be the same as the country of its registered address

$$\phi_7 := (rr(c_1, n) \wedge \text{REGISTERED_ADDRESS}(n, a) \quad (\text{B.35})$$

$$\wedge \text{LOCATED_IN}(a, c), \quad (\text{B.36})$$

$$c_1.\text{code} \neq c.\text{code} \rightarrow \perp) \quad (\text{B.37})$$

B.3.2 FAERS (Real)

Key Constraint: Two different cases cannot have the same event date, age, and gender

$$\phi_5 := (HAS_EVENT_DATE(c, d) \wedge HAS_AGE(c, a) \quad (B.38)$$

$$\wedge HAS_GENDER(c, g) \wedge HAS_EVENT_DATE(c_1, d) \quad (B.39)$$

$$\wedge HAS_AGE(c_1, a) \wedge HAS_GENDER(c_1, g), \quad (B.40)$$

$$id(c) \neq id(c_1) \rightarrow \perp) \quad (B.41)$$

Functional Dependency: The age of a case determines the corresponding Age Group

$$\phi_6 := (HAS_AGE(c, a) \wedge FALLS_UNDER(c, ag), \quad (B.42)$$

$$a.value < ag.minAge \vee a.value > ag.maxAge \rightarrow \perp) \quad (B.43)$$

Denial Constraint: A drug exposure cannot appear as both primary suspect and secondary suspect with the same drug name and dose amount in the same case

$$\phi_7 := (HAS_DRUG_EXPOSURE(c, d) \wedge HAS_ROLE(d, r) \quad (B.44)$$

$$\wedge HAS_DOSE_AMOUNT(d, a) \quad (B.45)$$

$$\wedge HAS_DRUG_EXPOSURE(c, d_1) \wedge HAS_ROLE(d_1, r_1) \quad (B.46)$$

$$\wedge HAS_DOSE_AMOUNT(d_1, a_1), \quad (B.47)$$

$$r.value = \text{"Primary Suspect"} \quad (B.48)$$

$$\wedge r_1.value = \text{"Secondary Suspect"} \quad (B.49)$$

$$\wedge d.drugName = d_1.drugName \quad (B.50)$$

$$\wedge id(d) \neq id(d_1) \quad (B.51)$$

$$\wedge a.value = a_1.value \rightarrow \perp) \quad (B.52)$$

Chapter C

Appendix for Ch. 6

C.1 Additional Experimental Results

This appendix reports the complete set of experimental results: the depth-scaling [figs/appendix/ch6](#) at every intervention size (the main paper reports the 10% sweep), the full grid over update kinds, absolute update sizes, and model depth, and the exactness gate over all three GNN tasks.

C.1.1 Depth Scaling at All Intervention Sizes

[figs/appendix/ch6 C.1](#) and [C.2](#) repeat the depth-scaling experiment of the main paper at the remaining intervention sizes, 5%, 15%, 20%, and 30% of the dataset (the 10% sweep is in the main paper). The protocol is unchanged: the intervention size is fixed, depth is swept over $L = 1-4$, and each panel plots the latency speedup of incremental evaluation over from-scratch (left axis) together with the size of the flip set (right axis, log scale), for RGCN and hGAT on all four datasets.

The trends are identical at every size. With $L = 1$ no internal activation separates the intervention from the output, the flip count is zero, and the speedup peaks; the first internal activation at $L = 2$ introduces the bulk of the flips and the speedup drops correspondingly; beyond $L = 2$ flips grow only mildly and the speedup decays gently, never below parity. Larger interventions start from a lower peak because their L -hop closure saturates the graph earlier, which is the same effect visible along the size axis of the main paper’s $L = 2$ comparison.

C.1.2 Absolute Latency at All Intervention Sizes

[figs/appendix/ch6 C.4–C.7](#) report the same depth sweep as [§C.1.1](#) in *absolute* terms: per-query latency (log scale) of incremental vs. from-scratch evaluation, for RGCN and hGAT on all four datasets, at every intervention size from 5% to 30%, for attribute updates ([figs/appendix/ch6 C.4](#) and [C.5](#)) and structure updates ([figs/appendix/ch6 C.6](#) and [C.7](#)). The speedup curves of [§C.1.1](#) are the ratio of these lines.

The absolute view separates the two cost regimes directly. From-scratch latency grows essentially linearly with L and is independent of the intervention size, so its curves coincide across the five sizes; the incremental curves start well below it, rise with L as the closure and

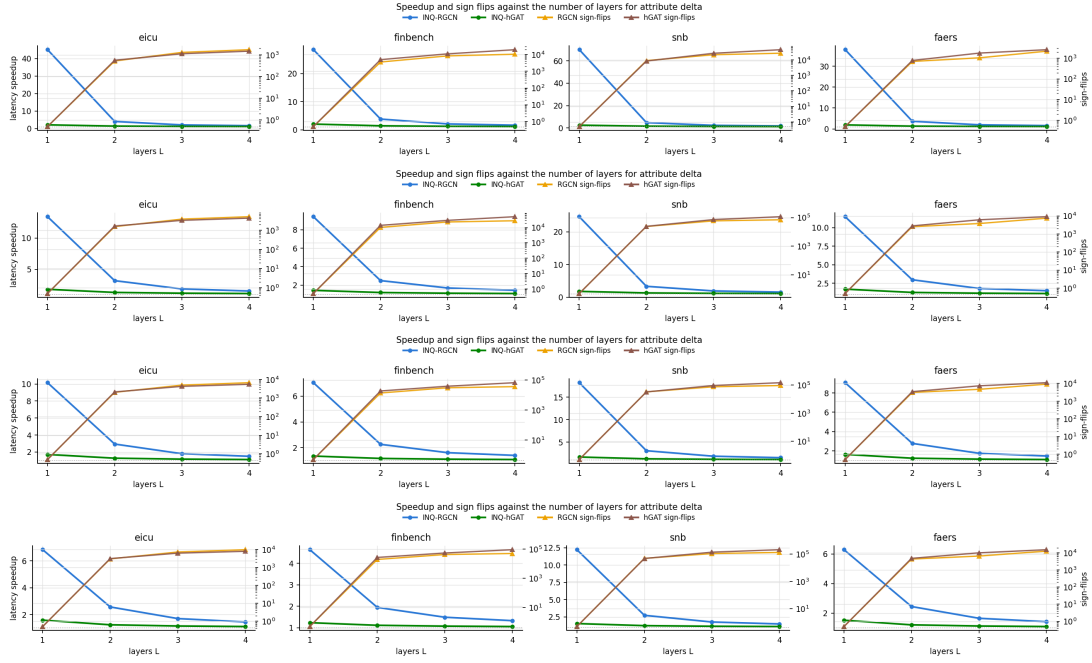


Figure C.1: Depth scaling for *attribute* updates at the remaining intervention sizes (5%, 15%, 20%, 30% of the label-type nodes; top to bottom). Speedup over from-scratch (left axis) and flip-set size (right axis, log), for RGCN and hGAT on all four datasets.

Table C.1: Maximum elementwise output deviation of incremental evaluation from from-scratch recomputation, per dataset and GNN task, over all architectures, depths, intervention kinds, and both maintenance paths

Dataset	Node classif.	Link pred.	Link regr.
FinBench	3.8×10^{-6}	6.1×10^{-5}	2.9×10^{-6}
FAERS	9.5×10^{-7}	1.3×10^{-5}	6.0×10^{-7}
eICU	3.8×10^{-6}	1.4×10^{-5}	2.9×10^{-6}
SNB	2.3×10^{-5}	1.1×10^{-5}	0

the flip set grow, and shift upward with the intervention size, approaching the from-scratch line exactly where the speedup plots of §C.1.1 approach parity (*faers* for structure updates, large sizes at $L = 4$ elsewhere) while never crossing above it.

C.1.3 Exactness Across All GNN Tasks

The main paper reports efficiency on node classification. The exactness gate is run on *all three* canonical GNN tasks: node classification, link prediction, and link regression. For every combination of dataset, architecture (RGCN, hGAT), depth $L = 1-4$, task, and intervention kind (feature, structure, mixed), three random schema-valid interventions are evaluated with both maintenance paths (low-rank and closure-based) and compared entrywise against a from-scratch forward pass on G^Δ , 924 configurations in total. All of them pass the tolerance gate of the main paper (absolute 10^{-5} or relative 10^{-4}); Table C.1 reports the maximum output deviation per dataset and task, all of which are float32 rounding on isolated coordinates.

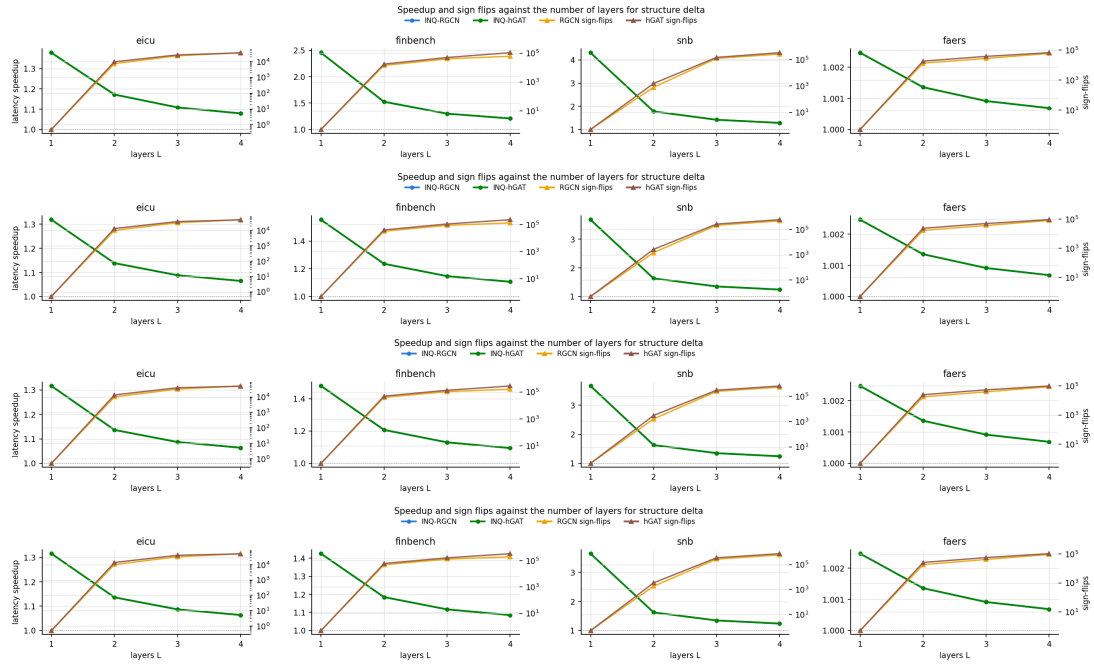


Figure C.2: Depth scaling for *structure* updates at the remaining intervention sizes (5%, 15%, 20%, 30% of the core-relation edges added and deleted; top to bottom). Same axes as Figure C.1.

Table C.2: Workload specification per dataset: the node type and relation the updates target, the update templates, and the evaluated query.

Dataset	Node type	Core relation	Attribute update $\Delta\kappa$	Edge update $\Delta E^\pm$	Query readout
FinBench	account	transfer	shift one account feature	add/delete transfer edges	class scores over accounts
FAERS	drug	causes	shift one drug feature	add/delete causes edges	class scores over drugs
eICU	stay	received	shift one stay or medication feature	add/delete received edges	high-risk fraction over a cohort
SNB	person	knows	shift the person age feature	add/delete knows edges	predicted-positive fraction over a cohort

C.2 Workload Specification

This section specifies, for each dataset, exactly which hypothetical updates are applied and which query is evaluated.

Every dataset designates a label node type (the type carrying the classification target, over which queries aggregate) and a core relation (the edge type on which structural updates and the link tasks are defined). Table C.2 lists both, together with the concrete updates and the query of the interactive sessions.

Updates in the efficiency sweeps. The sweeps draw updates from two seed-controlled templates, instantiated on the anchors of Table C.2:

- **Attribute update.** One feature coordinate k of the label node type is selected (fixed by the run seed) and shifted by $\delta = 1$ standard deviation of that coordinate, over a random cohort. The cohort size is the swept variable: a fraction (5–30%) of the label-type nodes

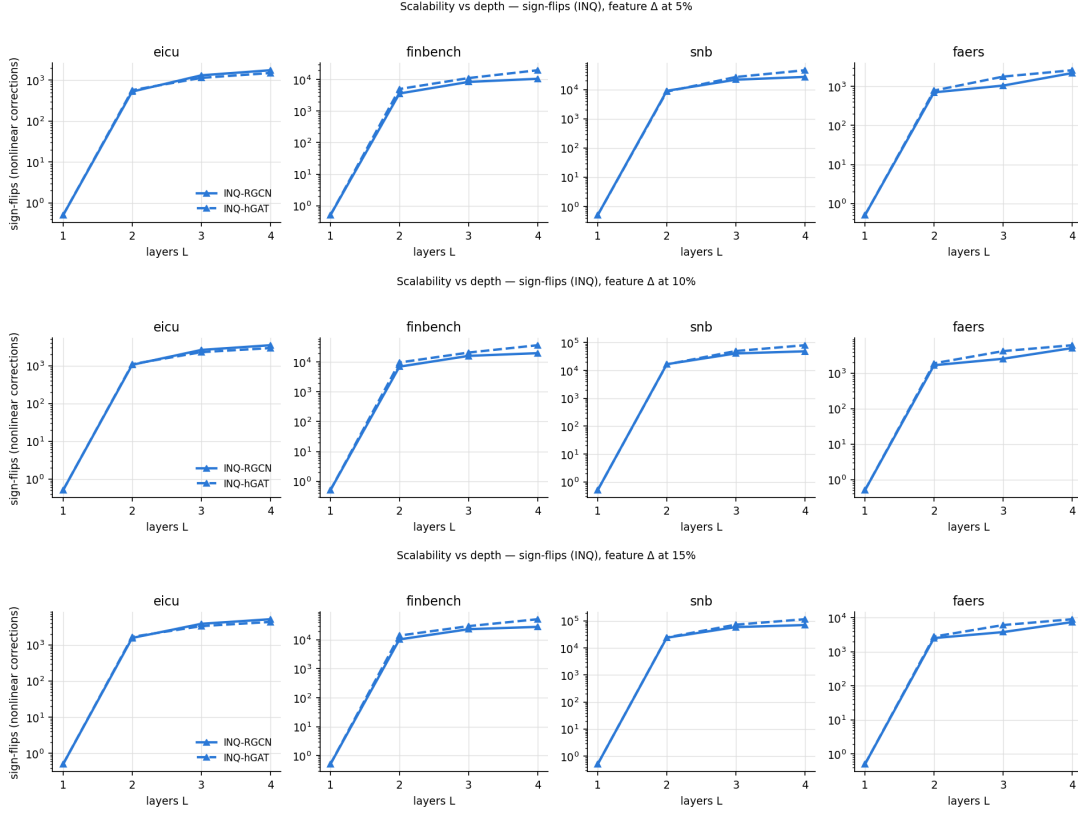


Figure C.3: Flip-set size vs. depth for attribute updates, at intervention sizes 5%, 10%, and 15% (top to bottom). Only the incremental method incurs flips; from-scratch recomputation has none.

in the scaling experiments in the grid and baseline experiments. This is the rank-one intervention $\Delta X = u e_k^\top$ of the main paper.

- **Structure update.** n edges are inserted and n deleted on the core relation, endpoints drawn uniformly over the admissible types, so every edit is schema-valid by construction; n follows the same fractional or absolute schedule (fractions are taken of the core-relation edge count). Edge additions, deletions, and swaps (paired delete+add) are reported separately in the grid.

The exactness experiments additionally use mixed updates, the union of one attribute and one structure update. In all sweeps the maintained quantity is gated on the full prediction vector of the model, the strongest possible readout, since any aggregate query Q evaluated on an exact output is exact.

C.3 Full Store-Backed Comparison

Figure C.8 completes the store-backed comparison of the main paper (reported there on **FinBench**): per-query time for from-scratch, Ripple, NeutronRT, and ours, on all four datasets (columns), for both stores (Neo4j and Memgraph) and both intervention kinds (rows), with both architectures side by side. Each bar decomposes into inference time (blue, bottom) and store-update time (orange, top: applying and rolling back Δ in the database); the one-time graph download is excluded for readability.

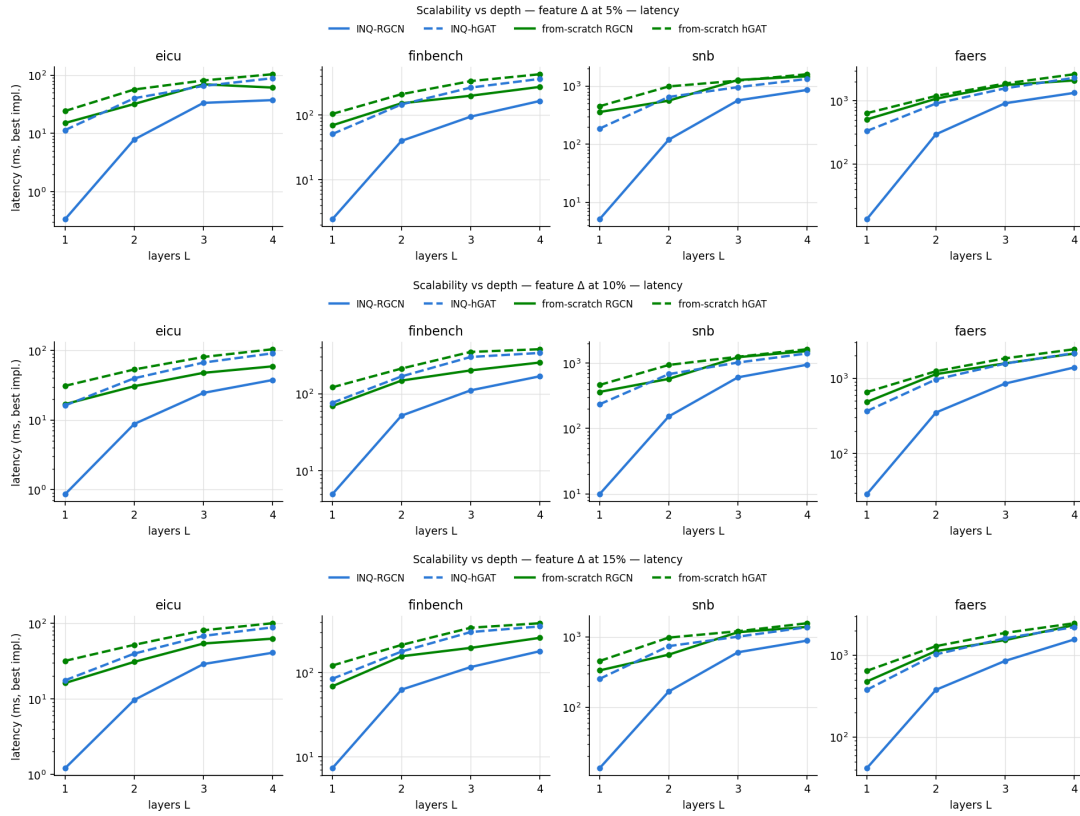


Figure C.4: Absolute latency vs. depth for *attribute* updates at intervention sizes 5%, 10%, and 15% (top to bottom); incremental vs. from-scratch, RGCN and hGAT, log scale.

The **FinBench** findings hold everywhere: the committed-timeline baselines pay an orange store-write bar that grows with the edit size and is paid on every hypothetical, on both stores, while our overlay never writes to the database and therefore carries no update bar at any size, on any dataset.

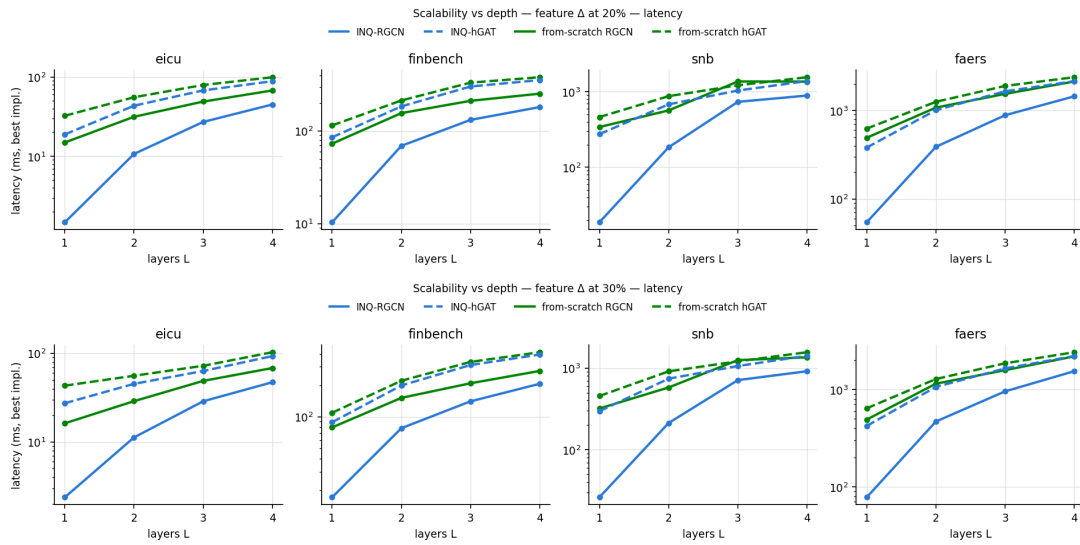


Figure C.5: Absolute latency vs. depth for *attribute* updates at intervention sizes 20% and 30% (top to bottom).

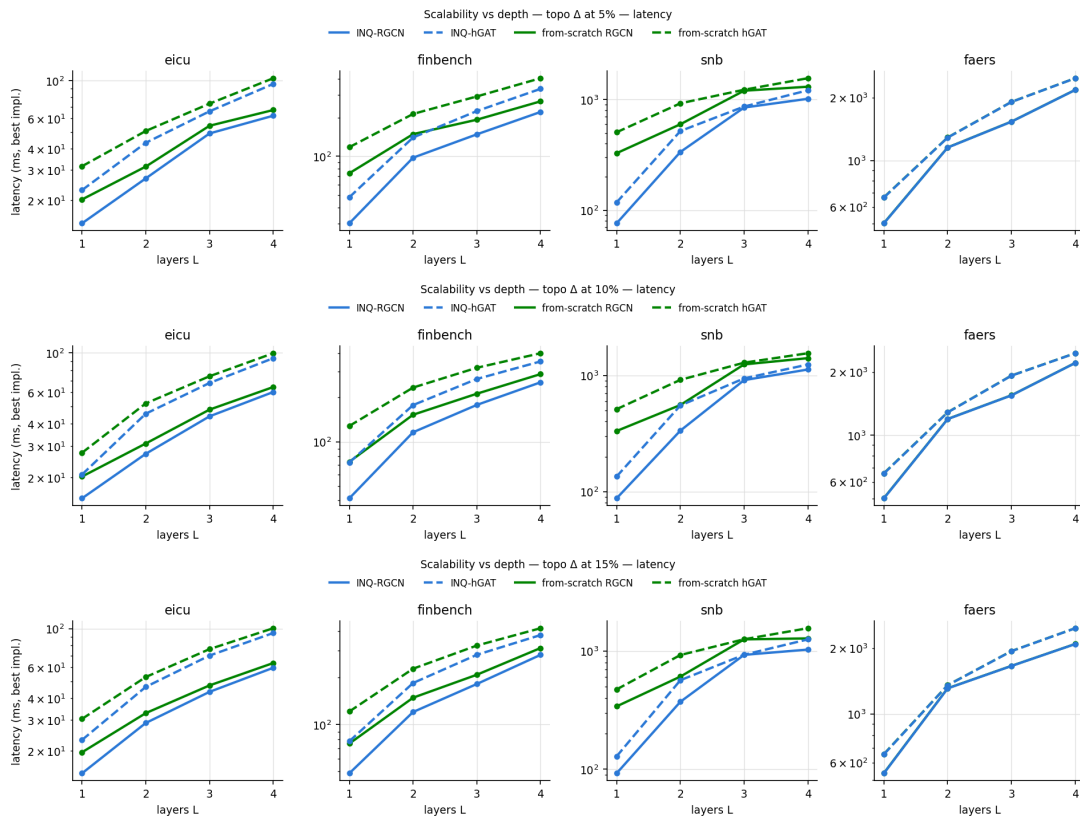


Figure C.6: Absolute latency vs. depth for *structure* updates at intervention sizes 5%, 10%, and 15% (top to bottom).

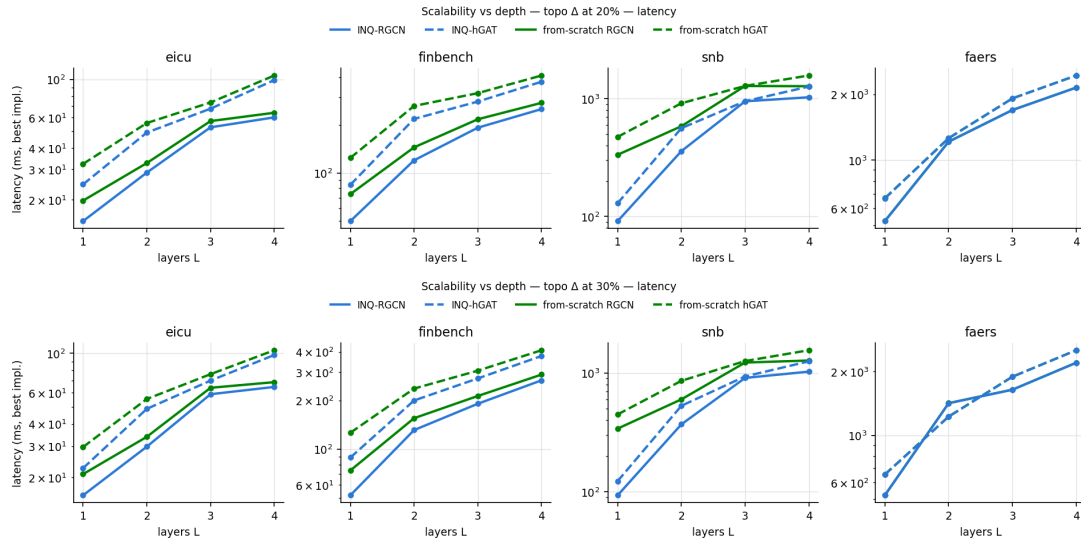


Figure C.7: Absolute latency vs. depth for *structure* updates at intervention sizes 20% and 30% (top to bottom).

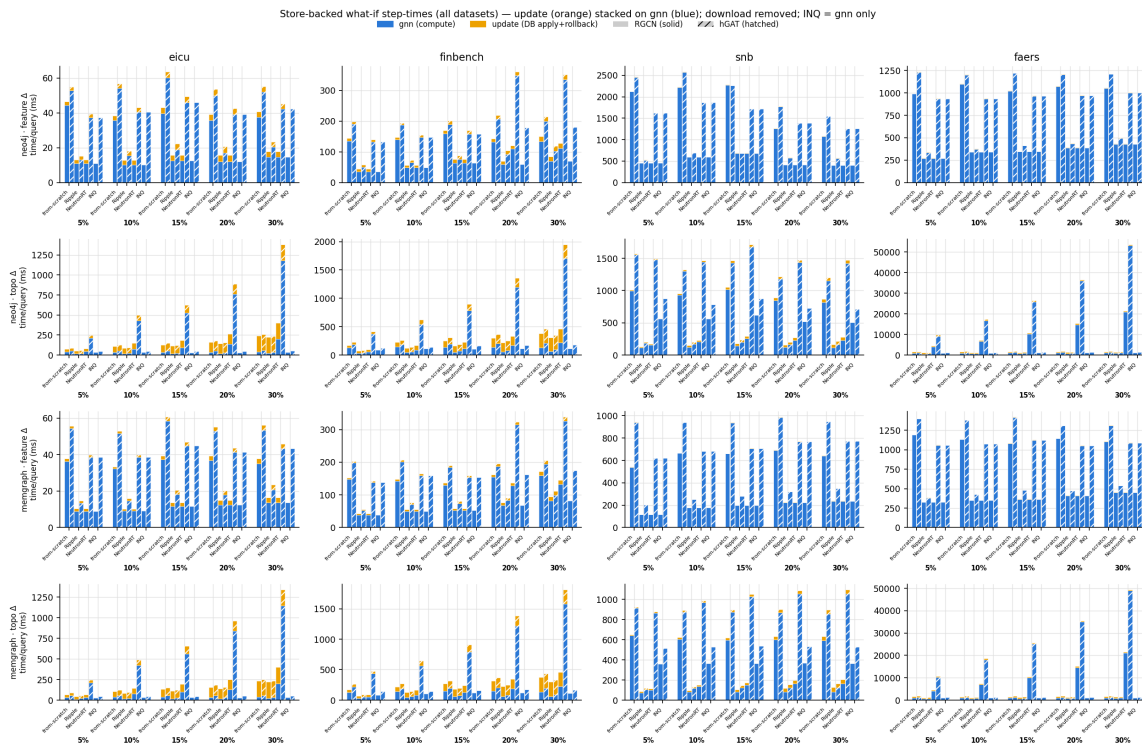


Figure C.8: Full store-backed comparison: per-query time (store update + inference, one-time download excluded) for every dataset (columns) and every store/intervention-kind combination (rows), both architectures.

Chapter D

Appendix for Ch. 9

D.1 Informed Consent

Researchers

Removed for anonymity purposes.

Purpose of the study:

You are invited to participate in a research study conducted by the University. This study is aimed at understanding how students comprehend various aspects of the Cypher language, identifying common misconceptions. This will require approximately 90 minutes. The findings will be used to improve Cypher teaching methods and potentially enhance the language for better accessibility.

Study Procedure

You will be tasked with writing solutions to different queries using the Neo4j desktop edition to test and submit answers. Correct answers will be submitted via a Google form, while other attempts should be documented in a separate file. At the beginning and end of the study, surveys may be required. Please avoid personal identifiers in surveys.

Benefits and Risks of Participation

Participation poses no risks beyond daily life or typical online activities. While there may be no direct benefits, the data could aid in designing better courses and more comprehensible query languages.

Withdrawal Procedures

Participation is voluntary, and you may withdraw at any time.

Data Collection and Processing

We will only collect data that you submit. The analysis results and collection methodology may be published in research papers or technical reports, aiming for open access. Data access rights will be supervised by the researchers.

Participant Rights

If you have concerns about the study, please contact the researchers listed at the beginning of this informed consent form.

Consent Options

- **Accept**
- **Don't Accept**

D.2 Demographic Information and Knowledge Self-Assessment

Demographic Information

- **Age Range:** 18-24, 25-34, 35-44, 45-54, 55+, Prefer not to say

Knowledge Assessment

- **Programming Languages:** List the programming languages you know, separated by commas.
- **Relational Database Knowledge:** Rate on a scale of 1 (never heard of it) to 5 (professional level)
- **NoSQL Database Knowledge:** Rate on a scale of 1 (never heard of it) to 5 (professional level)
- **Graph Structure Knowledge:** Rate on a scale of 1 (never heard of it) to 5 (professional level)

Graph Query Language Proficiency

- OpenCypher, GraphQL, AQL, SQL/PGQ, SPARQL, None

D.3 Queries Ground Truth

In this Section, we report the ground truth answer to the questions of our study. We manually write the correct query to answer each question. Note that there are multiple solutions to obtain the correct results. We did not directly compare our ground truth with the students' answers to label them, instead, we used the following queries and guidelines.

D.3.1 Schema Discovery Queries

- Q1: Write a Cypher query that returns the set of all distinct node labels. State also how many answers it returns.

```
MATCH (n)
RETURN DISTINCT labels(n)
//ANSWERS: 9
```

- Q2: Write a Cypher query that returns the set of all distinct relationship types. State also how many answers it returns.

```
MATCH ()-[r]->()
RETURN DISTINCT type(r)
//ANSWERS: 10
```

- Q3: Write a Cypher query that returns each distinct node label together with its corresponding set of properties. State also how many answers it returns.

```
MATCH (n)
RETURN DISTINCT labels(n) AS label, properties(n) AS properties
//ANSWERS: 18
```

- Q4: Write a Cypher query that returns each distinct relationship type together with its corresponding set of properties. State also how many answers it returns.

```
MATCH ()-[r]->()
RETURN DISTINCT type(r) AS type, properties(n) AS properties
//ANSWERS: 10
```

- Q5: Write at least three additional Cypher queries that are needed for you to understand the underlying schema structure of the dataset. You can also write more than three queries in case. For each of them, state also how many answers it returns.

```
MATCH (m)-[l]->(n)-[k]->(:Genome)
RETURN DISTINCT type(k), type(l), labels(n), labels(m)
//ANSWERS: 1
```

```
MATCH (m)-[l]->(n)-[k]->(:Family)
RETURN DISTINCT type(k), type(l), labels(n), labels(m)
//ANSWERS: 6
```

```
MATCH (m)-->()-->(n)-->(:Family)
RETURN DISTINCT labels(m), labels(n)
//ANSWERS: 2
```

D.3.2 Analytical Queries

- Q6: Write a Cypher query to return each pangenome, together with its associated genes. State also how many answers it returns.

```
MATCH (g:Gene)-[:IS_IN_FAMILY]->(f:Family)-[:IS_IN_PANGENOME]->(p:Pangenome)
RETURN distinct p.name, g.name
//ANSWERS: 5477
```

- Q7: Write a Cypher query to return each pangenome, together with its associated spots. State also how many answers it returns.

```
MATCH (sp:Spot)-[:IS_IN_SPOT]-()-[:IS_IN_RGP]-()-[:IS_IN_FAMILY]->()
-[:IS_IN_PANGENOME]->(p:Pangenome)
RETURN distinct p.name, sp.name
//ANSWERS: 656
```

- Q8: Write a Cypher query to return each pangenome and its associated modules that contain at least a pair of gene families that are more than 80% identical. State also how many answers it returns.

```
MATCH (p:Pangenome)->()->(m:Module)->-(f1:Family)-[:IS_SIMILAR]-(f2:Family)
WHERE f1 <> f2 AND r.coverage >= 0.8
RETURN p.name, COUNT(DISTINCT(m.name))
//ANSWERS: 2 - Enterobacter.cloacae (236 modules)
//and Acinetobacter.baumannii (278 modules)
```

- Q9: Write a Cypher query that returns pairs of inter-pangenome families that both contain annotations and that are more than 80% identical. State also how many answers it returns.

```
MATCH (p1:Pangenome)-[:IS_IN_PANGENOME]-(f1:Family)-[:IS_SIMILAR]-(f2:Family)
-[:IS_IN_PANGENOME]-(p2:Pangenome)
WHERE r1.identity >= 0.8
AND p1.taxid <> p2.taxid
AND f1.annotation IS NOT NULL
AND f2.annotation IS NOT NULL
RETURN f1.name, f2.name
//ANSWERS: 14
```

- Q10: Write a Cypher query that returns, for each pangenome, the neighborhood of its gene families containing annotations. State also how many answers it returns.

```
MATCH (f1:Family)-[:NEIGHBOR]-(f2:Family)-[:IS_IN_PANGENOME]->(p:Pangenome),
WHERE f1.annotation IS NOT NULL
AND f2.annotation IS NOT NULL
RETURN p, f1, f2
//ANSWERS: 153
```

D.3.3 Aggregation Queries

- Q11: For every pangenome, write a Cypher query that return its name, together with its number of families that contain annotations. State also how many answers it returns.

```
MATCH (p:Pangenome)-[:IS_IN_PANGENOME]-(f:Family)
WHERE f.annotation IS NOT NULL
RETURN p.name, count(f)
//ANSWERS: ("Enterobacter.cloacae": 110, "Acinetobacter.baumannii": 92)
```

- Q12: Write a Cypher query that returns all modules together with the average similarity of the gene families they are comprised of, in descending order. State also how many answers it returns.

```
MATCH (m:Module) <-[]-(a:Family)-[:IS_SIMILAR]-(f:Family)
RETURN m.name, COLLECT(f.name), AVG(r.identity) AS idnt
ORDER BY idnt DESC
//ANSWERS: 406
```

- Q13: Write a Cypher query that returns the names of the modules that contain the pairs of gene families that have the minimum and, respectively, the maximum similarity (as

indicated by the value of the "identity" property of the "IS_SIMILAR" edge connecting them). State also how many answers it returns.

```
MATCH (m1:Module)-[:IS_IN_MODULE]-(f:Family)-[sim:IS_SIMILAR]->(f2:Family)
-[:IS_IN_MODULE]->(m2:Module)
RETURN m1.name, m2.name, sim.identity
ORDER BY sim.identity DESC
LIMIT 1
UNION ALL
MATCH (m1:Module)-[:IS_IN_MODULE]-(f:Family)-[sim:IS_SIMILAR]->(f2:Family)
-[:IS_IN_MODULE]->(m2:Module)
RETURN m1.name, m2.name, sim.identity
ORDER BY sim.identity LIMIT 1
//ANSWERS: Modules 2 and 177 with similarities 40 and, respectively, 0.8
```

- Q14: Write a Cypher query that returns the top 10 gene families with the highest number of associated modules. State also how many answers it returns.

```
MATCH (f:Family)-[:IS_IN_MODULE]-(m:Module)
WITH f, count(m) AS num
RETURN f, num
ORDER BY num DESC
LIMIT 10
//ANSWERS: 10
```

- Q15: Return the names of the top 10 pairs of similar families (as defined in Q13), whose associated identity and coverage metrics surpass 0.8, together with the names of their partitions (as indicated by the value of the "subpartition" property associated to the nodes of type "Partition"). State also how many answers it returns.

```
MATCH (p:Partition)-[:HAS_PARTITION]-(f1:Family)-[r:IS_SIMILAR]->(f2:Family)
-[:HAS_PARTITION]->(p2:Partition)
WHERE r.identity >=0.8 AND r.coverage >=0.8
RETURN f1.name, f2.name, p.partition, p2.partition, r.identity
ORDER BY r.identity DESC
LIMIT 10
//ANSWERS: 10
```

D.4 Semi-Structured Interview

- **Q1:** Explain your understanding of the SCHEMA (in general). What is it and what is it used for?
- **Q2:** What is the use of the COLLECT keyword in Cypher?
- **Q3:** In which other programming language/tool is there something similar to COLLECT? Does it remind you of something you've used before?
- **Q4:** What is the use of the WITH keyword in Cypher?
- **Q5:** In which other programming language/tool is there something similar to WITH? Does it remind you of something you've used before?
- **Q6:** Given this question: Write a Cypher query that returns each distinct node label together with its corresponding set of properties. Which of the following queries is correct?

- MATCH (n) RETURN DISTINCT labels(n) AS labels, keys(n) AS properties
- MATCH (n) RETURN DISTINCT labels(n) AS labels, collect(distinct keys(n)) AS properties
- MATCH (n) RETURN DISTINCT labels(n) AS labels, collect(keys(n)) AS properties
- **Q7:** Explain the differences between the queries provided in **Q6**.
- **Q8:** Given the same question in **Q6**, explain the difference between the following queries:
 - MATCH (n) RETURN DISTINCT labels(n), collect(keys(n)) AS properties
 - MATCH (n) WITH DISTINCT labels(n) AS labels, n RETURN labels, collect(keys(n)) AS properties
- **Q9:** Explain the differences between the queries provided in **Q8**.
- **Q10:** Given this question: Write a Cypher query to return each pangenome, together with its associated genes. Which of the following queries is correct?
 - MATCH (p:Pangenome)-(f:Family)-(g:Gene) RETURN DISTINCT p.name, g.name;
 - MATCH (g:Gene)-[:IS_IN_FAMILY]->(f:Family)-[:IS_IN_PANGENOME]->(p:Pangenome) RETURN p.name AS pangenomeName, COLLECT(g.name) AS genes
 - MATCH (gene:Gene)-[:IS_IN_FAMILY]->(family:Family)-[:IS_IN_PANGENOME]->(pangenome:Pangenome) RETURN pangenome, gene
- **Q11:** Explain the differences between the queries provided in **Q10**.
- **Q12:** Given a similar query structure as in **Q10**, are both of the following queries correct?
 - MATCH (s:Spot)<-[:IS_IN_SPOT]-(:RGP)<-[:IS_IN_RGP]-(g:Gene)-[:IS_IN_FAMILY]->(f:Family)-[:IS_IN_PANGENOME]->(p:Pangenome) RETURN p.name AS pangenome, COLLECT(s.name) AS spots
 - MATCH (sp:Spot)<-[:IS_IN_SPOT]-(p:Pangenome)<-[:IS_IN_PANGENOME]-(f:Family)-[:IS_IN_FAMILY]->(g:Gene)-[:IS_IN_FAMILY]->(p:Pangenome) RETURN DISTINCT p.name, sp.name
- **Q13:** Explain the answer to the previous question.
- **Q14:** If a query has the return statement RETURN a, COLLECT(b), would you assume that the names of b are unique? Motivate your answer.
- **Q15:** Given this question: Write a Cypher query to return each pangenome and its associated modules with gene families more than 80% identical. Which of the following queries is correct?
 - MATCH (p:Pangenome)<-()->(m:Module)<-(f1:Family)<-[:IS_SIMILAR]-(f2:Family) WHERE f1 <> f2 AND r.coverage >= 0.8 RETURN p.name
 - MATCH (p:Pangenome)<-[:IS_IN_PANGENOME]-(f:Family)-[:IS_IN_MODULE]->(m:Module), (p:Pangenome) <-[:IS_IN_PANGENOME]-(f:Family)-[:IS_SIMILAR]-(f2:Family) WHERE sim.coverage >= 0.8 RETURN p.name AS Pan-genome

- **Q16:** Explain the differences between the queries provided in **Q15**.
- **Q17:** Given a query pattern with two nodes of the same label, for example:


```

      - MATCH (p:Pangenome)<-()->(m:Module)<-(f1:Family)<-[r:IS_SIMILAR]-(f2:Family)
      - MATCH (p:Pangenome)<-[:IS_IN_PANGENOME]-(f1:Family)-[:IS_IN_MODULE]->(m:Module),
        (p:Pangenome) <-[:IS_IN_PANGENOME]-(f1:Family)-[sim:IS_SIMILAR]->(f2:Family)
        WHERE sim.identity >= 0.8 RETURN p.name AS Pangenome
      
```

Could f1 and f2 be the same node? Motivate your answer.

- **Q18:** Given a query to return inter-pangenome family pairs with annotations that are over 80% identical, which query is correct?


```

      - MATCH (p:Pangenome)<-[:IS_IN_PANGENOME] -(f:Family) MATCH (p2:Pangenome)<-[:IS_IN_PANGENOME]-(f2:Family) MATCH (f)-[s:IS_SIMILAR]->(f2) WHERE s.identity >= 0.8 AND
        p <> p2 AND f IS NOT NULL AND f2 IS NOT NULL RETURN f.name, f2.name
      - MATCH (p1:Pangenome)-[:IS_IN_PANGENOME]-(f1:Family)-[r1:IS_SIMILAR]-(f2:Family)-[:IS_IN_PANGENOME] -(p2:Pangenome) WHERE r1.identity >= 0.8 AND p1.taxid
        <> p2.taxid AND f1.annotation IS NOT NULL AND f2.annotation IS NOT NULL
        RETURN f1.name, f2.name
      
```
- **Q19:** Explain the differences between the queries provided in **Q18**.

D.5 Table extended versions

Table D.1 is the extended version of Table 4 of the paper, reporting the *complications* and the *semantic return errors* performed by the students in the **schema discovery queries**. This extended table shows that none of the errors were related to advanced query concepts (as expected from the type of questions) such as the **LIMIT** and **ORDER BY** clauses. Moreover, none of the errors consists of a *semantic where error* or a *semantic pattern error*.

Table D.2 is the extended version of Table 5 of the paper, reporting the *complications* and the *semantic errors* performed by the students in the **analytical queries**. In this phase, the *complications* caused by the overuse of **WITH** and **UNWIND** are absent. None of the students used a *Wrong Edge Label* when defining the pattern of the queries. *Additional Condition* errors were also absent in this phase. Finally, as for the schema discovery queries, none of the students struggled with the **LIMIT** and **ORDER BY** clauses.

Table D.3 is the extended version of Table 6 of the paper, reporting the *complications* and the *semantic errors* performed by the students in the **aggregation queries**. In this last phase, none of the *complications* was due to the use of **COLLECT** and **UNWIND**. As for the analytical queries, none of the students used a *Wrong Edge Label* when defining the pattern of the queries. For what concerns the *semantic where errors*, none of the students put a condition on a *wrong property* or used a *wrong* or *additional condition*. Finally, among the *semantic return errors*, **Wrong DISTINCT** and **Missing DISTINCT** are absent. Differently from the previous phases, some of the errors (10) are due to **LIMIT** on the wrong variable, while none of them concerns the **ORDER BY** clause.

Category	Type	Count	Count / Category Total	Count / Total
Complications	Added COLLECT	49	0.422	0.395
	Added WITH	40	0.345	0.323
	Added UNWIND	27	0.233	0.218
	Additional information	0	0	0
	Total	116	1	0.935
Semantic Return Errors	Missing DISTINCT	6	0.75	0.048
	Missing Information	1	0.125	0.008
	Wrong Information	1	0.125	0.008
	Missing LIMIT	0	0	0
	Missing ORDER BY	0	0	0
	Wrong LIMIT	0	0	0
	Wrong ORDER BY	0	0	0
	Wrong DISTINCT	0	0	0
	Total	8	1	0.065
Total		124	1	1

Table D.1: Number of error categories and type occurred writing schema discovery queries. We report the absolute number, the percentage of occurrence both within the category and with respect to the whole set of errors.

Category	Type	Count	Count/ Category Total	Count / Total
Complications	Added COLLECT	26	0.963	0,096
	Added WITH	0	0	0
	Added UNWIND	0	0	0
	Additional Information	1	0.037	0,004
	Total	27	1	0.1
Semantic Pattern Errors	Wrong Pattern w.r.t. Schema	18	0.29	0,067
	Wrong Pattern	21	0.339	0,078
	Missing Node	4	0.065	0,015
	Additional Node	5	0.081	0,019
	Wrong Edge Direction	14	0.226	0,052
	Wrong Edge Label	0	0	0
	Total	62	1	0.230
Semantic Where Errors	Wrong Property	31	0.34	0,115
	Missing Condition	49	0.538	0,181
	Wrong Condition	3	0.033	0,011
	Wrong Operator	8	0.088	0,030
	Additional Condition	0	0	0
	Total	91	1	0.337
Semantic Return Errors	Missing DISTINCT	34	0.378	0,126
	Missing Information	13	0.144	0,048
	Wrong Information	41	0.456	0,152
	Wrong DISTINCT	2	0.022	0,007
	Missing LIMIT	0	0	0
	Missing ORDER BY	0	0	0
	Wrong LIMIT	0	0	0
	Wrong ORDER BY	0	0	0
	Total	90	1	0.333
Total		270	1	1

Table D.2: Number of error categories and type occurred writing analytical queries. We report the absolute number, the percentage of occurrence both within the category and with respect to the whole set of errors.

Category	Type	Count	Count/ Category Total	Count / Total
Complications	Added COLLECT	0	0	0
	Added WITH	2	0.5	0.038
	Added UNWIND	0	0	0
	Additional Information	2	0.5	0.038
	Total	4	1	0.075
Semantic Pattern Errors	Wrong Pattern w.r.t. Schema	3	0.143	0.057
	Wrong Pattern	1	0.048	0.019
	Missing Node	1	0.048	0.019
	Additional Node	3	0.0143	0.057
	Wrong Edge Direction	13	0.619	0.245
	Wrong Edge Label	0	0	0
	Total	21	1	0.396
Semantic Where Errors	Wrong Property	0	0	0
	Missing Condition	2	0.667	0.038
	Wrong Condition	0	0	0
	Wrong Operator	1	0.333	0.019
	Additional Condition	0	0	0
	Total	3	1	0.057
Semantic Return Errors	Missing DISTINCT	0	0	0
	Missing Information	14	0.56	0.264
	Wrong Information	1	0.040	0.019
	Wrong DISTINCT	0	0	0
	Missing LIMIT	0	0	0
	Missing ORDER BY	0	0	0
	Wrong LIMIT	10	0.4	0.189
	Wrong ORDER BY	0	0	0
	Total	25	1	0.472
Total		53	1	1

Table D.3: Number of error categories and type occurred in writing aggregate queries. We report the absolute number, the percentage of occurrence both within the category and with respect to the whole set of errors

Bibliography

- [1] International Organization for Standardization, “ISO/IEC 39075:2024 information technology — database languages — GQL.” ISO/IEC Standard, 2024.
- [2] R. Angles, A. Bonifati, S. Dumbrava, G. Fletcher, A. Green, J. Hidders, B. Li, L. Libkin, V. Marsault, W. Martens, F. Murlak, S. Plantikow, O. Savkovic, M. Schmidt, J. Sequeda, S. Staworko, D. Tomaszuk, H. Voigt, D. Vrgoc, M. Wu, and D. Zivkovic, “Pg-schema: Schemas for property graphs,” *Proc. ACM Manag. Data*, vol. 1, jun 2023.
- [3] W. Fan, Y. Wu, and J. Xu, “Functional dependencies for graphs,” in *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD ’16, (New York, NY, USA), p. 1843–1857, Association for Computing Machinery, 2016.
- [4] W. Fan and P. Lu, “Dependencies for graphs,” in *Proceedings of the 36th ACM SIGMOD-SIGACT-SIGAI Symposium on Principles of Database Systems*, PODS ’17, (New York, NY, USA), p. 403–416, Association for Computing Machinery, 2017.
- [5] S. Sahu, A. Mhedhbi, S. Salihoglu, J. Lin, and M. T. Özsu, “The ubiquity of large graphs and surprising challenges of graph processing: extended survey,” *VLDB J.*, vol. 29, no. 2-3, pp. 595–618, 2020.
- [6] A. Zaveri, A. Rula, A. Maurino, R. Pietrobon, J. Lehmann, and S. Auer, “Quality assessment methodologies for linked open data: A systematic literature review and conceptual framework,” *Semantic Web Journal*, 11 2013.
- [7] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, “HoloClean: Holistic data repairs with probabilistic inference,” *Proceedings of the VLDB Endowment (PVLDB)*, vol. 10, no. 11, pp. 1190–1201, 2017.
- [8] T. N. Kipf and M. Welling, “Semi-supervised classification with graph convolutional networks,” in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2017.
- [9] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *ICLR*, 2018.
- [10] J. Pearl and D. Mackenzie, *The Book of Why: The New Science of Cause and Effect*. Basic Books, 2018.

- [11] J. Pearl, “The seven tools of causal inference, with reflections on machine learning,” *Communications of the ACM (CACM)*, vol. 62, no. 3, pp. 54–60, 2019.
- [12] A. Pachera, A. Bonifati, and A. Mauri, “Towards user-centric graph repairs,” in *Proceedings of the 40th IEEE International Conference on Data Engineering Workshops (ICDEW 2024)*, pp. 375–376, IEEE, 2024.
- [13] A. Pachera, A. Bonifati, and A. Mauri, “User-centric property graph repairs,” *Proceedings of the ACM on Management of Data*, vol. 3, no. 1, pp. 85:1–85:27, 2025.
- [14] A. Pachera, A. Bonifati, and A. Mauri, “Grafixer: Enabling user-centric repairs for property graphs,” in *Companion of the 2025 International Conference on Management of Data (SIGMOD/PODS 2025)*, pp. 199–202, ACM, 2025.
- [15] A. Pachera, A. Bonifati, L. V. Lakshmanan, , and A. Mauri, “Budgeted interactive property graph repair with graph neural networks,” in *Proceedings of the 2027 ACM SIGMOD International Conference on Management of Data (SIGMOD 2027)*, ACM, 2027. To appear.
- [16] A. Pachera, M. Palmiotto, A. Bonifati, and A. Mauri, “WhatIf: Causal analysis with graph databases,” *Proceedings of the VLDB Endowment*, vol. 18, no. 11, pp. 4009–4016, 2025.
- [17] D. Qiu, Y. Zhou, A. Pachera, A. Bonifati, and A. Mauri, “Toward a causal data management ecosystem for decision making and agentic AI,” *CoRR*, vol. abs/2608.07214, 2026.
- [18] A. Pachera, S. Dumbrava, A. Bonifati, and A. Mauri, “Understanding student errors in graph query formulation,” *ACM Transactions on Computing Education*, vol. 25, no. 3, pp. 1–35, 2025.
- [19] A. Pachera, A. Mauri, K. Imteyaz, J. Yang, E. Umuhoza, A. Bonifati, M. Lahav, and N. Goyal, “Co-data: Cultivating effective human-LLM collaboration for collaborative data processing,” in *Extended Abstracts of the 2026 CHI Conference on Human Factors in Computing Systems (CHI EA ’26)*, pp. 1–7, ACM, 2026.
- [20] A. Bonifati, G. H. L. Fletcher, H. Voigt, and N. Yakovets, *Querying Graphs*. Synthesis Lectures on Data Management, Morgan & Claypool Publishers, 2018.
- [21] R. Angles, M. Arenas, P. Barceló, P. Boncz, G. Fletcher, C. Gutierrez, T. Lindaaker, M. Paradies, S. Plantikow, J. Sequeda, O. van Rest, and H. Voigt, “G-CORE: A core for future graph query languages,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*, pp. 1421–1432, 2018.
- [22] R. Angles and C. Gutierrez, “Survey of graph database models,” *ACM Comput. Surv.*, vol. 40, feb 2008.
- [23] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, and A. Taylor, “Cypher: An evolving query language for property graphs,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*, pp. 1433–1445, 2018.

- [24] International Organization for Standardization, “ISO/IEC 39075:2024: Information technology — Database languages — GQL,” Apr. 2024.
- [25] World Wide Web Consortium, “Sparql 1.1 overview.” W3C Recommendation, <https://www.w3.org/TR/sparql11-overview/>, 2013.
- [26] W. Fan, F. Geerts, X. Jia, and A. Kementsietsidis, “Conditional functional dependencies for capturing data inconsistencies,” *ACM Trans. Database Syst.*, vol. 33, no. 1, pp. 6:1–6:48, 2008.
- [27] W. Fan, H. Gao, X. Jia, J. Li, and S. Ma, “Dynamic constraints for record matching,” *VLDB J.*, vol. 20, no. 4, pp. 495–520, 2011.
- [28] M. Arenas, L. Bertossi, and J. Chomicki, “Consistent query answers in inconsistent databases,” in *Proceedings of the ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS)*, pp. 68–79, 1999.
- [29] X. Chu, I. F. Ilyas, and P. Papotti, “Discovering denial constraints,” *Proceedings of the VLDB Endowment*, vol. 6, no. 13, pp. 1498–1509, 2013.
- [30] W. Fan, K. Pang, P. Lu, and C. Tian, “Making it tractable to detect and correct errors in graphs,” *ACM Trans. Database Syst.*, vol. 49, no. 4, pp. 16:1–16:75, 2024.
- [31] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.
- [32] R. Angles, A. Bonifati, S. Dumbrava, G. Fletcher, K. W. Hare, J. Hidders, V. E. Lee, B. Li, L. Libkin, W. Martens, F. Murlak, J. Perryman, O. Savković, M. Schmidt, J. Sequeda, S. Staworko, and D. Tomaszuk, “PG-Keys: Keys for property graphs,” in *SIGMOD*, pp. 2423–2436, 2021.
- [33] D. Kontokostas and H. Knublauch, “Shapes constraint language (SHACL),” w3c recommendation, W3C, 2017.
- [34] S. Staworko, I. Boneva, J. E. L. Gayo, S. Hym, E. G. Prud’hommeaux, and H. R. Solbrig, “Complexity and expressiveness of ShEx for RDF,” in *ICDT*, pp. 195–211, 2015.
- [35] W. Fan, C. Hu, X. Liu, and P. Lu, “Discovering graph functional dependencies,” *ACM Trans. Database Syst.*, vol. 45, no. 3, pp. 15:1–15:42, 2020.
- [36] W. Fan, W. Fu, R. Jin, P. Lu, and C. Tian, “Discovering association rules from big graphs,” *Proc. VLDB Endow.*, vol. 15, no. 7, pp. 1479–1492, 2022.
- [37] M. Elseidy, E. Abdelhamid, S. Skiadopoulos, and P. Kalnis, “GRAMI: Frequent subgraph and pattern mining in a single large graph,” *Proc. VLDB Endow.*, vol. 7, no. 7, pp. 517–528, 2014.
- [38] W. Fan, Z. Fan, C. Tian, and X. L. Dong, “Keys for graphs,” *Proceedings of the VLDB Endowment (PVLDB)*, vol. 8, no. 12, pp. 1590–1601, 2015.

- [39] W. Fan and P. Lu, “Dependencies for graphs,” *ACM Trans. Database Syst.*, vol. 44, no. 2, pp. 5:1–5:40, 2019.
- [40] W. Fan, X. Liu, P. Lu, and C. Tian, “Catching numeric inconsistencies in graphs,” *ACM Trans. Database Syst.*, vol. 45, no. 2, pp. 9:1–9:47, 2020.
- [41] W. Fan, R. Jin, M. Liu, P. Lu, C. Tian, and J. Zhou, “Capturing associations in graphs,” *Proc. VLDB Endow.*, vol. 13, no. 11, pp. 1863–1876, 2020.
- [42] W. Fan, W. Fu, R. Jin, M. Liu, P. Lu, and C. Tian, “Making it tractable to catch duplicates and conflicts in graphs,” *Proc. ACM Manag. Data*, vol. 1, no. 1, pp. 86:1–86:28, 2023.
- [43] C. Spinrath, A. Bonifati, and R. Echahed, “Repairing property graphs under PG-Constraints,” *Proc. VLDB Endow.*, vol. 19, no. 6, pp. 1212–1225, 2026.
- [44] S. Song, B. Liu, H. Cheng, J. X. Yu, and L. Chen, “Graph repairing under neighborhood constraints,” *The VLDB Journal*, vol. 26, no. 5, pp. 611–635, 2017.
- [45] P. Lin, Q. Song, Y. Wu, and J. Pi, “Repairing entities using star constraints in multirelational graphs,” in *36th IEEE International Conference on Data Engineering, ICDE 2020, Dallas, TX, USA, April 20-24, 2020*, pp. 229–240, IEEE, 2020.
- [46] B. Motik, Y. Nenov, R. Piro, and I. Horrocks, “Maintenance of datalog materialisations revisited,” *Artif. Intell.*, vol. 269, pp. 76–136, 2019.
- [47] L. C. Shimomura, G. Fletcher, and N. Yakovets, “GGDs: Graph generating dependencies,” in *CIKM*, 2020.
- [48] T. Bleifuß, S. Kruse, and F. Naumann, “Efficient denial constraint discovery with hydra,” *Proc. VLDB Endow.*, vol. 11, no. 3, pp. 311–323, 2017.
- [49] S. Song, H. Cheng, J. X. Yu, and L. Chen, “Repairing vertex labels under neighborhood constraints,” *Proc. VLDB Endow.*, vol. 7, no. 11, pp. 987–998, 2014.
- [50] B. Perozzi, R. Al-Rfou, and S. Skiena, “DeepWalk: Online learning of social representations,” in *KDD*, pp. 701–710, 2014.
- [51] A. Grover and J. Leskovec, “node2vec: Scalable feature learning for networks,” in *KDD*, pp. 855–864, 2016.
- [52] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, “Neural message passing for quantum chemistry,” in *ICML*, pp. 1263–1272, 2017.
- [53] W. L. Hamilton, R. Ying, and J. Leskovec, “Inductive representation learning on large graphs,” in *NeurIPS*, pp. 1024–1034, 2017.
- [54] Z. Zhang, P. Cui, and W. Zhu, “Deep learning on graphs: A survey,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 249–270, 2020.

- [55] M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, “Modeling relational data with graph convolutional networks,” in *ESWC*, vol. 10843 of *LNCS*, pp. 593–607, 2018.
- [56] T. Yang, L. Hu, C. Shi, H. Ji, X. Li, and L. Nie, “HGAT: Heterogeneous graph attention networks for semi-supervised short text classification,” *ACM Transactions on Information Systems*, vol. 39, no. 3, pp. 1–29, 2021. Conference version: EMNLP-IJCNLP 2019, pp. 4821–4830.
- [57] X. Wang, H. Ji, C. Shi, B. Wang, Y. Ye, P. Cui, and P. S. Yu, “Heterogeneous graph attention network,” in *WWW*, pp. 2022–2032, 2019.
- [58] J. Pearl, *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2nd ed., 2009.
- [59] H. Reichenbach, *The Direction of Time*. University of California Press, 1956.
- [60] B. Salimi, J. Gehrke, and D. Suciu, “Bias in OLAP queries: Detection, explanation, and removal,” in *Proc. SIGMOD*, 2018.
- [61] E. Bareinboim, J. D. Correa, D. Ibeling, and T. Icard, “On Pearl’s hierarchy and the foundations of causal inference,” in *Probabilistic and Causal Inference: The Works of Judea Pearl*, pp. 507–556, ACM Books, 2022.
- [62] D. Z. Wang, E. Michelakis, M. Garofalakis, and J. M. Hellerstein, “Bayesstore: managing large, uncertain data repositories with probabilistic graphical models,” *Proceedings of the VLDB Endowment*, vol. 1, no. 1, pp. 340–351, 2008.
- [63] R. Dechter and J. Pearl, “Directed constraint networks: A relational framework for causal modeling,” in *Proceedings of the 12th International Joint Conference on Artificial Intelligence. Sydney, Australia, August 24-30, 1991*, pp. 1164–1170, 1991.
- [64] P. Spirtes, C. Glymour, and R. Scheines, *Causation, Prediction, and Search*. MIT Press, 2nd ed., 2000.
- [65] J. Pearl, “Causal diagrams for empirical research,” *Biometrika*, vol. 82, no. 4, pp. 669–688, 1995.
- [66] I. Shpitser and J. Pearl, “Identification of joint interventional distributions in recursive semi-Markovian causal models,” in *Proc. AAAI*, 2006.
- [67] Y. Huang and M. Valtorta, “Pearl’s calculus of intervention is complete,” in *Proc. UAI*, 2006.
- [68] A. Balke and J. Pearl, “Counterfactual probabilities: Computational methods, bounds and applications,” in *Uncertainty in artificial intelligence*, pp. 46–54, Elsevier, 1994.
- [69] J. Tian and J. Pearl, “Probabilities of causation: Bounds and identification,” *Annals of Mathematics and Artificial Intelligence*, vol. 28, pp. 287–313, 2000.

- [70] D. B. Rubin, “Estimating causal effects of treatments in randomized and nonrandomized studies,” *Journal of Educational Psychology*, vol. 66, no. 5, pp. 688–701, 1974.
- [71] G. W. Imbens and D. B. Rubin, *Causal Inference for Statistics, Social, and Biomedical Sciences*. Cambridge University Press, 2015.
- [72] S. Amershi, M. Cakmak, W. B. Knox, and T. Kulesza, “Power to the people: The role of humans in interactive machine learning,” *AI Magazine*, vol. 35, no. 4, pp. 105–120, 2014.
- [73] J. Heer, “Agency plus automation: Designing artificial intelligence into interactive systems,” *Proceedings of the National Academy of Sciences*, vol. 116, no. 6, pp. 1844–1850, 2019.
- [74] X. Wu, L. Xiao, Y. Sun, J. Zhang, T. Ma, and L. He, “A survey of human-in-the-loop for machine learning,” *Future Generation Computer Systems*, vol. 135, pp. 364–381, 2022.
- [75] A. J. Quinn and B. B. Bederson, “Human computation: A survey and taxonomy of a growing field,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 2011)*, pp. 1403–1412, ACM, 2011. Page range to be verified.
- [76] R. Snow, B. O’Connor, D. Jurafsky, and A. Y. Ng, “Cheap and fast but is it good? evaluating non-expert annotations for natural language tasks,” in *Proceedings of the 2008 Conference on Empirical Methods in Natural Language Processing (EMNLP 2008)*, pp. 254–263, 2008. Page range to be verified.
- [77] M. Yakout, A. K. Elmagarmid, J. Neville, M. Ouzzani, and I. F. Ilyas, “Guided data repair,” *Proc. VLDB Endow.*, vol. 4, p. 279–289, feb 2011.
- [78] A. G. Parameswaran, H. Garcia-Molina, H. Park, N. Polyzotis, A. Ramesh, and J. Widom, “CrowdScreen: Algorithms for filtering data with humans,” in *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data (SIGMOD 2012)*, pp. 361–372, ACM, 2012.
- [79] A. Ratner, S. H. Bach, H. Ehrenberg, J. Fries, S. Wu, and C. Ré, “Snorkel: Rapid training data creation with weak supervision,” *Proceedings of the VLDB Endowment*, vol. 11, no. 3, pp. 269–282, 2017.
- [80] M. J. Franklin, D. Kossmann, T. Kraska, S. Ramesh, and R. Xin, “Crowddb: Answering queries with crowdsourcing,” in *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data, SIGMOD ’11*, (New York, NY, USA), p. 61–72, Association for Computing Machinery, 2011.
- [81] V. Raman and J. M. Hellerstein, “Potter’s wheel: An interactive data cleaning system,” in *Proceedings of the 27th International Conference on Very Large Data Bases (VLDB 2001)*, pp. 381–390, Morgan Kaufmann, 2001.
- [82] S. Kandel, A. Paepcke, J. M. Hellerstein, and J. Heer, “Wrangler: Interactive visual specification of data transformation scripts,” in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI 2011)*, pp. 3363–3372, ACM, 2011.

- [83] S. Idreos, O. Papaemmanouil, and S. Chaudhuri, “Overview of data exploration techniques,” in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD 2015)*, pp. 277–281, ACM, 2015.
- [84] B. Settles, “Active learning literature survey,” Computer Sciences Technical Report 1648, University of Wisconsin-Madison, 2009.
- [85] V. S. Sheng, F. Provost, and P. G. Ipeirotis, “Get another label? improving data quality and data mining using multiple, noisy labelers,” in *Proceedings of the 14th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD 2008)*, pp. 614–622, ACM, 2008.
- [86] B. Green and Y. Chen, “The principles and limits of algorithm-in-the-loop decision making,” *Proceedings of the ACM on Human-Computer Interaction*, vol. 3, no. CSCW, pp. 50:1–50:24, 2019.
- [87] A. Bonifati, U. Comignani, E. Coquery, and R. Thion, “Interactive mapping specification with exemplar tuples,” *ACM Transactions on Database Systems*, vol. 44, no. 3, pp. 10:1–10:44, 2019. Conference version: SIGMOD 2017, pages 667–682, doi 10.1145/3035918.3064028.
- [88] S. R. Jeffery, M. J. Franklin, and A. Y. Halevy, “Pay-as-you-go user feedback for database systems,” in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD 2008)*, pp. 847–860, ACM, 2008. Page range to be verified.
- [89] J. Sweller, “Cognitive load during problem solving: Effects on learning,” *Cognitive Science*, vol. 12, no. 2, pp. 257–285, 1988.
- [90] P. G. Ipeirotis, F. Provost, and J. Wang, “Quality management on Amazon Mechanical Turk,” in *Proceedings of the ACM SIGKDD Workshop on Human Computation (HCOMP 2010)*, pp. 64–67, ACM, 2010.
- [91] A. P. Dawid and A. M. Skene, “Maximum likelihood estimation of observer error-rates using the EM algorithm,” *Journal of the Royal Statistical Society, Series C (Applied Statistics)*, vol. 28, no. 1, pp. 20–28, 1979.
- [92] A. Marcus, E. Wu, D. R. Karger, S. Madden, and R. C. Miller, “Crowdsourced databases: Query processing with people,” in *Proceedings of the 5th Biennial Conference on Innovative Data Systems Research (CIDR 2011)*, pp. 211–214, 2011.
- [93] H. Park, R. Pang, A. G. Parameswaran, H. Garcia-Molina, N. Polyzotis, and J. Widom, “Deco: A system for declarative crowdsourcing,” *Proceedings of the VLDB Endowment*, 2012. Volume, number, and pages to be completed.
- [94] B. Trushkowsky, T. Kraska, M. J. Franklin, and P. Sarkar, “Crowdsourced enumeration queries,” in *Proceedings of the 29th IEEE International Conference on Data Engineering (ICDE 2013)*, pp. 673–684, IEEE, 2013. Page range to be verified.

- [95] G. Li, J. Wang, Y. Zheng, and M. J. Franklin, “Crowdsourced data management: A survey,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 28, no. 9, pp. 2296–2319, 2016.
- [96] A. Marcus and A. Parameswaran, “Crowdsourced data management: Industry and academic perspectives,” *Foundations and Trends in Databases*, vol. 6, no. 1-2, pp. 1–161, 2015.
- [97] A. Doan, R. Ramakrishnan, and A. Y. Halevy, “Crowdsourcing systems on the world-wide web,” *Communications of the ACM*, vol. 54, no. 4, pp. 86–96, 2011.
- [98] J. Wang, T. Kraska, M. J. Franklin, and J. Feng, “CrowdER: Crowdsourcing entity resolution,” *Proceedings of the VLDB Endowment*, vol. 5, no. 11, pp. 1483–1494, 2012.
- [99] A. Marcus, E. Wu, D. R. Karger, S. Madden, and R. C. Miller, “Human-powered sorts and joins,” *Proceedings of the VLDB Endowment*, vol. 5, no. 1, pp. 13–24, 2011.
- [100] S. E. Whang, P. Lofgren, and H. Garcia-Molina, “Question selection for crowd entity resolution,” *Proceedings of the VLDB Endowment*, vol. 6, no. 6, pp. 349–360, 2013.
- [101] C. Chai, G. Li, J. Li, D. Deng, and J. Feng, “A partial-order-based framework for cost-effective crowdsourced entity resolution,” *The VLDB Journal*, vol. 27, no. 6, pp. 745–770, 2018.
- [102] X. Chu, J. Morcos, I. F. Ilyas, M. Ouzzani, P. Papotti, N. Tang, and Y. Ye, “KATARA: A data cleaning system powered by knowledge bases and crowdsourcing,” in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data (SIGMOD 2015)*, pp. 1247–1261, ACM, 2015.
- [103] G. Demartini, D. E. Difallah, and P. Cudré-Mauroux, “ZenCrowd: Leveraging probabilistic reasoning and crowdsourcing techniques for large-scale entity linking,” in *Proceedings of the 21st International Conference on World Wide Web (WWW 2012)*, pp. 469–478, ACM, 2012.
- [104] Y. Amsterdamer, Y. Grossman, T. Milo, and P. Senellart, “Crowd mining,” in *Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (SIGMOD 2013)*, pp. 241–252, ACM, 2013.
- [105] Y. Zheng, G. Li, Y. Li, C. Shan, and R. Cheng, “Truth inference in crowdsourcing: Is the problem solved?,” *Proceedings of the VLDB Endowment*, vol. 10, no. 5, pp. 541–552, 2017.
- [106] E. Rahm and P. A. Bernstein, “A survey of approaches to automatic schema matching,” *The VLDB Journal*, vol. 10, no. 4, pp. 334–350, 2001.
- [107] Q. V. H. Nguyen, T. T. Nguyen, Z. Miklós, K. Aberer, A. Gal, and M. Weidlich, “Pay-as-you-go reconciliation in schema matching networks,” in *Proceedings of the 30th IEEE International Conference on Data Engineering (ICDE 2014)*, pp. 220–231, IEEE, 2014.
- [108] C. Gokhale, S. Das, A. Doan, J. F. Naughton, N. Rampalli, J. Shavlik, and X. Zhu, “Corleone: Hands-off crowdsourcing for entity matching,” in *Proceedings of the 2014 ACM*

- SIGMOD International Conference on Management of Data (SIGMOD 2014)*, pp. 601–612, ACM, 2014.
- [109] D. Mottin, M. Lissandrini, Y. Velegrakis, and T. Palpanas, “Exemplar queries: Give me an example of what you need,” *Proceedings of the VLDB Endowment*, vol. 7, no. 5, pp. 365–376, 2014.
 - [110] A. Bonifati, R. Ciucanu, and S. Staworko, “Learning join queries from user examples,” *ACM Transactions on Database Systems*, vol. 40, no. 4, pp. 24:1–24:38, 2016.
 - [111] J. He, E. Veltri, D. Santoro, G. Li, G. Mecca, P. Papotti, and N. Tang, “Interactive and deterministic data cleaning,” in *Proceedings of the 2016 International Conference on Management of Data*, SIGMOD ’16, (New York, NY, USA), p. 893–907, Association for Computing Machinery, 2016.
 - [112] M. T. Özsu, “A survey of RDF data management systems,” *Frontiers Comput. Sci.*, vol. 10, no. 3, pp. 418–432, 2016.
 - [113] X. Dong, E. Gabrilovich, G. Heitz, W. Horn, N. Lao, K. Murphy, T. Strohmann, S. Sun, and W. Zhang, “Knowledge vault: a web-scale approach to probabilistic knowledge fusion,” in *The 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD ’14, New York, NY, USA - August 24 - 27, 2014* (S. A. Macskassy, C. Perlich, J. Leskovec, W. Wang, and R. Ghani, eds.), pp. 601–610, ACM, 2014.
 - [114] L. Bertossi, *Database Repairs and Consistent Query Answering*. Morgan & Claypool Publishers, 2011.
 - [115] A. Pirhadi, M. H. Moslemi, A. Cloninger, M. Milani, and B. Salimi, “Otclean: Data cleaning for conditional independence violations using optimal transport,” *Proc. ACM Manag. Data*, vol. 2, May 2024.
 - [116] Y. Cheng, L. Chen, Y. Yuan, and G. Wang, “Rule-based graph repairing: Semantic and efficient repairing methods,” in *34th IEEE International Conference on Data Engineering, ICDE 2018, Paris, France, April 16-19, 2018*, pp. 773–784, IEEE Computer Society, 2018.
 - [117] S. Staworko, J. Chomicki, and J. Marcinkowski, “Prioritized repairing and consistent query answering in relational databases,” *Ann. Math. Artif. Intell.*, vol. 64, no. 2-3, pp. 209–246, 2012.
 - [118] I. F. Ilyas and X. Chu, *Data Cleaning*, vol. 28 of *ACM Books*. ACM, 2019.
 - [119] A. Arioua and A. Bonifati, “User-guided repairing of inconsistent knowledge bases,” in *EDBT: Extending Database Technology*, pp. 133–144, OpenProceedings.org, 2018.
 - [120] M. Loster, D. Mottin, P. Papotti, J. Ehmüller, B. Feldmann, and F. Naumann, “Few-shot knowledge validation using rules,” in *Proceedings of the Web Conference 2021, WWW ’21*, (New York, NY, USA), p. 3314–3324, Association for Computing Machinery, 2021.

- [121] P. Juillard, A. Bonifati, and A. Mauri, “Interactive graph repairs for neighborhood constraints,” in *Proceedings of the 27th International Conference on Extending Database Technology (EDBT 2024)*, pp. 2:175–2:187, OpenProceedings.org, 2024.
- [122] J. Chomicki and J. Marcinkowski, “Minimal-change integrity maintenance using tuple deletions,” *Information and Computation*, vol. 197, no. 1-2, pp. 90–121, 2005.
- [123] J. Wijsen, “Database repairing using updates,” *ACM Transactions on Database Systems (TODS)*, vol. 30, no. 3, pp. 722–768, 2005.
- [124] P. A. Bernstein, V. Hadzilacos, and N. Goodman, *Concurrency Control and Recovery in Database Systems*. Addison-Wesley, 1987.
- [125] U. Halıcı and A. Doğaç, “A graph-based method for database concurrency control,” in *Proceedings of the 19th Annual Conference on Computer Science, CSC ’91*, (New York, NY, USA), p. 652, Association for Computing Machinery, 1991.
- [126] N. M. Preguiça, C. Baquero, and M. Shapiro, “Conflict-free replicated data types (CRDTs),” in *Encyclopedia of Big Data Technologies*, pp. 1–10, Cham: Springer, 2018.
- [127] X. Chu, I. F. Ilyas, and P. Papotti, “Holistic data cleaning: Putting violations into context,” in *2013 IEEE 29th International Conference on Data Engineering (ICDE)*, pp. 458–469, IEEE, 2013.
- [128] R. Shrestha, O. Habibelahian, A. Termehchy, and P. Papotti, “Exploratory training: When annotators learn about data,” *Proceedings of the ACM on Management of Data*, vol. 1, no. 2, pp. 1–25, 2023.
- [129] S. Thirumuruganathan, L. Berti-Equille, M. Ouzzani, J.-A. Quiane-Ruiz, and N. Tang, “Uguide: User-guided discovery of fd-detectable errors,” in *Proceedings of the 2017 ACM International Conference on Management of Data, SIGMOD ’17*, (New York, NY, USA), p. 1385–1397, Association for Computing Machinery, 2017.
- [130] W. Fan, P. Lu, C. Tian, and J. Zhou, “Deducing certain fixes to graphs,” *Proceedings of the VLDB Endowment*, vol. 12, no. 7, pp. 752–765, 2019.
- [131] M. Mahdavi and Z. Abedjan, “Semi-supervised data cleaning with raha and baran.,” in *CIDR*, 2021.
- [132] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, “HoloClean: Holistic data repairs with probabilistic inference,” *Proceedings of the VLDB Endowment*, vol. 10, no. 11, pp. 1190–1201, 2017.
- [133] H. Eto, F. Guo, and E. Miyano, “Distance- d independent set problems for bipartite and chordal graphs,” *Journal of Combinatorial Optimization*, vol. 27, no. 1, pp. 88–99, 2014. Preliminary version in COCOA 2012, LNCS 7402, pages 234–244.
- [134] R. M. Karp, “Reducibility among combinatorial problems,” in *Complexity of Computer Computations* (R. E. Miller and J. W. Thatcher, eds.), pp. 85–103, New York: Plenum Press, 1972.

- [135] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. San Francisco: W. H. Freeman, 1979.
- [136] M. Xiao and H. Nagamochi, “Exact algorithms for maximum independent set,” *Information and Computation*, vol. 255, pp. 126–146, 2017.
- [137] M. Xiao and H. Nagamochi, “Confining sets and avoiding bottleneck cases: A simple maximum independent set algorithm in degree-3 graphs,” *Theoretical Computer Science*, vol. 469, pp. 92–104, 2013.
- [138] M. Halldórsson and J. Radhakrishnan, “Greed is good: Approximating independent sets in sparse and bounded-degree graphs,” in *Proceedings of the twenty-sixth annual ACM symposium on Theory of computing*, pp. 439–448, 1994.
- [139] K. Casel and M. L. Schmid, “Fine-Grained Complexity of Regular Path Queries,” in *24th International Conference on Database Theory (ICDT 2021)* (K. Yi and Z. Wei, eds.), vol. 186 of *Leibniz International Proceedings in Informatics (LIPIcs)*, (Dagstuhl, Germany), pp. 19:1–19:20, Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021.
- [140] I. Neo4j, “Neo4j graph database,” 2024.
- [141] I. Neo4j, “Neo4j auradb,” 2024.
- [142] A. M. Amedeo Pachera, Angela Bonifati, “Code repository,” 2024.
- [143] I. Neo4j, “Star wars,” 2022.
- [144] I. Neo4j, “Stackoverflow,” 2020.
- [145] I. Neo4j, “Wwc2019,” 2020.
- [146] I. Neo4j, “Fincen,” 2020.
- [147] J. Walonoski, M. Kramer, J. Nichols, A. Quina, C. Moesel, D. Hall, C. Duffett, K. Dube, T. Gallagher, and S. McLachlan, “Synthea: An approach, method, and software mechanism for generating synthetic patients and the synthetic electronic health care record,” *Journal of the American Medical Informatics Association*, vol. 25, pp. 230–238, 08 2017.
- [148] I. Neo4j, “Icij offshore leaks,” 2021.
- [149] A. Bonifati, W. Martens, and T. Timm, “DARQL: deep analysis of SPARQL queries,” in *Companion of the The Web Conference 2018 on The Web Conference 2018, WWW 2018, Lyon , France, April 23-27, 2018* (P. Champin, F. Gandon, M. Lalmas, and P. G. Ipeirotis, eds.), pp. 187–190, ACM, 2018.
- [150] H. Paulheim, “Knowledge graph refinement: A survey of approaches and evaluation methods,” *Semantic Web*, vol. 8, no. 3, pp. 489–508, 2017.
- [151] Y. Cheng, L. Chen, Y. Yuan, G. Wang, B. Li, and F. Jin, “Strict and flexible rule-based graph repairing,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 7, pp. 3521–3535, 2022.

- [152] J. Lajus, L. Galárraga, and F. Suchanek, “Fast and exact rule mining with amie 3,” in *The Semantic Web: 17th International Conference, ESWC 2020, Heraklion, Crete, Greece, May 31–June 4, 2020, Proceedings 17*, pp. 36–52, Springer, 2020.
- [153] P. C. Ecosystem, “Flask.”
- [154] Neo4j, “APOC - Awesome Procedures On Cypher.” <https://neo4j.com/labs/apoc/>, 2025. Last accessed: 2025-07-08.
- [155] Neo4j, “GDS - Graph Data Science.” <https://neo4j.com/docs/graph-data-science/current/>, 2025. Last accessed: 2025-07-08.
- [156] A. M. Amedeo Pachera, Angela Bonifati, “Code repository,” 2025.
- [157] G. Li, Y. Zheng, J. Fan, J. Wang, and R. Cheng, “Crowdsourced data management: Overview and challenges,” in *Proceedings of the 2017 ACM international conference on Management of Data*, pp. 1711–1716, 2017.
- [158] K. Huang, H. E. Chua, S. S. Bhowmick, B. Choi, and S. Zhou, “Catapult: data-driven selection of canned patterns for efficient visual graph query formulation,” in *Proceedings of the 2019 International Conference on Management of Data*, pp. 900–917, 2019.
- [159] J. Yang, J. Redi, G. Demartini, and A. Bozzon, “Modeling task complexity in crowdsourcing,” in *Proceedings of the AAAI Conference on Human Computation and Crowdsourcing*, vol. 4, pp. 249–258, 2016.
- [160] S. Basu Roy, I. Lykourantzou, S. Thirumuruganathan, S. Amer-Yahia, and G. Das, “Task assignment optimization in knowledge-intensive crowdsourcing,” *The VLDB Journal*, vol. 24, no. 4, pp. 467–491, 2015.
- [161] H. Rahman, S. B. Roy, S. Thirumuruganathan, S. Amer-Yahia, and G. Das, “Task assignment optimization in collaborative crowdsourcing,” in *2015 IEEE International Conference on Data Mining*, pp. 949–954, IEEE, 2015.
- [162] D. Hettiachchi, V. Kostakos, and J. Goncalves, “A survey on task assignment in crowdsourcing,” *ACM Computing Surveys (CSUR)*, vol. 55, no. 3, pp. 1–35, 2022.
- [163] H. Rahman, S. Thirumuruganathan, S. B. Roy, S. Amer-Yahia, and G. Das, “Worker skill estimation in team-based tasks,” *Proceedings of the VLDB Endowment*, vol. 8, no. 11, pp. 1142–1153, 2015.
- [164] S. Abadal, A. Jain, R. Guirado, J. López-Alonso, and E. Alarcón, “Computing graph neural networks: A survey from algorithms to accelerators,” *ACM Computing Surveys (CSUR)*, vol. 54, no. 9, pp. 1–38, 2021.
- [165] H. Wu, T. Ma, L. Wu, F. Xu, and S. Ji, “Exploiting heterogeneous graph neural networks with latent worker/task correlation information for label aggregation in crowdsourcing,” *ACM Transactions on Knowledge Discovery from Data (TKDD)*, vol. 16, no. 2, pp. 1–18, 2021.

- [166] L. Akoglu, H. Tong, and D. Koutra, “Graph based anomaly detection and description: a survey,” *Data mining and knowledge discovery*, vol. 29, no. 3, pp. 626–688, 2015.
- [167] A. Roy, J. Shu, J. Li, C. Yang, O. Elshocht, J. Smeets, and P. Li, “Gad-nr: Graph anomaly detection via neighborhood reconstruction,” in *Proceedings of the 17th ACM international conference on web search and data mining*, pp. 576–585, 2024.
- [168] Y. Huang, L. Wang, F. Zhang, and X. Lin, “Unsupervised graph outlier detection: Problem revisit, new insight, and superior method,” in *2023 IEEE 39th International Conference on Data Engineering (ICDE)*, pp. 2565–2578, IEEE, 2023.
- [169] S. Bandyopadhyay, L. N. S. V. Vivek, and M. N. Murty, “Outlier resistant unsupervised deep architectures for attributed network embedding,” in *Proceedings of the 13th international conference on web search and data mining*, pp. 25–33, 2020.
- [170] S. Liang, Y. Ding, Z. Li, B. Liang, S. Zhang, Y. Wang, and F. Chen, “Can gnns learn link heuristics? a concise review and evaluation of link prediction methods,” *IEEE Transactions on Big Data*, 2025.
- [171] O. C. et al., “A cognitive load theory approach to defining and measuring task complexity through element interactivity,” *Educational Psych. Rev.*, vol. 35, no. 2, 2023.
- [172] V. Y. et al., “Scalability of network visualisation from a cognitive load perspective,” *IEEE Trans. Visualization & Computer Graphics*, vol. 27, no. 2, pp. 1677–1687, 2021.
- [173] B. D. et al., “Data quality in online human-subjects research: Comparisons between mturk, prolific, cloudresearch, qualtrics, and sona,” *Plos one*, vol. 18, no. 3, 2023.
- [174] M. L. Fisher, “The lagrangian relaxation method for solving integer programming problems,” *Management science*, vol. 27, no. 1, pp. 1–18, 1981.
- [175] R. K. Iyer and J. A. Bilmes, “Submodular optimization with submodular cover and submodular knapsack constraints,” *Advances in neural information processing systems*, vol. 26, 2013.
- [176] A. Pachera, A. Bonifati, L. V. Lakshmanan, , and A. Mauri, “Code repository,” 2026.
- [177] I. Neo4j, “Fda adverse event reporting system,” 2022.
- [178] S. Qi, H. Lin, Z. Guo, G. Szárnyas, B. Tong, Y. Zhou, B. Yang, J. Zhang, Z. Wang, Y. Shen, C. Wang, P. Peiravi, H. Gabb, and B. Steer, “The ldbc financial benchmark,” 2023.
- [179] R. Angles, J. B. Antal, A. Averbuch, A. Birler, P. Boncz, M. Búr, O. Erling, A. Gubichev, V. Haprian, M. Kaufmann, J. L. L. Pey, N. Martínez, J. Marton, M. Paradies, M.-D. Pham, A. Prat-Pérez, D. Püroja, M. Spasić, B. A. Steer, D. Szakállas, G. Szárnyas, J. Waudby, M. Wu, and Y. Zhang, “The ldbc social network benchmark,” 2024.
- [180] M. D. Reckase, “18 multidimensional item response theory,” *Handbook of statistics*, vol. 26, pp. 607–642, 2006.

- [181] J. Li, H. Dani, X. Hu, and H. Liu, “Radar: Residual analysis for anomaly detection in attributed networks,” in *IJCAI*, vol. 17, pp. 2152–2158, 2017.
- [182] L. Huang, Y. Zhu, Y. Gao, T. Liu, C. Chang, C. Liu, Y. Tang, and C.-D. Wang, “Hybrid-order anomaly detection on attributed networks,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, no. 12, pp. 12249–12263, 2021.
- [183] X. Dong, X. Zhang, Y. Sun, L. Chen, M. Yuan, and S. Wang, “Smoothgcn: Smoothing-aware gnn for unsupervised node anomaly detection,” in *Proceedings of the ACM on Web Conference 2025*, WWW ’25, (New York, NY, USA), p. 1225–1236, Association for Computing Machinery, 2025.
- [184] D. E. Difallah, G. Demartini, and P. Cudré-Mauroux, “Pick-a-crowd: tell me what you like, and i’ll tell you what to do,” in *Proceedings of the 22nd international conference on World Wide Web*, pp. 367–374, 2013.
- [185] Meta AI, “Meta Llama 3.” <https://huggingface.co/meta-llama/Meta-Llama-3-8B>, 2024. Model card and weights.
- [186] E. L. et al., “Computing optimal repairs for functional dependencies,” *ACM TODS*, vol. 45, no. 1, pp. 1–46, 2020.
- [187] A. Q. et al., “Pattern functional dependencies for data cleaning,” *PVLDB*, vol. 13, no. 5, pp. 684–697, 2020.
- [188] H. T. Le, A. Bonifati, and A. Mauri, “Graph consistency rule mining with LLMs: an exploratory study,” in *Proceedings of the 28th International Conference on Extending Database Technology (EDBT 2025)*, pp. 748–754, OpenProceedings.org, 2025.
- [189] H. Terdalkar, A. Bonifati, and A. Mauri, “Graph repairs with large language models: An empirical study,” in *Proceedings of the 8th Joint Workshop on Graph Data Management Experiences and Systems (GRADES) and Network Data Analytics (NDA)*, 2025. Also available as arXiv:2507.03410.
- [190] B. Rahmanian and J. Davis, “User interface design for crowdsourcing systems,” *Proceedings of the Workshop on Advanced Visual Interfaces AVI*, 05 2014.
- [191] U. Gadiraju, G. Demartini, R. Kawase, and S. Dietze, “Crowd anatomy beyond the good and bad: Behavioral traces for crowd worker modeling and pre-selection,” *Comput. Supported Coop. Work*, vol. 28, p. 815–841, sep 2019.
- [192] S. Meng, J. Zhou, X. Chen, Y. Liu, F. Lu, and X. Huang, “Structure-information-based reasoning over the knowledge graph: A survey of methods and applications,” *ACM Trans. Knowl. Discov. Data*, vol. 18, Aug. 2024.
- [193] Y. Li, B. Qian, X. Zhang, and H. Liu, “Graph neural network-based diagnosis prediction,” *Big data*, vol. 8, no. 5, pp. 379–390, 2020.

- [194] D. Cheng, F. Yang, S. Xiang, and J. Liu, “Financial time series forecasting with multi-modality graph neural network,” *Pattern Recognition*, vol. 121, p. 108218, 2022.
- [195] S. X. Rao, S. Zhang, Z. Han, Z. Zhang, W. Min, Z. Chen, Y. Shan, Y. Zhao, and C. Zhang, “xfraud: explainable fraud transaction detection,” *Proc. VLDB Endow.*, vol. 15, p. 427–436, Nov. 2021.
- [196] S. Wang, L. Hu, Y. Wang, X. He, Q. Z. Sheng, M. A. Orgun, L. Cao, F. Ricci, and P. S. Yu, “Graph learning based recommender systems: a review,” in *30th International Joint Conference on Artificial Intelligence, IJCAI 2021*, pp. 4644–4652, International Joint Conferences on Artificial Intelligence, 2021.
- [197] J. Ma, R. Guo, S. Mishra, A. Zhang, and J. Li, “Clear: Generative counterfactual explanations on graphs,” *Advances in neural information processing systems*, vol. 35, pp. 25895–25907, 2022.
- [198] R. Ying, D. Bourgeois, J. You, M. Zitnik, and J. Leskovec, “GNNEExplainer: Generating explanations for graph neural networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.
- [199] A. Meliou, W. Gatterbauer, K. F. Moore, and D. Suciu, “The complexity of causality and responsibility for query answers and non-answers,” *Proceedings of the VLDB Endowment (PVLDB)*, vol. 4, no. 1, pp. 34–45, 2010.
- [200] S. Wachter, B. Mittelstadt, and C. Russell, “Counterfactual explanations without opening the black box: Automated decisions and the GDPR,” *Harvard Journal of Law and Technology*, vol. 31, no. 2, pp. 841–887, 2018.
- [201] A. Balmin, T. Papadimitriou, and Y. Papakonstantinou, “Hypothetical queries in an olap environment,” in *VLDB*, vol. 220, p. 231, 2000.
- [202] D. Deutch, Y. Moskovitch, and N. Rinetzky, “Hypothetical reasoning via provenance abstraction,” in *Proceedings of the 2019 International Conference on Management of Data*, pp. 537–554, 2019.
- [203] J. Su, D. Zou, Z. Zhang, and C. Wu, “Towards robust graph incremental learning on evolving graphs,” in *International Conference on Machine Learning*, pp. 32728–32748, PMLR, 2023.
- [204] A. Gupta, I. S. Mumick, and V. S. Subrahmanian, “Maintaining views incrementally,” *SIGMOD Rec.*, vol. 22, p. 157–166, June 1993.
- [205] X. Shi, B. Cui, Y. Shao, and Y. Tong, “Tornado: A system for real-time iterative analysis over evolving data,” in *Proceedings of the 2016 International Conference on Management of Data*, pp. 417–430, 2016.
- [206] T. A, P. SE, V. B. E, T. N, C. N, K.-C. A, K. D, K. M, C. M, I. B, T. A, S. D, B. G, C. A, K. E, P. S, P. H, W. A, L. TF, and V. PE, “European society of cardiology: cardiovascular disease statistics 2025,” *Eur Heart J.*, May 2026.

- [207] I. of Medicine (US) Committee on a National Surveillance System for Cardiovascular and S. C. Diseases, *A Nationwide Framework for Surveillance of Cardiovascular and Chronic Lung Diseases*. National Academies Press, Washington, DC, US, 2011.
- [208] J. A. Blakeley, P.-Å. Larson, and F. W. Tompa, “Efficiently updating materialized views,” in *SIGMOD*, pp. 61–71, 1986.
- [209] A. Meliou and D. Suciu, “Tiresias: the database oracle for how-to queries,” in *Proceedings of the 2012 ACM SIGMOD International Conference on Management of Data*, pp. 337–348, 2012.
- [210] C. Koch, Y. Ahmad, O. Kennedy, M. Nikolic, A. Nötzli, D. Lupei, and A. Shaikhha, “Dbtoaster: higher-order delta processing for dynamic, frequently fresh views,” *The VLDB Journal*, vol. 23, no. 2, pp. 253–278, 2014.
- [211] M. Nikolic, M. Elseidy, and C. Koch, “LINVIEW: incremental view maintenance for complex analytical queries,” in *SIGMOD*, pp. 253–264, 2014.
- [212] M. Mariappan and K. Vora, “GraphBolt: Dependency-driven synchronous processing of streaming graphs,” in *EuroSys*, pp. 25:1–25:16, 2019.
- [213] K. Vora, R. Gupta, and G. Xu, “KickStarter: Fast and accurate computations on streaming graphs via trimmed approximations,” in *ASPLOS*, pp. 237–251, 2017.
- [214] S. Ko, T. Lee, K. Hong, W. Lee, I. Seo, J. Seo, and W.-S. Han, “iTurboGraph: Scaling and automating incremental graph analytics,” in *SIGMOD*, pp. 977–990, 2021.
- [215] S. Gong, C. Tian, Q. Yin, W. Yu, Y. Zhang, L. Geng, S. Yu, G. Yu, and J. Zhou, “Automating incremental graph processing with flexible memoization,” *Proc. VLDB Endow.*, vol. 14, no. 9, pp. 1613–1625, 2021.
- [216] D. Wu, Z. Li, and T. Mitra, “Inkstream: Instantaneous gnn inference on dynamic graphs via incremental update,” in *2025 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 1273–1285, IEEE, 2025.
- [217] P. Naman and Y. Simmhan, “Ripple: Scalable incremental gnn inferencing on large streaming graphs,” in *2025 IEEE 45th International Conference on Distributed Computing Systems (ICDCS)*, pp. 857–867, IEEE, 2025.
- [218] Q. Wang, H. Lv, Y. Zhang, W.-F. Wong, and B. He, “Incremental GNN embedding computation on streaming graphs,” *CoRR*, vol. abs/2603.20622, 2026.
- [219] L. V. Lakshmanan, A. Russakovsky, and V. Sashikanth, “What-if olap queries with changing dimensions,” in *ICDE*, pp. 1334–1336, 2008.
- [220] H. Herodotou and S. Babu, “Profiling, what-if analysis, and cost-based optimization of mapreduce programs,” *Proceedings of the VLDB Endowment*, vol. 4, no. 11, pp. 1111–1122, 2011.

- [221] Q. V. H. Nguyen, K. Zheng, M. Weidlich, B. Zheng, H. Yin, T. T. Nguyen, and B. Stantic, “What-if analysis with conflicting goals: Recommending data ranges for exploration,” in *2018 IEEE 34th International Conference on Data Engineering (ICDE)*, pp. 89–100, IEEE, 2018.
- [222] E. Wu and S. Madden, “Scorpion: Explaining away outliers in aggregate queries,” *Proc. VLDB Endow.*, vol. 6, no. 8, pp. 553–564, 2013.
- [223] S. Roy and D. Suciu, “A formal approach to finding explanations for database queries,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 1579–1590, 2014.
- [224] A. Meliou, S. Roy, and D. Suciu, “Causality and explanations in databases,” *Proceedings of the VLDB Endowment (PVLDB)*, vol. 7, no. 13, pp. 1715–1716, 2014.
- [225] S. Galhotra, A. Gilad, S. Roy, and B. Salimi, “Hyper: Hypothetical reasoning with what-if and how-to queries using a probabilistic causal approach,” in *Proceedings of the 2022 International Conference on Management of Data*, pp. 1598–1611, 2022.
- [226] A. Johnson, T. Pollard, O. Badawi, and J. Raffa, “eICU Collaborative Research Database Demo,” *PhysioNet*, May 2021. Version 2.0.1.
- [227] B. Khemani, S. Patil, K. Kotecha, and S. Tanwar, “A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions,” *Journal of Big Data*, vol. 11, no. 1, p. 18, 2024.
- [228] H. Zeng, M. Zhang, Y. Xia, A. Srivastava, A. Malevich, R. Kannan, V. Prasanna, L. Jin, and R. Chen, “Decoupling the depth and scope of graph neural networks,” *Advances in neural information processing systems*, vol. 34, pp. 19665–19679, 2021.
- [229] M. Ltd, “Memgraph graph engine,” 2026.
- [230] R. McNamee, “Confounding and confounders,” *Occupational and environmental medicine*, vol. 60, no. 3, pp. 227–234, 2003.
- [231] G. Chi, M. E. Roberts, B. M. Stewart, *et al.*, “How to make causal inferences using texts,” *Science Advances*, vol. 8, no. 44, p. eabg2652, 2022.
- [232] K. Cho, “A brief introduction to causal inference in machine learning,” *arXiv preprint arXiv:2405.08793*, 2024.
- [233] B. Salimi, H. Parikh, M. Kayali, L. Getoor, S. Roy, and D. Suciu, “Causal relational learning,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pp. 241–256, 2020.
- [234] B. Salimi, C. Cole, D. R. K. Ports, and D. Suciu, “Zaliql: Causal inference from observational data at scale,” *Proc. VLDB Endow.*, vol. 10, no. 12, pp. 1957–1960, 2017.

- [235] F. Shen, K. Heravi, O. Gomez, S. Galhotra, A. Gilad, S. Roy, and B. Salimi, “Causal what-if and how-to analysis using hyper,” in *39th IEEE International Conference on Data Engineering, ICDE 2023, Anaheim, CA, USA, April 3-7, 2023*, pp. 3663–3666, IEEE, 2023.
- [236] B. Youngmann, M. Cafarella, A. Gilad, and S. Roy, “Summarized causal explanations for aggregate views,” *Proc. ACM Manag. Data*, vol. 2, Mar. 2024.
- [237] L. E. Bertossi and B. Salimi, “From causes for database queries to repairs and model-based diagnosis and back,” *Theory Comput. Syst.*, vol. 61, no. 1, pp. 191–232, 2017.
- [238] L. Bertossi, “Specifying and computing causes for query answers in databases via database repairs and repair-programs,” *Knowl. Inf. Syst.*, vol. 63, p. 199–231, Jan. 2021.
- [239] L. E. Bertossi and B. Salimi, “Causes for query answers from databases: Datalog abduction, view-updates, and integrity constraints,” *Int. J. Approx. Reason.*, vol. 90, pp. 226–252, 2017.
- [240] B. Youngmann, M. J. Cafarella, B. Salimi, and A. Zeng, “Causal data integration,” *Proc. VLDB Endow.*, vol. 16, no. 10, pp. 2659–2665, 2023.
- [241] S. Sakr, A. Bonifati, H. Voigt, A. Iosup, K. Ammar, R. Angles, W. Aref, M. Arenas, M. Besta, P. A. Boncz, K. Daudjee, E. D. Valle, S. Dumbrava, O. Hartig, B. Haslhofer, T. Hegeman, J. Hidders, K. Hose, A. Iamnitich, V. Kalavri, H. Kapp, W. Martens, M. T. Özsu, E. Peukert, S. Plantikow, M. Ragab, M. R. Ripeanu, S. Salihoglu, C. Schulz, P. Selmer, J. F. Sequeda, J. Shnavier, G. Szárnyas, R. Tommasini, A. Tumeo, A. Uta, A. L. Varbanescu, H.-Y. Wu, N. Yakovets, D. Yan, and E. Yoneki, “The future is big graphs: A community view on graph processing systems,” *Commun. ACM*, vol. 64, p. 62–71, aug 2021.
- [242] AWS, “Amazon neptune.” <https://aws.amazon.com/neptune/>, 2023.
- [243] “Oracle pgx: Parallel graph analytix.” Accessed: 2024-12-04.
- [244] “Sap hana graph.” Accessed: 2024-12-04.
- [245] “Redisgraph module.” Accessed: 2024-12-04.
- [246] “Sparksee graph database.” Accessed: 2024-12-04.
- [247] “Kuzu graph database.” Accessed: 2024-12-04.
- [248] R. Angles, M. Arenas, P. Barceló, A. Hogan, J. L. Reutter, and D. Vrgoc, “Foundations of modern query languages for graph databases,” *ACM Comput. Surv.*, vol. 50, no. 5, pp. 68:1–68:40, 2017.
- [249] openCypher Project, “openCypher Implementation Proposal (CIP).” <https://github.com/opencypher/openCypher/tree/main/cip>, 2025. Last accessed: 2025-07-10.

- [250] A. Deutsch, N. Francis, A. Green, K. Hare, B. Li, L. Libkin, T. Lindaaker, V. Marsault, W. Martens, J. Michels, F. Murlak, S. Plantikow, P. Selmer, O. van Rest, H. Voigt, D. Vrgoč, M. Wu, and F. Zemke, “Graph pattern matching in GQL and SQL/PGQ,” in *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*, pp. 2246–2258, 2022.
- [251] R. Angles, A. Bonifati, R. García, and D. Vrgoč, “Path-based algebraic foundations of graph query languages,” in *Proceedings of the 28th International Conference on Extending Database Technology (to appear)*, 2025.
- [252] S. Sadoughi, N. Yakovets, and G. Fletcher, “Breaking down the data-metadata barrier for effective property graph data management,” in *Proceedings 28th International Conference on Extending Database Technology, EDBT 2025, Barcelona, Spain, March 25-28, 2025*, pp. 978–984, 2025.
- [253] S. Han and Z. G. Ives, “Implementation strategies for views over property graphs,” *Proc. ACM Manag. Data*, vol. 2, May 2024.
- [254] A. Bonifati, F. Murlak, and Y. Ramusat, “Transforming property graphs,” *Proc. VLDB Endow.*, vol. 17, no. 11, pp. 2906–2918, 2024.
- [255] International Organization for Standardization, “ISO/IEC 9075-16:2023 information technology — database languages — SQL — part 16: Property graph queries (SQL/PGQ).” ISO/IEC Standard, 2023.
- [256] J. Pearl, “Direct and indirect effects,” in *Proceedings of the Seventeenth Conference on Uncertainty in Artificial Intelligence*, UAI’01, (San Francisco, CA, USA), p. 411–420, Morgan Kaufmann Publishers Inc., 2001.
- [257] Neo4j, “Cypher Manual: Reference Pattern Usage.” <https://neo4j.com/docs/cypher-manual/current/patterns/reference/>, 2025. Last accessed: 2025-07-08.
- [258] J. Pearl, “Causal inference in statistics: An overview,” *Statistics Surveys*, vol. 3, pp. 96–146, 2009.
- [259] I. Shpitser and J. Pearl, “What counterfactuals can be tested,” in *Proceedings of the Twenty-Third Conference on Uncertainty in Artificial Intelligence*, UAI’07, (Arlington, Virginia, USA), p. 352–359, AUAI Press, 2007.
- [260] S. Shimizu, P. O. Hoyer, A. Hyvärinen, A. Kerminen, and M. Jordan, “A linear non-gaussian acyclic model for causal discovery,” *Journal of Machine Learning Research*, vol. 7, no. 10, 2006.
- [261] Y. Zhuge and H. Garcia-Molina, “Graph structured views and their incremental maintenance,” in *Proceedings 14th International Conference on Data Engineering*, pp. 116–125, 1998.
- [262] S. Han and Z. G. Ives, “Implementing views for property graphs,” *SIGMOD Record*, vol. 54, no. 1, 2025.

- [263] S. Ceri, A. Bernasconi, and A. Gagliardi, “Reactive knowledge management,” in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*, pp. 5574–5582, 2024.
- [264] S. Ceri, A. Bernasconi, A. Gagliardi, D. Martinenghi, L. Bellomarini, and D. Magnanini, “Pg-triggers: Triggers for property graphs,” in *Companion of the 2024 International Conference on Management of Data*, pp. 373–385, 2024.
- [265] P. organization, “Pywhy.” <https://www.pywhy.org>, 2025. Accessed: 2025-05-03.
- [266] D. Arpit, M. Fernandez, I. Feigenbaum, W. Yao, C. Liu, W. Yang, P. Josel, S. Heinecke, E. Hu, H. Wang, S. Hoi, C. Xiong, K. Zhang, and J. C. Nibbles, “Salesforce causalai library: A fast and scalable framework for causal analysis of time series and tabular data,” *arXiv preprint arXiv:2301.10859*, 2023.
- [267] J. Pearl and E. Bareinboim, “Transportability of causal and statistical relations: A formal approach,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 25, pp. 247–254, 2011.
- [268] DoWhy, “DoWhy v0.12 - Causal Inference Library,” 2025. Accessed: February 27, 2025.
- [269] Neo4j, “Neo4j Download,” 2025. Accessed: February 27, 2025.
- [270] B. Schölkopf, F. Locatello, S. Bauer, N. R. Ke, N. Kalchbrenner, A. Goyal, and Y. Bengio, “Toward causal representation learning,” *Proceedings of the IEEE*, vol. 109, no. 5, pp. 612–634, 2021.
- [271] L. Jiao, Y. Wang, X. Liu, L. Li, F. Liu, W. Ma, Y. Guo, P. Chen, S. Yang, and B. Hou, “Causal inference meets deep learning: A comprehensive survey,” *Research*, vol. 7, p. 0467, 2024.
- [272] Y. LeCun, “A path towards autonomous machine intelligence,” tech. rep., OpenReview, 2022. Position paper, version 0.9.2.
- [273] H. Nam, Q. Le Lidec, L. Maes, Y. LeCun, and R. Balestriero, “Causal-JEPA: Learning world models through object-level latent masking,” in *Proceedings of the 43rd International Conference on Machine Learning (ICML)*, 2026. arXiv:2602.11389.
- [274] J. Kaddour, A. Lynch, Q. Liu, M. J. Kusner, and S. Ricardo, “Causal machine learning: A survey and open problems,” *Foundations and Trends in Optimization*, vol. 9, no. 1-2, pp. 1–247, 2025.
- [275] M. Assran, Q. Duval, I. Misra, P. Bojanowski, P. Vincent, M. Rabbat, Y. LeCun, and N. Ballas, “Self-supervised learning from images with a joint-embedding predictive architecture,” in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 15619–15629, 2023.
- [276] A. Bardes, J. Ponce, and Y. LeCun, “MC-JEPA: A joint-embedding predictive architecture for self-supervised learning of motion and content features,” *arXiv preprint arXiv:2307.12698*, 2023.

- [277] D. Chen, M. Shukor, T. Moutakanni, W. Chung, T. Kasarla, Y. Bang, A. Bolourchi, Y. LeCun, and P. Fung, “VL-JEPA: Joint embedding predictive architecture for vision-language,” *arXiv preprint arXiv:2512.10942*, 2025.
- [278] M. Lenzerini, “Data integration: A theoretical perspective,” in *Proceedings of the ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pp. 233–246, 2002.
- [279] A. Y. Halevy, “Answering queries using views: A survey,” *The VLDB Journal*, vol. 10, no. 4, pp. 270–294, 2001.
- [280] E. Bareinboim and J. Pearl, “Causal inference and the data-fusion problem,” *Proceedings of the National Academy of Sciences (PNAS)*, vol. 113, no. 27, pp. 7345–7352, 2016.
- [281] S. Beckers and J. Y. Halpern, “Abstracting causal models,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2019.
- [282] D. M. Chickering, “Optimal structure identification with greedy search,” *Journal of Machine Learning Research*, vol. 3, pp. 507–554, 2002.
- [283] T. Richardson and P. Spirtes, “Ancestral graph Markov models,” *The Annals of Statistics*, vol. 30, no. 4, pp. 962–1030, 2002.
- [284] X. Zheng, B. Aragam, P. Ravikumar, and E. P. Xing, “Dags with no tears: Continuous optimization for structure learning,” in *Advances in Neural Information Processing Systems*, vol. 31, 2018.
- [285] I. Shpitser and J. Pearl, “Complete identification methods for the causal hierarchy,” *Journal of Machine Learning Research*, vol. 9, pp. 1941–1979, 2008.
- [286] P. K. Rubenstein, S. Weichwald, S. Bongers, J. M. Mooij, D. Janzing, M. Grosse-Wentrup, and B. Schölkopf, “Causal consistency of structural equation models,” in *Proceedings of the 33rd Conference on Uncertainty in Artificial Intelligence*, 2017.
- [287] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, *et al.*, “V-jepa 2: Self-supervised video models enable understanding, prediction and planning,” *arXiv preprint arXiv:2506.09985*, 2025.
- [288] M. Arjovsky, L. Bottou, I. Gulrajani, and D. Lopez-Paz, “Invariant risk minimization,” *arXiv preprint arXiv:1907.02893*, 2019.
- [289] J. Peters, P. Bühlmann, and N. Meinshausen, “Causal inference by using invariant prediction: Identification and confidence intervals,” *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, vol. 78, no. 5, pp. 947–1012, 2016.
- [290] P. Humane, P. Cudrano, D. Z. Kaplan, M. Matteucci, S. Chakraborty, and I. Rish, “Influence functions for efficient data selection in reasoning,” *arXiv preprint arXiv:2510.06108*, 2025.

- [291] M. Kocaoglu, C. Snyder, A. G. Dimakis, and S. Vishwanath, “Causalgan: Learning causal implicit generative models with adversarial training,” in *International Conference on Learning Representations*, 2018.
- [292] A. Geiger, Z. Wu, H. Lu, J. Rozner, E. Kreiss, T. Icard, N. Goodman, and C. Potts, “Inducing causal structure for interpretable neural networks,” in *International Conference on Machine Learning*, pp. 7324–7338, PMLR, 2022.
- [293] Y. Zhang, Y. Zhang, Y. Zheng, B. Raducanu, and D. Liu, “Causal-tune: mining causal factors from vision foundation models for domain generalized semantic segmentation,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, pp. 12916–12924, 2026.
- [294] A. Zeng, M. Cafarella, B. Kenig, M. Markakis, B. Youngmann, and B. Salimi, “Causal dag summarization,” *Proceedings of the VLDB Endowment*, vol. 18, no. 6, pp. 1933–1947, 2025.
- [295] A. Mansouri, J. Hartford, Y. Zhang, and Y. Bengio, “Object centric architectures enable efficient causal representation learning,” in *International Conference on Learning Representations*, vol. 2024, pp. 830–853, 2024.
- [296] S. Yaramakala and D. Margaritis, “Speculative markov blanket discovery for optimal feature selection,” in *Fifth IEEE International Conference on Data Mining (ICDM’05)*, pp. 4–pp, IEEE, 2005.
- [297] X. Chen, R. Sun, U. Saluz, S. Schiavon, and P. Geyer, “Using causal inference to avoid fallouts in data-driven parametric analysis: A case study in the architecture, engineering, and construction industry,” *Developments in the Built Environment*, vol. 17, p. 100296, 2024.
- [298] R. Angles, “The property graph database model,” in *AMW*, vol. 2100 of *CEUR Workshop Proceedings*, CEUR-WS.org, 2018.
- [299] A. Ahadi, J. Prior, V. Behbood, and R. Lister, “Students’ semantic mistakes in writing seven different types of sql queries,” in *Proceedings of the 2016 ACM Conference on Innovation and Technology in Computer Science Education*, ITiCSE ’16, (New York, NY, USA), p. 272–277, Association for Computing Machinery, 2016.
- [300] A. Ahadi, V. Behbood, A. Vihavainen, J. Prior, and R. Lister, “Students’ syntactic mistakes in writing seven different types of sql queries and its application to predicting students’ success,” in *Proceedings of the 47th ACM Technical Symposium on Computing Science Education*, SIGCSE ’16, (New York, NY, USA), p. 401–406, Association for Computing Machinery, 2016.
- [301] R. B. Buitendijk, “Logical errors in database sql retrieval queries,” *Comput. Sci. Econ. Manage.*, vol. 1, p. 79–96, jun 1988.
- [302] S. Brass and C. Goldberg, “Semantic errors in sql queries: A quite complete list,” *Journal of Systems and Software*, vol. 79, no. 5, pp. 630–644, 2006. Quality Software.

- [303] D. Miedema, E. Aivaloglou, and G. Fletcher, “Identifying sql misconceptions of novices: Findings from a think-aloud study,” in *Proceedings of the 17th ACM Conference on International Computing Education Research*, ICER 2021, (New York, NY, USA), p. 355–367, Association for Computing Machinery, 2021.
- [304] D. Miedema, G. Fletcher, and E. Aivaloglou, “So many brackets! an analysis of how sql learners (mis)manage complexity during query formulation,” in *Proceedings of the 30th IEEE/ACM International Conference on Program Comprehension*, ICPC ’22, (New York, NY, USA), p. 122–132, Association for Computing Machinery, 2022.
- [305] D. Miedema, G. Fletcher, and E. Aivaloglou, “Expert perspectives on student errors in sql,” *ACM Trans. Comput. Educ.*, vol. 23, dec 2022.
- [306] M. Chen, S. Poulsen, R. Alkhabaz, and A. Alawini, “A quantitative analysis of student solutions to graph database problems,” in *Proceedings of the 26th ACM Conference on Innovation and Technology in Computer Science Education V. 1*, ITiCSE ’21, (New York, NY, USA), p. 283–289, Association for Computing Machinery, 2021.
- [307] R. Alkhabaz, S. Poulsen, M. Chen, and A. Alawini, “Insights from student solutions to mongodb homework problems,” in *Proceedings of the 26th ACM Conference on Innovation and Technology in Computer Science Education V. 1*, ITiCSE ’21, (New York, NY, USA), p. 276–282, Association for Computing Machinery, 2021.
- [308] R. Alkhabaz, Z. Li, S. Yang, and A. Alawini, “Student’s learning challenges with relational, document, and graph query languages,” in *Proceedings of the 2nd International Workshop on Data Systems Education: Bridging Education Practice with Education Research*, DataEd ’23, (New York, NY, USA), p. 30–36, Association for Computing Machinery, 2023.
- [309] solidIT, “Db-engines ranking.” https://db-engines.com/en/ranking_trend/graph+dbms, 2023.
- [310] M. Paradies, “Graph pattern matching in sap hana.” <https://tinyurl.com/ycxu54pr>, 2017.
- [311] R. Labs, “Redisgrap.” <https://oss.redislabs.com/redisgraph/>, 2017.
- [312] B. G. Inc., “Agensgraph.” <https://bitnine.net/agensgraph/>, 2023.
- [313] A. D. Fekete and U. Röhm, “Teaching about data and databases: Why, what, how?,” *SIGMOD Rec.*, vol. 51, no. 2, pp. 52–60, 2022.
- [314] T. Taipalus, M. Siponen, and T. Vartiainen, “Errors and complications in sql query formulation,” *ACM Trans. Comput. Educ.*, vol. 18, aug 2018.
- [315] J. Melton and A. Simon, *SQL:1999: Understanding Relational Language Components*. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2001.
- [316] A. Migler and A. Dekhtyar, “Mapping the sql learning process in introductory database courses,” in *Proceedings of the 51st ACM Technical Symposium on Computer Science*

- Education*, SIGCSE '20, (New York, NY, USA), p. 619–625, Association for Computing Machinery, 2020.
- [317] Z. Li, S. Yang, K. Cunningham, and A. Alawini, “Assessing student learning across various database query languages,” in *2023 IEEE Frontiers in Education Conference (FIE)*, pp. 1–9, 2023.
 - [318] E. Tshukudu and Q. I. Cutts, “Semantic transfer in programming languages: Exploratory study of relative novices,” in *ITiCSE*, pp. 307–313, ACM, 2020.
 - [319] A. Bonifati, G. Fletcher, H. Voigt, and N. Yakovets, *Data Models*, pp. 3–14. Cham: Springer International Publishing, 2018.
 - [320] J. Arnoux, A. Bonifati, A. Calteau, S. Dumbrava, and G. Gautreau, “Integrating complex pangenome graphs,” in *ICDEW*, pp. 350–354, IEEE, 2024.
 - [321] Y. Peng, S. Tang, D. Wang, H. Zhong, H. Jia, X. Cai, Z. Zhang, M. Xiao, H. Yang, J. Wang, K. Kristiansen, X. Xu, and J. Li, “MetaPGN: a pipeline for construction and graphical visualization of annotated pangenome networks,” *Gigascience*, vol. 7, Nov. 2018.
 - [322] PGQL, “Property graph query language,” 2025. Accessed: 2026-09-09.
 - [323] L. Shi, Z. Tang, N. Zhang, X. Zhang, and Z. Yang, “A survey on employing large language models for text-to-sql tasks,” *ACM Computing Surveys*, vol. 58, no. 2, pp. 1–37, 2025.
 - [324] R. C. Fernandez, A. J. Elmore, M. J. Franklin, S. Krishnan, and C. Tan, “How large language models will disrupt data management,” *Proceedings of the VLDB Endowment*, vol. 16, no. 11, pp. 3302–3309, 2023.
 - [325] Z. Hong, Z. Yuan, Q. Zhang, H. Chen, J. Dong, F. Huang, and X. Huang, “Next-generation database interfaces: A survey of LLM-based text-to-SQL,” *IEEE Transactions on Knowledge and Data Engineering*, 2025. arXiv:2406.08426.
 - [326] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin, V. Zhong, C. Xiong, R. Sun, Q. Liu, S. I. Wang, and T. Yu, “Spider 2.0: Evaluating language models on real-world enterprise text-to-SQL workflows,” in *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, 2025. Oral presentation; arXiv:2411.07763.
 - [327] T. Jin, Y. Choi, Y. Zhu, and D. Kang, “Text-to-SQL benchmarks are broken: An in-depth analysis of annotation errors,” in *Proceedings of the 16th Annual Conference on Innovative Data Systems Research (CIDR)*, 2026.
 - [328] M. G. Ozsoy, L. Messallem, J. Besga, and G. Minneci, “Text2cypher: Bridging natural language and graph databases,” in *Proceedings of the Workshop on Generative AI and Knowledge Graphs (GenAIK)*, pp. 100–108, 2025. arXiv:2412.10064.
 - [329] Y. Feng, S. Papicchio, and S. Rahman, “CypherBench: Towards precise retrieval over full-scale modern knowledge graphs in the LLM era,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025. arXiv:2412.18702.

- [330] V. Chauhan, S. Raj, S. Mujumdar, A. Saha, and A. Jain, “Mind the query: A benchmark dataset towards Text2Cypher task,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 1890–1905, 2025.
- [331] Y. Luo, G. Li, J. Fan, and N. Tang, “Data agents: Levels, state of the art, and open problems,” in *Companion of the 2026 International Conference on Management of Data (SIGMOD/PODS ’26)*, ACM, 2026. Tutorial; arXiv:2602.04261.
- [332] Anthropic, “Model context protocol.” <https://modelcontextprotocol.io>, 2024.
- [333] A. Biswal, L. Patel, S. Jha, A. Kamsetty, S. Liu, J. E. Gonzalez, C. Guestrin, and M. Zaharia, “Text2sql is not enough: Unifying ai and databases with tag,” *arXiv preprint arXiv:2408.14717*, 2024.